
[野火]Python 应用开发实战指南

—基于 LubanCat-RK 系列板卡

EmbedFire
野火电子

2025 年 09 月 09 日

Contents

关于本项目.....	1
鲁班猫 (LubanCat)	1
文档说明	1
板卡其他配套在线文档及教程	2
关于野火	3
开源共享, 共同进步	3
项目定制	3
联系方式	3
第 1 章 鲁班猫系列板卡与 Python	5
1.1 Python 简介	5
1.2 鲁班猫简介	5
1.3 鲁班猫 +Python=?	6
1.4 参考资料	6
第 2 章 安装 Python	7
2.1 关于 Python 版本	7
2.2 安装 python 和 pip	8
2.3 python 和 pip 的默认版本切换	10
2.3.1 手动创建链接	10
2.3.2 使用 update-alternatives	11
2.4 软件库安装方法	13
2.4.1 pip 下载加速	13
2.4.2 用 apt 代替 pip 安装软件包	13
2.4.3 用 setuptools 工具安装软件包	14
2.4.4 UV	15
2.5 参考资料	19
第 3 章 运行 Python	20
3.1 使用 python 终端交互式环境	20

3.2	使用 python 脚本	22
3.3	使用 python IDE	24
3.4	import 其它软件包	27
3.5	参考资料	30
第 4 章	Python 基础速览	31
4.1	注释	31
4.2	缩进	31
4.3	变量和数据类型	32
4.3.1	String(字符串)	33
4.3.2	List(列表)	34
4.3.3	Tuple(元组)	34
4.4	流程控制相关	35
4.4.1	条件控制	35
4.4.2	循环语句	35
4.4.3	迭代	36
4.5	函数	36
4.6	模块和包	37
4.6.1	模块	37
4.6.2	包	38
4.7	虚拟环境	38
4.8	参考文档	39
第 5 章	控制 GPIO	40
5.1	libgpiod 基本概念	40
5.2	实验准备	41
5.3	方式一：使用 python3-libgpiod	42
5.3.1	安装 python3-libgpiod	42
5.3.2	libgpiod 输出	42
5.3.3	libgpiod 输入输出	45
5.4	方式二：使用 python-periphery	46
5.4.1	安装 python-periphery	46
5.4.2	periphery 输入输出	46

5.5	方式三：使用 Adafruit Blinka	48
5.5.1	安装 Adafruit Blinka	48
5.5.2	Adafruit Blinka 板卡支持测试	49
5.5.3	Adafruit-Blinka 控制 IO	51
5.6	参考资料	53
第 6 章	PWM 输出	54
6.1	PWM 实验	54
6.2	实验准备	55
6.3	使用 python-periphery	56
6.3.1	安装 python-periphery	56
6.3.2	periphery 输出 PWM	57
第 7 章	UART 通讯	60
7.1	实验准备	60
7.1.1	添加 UART 资源	60
7.1.2	硬件连接	62
7.2	方式一：使用 pyserial 库	62
7.2.1	安装 pyserial	63
7.2.2	使用 pyserial	63
7.3	方式二：使用 python-periphery	66
7.3.1	安装 python-periphery	67
7.3.2	periphery 使用 UART 功能	67
第 8 章	I2C 通讯	71
8.1	I2C 实验	71
8.2	实验准备	72
8.2.1	板卡添加 I2C 资源	72
8.2.2	硬件连接	73
8.3	使用 python-periphery	74
8.3.1	安装 python-periphery	74
8.3.2	periphery 使用 I2C 功能	74
8.4	方式二：使用 Adafruit Blinka	80
8.4.1	安装 Adafruit Blinka	80

8.4.2	Adafruit-Blinka 使用 I2C 主机	80
8.4.3	Adafruit_CircuitPython_SSD1306	84
第 9 章	SPI 通讯	88
9.1	SPI 实验	88
9.2	实验准备	88
9.2.1	添加 SPI 资源	88
9.2.2	硬件连接	90
9.3	使用 python-periphery	90
9.3.1	安装 python-periphery	91
9.3.2	periphery 使用 SPI 功能	91
9.4	方式二：使用 Adafruit Blinka	93
9.4.1	安装 Adafruit Blinka	93
9.4.2	Adafruit-Blinka 使用 SPI 主机	93
第 10 章	屏幕显示-Pygame	96
10.1	Pygame 库简介	96
10.2	实验准备	96
10.2.1	添加屏幕资源	96
10.2.2	硬件连接	97
10.2.3	Pygame 库安装	97
10.2.4	Pygame 库使用	98
10.3	测试代码一	98
10.3.1	实验步骤	100
10.4	测试代码二	100
10.4.1	实验步骤	103
10.5	参考	103
第 11 章	音频播放-Pygame	104
11.1	Pygame 库简介	104
11.2	实验准备	104
11.2.1	板卡上的 Sound 资源	104
11.2.2	硬件连接	105
11.2.3	Pygame 库安装	105

11.2.4	Pygame 库使用	105
11.3	测试代码	105
11.3.1	实验步骤	107
11.3.2	参考	108
第 12 章	摄像头使用-Opencv	109
12.1	opencv-python 库简介	109
12.2	鲁班猫板卡添加 camera 资源	109
12.3	相关工具及库安装	110
12.4	摄像头拍照	111
12.5	摄像头捕获、录制视频	114
12.6	参考	116
第 13 章	Jupyter Notebook	117
13.1	Jupyter Notebook 简介	117
13.2	Jupyter Notebook 安装	118
13.2.1	软件安装	118
13.2.2	配置 Jupyter	118
13.2.3	安全登录配置	119
13.3	Jupyter Notebook 使用前注意事项	120
13.4	Jupyter Notebook 使用	121
13.4.1	启动软件	121
13.4.2	上传代码	123
13.5	参考链接	124
第 14 章	PIL 库	125
14.1	PIL 库简介	125
14.2	PIL 库安装	125
14.3	PIL 库使用	125
第 15 章	Numpy 库	129
15.1	Numpy 库简介	129
15.2	NumPy 库安装	129
15.3	NumPy 库使用	129

第 16 章	Pandas 库	131
16.1	Pandas 库简介	131
16.2	Pandas 库安装	131
16.3	Pandas 库使用	131
第 17 章	Web 库 - Flask	133
17.1	Flask 库简介	133
17.2	Flask 库安装	133
17.3	Flask 库使用	134
17.3.1	新建工程目录	134
17.3.2	添加代码文件	134
17.3.3	运行 webapp	136
17.3.4	参考	137
第 18 章	Web 库 - Django	138
18.1	Django 库简介	138
18.2	Django 库安装	138
18.3	Django 库使用	138
18.3.1	新建工程目录	138
18.3.2	添加代码文件及配置	139
18.3.3	启动 webapp	140
18.3.4	Django 版本注意事项	141
第 19 章	Modbus 协议-pymodbus 库	142
19.1	Modbus 协议实验	142
19.2	python3-pymodbus 库	142
19.3	实验准备	143
19.3.1	添加板卡 uart 资源	143
19.3.2	硬件连接	145
19.3.3	pymodbus 库安装	146
19.4	测试代码	146
19.4.1	RTU 从机 slave_server.py	146
19.4.2	实验步骤	152
19.4.3	现象简析	158

19.4.4 参考	159
第 20 章 GUI 库 - PyQt5	160
20.1 PyQt5 库简介	160
20.2 PyQt5 库安装	161
20.2.1 Qt 库安装	161
20.3 PyQt5 库基础使用	162
20.3.1 导入需要模块	162
20.3.2 创建一个类将作为主窗口	163
20.3.3 创建应用程序	164
20.3.4 整体程序	165
20.4 Qt Designer 使用	168
20.4.1 创建一个界面 UI 文件	168
20.4.2 界面构建	168
20.4.3 调用 ui 文件	173
20.5 参考	176
第 21 章 NPU 使用	177
21.1 rknn-Toolkit2	177
21.1.1 rknn-Toolkit2 安装	178
21.1.2 模型转换和模型推理	179
21.1.3 性能和内存评估	180
21.2 RKNN Toolkit Lite2	184
21.2.1 板卡上安装 RKNN Toolkit Lite2	184
21.2.2 板端部署推理	186
21.3 相关频率设置 (rk356X)	190
21.4 参考	191
第 22 章 手写数字识别—飞桨 (PaddlePaddle)	192
22.1 飞桨 (PaddlePaddle) 简介	192
22.2 安装飞桨 (PaddlePaddle)	193
22.3 手写数字识别任务	194
22.3.1 准备训练集和测试集	194
22.3.2 模型组网	195

22.3.3	模型训练与评估	196
22.3.4	模型保存和导出 ONNX 模型	197
22.3.5	完整程序	198
22.3.6	模拟推理和导出 RKNN 模型	201
22.4	板端部署推理	203
22.4.1	安装 RKNN Toolkit Lite2 和相关库	203
22.4.2	推理测试	204
22.5	总结	206
22.6	参考链接	206
第 23 章	花卉图像分类-TensorFlow	207
23.1	安装 tensorflow 环境	207
23.2	图像分类	208
23.2.1	准备数据集	208
23.2.2	构建和训练模型	210
23.2.3	测试模型	216
23.2.4	TensorFlow Lite 模型	216
23.2.5	模型转换和模拟测试	217
23.3	部署推理测试	220
23.4	总结	223
23.5	参考链接	223
第 24 章	ResNet18 网络-PyTorch	224
24.1	PyTorch 和 ResNet18	224
24.1.1	PyTorch 安装	224
24.1.2	ResNet18 结构简介	226
24.1.3	PyTorch 中的 ResNet18 实现	227
24.2	ResNet18 实现	227
24.2.1	数据集准备和数据预处理	227
24.2.2	构建模型	229
24.2.3	训练和测试模型	235
24.2.4	保存为 onnx 模型	237
24.2.5	导出 RKNN 模型和模拟测试	238

24.2.6	板端部署测试	240
24.3	参考	245
第 25 章	YOLOv5	246
25.1	YOLOv5 环境安装	246
25.2	YOLOv5 简单使用	247
25.2.1	获取预训练权重文件	247
25.2.2	YOLOv5 简单测试	247
25.2.3	转换为 rknn 模型	249
25.2.4	部署到鲁班猫板卡	259
25.3	airockchip/yolov5 简单测试	262
25.3.1	导出 rknn 模型	264
25.4	参考链接	267
第 26 章	YOLOv10	268
26.1	YOLOv10 简单测试	269
26.1.1	环境配置	269
26.1.2	模型推理测试	269
26.2	板卡上部署测试	270
26.2.1	导出适配的 onnx 模型	270
26.2.2	导出 rknn 模型	271
26.2.3	部署测试	274
26.3	参考链接	276
第 27 章	SenseVoice	277
27.1	RKNN 模型转换	277
27.2	Lubancat 板卡上测试	280
27.2.1	测试语音文件	280
27.3	参考链接	283
第 28 章	鲁班猫监控检测	284
28.1	依赖工具及库安装	284
28.2	视频流服务器和摄像头获取帧	285
28.2.1	部署视频流服务器	285
28.2.2	摄像头中获取帧	288

28.3	NPU 处理图像	289
28.4	适配具体环境	291
28.5	测试实验	293
28.6	参考链接	297
第 29 章	问题反馈	298
版权说明		299

关于本项目

鲁班猫 (LubanCat)

- 1、鲁班猫是野火推出的运行 Linux、Android 的卡片电脑品牌。该系列卡片电脑硬件型号丰富，操作系统适配度高，开源教材资料众多，应用极其简单；
- 2、鲁班猫卓越的性能及其丰富的硬件型号，覆盖了教育、商业应用、工业控制等领域，具备广泛的应用场景：卡片电脑、Linux 服务器、家庭智能化中枢、工业板卡；
- 3、鲁班猫支持 Ubuntu、Debian、Android 等系统，提供多套教材，覆盖纯应用层用户以及系统开发用户，即使初入行业的嵌入式小白，也能根据我们的教程完成开发，而对资深的嵌入式老鸟，则能加速产品二次开发过程。

文档说明

点击右侧链接可在线阅读本项目文档：《[鲁班猫 Python 应用开发实践—基于 LubanCat_RK 系列板卡](#)》

本教程旨在引导开发者在鲁班猫上使用 Python 开发应用程序、控制外围硬件等等。

本教程使用的开发环境说明如下：

- python 版本：Python3 以上
- 开发板系统：适配板卡野火定制的鲁班猫系统镜像（ubuntu、debian 等）。

板卡其他配套在线文档及教程

《快速使用手册—基于 LubanCat-RK356x 系列板卡》

《快速使用手册—基于 LubanCat-RK3588 系列板卡》

《嵌入式 Linux 镜像构建与部署—基于 LubanCat-RK 系列板卡》

《LubanCat-RK 系列板卡应用开发手册》

《嵌入式 Linux 驱动开发实战指南—基于 LubanCat_RK 系列板卡》

《嵌入式 AI 应用开发实战指南—基于 LubanCat-RK 系列板卡》

《LubanCat-RK 系列板卡 Android 用户手册》

关于野火

开源共享，共同进步

野火在发布第一块 STM32 开发板之初，就喊出 **开源共享，共同进步** 的口号，把代码和文档教程都免费提供给用户下载，而我们也一直把这个理念贯穿至今。

目前我们的产品已经包括 *STM32*、*i.MX RT* 系列、*GD32V*、*FPGA*、*Linux*、*emXGUI*、操作系统、网络、下载器等分支，覆盖电子工程应用领域的各种常用技术，其中教学类产品的代码和文档一直保持着开源的姿态发布到网络上，为电子工程师排忧解难，让嵌入式没有难用的技术是我们最大的愿望。

项目定制

野火接受针对 STM32、i.MX RT、i.MX6、FPGA 主控芯片的各种硬件、软件项目定制，目前已有丰富的成功案例，有需要可以直接发送邮件咨询：embedfire@embedfire.com。

联系方式

- 官网：<http://www.embedfire.com>
- 论坛：<http://www.firebbs.cn>
- github 主页：<https://github.com/Embedfire>
- gitee 主页：<https://gitee.com/Embedfire>
- 淘宝：<https://yehuosm.tmall.com>
- 邮箱：embedfire@embedfire.com

- 电话：0769-33894118

第 1 章 鲁班猫系列板卡与 Python

1.1 Python 简介

对于 Python 是什么，此处简要概括如下：

- Python 是一种跨平台的解析型编程语言
- Python 易学、易用、功能强大，被大量应用在 AI、数据处理、编程教育中
- 大量的开发者提供了各种 Python 代码库，其他人可以使用这些库轻松开发自己的程序

关于 Python 更详细的教程，请参考：

- 《廖雪峰-Python 教程》
- 《和孩子一起学编程》
- 《趣学 Python》

1.2 鲁班猫简介

鲁班猫是野火推出的高性能卡片电脑品牌，以鲁班为名，勉励工程师传承鲁班那样的工匠创新精神，争当当代鲁班；以小猫为形，期盼大家如孩童如猫咪般充满好奇心，探索精神不止步，永远保持童心。

鲁班猫系列卡片电脑，硬件型号丰富，操作系统适配度高，开源教材资料众多，应用极其简单，并且为 AI、工控、物联网、机器人以及编程教育等应用场景提供了便捷的示例和应用。

鲁班猫系统是野火基于 Ubuntu、Debian、Android 等系统制作的 Linux 发行版，可以直接使用 APT 包管理工具直接安装、维护软件，如 Python、Opencv、Nginx 以及 Docker 等软件，能轻松部署各种应用程序。

1.3 鲁班猫 +Python= ?

使用传统的 PC 开发与运行纯软件类型的 Python 应用程序也非常方便，但想要控制外部硬件或电气设备时，就需要扩展 IO 卡、运动控制卡等专用硬件，昂贵且复杂。

而手机尺寸大小的鲁班猫系列板卡，集成了 IO、PWM、I2C、USB、网络等各种硬件控制与通讯接口，能轻松与外部设备进行交互，而且功耗低、工业级的稳定性、性价比高等特点，非常适用于创客 DIY、工控产品中。

野火将基于鲁班猫板卡，提供以下 Python 应用开发示例：

- 创建个人网站
- 制作智能家居数据中心
- 制作 AI 智能音箱
- 制作机器人
- 搭建编程教学环境
- ...

1.4 参考资料

- [python 官网](#)
- 《廖雪峰-Python 教程》
- 《和孩子一起学编程》
- 《趣学 Python》

第 2 章 安装 Python

本章讲解如何在鲁班猫板卡上安装 Python 环境。

重要：教程一般基于 Python 3.8.10 版本 (镜像系统是 Ubuntu20.04) 进行实验及讲解。

2.1 关于 Python 版本

鲁班猫镜像是基于 Debian、Ubuntu 等 Linux 发行版本, 这些发行版本默认都附带了 python, 一般附带 python2* 较旧的版本和 python3* 较新的版本 (ubuntu20.04 以上默认不安装 python2), 可使用如下命令查看默认安装的版本 (ubuntu20.04) :

```
# 查看 python3 版本
cat@lubancat:~$ python3 --version
Python 3.8.10

# 查看 python 版本
cat@lubancat:~$ python2 --version
Python 2.7.18

# 查看 pip3 版本
cat@lubancat:~$ pip3 --version
pip 20.0.2 from /usr/lib/python3/dist-packages/pip (python 3.8)
```

提示：Debian10 镜像默认安装的是 python3.7.3; Debian11 是 python3.9.2; ubuntu22.04 是 Python 3.10.12; ubuntu24.04 是 Python 3.12.3

2.2 安装 python 和 pip

鲁班猫系统镜像基于 Debian、Ubuntu 等 Linux 发行版本，默认已经安装了 python，如果没有安装可以使用 apt 命令安装和更新：

```
# 确认连接网络，第一次使用 apt 需要更新
sudo apt update

# 安装 python3 (默认系统已经安装可以不执行)
sudo apt -y install python3

# 安装 pip 工具
sudo apt -y install python3-pip

# 升级 pip 工具
python3 -m pip install --upgrade pip

# 配置 pip 源 (清华源)
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

如果需要其他版本的 python，Ubuntu 系统可以使用 PPA（个人软件包存档），PPA 允许应用程序开发人员和 Linux 用户创建自己的软件仓库来分发软件，通过 PPA 可以轻松地获得更加新的软件版本或获得官方 Ubuntu 仓库以外的软件，但需要注意这些软件的不稳定和安全性等。

下面我们测试在鲁班猫 ubuntu20.04 系统上使用 [deadsnakes 团队](#) 维护的 PPA，添加该 PPA 后安装最新版本的 Python 及附加模块。

```
# 添加 PPA，需要回车确认
sudo add-apt-repository ppa:deadsnakes/ppa

# 更新更新下 apt 缓存
sudo apt update
```

回车后确认添加，会更新下 apt 缓存，在输出信息中我们可以看到支持的列表：

```
Supported Ubuntu and Python Versions
```

(下页继续)

(续上页)

```
=====

- Ubuntu 20.04 (focal) Python3.5 - Python3.7, Python3.9 - Python3.13
- Ubuntu 22.04 (jammy) Python3.7 - Python3.9, Python3.11 - Python3.13
- Ubuntu 24.04 (noble) Python3.7 - Python3.11, Python3.13
- Note: Python2.7 (focal, jammy), Python 3.8 (focal), Python 3.10 (jammy), ↵
  ↵Python3.12 (noble)
are not provided by deadsnakes as upstream ubuntu provides those packages.

Why some packages aren't built:
- Note: for focal, older python versions require libssl<1.1 so they are not ↵
  ↵currently built
- Note: for jammy and noble, older python versions require libssl<3 so they ↵
  ↵are not currently built
- If you need these, reach out to asottile to set up a private ppa
```

以上信息也可以到 [deadsnakes](#) 查看。

然后测试安装指定版本 python:

```
# 安装 python3.13
sudo apt install python3.13

# 检测版本
python3.13 --version
Python 3.13.0b2

# 安装其他附加模块 (可选)
sudo apt install python3.13-full

# 或者单独安装 python 相关包 (可选)
sudo apt install python3.13-dev python3.13-venv
```


最后，python 安装还可以去官网下载 [python 源码包](#)，然后自行编译安装。

2.3 python 和 pip 的默认版本切换

2.3.1 手动创建链接

可以注意到上面命令中 python3、pip3 都带了版本号，主要是由于 python 的历史原因，这样使用能更好地与 python2 区分开来，而 python2 目前已经停止维护了，我们不建议大家继续使用。使用我们提供的镜像默认是安装了 python2 和 python3，可以设置当前系统默认的使用版本。

通过以下命令可以设置 python、pip 命令默认使用 python3（可选）：

```
# 设置软链接, python 默认使用 python3
sudo ln -sf /usr/bin/python3 /usr/bin/python

# 设置软链接, pip 默认使用 pip3
sudo ln -sf /usr/bin/pip3 /usr/bin/pip
```

设置后即可直接使用 python 或 pip 命令，检查当前系统 python 版本（可选）：

```
# 查看 python 路径
cat@lubancat:~$ which python
/usr/bin/python

# 查看 python 版本
cat@lubancat:~$ python --version
Python 3.8.10

# 查看 pip 版本
cat@lubancat:~$ pip --version
pip 20.0.2 from /usr/lib/python3/dist-packages/pip (python 3.8)
```

为了更明确地区分版本，本书后面依然直接使用 python3 或 pip3 命令进行说明。

2.3.2 使用 update-alternatives

update-alternatives 是专门维护系统命令链接符的工具，通过 update-alternatives 命令修改连接指向不同的实际程序或者动态库，实现多版本共存，动态切换的效果。

添加 python 方案链接，以前面安装了 python3.13 和 python3.8 的鲁班猫 ubuntu20.04 系统为例：

```
# 创建一个 python 方案链接，添加 python3.13 和 python3.8，最后一个数字是优先级，数值越大，优先级越高
sudo update-alternatives --install /usr/bin/python python /usr/bin/python3.13 1
sudo update-alternatives --install /usr/bin/python python /usr/bin/python3.8 2
```

查看可选 python 版本，输入对应可选数字设置默认的 python 版本，这里测试设置 python3.13 为默认版本：

```
# 查看当前的 python 版本
cat@lubancat:~$ python --version
Python 3.8.10

# 然后列出系统上已安装的所有 Python 配置可选版本，这里重新设置 python3.13 为默认版本：
cat@lubancat:~$ sudo update-alternatives --config python
There are 2 choices for the alternative python (providing /usr/bin/python).

Selection    Path                                Priority    Status
-----
* 0           /usr/bin/python3.8                  2          auto mode
 1           /usr/bin/python3.13                 1          manual mode
 2           /usr/bin/python3.8                  2          manual mode
Press <enter> to keep the current choice[*], or type selection number: 1
update-alternatives: using /usr/bin/python3.13 to provide /usr/bin/python
(python) in manual mode
```

(下页继续)

(续上页)

```
# 查看重新设置后的 python
cat@lubancat:~$ sudo update-alternatives --query python
Name: python
Link: /usr/bin/python
Status: manual
Best: /usr/bin/python3.8
Value: /usr/bin/python3.13

Alternative: /usr/bin/python3.13
Priority: 1

Alternative: /usr/bin/python3.8
Priority: 2

# 重新设置 python 版本后, 查看版本
cat@lubancat:~$ python --version
Python 3.13.0b2
```

如果不需要 update-alternatives 管理 python, 删除请使用命令:

```
# 删除单个 python 链接
sudo update-alternatives --remove python /usr/bin/python3.13

# 删除所有 python 方案链接
sudo update-alternatives --remove-all python
```

2.4 软件库安装方法

2.4.1 pip 下载加速

在后面使用 Python 时，可能会需要使用 pip 工具下载安装依赖软件包，而官方下载站 [pypi](#) 在国内的访问较慢，参考如下网页说明设置镜像网站：[Pypi 镜像使用帮助](#)。

例如，使用 pip 安装软件包 `adafruit-circuitpython-ssd1306`：

```
# 使用 apt 安装 python 的 adafruit-circuitpython-ssd1306 包
sudo pip3 install adafruit-circuitpython-ssd1306

# 使用 pip list 查看安装的包
pip3 list | grep adafruit-circuitpython-ssd1306

# 使用 pip uninstall 卸载包
sudo pip3 uninstall adafruit-circuitpython-ssd1306
```

2.4.2 用 apt 代替 pip 安装软件包

使用 pip 工具安装软件包时，通常是在本机中进行编译的，部分鲁班猫板卡的性能不高导致编译时间非常长，还可能因为缺少某些库文件而安装失败。

所以在性能不高的板卡安装依赖包时，我们建议先直接搜索一下是否能使用 apt 工具安装，它会从软件库中下载预先编译好的软件包，安装时间基本只取决于网络速度。需要安装更新版本或者 apt 找不到的包时才使用 pip 安装。

例如，python 常用的 numpy 数据科学库，如下使用如下 apt 命令安装，很快就完成了：

```
# 使用 apt 安装 python 的 numpy 包
sudo apt -y install python3-numpy
```

至于其它具体软件包在 apt 工具中的名字，可以在网络中搜索“apt 安装 XXX（如 numpy）”之类

的内容，如果你使用的是 debian 系统，在 debian 的软件列表库中查找关键字如 “numpy”：[查找 debian 官方软件包](#)。

2.4.3 用 setuptools 工具安装软件包

在有些情况下，我们用户需要使用到的一些 Python 库包。由于一些原因，我们只获取到了这些库包的源码。那么我们用 pip 工具或者 apt 工具就没有办法去安装了。

这时我们可以使用 python 的 setuptools 工具，来通过库包源码进行 Python 库包的安装。

setuptools 工具安装方法如下：

```
# 在终端中输入如下命令：  
sudo apt -y install python3-setuptools
```

下面我们来使用一下 setuptools 工具。

以 Python 下的 jieba（结巴）库为例，该库可以用于中文分词，是 Github 一个非常热门的开源 Python 项目。

其 Github 仓库为：[jieba](#)

我们可以从其仓库拉取源码或者下载其 Releases 压缩包并安装。

源码目录如下：

```
cat@lubancat:~/jieba$ tree -L 1 ../jieba/  
../jieba/  
├── Changelog  
├── LICENSE  
├── MANIFEST.in  
├── README.md  
├── extra_dict  
├── jieba  
├── setup.py  
└── test  
  
3 directories, 5 files
```

进入源码目录，使用 setuptools 工具通过源码安装库：

```
# 拉取源码
git clone https://github.com/fxsjy/jieba.git

# 进入 jieba/目录, 在终端中输入如下命令:
sudo python3 setup.py install
```

等待软件包安装完成即可。

输入下面命令, 可以看到, jieba 库已经可以正常使用了, 分词效果如下:

```
# 登录系统终端, 输入 python3 或者 ipython, 进入 python 交互模式, 然后使用命令:
import jieba
set_list = jieba.cut("欢迎来到野火 Python 实践教程", cut_all=False)
print("Default Mode:" + "/".join(set_list))
```

```
cat@lubancat:~$ ipython
Python 3.8.10 (default, Jun 22 2022, 20:18:18)
Type 'copyright', 'credits' or 'license' for more information
IPython 8.6.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: import jieba

In [2]: set_list = jieba.cut("欢迎来到野火Python实践教程", cut_all=False)

In [3]: print("Default Mode:" + "/".join(set_list))
Building prefix dict from the default dictionary ...
Loading model from cache /tmp/jieba.cache
Loading model cost 3.286 seconds.
Prefix dict has been built successfully.
Default Mode:欢迎/来到/野火/Python/实践/教程

In [4]: █
```

2.4.4 UV

UV 是一个用 Rust 编写的极速 Python 包和项目管理工具。

鲁班猫板卡上安装 UV (测试系统为 ubuntu24.04), 测试使用独立安装程序下载和安装:

```
# 安装 UV
cat@lubancat:~$ curl -LsSf https://astral.sh/uv/install.sh | sh
downloading uv 0.8.10 aarch64-unknown-linux-gnu
no checksums to verify
installing to /home/cat/.local/bin
uv
uvx
everything's installed!

To add $HOME/.local/bin to your PATH, either restart your shell or run:
    source $HOME/.local/bin/env (sh, bash, zsh)
    source $HOME/.local/bin/env.fish (fish)

# uv help 查看命令使用
uv help

# 升级 UV
uv self update
```

之后重新打开一个终端，或者执行上面提示的命令：`source $HOME/.local/bin/env (sh, bash, zsh)`。

1、安装和管理 Python

```
# 安装指定的 python 版本
cat@lubancat:~$ uv python install 3.11
Installed Python 3.11.13 in 56.49s
+ cpython-3.11.13-linux-aarch64-gnu (python3.11)

# 查看安装的 python 版本, python3.12 是系统默认安装
cat@lubancat:~$ uv python list --only-installed
cpython-3.12.3-linux-aarch64-gnu      /usr/bin/python3.12
cpython-3.12.3-linux-aarch64-gnu      /usr/bin/python3 -> python3.12
```

(下页继续)

(续上页)

```
cpython-3.11.13-linux-aarch64-gnu      .local/bin/python3.11 -> .local/share/
↳ uv/python/cpython-3.11.13-linux-aarch64-gnu/bin/python3.11
cpython-3.11.13-linux-aarch64-gnu      .local/share/uv/python/cpython-3.11.13-
↳ linux-aarch64-gnu/bin/python3.11

# 执行程序指定 python 版本
```

2、虚拟环境

```
# 创建一个名称为 testvenv 虚拟环境, 指定 python 版本为 python3.11
cat@lubancat:~$ uv venv --python python3.11 testvenv
Using CPython 3.11.13
Creating virtual environment at: testvenv
Activate with: source testvenv/bin/activate

# 手动激活环境 testvenv
cat@lubancat:~$ source testvenv/bin/activate
(testvenv) cat@lubancat:~$

# 测试安装 jieba 库
(testvenv) cat@lubancat:~$ uv pip install jieba
Using Python 3.11.13 environment at: testvenv
Resolved 1 package in 33.06s
   Built jieba==0.42.1
Prepared 1 package in 4.96s
Installed 1 package in 4ms
+ jieba==0.42.1
```

3、工程依赖管理

UV 支持完整的 Python 项目结构和现代依赖管理流程, 天然支持 pyproject.toml, 可完全替代 pip、virtualenv、requirements.txt、pip-tools、poetry 等工具组合。


```
# 创建一个名称为 test_project 的空工程，会在当前目录下创建 test_project 文件，并添加
pyproject.toml 等等文件
cat@lubancat:~$ uv init test_project
Initialized project `test-project` at `/home/cat/test-project`

cat@lubancat:~$ tree test_project/
test_project/
├── main.py
├── pyproject.toml
└── README.md

1 directory, 3 files

# 添加软件 httpx，会自动在该目录中创建 .venv 虚拟环境用于包管理。
cat@lubancat:~$ cd test_project/
cat@lubancat:~/test_project$ uv add httpx
Using CPython 3.11.13
Creating virtual environment at: .venv
Resolved 9 packages in 1.32s
Prepared 8 packages in 475ms
Installed 8 packages in 13ms
+ anyio==4.10.0
+ certifi==2025.8.3
+ h11==0.16.0
+ httpcore==1.0.9
+ httpx==0.28.1
+ idna==3.10
+ sniffio==1.3.1
+ typing-extensions==4.14.1

# 运行程序
cat@lubancat:~/test_project$ uv run main.py
```

(下页继续)

(续上页)

```
Hello from test-project!
```

UV 还可以直接运行脚本工具等等，相关的更多说明请参考：<https://docs.astral.sh/uv>。

2.5 参考资料

<https://launchpad.net/~deadsnakes>

<https://www.python.org/downloads/source/>

<https://mirrors.tuna.tsinghua.edu.cn/help/pypi/>

https://www.debian.org/distrib/packages#search_packages

<https://github.com/astral-sh/uv>

第 3 章 运行 Python

本章讲解如何在鲁班猫板卡上简单使用 Python，它的使用方式与 PC 上并无区别。

3.1 使用 python 终端交互式环境

我们可以直接运行 python 命令进入 Python 的交互式编程，交互式编程不需要创建脚本文件，是通过 Python 解释器的交互模式进来编写代码。

在交互环境可以做一些简单的操作，比如在终端输出”hello world!”，做一些算术运算等，最后通过 exit() 退出：

```
# 在终端输入
python3
# 输出内容
Python 3.8.10 (default, Jun 22 2022, 20:18:18)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>

# 输入并回车
>>> print("hello world!")
# 输出
hello world!
>>>

# 输入
>>> 8+2
# 输出
```

(下页继续)

(续上页)

```
10

# 退出交互模式, 也可使用 quit() 或者组合键 Ctrl+D 退出
>>> exit()
cat@lubancat:~$
```

另外, 在终端使用 python, 也可以安装 ipython (增强的交互式 Python shell):

```
# 安装 ipython
sudo pip3 install ipython

# 使用
cat@lubancat:~$ ipython
Python 3.8.10 (default, Jun 22 2022, 20:18:18)
Type 'copyright', 'credits' or 'license' for more information
IPython 8.6.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]:

# 输入 8+2
In [1]: 8+2
Out[1]: 10

In [2]:

# 使用 exit 退出交互
```

3.2 使用 python 脚本

我们也可以编写一个简单的 Python 代码，保存为 `hello.py` 文件并执行。

可以使用串口登录板卡系统使用 `vim` 编辑器编写，或者 `vscode ssh` 登录板卡进行编写：

`hello.py` 代码文件内容如下：

列表 1: `base/hello.py` 文件内容

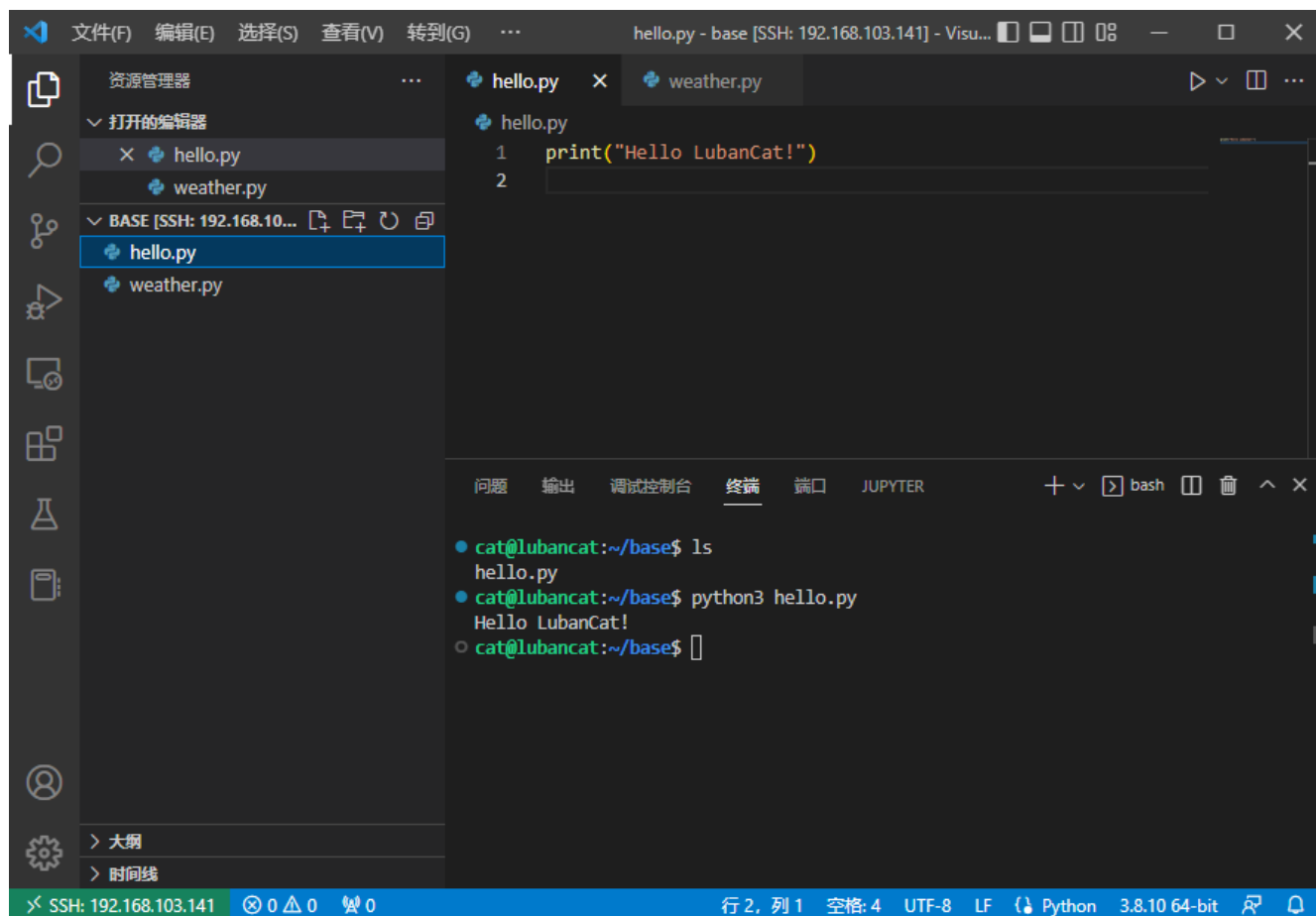
```
print("Hello LubanCat!")
```

然后运行：

```
# 编写 hello.py
# 使用 ls 命令确认目录下存在该文件
ls
# 以下为输出
hello.py

# 运行
python3 hello.py
# 输出
Hello LubanCat!
```

以下为使用 VS Code remote 编写及运行的情景：



```
hello.py
1 print("Hello LubanCat!")
2

cat@lubancat:~/base$ ls
hello.py
cat@lubancat:~/base$ python3 hello.py
Hello LubanCat!
cat@lubancat:~/base$
```

使用脚本，也可以在脚本内指定解释器的路径，比如：

列表 2: base/hello.py 文件内容

```
#!/usr/bin/python
print("Hello LubanCat!")
```

然后运行:

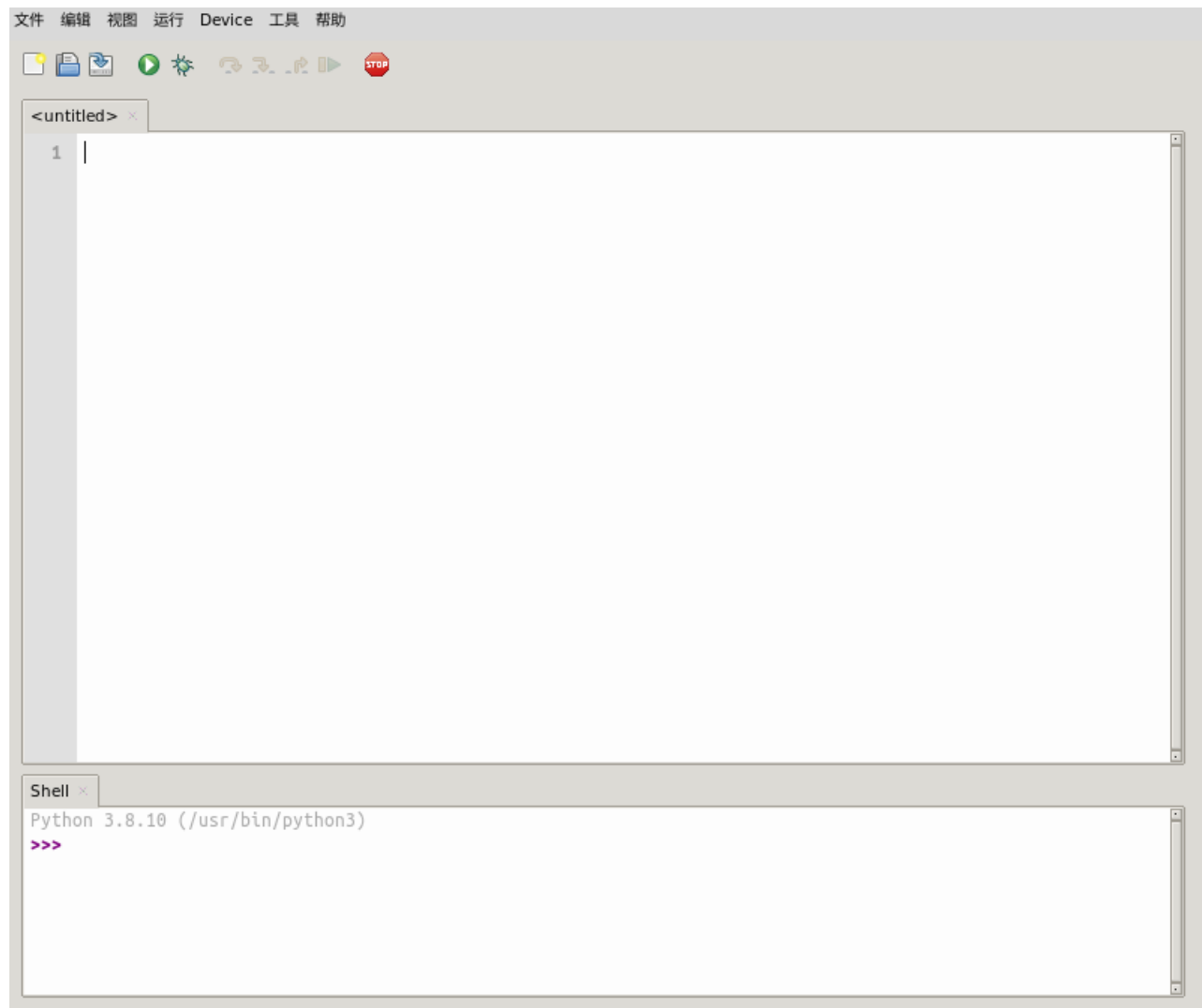
```
# 运行
cat@lubancat:~$ ./hello.py
Hello LubanCat!
cat@lubancat:~$
```

3.3 使用 python IDE

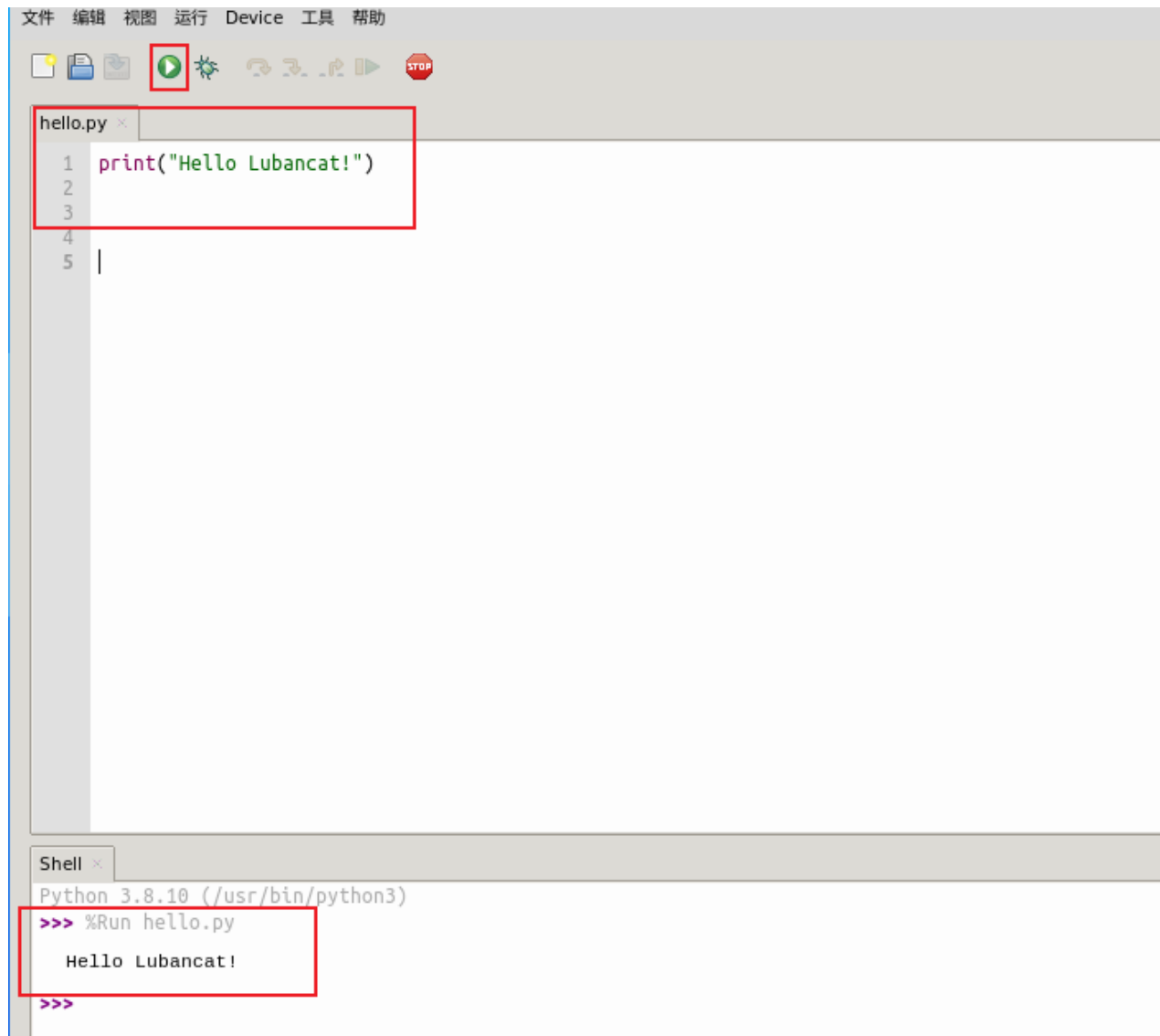
如果使用带桌面的系统，并且有屏幕显示，可以选择安装 Python IDE 进行学习，以安装 Thonny IDE 为例:

```
# 使用 apt 包管理器安装
sudo apt install thonny
```

安装之后打开显示 (以 xfce4 桌面为例显示):



测试, 点击运行:



Thonny 是面向 Python 初学者的集成开发环境（IDE），支持显示语法错误并解释作用域、内置调试器和逐步执行评估、GUI 界面来管理 python 软件包等。

3.4 import 其它软件包

除了运行基础的 Python 代码库，它也可以在代码中导入第三方软件包，运行更复杂的应用程序。以示例程序进行说明，我们新建一个 `get_info.py`，写入以下代码：

列表 3: 配套代码 `base/get_info.py` 文件内容

```
# 导入 psutil 包
import psutil
import datetime

# 获取当前用户信息，输出的 name 用户名、terminal 终端、host 主机地址、started 登录时间、pid 进程 id
print("当前用户:", psutil.users())

# 获取当前系统时间，转换成自然时间格式
print("当前系统时间:", datetime.datetime.fromtimestamp(psutil.boot_time()).
    ↳ strftime("%Y-%m-%d %H: %M: %S"))

print("\n-----内存信息-----")
mem = psutil.virtual_memory()
print("系统内存全部信息:", mem)
mem_total = float(mem.total)
mem_used = float(mem.used)
mem_free = float(mem.free)
mem_percent = float(mem.percent)
print(f"系统总计内存: {mem_total}")
print(f"系统已经使用内存: {mem_used}")
print(f"系统空闲内存: {mem_free}")
print(f"系统内存使用率: {mem_percent}")

print("\n-----CPU 信息-----")
```

(下页继续)

(续上页)

```
print("CPU 汇总信息:", psutil.cpu_times())
print("cpu 逻辑个数:", psutil.cpu_count())
print("cpu 频率:", psutil.cpu_freq())
```

注解：Python 的 psutil (process and system utilities) 库是一个跨平台的第三方库，能够轻松实现获取系统运行的进程和系统利用率（包括 CPU、内存、磁盘、网络、进程等）信息。它主要用于系统监控、分析、限制系统资源和进程的管理。

保存文件并尝试运行 get_info.py 程序：

```
# 运行脚本
python3 get_info.py

# 输出
当前用户: [suser(name='cat', terminal='ttyFIQ0', host='', started=1662631296.
↪0, pid=742)]
当前系统时间: 2022-11-01 16: 28: 20

-----内存信息-----
系统内存全部信息: svmem(total=2059280384, available=892481536, percent=56.7,
↪used=1123770368, free=30068736, active=1374724096, inactive=373829632,
↪buffers=83066880, cached=822374400, shared=17391616, slab=232140800)
系统总计内存: 2059280384.0
系统已经使用内存: 1123770368.0
系统空闲内存: 30068736.0
系统内存使用率: 56.7

-----CPU 信息-----
CPU 汇总信息: scputimes(user=2365.82, nice=3.25, system=765.33, idle=335360.
↪41, iowait=79.58, irq=0.0, softirq=50.22, steal=0.0, guest=0.0, guest_
↪nice=0.0)
```

(下页继续)

(续上页)

```
cpu 逻辑个数: 4
cpu 频率: scpuufreq(current=1992.0, min=408.0, max=1992.0)
```

对于一些没有安装的包，我们可以通过下面方式安装（以 requests 软件包为例）：

```
# 方式一：直接使用 pip 安装
sudo pip3 install requests

# 方式二：使用 apt 安装
sudo apt -y install python3-requests
```

一些依赖包也是 debian 支持的，所以可以直接使用 apt 安装，对于标准的依赖包，我们建议使用 apt 安装，而 pip 的安装方式主要用于安装 apt 找不到的第三方软件包。

如果使用 Thonny，可以点击界面的工具-> 管理包，查询和安装软件包：



3.5 参考资料

- 《Python 基础教程》

第 4 章 Python 基础速览

Python 是一门简单易学且功能强大的编程语言, 本章将简单浏览下 Python 的基础语法, 详细可以看参考文档。

4.1 注释

Python 中单行注释以 # 开头, 多行可以用多个 # 号, 还有 ''' 和 """, 例如:

```
1 # 这是注释
2
3 '''
4 这也是注释
5 '''
6
7 """
8 这还是注释
9 """
10 print ("Hello!")
```

4.2 缩进

python 是使用缩进来表示代码块, 缩进的空格数是可变的, 但是同一个代码块的语句必须包含相同的缩进空格数, 否则会导致运行错误。实例如下:

```
1 #for 循环
2 for i in range(5):
```

(下页继续)

(续上页)

```
3     print("in")
4 print("out")
5
6 # 输出结果
7 in
8 in
9 in
10 in
11 in
12 out
```

4.3 变量和数据类型

Python 中的变量和变量类型不需要声明，是动态的，每个变量在使用前都必须赋值，变量赋值以后该变量才会被创建，而变量的类型由赋予它的值来决定。例如：

```
1 a = 10                # 整型变量
2 print(type(a))
3 a = 10.0              # 浮点型变量
4 print(type(a))
5 a = "test"            # 字符串
6 print(type(a))
```

```
1 # 输出
2 <class 'int'>
3 <class 'float'>
4 <class 'str'>
```

Python 中常见几种标准数据类型：不可变数据: Number(数字)、String(字符串)、Tuple(元组)；可变数据: List(列表)、Dictionary(字典)、Set(集合)。下面简单列举几种：

4.3.1 String(字符串)

Python 中的字符串(无字符类型)用单引号 ' ' 或双引号 " " 括起来, 一些特殊字符使用反斜杠 \ 转义。例如:

```
1  #!/usr/bin/python3
2  str0 = 'Python'
3
4  '''
5  字符串索引和截取
6  +---+---+---+---+---+---+
7  | P | y | t | h | o | n |
8  +---+---+---+---+---+---+
9      0   1   2   3   4   5   (从前面开始索引)
10     -6  -5  -4  -3  -2  -1   (从后面开始索引)
11     :   1   2   3   4   5   : (从前面开始截取)
12     :  -5  -4  -3  -2  -1   : (从后面开始截取)
13  '''
14  print (str0)           # 输出整个 str0 字符串
15  print (str0[0])        # 输出字符串第一个字符
16  print (str0[-6])       # 输出字符串第一个字符
17  print (str0[:])        # 整个 str0 字符串
18  print (str0[2:])       # 输出从第三个开始后的所有字符
19  print (str0[:-2])      # 输出从第一个到第四个所有字符
20  print (str0 * 2)       # 输出字符串两次, 也可以写成 print (2 * str0)
21  print (str0 + "TEST") # 连接字符串
```

字符串可以被索引或者截取, 但是索引是不可以修改, 重新赋值等。

4.3.2 List(列表)

列表可以完成大多数集合类的数据结构实现。列表中元素的类型可以不相同，它支持数字，字符串甚至可以包含列表（所谓嵌套）。列表是写在方括号 [] 之间、用逗号分隔开的元素列表。与字符串的索引一样，列表索引从 0 开始，第二个索引是 1，依此类推。

```
1  #!/usr/bin/python3
2
3  list0 = [ 'python', "test", 10, 10.0 ]
4  list1 = [ 123, 'Python' ]
5
6  print (list0)           # 输出完整列表
7  print (list0[0])        # 输出列表第一个元素
8  print (list[1:3])       # 从第二个开始输出到第三个元素
9  print (list[2:])        # 输出从第三个元素开始的所有元素
10 print (tinylis * 2)     # 输出两次列表
11 print (list0 + list1)   # 连接列表
```

4.3.3 Tuple(元组)

Python 的元组与列表类似，不同之处在于元组的元素不能修改。元组使用小括号 ()，列表使用方括号 []。元组创建很简单，只需要在括号中添加元素，并使用逗号隔开即可。

使用举例：

```
1  #!/usr/bin/python3
2
3  tup1 = ('Pyrhon', 'Test', 1997, 2000)
4  tup2 = (1, 2, 3, 4, 5, 6, 7 )
5
6  print ("tup1[0]: ", tup1[0])
7  print ("tup2[1:4]: ", tup2[1:5])
```

4.4 流程控制相关

4.4.1 条件控制

Python 条件语句是通过一条或多条语句的执行结果（True 或者 False）来决定执行的代码块。

```
1  #!/usr/bin/python3
2
3  # 数字猜谜游戏
4  number = 20
5  guess = 0
6  print("数字猜谜游戏!")
7  while guess != number:
8      guess = int(input("请输入你猜的数字 (0-100): "))
9
10     if guess == number:
11         print("恭喜, 你猜对了!")
12     elif guess < number:
13         print("猜的数字小了...")
14     elif guess > number:
15         print("猜的数字大了...")
```

4.4.2 循环语句

Python 中的循环语句有 for 和 while。例如：

```
1  #!/usr/bin/env python3
2
3  # 使用 while 求和
4  n = 100
5  sum = 0
```

(下页继续)

(续上页)

```
6 counter = 1
7 while counter <= n:
8     sum = sum + counter
9     counter += 1
10
11 print("1 到 %d 之和为: %d" % (n, sum))
12
13 # 内置 range() 函数。它会生成数列
14 for i in range(4):
15     print(i)
```

4.4.3 迭代

迭代器是一个可以记住遍历的位置的对象。迭代器对象从集合的第一个元素开始访问，直到所有的元素被访问完结束。迭代器只能往前不会后退。

例如：

```
1 #!/usr/bin/python3
2
3 list=[1,2,3,4]
4 it = iter(list)    # 创建迭代器对象，for 语句进行遍历
5 for x in it:
6     print (x, end=" ")
```

4.5 函数

函数能提高应用的模块性，和代码的重复利用率。Python 提供了许多内建函数，比如 `print()`，也可以用户自定义函数。

在 Python 中，使用 `def` 关键字定义函数，一般格式如下：

def 函数名 (参数列表) : 函数体

使用举例:

```
1  #!/usr/bin/python3
2  # 定义函数 my_function
3  def my_function(test):
4      print("Hello!" + test)
5
6  # 使用函数 my_function
7  my_function(' Python')
```

4.6 模块和包

4.6.1 模块

Python 提供了一个方法可以从文件中获取定义, 在脚本或者解释器的一个交互式实例中使用。这样的文件被称为模块; 模块中的定义可以导入到另一个模块或主模块中, 使用 `import` 语句导入。

例如:

```
1  #!/usr/bin/python3
2
3  # 使用 python 标准模块: sys, 使用 import 语句
4  import sys
5  print('\nPython 环境路径: ', sys.path)
6
7  # 使用 from ... import 语句
8  from periphery import I2C
```

4.6.2 包

包是一种管理 Python 模块命名空间的形式，采用”点模块名称”。

比如一个模块的名称是 A.B，那么他表示一个包 A 中的子模块 B。就好像使用模块的时候，你不用担心不同模块之间的全局变量相互影响一样，采用点模块名称这种形式也不用担心不同库之间的模块重名的情况。

包的安装下载可以参考下前面 安装 python 章节。

4.7 虚拟环境

Python 应用程序通常会使用不属于标准库包和模块，有时会需要特定版本的包或者模块，因为应用程序可能需要修复特定错误，或者应用程序使用的库接口是以前版本。例如：如果应用程序 A 需要模块的版本 1.0，而应用程序 B 需要版本 2.0，则这些需求发生冲突，安装版本 1.0 或 2.0 将导致一个应用程序无法运行，这个问题可以通过创建一个虚拟环境来解决。

虚拟环境会创建一个自包含的目录树，其中包含用于特定版本 Python 的 Python 安装，以及许多附加包。不同的应用程序可以使用不同的虚拟环境，安装不同版本的包和模块。

Python 从 3.3 版本开始，自带了一个虚拟环境 venv，安装 venv 使用：

```
1  # 安装虚拟环境
2  cat@lubancat:~$ sudo apt-get install python3-venv
3
4  # 创建并进入 project-test 目录，然后创建一个独立的 Python 运行环境:test_env
5  cat@lubancat:~$ mkdir project-test && cd project-test
6  cat@lubancat:~/project-test$ python3 -m venv .test_env
7
8  cat@lubancat:~/project-test$ ls -al
9  total 12
10 drwxr-xr-x  3 cat  cat 4096 Dec  7 10:38 .
11 drwxr-xr-x 17 cat  cat 4096 Dec  7 10:38 ..
```

(下页继续)

(续上页)

```
12 drwxr-xr-x  6 cat cat 4096 Dec  7 10:38 .test_env
13 cat@lubancat:~/project-test$
14
15 # 激活进入环境
16 cat@lubancat:~/project-test$ source .test_env/bin/activate
17 (.test_env) cat@lubancat:~/project-test$
18 (.test_env) cat@lubancat:~/project-test$ pip list
19 Package          Version
20 -----
21 pip               18.1
22 pkg-resources     0.0.0
23 setuptools        40.8.0
24 (.test_env) cat@lubancat:~/project-test$
25
26 # 导出当前虚拟环境的包和模块版本号
27 (.test_env) cat@lubancat:~/project-test$ pip3 freeze > requirements.txt
28 # 安装需求的包和模块版本号
29 (.test_env) cat@lubancat:~/project-test$ pip3 install -r requirements.txt
30
31 # 退出环境
32 (.test_env) cat@lubancat:~/project-test$ deactivate
33 cat@lubancat:~/project-test$
```

更多虚拟环境相关知识，可以搜索下 `virtualenv`, `venv` 和 `pipenv` 关键字。

4.8 参考文档

更多基础和细节参考：<https://docs.python.org/3.8/tutorial/index.html>

<https://www.runoob.com/python3/python3-basic-syntax.html>

第 5 章 控制 GPIO

重要： 系统控制板卡外设接口需要 root 权限，可以更改设备文件权限或者使用 root 用户下安装 python 模块然后测试。

在鲁班猫板卡上开发纯软件类型的 Python 应用程序与 PC 上并无区别，接下来的章节着重讲解如何使用 Python 控制板卡上的 GPIO、PWM、ADC、I2C 及 SPI 等纯 PC 编程中很少用到的外围接口。

控制这些外围接口时，可以 Python 包：

- [python3-libgpiod](#)：标准 GPIO libgpiod 库的 python 版本，只支持控制 IO 输入输出。
- [python-periphery](#) 支持 GPIO、PWM、I2C、SPI、UART 等多种接口的基础控制。
- [Adafruit Blinka](#)：支持 GPIO、PWM、I2C、SPI、UART 等，还带有一些常用传感器、OLED 屏的应用示例。

接下来我们将简单介绍下这些包的使用。

5.1 libgpiod 基本概念

GPIO 主要用来对外输出高低电平，控制 GPIO 时，基本都会涉及到 libgpiod 的控制，我们主要需要知道板卡引脚的命名方式即可。

CPU 的 GPIO 引脚使用 (chip, line) 的方式命名，使用以下命令可以查看：

```
# 在板卡上执行以下命令  
sudo gpioinfo
```

(下页继续)

(续上页)

```
# 若提示找不到命令，使用如下方式安装
sudo apt -y install gpiod libgpiod-dev

# 以下为 gpioinfo 的输出
gpiochip0 - 32 lines:
    line 0:      unnamed      unused   input   active-high
    line 1:      unnamed      unused   input   active-high
    line 2:      unnamed      unused   input   active-high
    line 3:      unnamed      unused   input   active-high
    line 4:      unnamed      unused   input   active-high
    line 5:      unnamed "headset_gpio" input active-high [used]
# ...

    line 31:     unnamed      unused   input   active-high
gpiochip1 - 32 lines:
    line 0:      unnamed      unused   input   active-high
    line 1:      unnamed      unused   input   active-high
# ...

    line 31:     unnamed      unused   input   active-high
```

而板卡引出的排针接口与 GPIO (chip, line) 的对应方式可查看板卡具体的说明。

关于 libgpiod 更详细的说明请参考右侧说明：《使用 libgpiod 控制 IO》

5.2 实验准备

在板卡上的部分 GPIO 可能会被用做其他功能，可注释掉某些设备树节点或者设备树插件的加载，重启系统，释放相应的 GPIO 引脚，当然也可以换个引脚测试，LubanCat_RK 系列板卡的引脚参考下：《LubanCat-RK 系列-40pin 引脚对照图》

大部分操作硬件外设的功能，一般都需要 root 用户权限，简单的解决方案是临时修改下设备文件的权限或者使用 root 用户权限运行程序。

5.3 方式一：使用 python3-libgpiod

5.3.1 安装 python3-libgpiod

利用 `python3-libgpiod` 软件包，可以轻松使用 Python 控制 GPIO 引脚。目前 `python3-libgpiod` 没有官方文档说明，可使用 `import` 该软件包后使用 `help` 查看帮助，它的安装及查看帮助方式如下：

```
# 在板卡使用如下命令安装
sudo apt -y install python3-libgpiod

# 进入 python 交互模式，测试及查看帮助
python3
import gpiod
help(gpiod)

# 以下为输出的帮助说明
NAME
    gpiod - Python bindings for libgpiod.

DESCRIPTION
    This module wraps the native C API of libgpiod in a set of python_
    ↪classes.

# ...
```

5.3.2 libgpiod 输出

使用这个 `python3-libgpiod` 包控制 GPIO 点灯（输出）的示例代码如下：

列表 1: 配套代码 io/gpio/libgpiod_io0.py 文件内容

```
1 import time
2 import gpiod
3
4 # 根据具体板卡的 LED 灯连接修改使用的 Chip 和 Line, 没有 LED 可以自行外接
5 LINE_OFFSET = 8
6
7 chip0 = gpiod.Chip("0", gpiod.Chip.OPEN_BY_NUMBER)
8
9 gpio0_b0 = chip0.get_line(LINE_OFFSET)
10 gpio0_b0.request(consumer="gpio", type=gpiod.LINE_REQ_DIR_OUT, default_
    ↪vals=[0])
11
12 print(gpio0_b0.consumer())
13
14 try:
15     while True:
16         gpio0_b0.set_value(1)
17         time.sleep(0.5)
18         gpio0_b0.set_value(0)
19         time.sleep(0.5)
20 finally:
21     gpio0_b0.set_value(1)
22     gpio0_b0.release()
```

代码说明:

- 第 7 行, 创建了一个 chip ID 为 0 的 gpiod.Chip 对象 chip0
- 第 9 行, 设置使用 chip0 对象的 line8 作为引脚
- 第 10 行, 申请 gpio, 设置为输出, 默认输出低电平

后面的代码直接使用 gpio0_b0 对象控制选定的 GPIO 输出高低电平, 从而达到控制 LED 灯的亮

灭。

本示例以及后面的内容都是以 LubanCat 2 板卡作为实验对象，所以代码中的 (chip,line)=(0, 8) 是根据该板卡的引脚定义选择的，若使用其它板卡只要修改相应的编号即可。

5.3.2.1 实验操作

示例代码使用 LubanCat 2 板卡，源码可以参考配套例程，操作如下：

```
# 确认已安装 python3-libgpiod 包
# 在板卡 blink.py 所在的目录执行如下命令，需要 root 权限
python3 blink.py

# 如果这个引脚外接了 LED，那么灯就会闪烁
```

使用另外一个终端，用 gpioinfo 命令，可以看到 gpio (0, 8) 引脚已经被申请使用：

```
cat@lubancat:~$ sudo gpioinfo
gpiochip0 - 32 lines:
  line 0:      unnamed            unused    input    active-high
  line 1:      unnamed            unused    input    active-high
  line 2:      unnamed            unused    input    active-high
  line 3:      unnamed            unused    input    active-high
  line 4:      unnamed            unused    input    active-high
  line 5:      unnamed "headset_gpio" input    active-high [used]
  line 6:      unnamed            unused    input    active-high
  line 7:      unnamed            unused    input    active-high
  line 8:      unnamed            "gpio"    output    active-high [used]
  line 9:      unnamed            unused    input    active-high
  line 10:     unnamed            unused    input    active-high
  line 11:     unnamed            unused    input    active-high
  line 12:     unnamed            unused    input    active-high
  line 13:     unnamed            unused    input    active-high
  line 14:     unnamed            unused    input    active-high
```

5.3.3 libgpiod 输入输出

类似地，使用这个 `python3-libgpiod` 包检测 GPIO 输入（按键）的示例代码如下：

列表 2: 配套代码 `io/gpio/libgpiod_io1.py`

```
1 import gpiod
2
3 # 根据具体板卡的 LED 灯和按键连接修改使用的 Chip 和 Line
4
5 # 这里以 LubanCat 2 为例，使用 GPIO0_B0 接 LED，GPIO0_C2 接按键
6 LED_LINE_OFFSET = 8
7 BUTTON_LINE_OFFSET = 18
8
9 chip0_led = gpiod.Chip("0", gpiod.Chip.OPEN_BY_NUMBER)
10 chip0_button = gpiod.Chip("0", gpiod.Chip.OPEN_BY_NUMBER)
11
12 led = chip0_led.get_line(LED_LINE_OFFSET)
13 led.request(consumer="LED", type=gpiod.LINE_REQ_DIR_OUT, default_vals=[0])
14
15 button = chip0_button.get_line(BUTTON_LINE_OFFSET)
16 button.request(consumer="BUTTON", type=gpiod.LINE_REQ_DIR_IN)
17
18 print(led.consumer())
19 print(button.consumer())
20
21 try:
22     while True:
23         led.set_value(button.get_value())
24 finally:
25     led.set_value(1)
26     led.release()
27     button.release()
```

代码说明：

- 第 12 行，设置 LED 的 GPIO 控制方向为输出
- 第 15 行，设置按键的 GPIO 控制方向为输入
- 第 23 行，读取按键引脚的输入值来控制 LED

5.4 方式二：使用 python-periphery

5.4.1 安装 python-periphery

`python-periphery` 与 `python3-libgpiod` 的功能类似，但 `periphery` 除了支持 GPIO 输入输出控制外，还支持 I2C、SPI 等总线协议。

`python-periphery` 的安装方式如下：

```
# 在板卡使用如下命令安装
pip3 install python-periphery
```

5.4.2 periphery 输入输出

使用这个 `python-periphery` 进行输入输出的示例代码如下：

列表 3: 配套代码 `io/gpio/periphery_io.py` 文件内容

```
1 from periphery import GPIO
2
3 # 根据具体板卡的 LED 灯和按键连接修改使用的 Chip 和 Line
4 # 这里以 LubanCat 2 为例，使用 GPIO0_B0 接 LED，GPIO0_C2 接按键
5 LED_CHIP = "/dev/gpiocchip0"
6 LED_LINE_OFFSET = 8
7
```

(下页继续)

(续上页)

```
8 BUTTON_CHIP = "/dev/gpiochip0"
9 BUTTON_LINE_OFFSET = 18
10
11 led = GPIO(LED_CHIP, LED_LINE_OFFSET, "out")
12 button = GPIO(BUTTON_CHIP, BUTTON_LINE_OFFSET, "in")
13
14 try:
15     while True:
16         led.write(button.read())
17 finally:
18     led.write(True)
19     led.close()
20     button.close()
```

代码说明：

- 第 5~9 行，定义了 LED、按键的 chip 和 line 编号
- 第 11~12 行，分别创建了 led 和 button 的 GPIO 输出、输入对象
- 第 16 行，读取按键引脚的输入值来控制 LED

使用另外一个终端，用 `gpioinfo` 命令，可以看到两个引脚已经被申请使用：

```
cat@lubancat:~$ sudo gpioinfo
gpiochip0 - 32 lines:
    line 0:      unnamed      unused    input    active-high
    line 1:      unnamed      unused    input    active-high
    line 2:      unnamed      unused    input    active-high
    line 3:      unnamed      unused    input    active-high
    line 4:      unnamed      unused    input    active-high
    line 5:      unnamed      "headset_gpio" input    active-high [used]
    line 6:      unnamed      unused    input    active-high
    line 7:      unnamed      unused    input    active-high
    line 8:      unnamed      "periphery" output    active-high [used]
    line 9:      unnamed      unused    input    active-high
    line 10:     unnamed      unused    input    active-high
    line 11:     unnamed      unused    input    active-high
    line 12:     unnamed      unused    input    active-high
    line 13:     unnamed      unused    input    active-high
    line 14:     unnamed      unused    input    active-high
    line 15:     unnamed      unused    input    active-high
    line 16:     unnamed      unused    output    active-high
    line 17:     unnamed      unused    output    active-high
    line 18:     unnamed      "periphery" input    active-high [used]
    line 19:     unnamed      unused    input    active-high
    line 20:     unnamed      unused    input    active-high
    line 21:     unnamed      unused    input    active-high
    line 22:     unnamed      unused    input    active-high
```

5.5 方式三：使用 Adafruit Blinka

5.5.1 安装 Adafruit Blinka

Adafruit Blinka 与 `python-periphery` 功能类似，除支持 GPIO 输入输出控制外，都支持 I2C、SPI 等总线协议，但 Adafruit 基于 blinka 还提供了更丰富的应用 demo，如控制屏幕等。

Adafruit Blinka 的安装方式如下：

```
# 在板卡使用如下命令安装
sudo apt -y install python3-libgpiod # 注：如在方法一中已安装，忽略此步
pip3 install Adafruit-Blinka
```

5.5.2 Adafruit Blinka 板卡支持测试

鲁班猫 RK 系列板卡已经适配了 Adafruit-Blinka，但是部分板卡引脚功能不支持，如果想知道自己的板上是否支持一些功能的时候，可以查询板上资源定义，参考如下：

查询前确保安装了 Adafruit-Blinka 库（烧录的系统默认已经安装）。

1. 输入 `python3` 进入 `python3` 终端。
2. 输出 `from adafruit_platformdetect import Detector` 添加 `adafruit_platformdetect` 包；
3. 输入 `detector = Detector()` 接着输入 `print("Chip id: ", detector.chip.id)` 和 `print("Board id: ", detector.board.id)` 检测板子和芯片 id；
4. 在 `python3` 终端中，输入 `import board` 导入 `board` 软件包。
5. 输入 `board.` 并双击 `Tab` 键，出现补全信息，补全信息中含有板卡定义的资源内容。

提示：如果测试 Adafruit Blinka 过程中出现 `PermissionError: [Errno 13] Permission denied`，请使用命令 `sudo chmod a+rw /dev/gpiochip*` 临时更改下权限，然后重新测试。

lubancat-zero:

```
>>> from adafruit_platformdetect import Detector
>>> detector = Detector()
>>> print("Chip id: ", detector.chip.id)
Chip id: RK3566
>>> print("Board id: ", detector.board.id)
Board id: LUBANCAT_ZERO
>>>
```

LubanCat-1:

```
>>> from adafruit_platformdetect import Detector
>>> detector = Detector()
>>> print("Chip id: ", detector.chip.id)
Chip id: RK3566
>>> print("Board id: ", detector.board.id)
Board id: LUBANCAT1
>>>
```


LubanCat-2:

```
cat@lubancat:~$ python
Python 3.8.10 (default, Jun 22 2022, 20:18:18)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from adafruit_platformdetect import Detector
>>> detector = Detector()
>>> print("Chip id: ", detector.chip.id)
Chip id: RK3568
>>> print("Board id: ", detector.board.id)
Board id: LUBANCAT2
>>>
>>> import board
>>> board.
board.CS0      board.GPIO12  board.GPIO18  board.GPIO23  board.GPIO28  board.GPIO32  board.GPIO37  b
board.CS1      board.GPIO13  board.GPIO19  board.GPIO24  board.GPIO29  board.GPIO33  board.GPIO38  b
board.GPIO10   board.GPIO15  board.GPIO21  board.GPIO26  board.GPIO3   board.GPIO35  board.GPIO40  b
board.GPIO11   board.GPIO16  board.GPIO22  board.GPIO27  board.GPIO31  board.GPIO36  board.GPIO5    b
>>> board.█
```

LubanCat-4:

```
root@lubancat:~# python3
Python 3.9.2 (default, Feb 28 2021, 17:03:44)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from adafruit_platformdetect import Detector
>>> detector = Detector()
>>> print("Chip id: ", detector.chip.id)
Chip id: RK3588
>>> print("Board id: ", detector.board.id)
Board id: LUBANCAT4
>>> import board
>>> board
board
>>> board.
board.CS0      board.GPIO13  board.GPIO21  board.GPIO27  board.GPIO32  board.GPIO38  board.I2C5_SCL
board.CS1      board.GPIO15  board.GPIO22  board.GPIO28  board.GPIO33  board.GPIO40  board.I2C5_SDA
board.GPIO10   board.GPIO16  board.GPIO23  board.GPIO29  board.GPIO35  board.GPIO5    board.I2C6_SCL
board.GPIO11   board.GPIO18  board.GPIO24  board.GPIO3   board.GPIO36  board.GPIO7    board.I2C6_SDA
board.GPIO12   board.GPIO19  board.GPIO26  board.GPIO31  board.GPIO37  board.GPIO8    board.MISO
>>> board.█
```

LubanCat-5:

```
cat@lubancat:~$ python3
Python 3.8.10 (default, Nov 22 2023, 10:22:35)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from adafruit_platformdetect import Detector
>>> detector = Detector()
>>> print("Chip id: ", detector.chip.id)
Chip id: RK3588
>>> print("Board id: ", detector.board.id)
Board id: LUBANCAT5
>>> import board
>>> board.
board.CS0      board.GPI015  board.GPI022  board.GPI028  board.GPI033  board.GPI040
board.GPI010   board.GPI016  board.GPI023  board.GPI029  board.GPI035  board.GPI05
board.GPI011   board.GPI018  board.GPI024  board.GPI03   board.GPI036  board.GPI07
board.GPI012   board.GPI019  board.GPI026  board.GPI031  board.GPI037  board.GPI08
board.GPI013   board.GPI021  board.GPI027  board.GPI032  board.GPI038  board.I2C3_SCL
>>> board.
```

注意此处查询到的资源仅是我们为适配 Adafruit-Blinka 库添加的资源，具体资源能否使用，还是根据板卡情况决定，大家也可以自行尝试。

提示：添加的板卡资源是引出的 40pin，设置的规则是第几个引脚对应第几个 GPIO，例如 GPIO3 对应 40pin 的第三个物理引脚，如果物理引脚是接地或者电源是没有的，比如没有 GPIO1。

5.5.3 Adafruit-Blinka 控制 IO

使用这个 [Adafruit Blinka](#) 进行输入输出的示例代码如下 (这里以 LubanCat 1 为例，其他系列需要自行修改下引脚等)：

列表 4: 配套代码 io/gpio/digital_io.py 文件内容

```
1 import board
2 import digitalio
3 import time
4
5 # 根据具体板卡修改使用的 GPIO
6 # 以 LubanCat 1 board 为例，board.GPIO11 就是引出的 40Pin 的第 11 个物理引脚，
board.GPIO11 = GPIO3_A5 = Pin 101
```

(下页继续)

(续上页)

```
7 led = digitalio.DigitalInOut(board.GPIO11)
8 led.direction = digitalio.Direction.OUTPUT
9
10 try:
11     while True:
12         led.value=1
13         time.sleep(0.5)
14         led.value=0
15         time.sleep(0.5)
16
17 finally:
18     led.value = True
19     led.deinit()
```

代码说明：

- 第 1~2 行，board 及 digitalio 都是 Blinka 包提供的库
- 第 6~7 行，定义使用的引脚 board.GPIO11，并设置为输出方向
- 第 9-14 行，使引脚输出高低电平

代码中的 board.GPIO11 引脚是在 Blinka 库定义好的 LubanCat 1 引脚, 使用命令：

```
python3 digital_io.py
```

如果对应引脚外接一个 LED 灯，就可以看到 LED 闪烁。

当我们申请对应的板卡资源的时候，它会检测板卡型号，然后根据板卡信息加载相应的资源定义，所以我们可以直接使用定义好的资源名，使用起来相对 [python-periphery](#) 会更直观。

更详细的引脚定义可直接查看 [Adafruit Blinka](#) 的源码：

- [Adafruit Blinka 中 LubanCat 系列板卡引脚定义](#)
- [Adafruit Blinka 中 LubanCat RK 系列芯片引脚定义](#)

用户也可以自行拉取 Adafruit Blinka 库然后修改自定义的引脚，独自安装使用。

5.6 参考资料

事实上，还有更多的 Python 库支持控制外围硬件，感兴趣可到 [github](#) 或 [pypi](#) 查找。

- [《CircuitPython》](#)
- [python-periphery 源码](#)
- [Adafruit Blinka 源码](#)

第 6 章 PWM 输出

在前面章节中，我们通过 `python3-libgpiod`、`python-periphery` 这几个 Python 库，编写了操作 GPIO 外设的 Python 应用程序，最终实现了在鲁班猫板卡上通过 Python 应用程序，点亮了板卡上的 LED 灯以及使用了按键资源。那么在本小节中，我们接着为大家介绍上述库中提供的另外一项功能，PWM 波形输出。

本节实验中，会使用到控制 LED 灯 GPIO 引脚，做 PWM 输出。届时大家可以通过 LED 灯的明暗变化查看到实验现象 (板卡可能需要外接 LED)。

6.1 PWM 实验

通过控制 GPIO 引进的高低电平变化，我们可以做些一些简单的控制，比如控制 LED 灯的亮灭。如果我们进一步去调整 GPIO 引脚电平变化的频率、持续的时间，那么基于此，我们可以做的控制就可以变得更加复杂，这就是 PWM (脉冲宽度调制)。

通常我们可以利用 PWM 波形输出实现许多功能，比如控制 LED 灯的光亮程度、无源蜂鸣器的响度、甚至可以控制小舵机。本节实验中我们就通过输出 PWM，来控制 LED 灯实现 LED 呼吸灯效果 (以 LubanCat 2 板卡为例，外接 LED)。

关于 PWM 究竟是如何产生作用的，我们在此不作过多探究，大家可以自行搜索 PWM 的原理。本节重点是提供 PWM 输出的实例，当然这离不开我们前面介绍的 Python 库。

重要： 若无特殊提及，本书教程基于 Python 3.8.10 版本 (extboot 分区 Ubuntu20.04 镜像) 进行实验及讲解。

6.2 实验准备

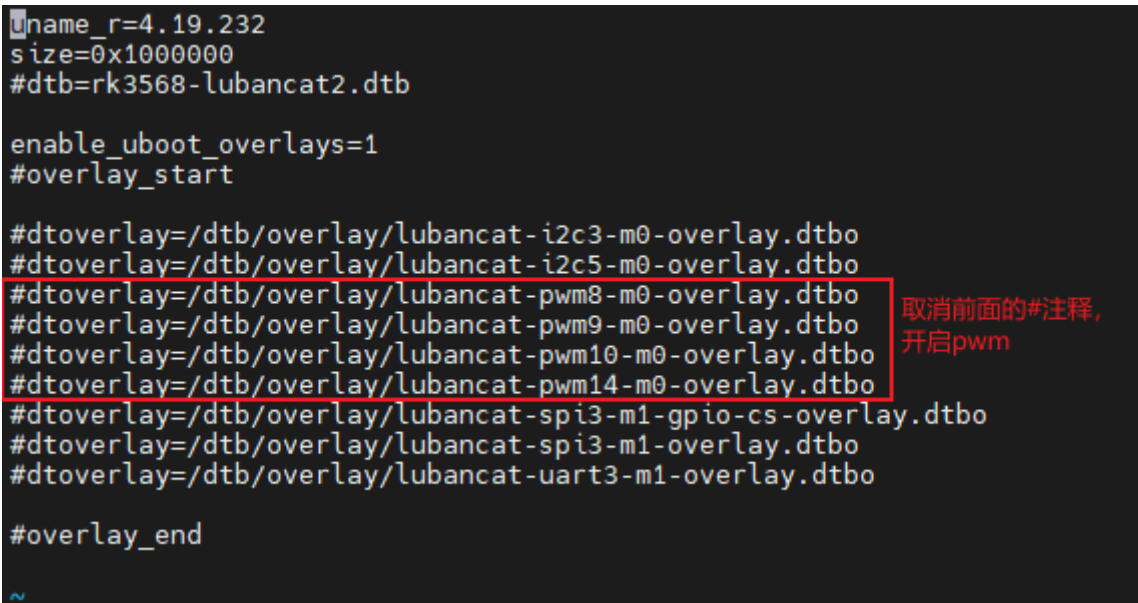
在板卡上的部分 GPIO 可能会被系统占用，可注释掉某些设备树节点或者设备树插件的加载，重启系统，释放相应的 GPIO 引脚，当然也可以换个引脚测试，LubanCat_RK 系列板卡的引脚参考下：《LubanCat-RK 系列-40pin 引脚对照图》

如出现 Permission denied 或类似字样，请注意用户权限，大部分操作硬件外设的功能，几乎都需要 root 用户权限，简单的解决方案是在执行语句前加入 sudo 或以 root 用户运行程序。

pwm 设备树插件的开启方法（以 LubanCat 2 为例）：

```
# 不同板卡的配置文件和 pwm 不同，以 LubanCat 2 为例，配置文件为 uEnvLubanCat 2.txt，
# 可以开启 pwm8、pwm9、pwm10、pwm14
# 登录系统终端，打开/boot/uEnv/uEnvLubanCat 2.txt 文件

sudo vim /boot/uEnv/uEnvLubanCat 2.txt
# 进入编辑模式，取消 pwm 前面的注释，保存并退出文件，重启系统
```



```
uname_r=4.19.232
size=0x1000000
#dtb=rk3568-lubancat2.dtb

enable_uboot_overlays=1
#overlay_start

#dtoverlay=/dtb/overlay/lubancat-i2c3-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-i2c5-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm8-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm9-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm10-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm14-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-spi3-m1-gpio-cs-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-spi3-m1-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-uart3-m1-overlay.dtbo

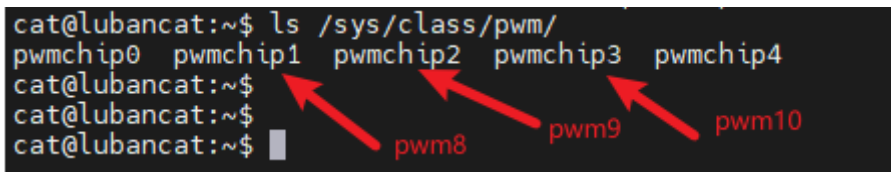
#overlay_end

~
```

```
# 在终端中输入如下命令，可以查看到当前板卡的 PWM 资源：
ls /sys/class/pwm/
```

板卡上的 PWM 资源示例，仅供参考：

```
cat@lubancat:~$ ls /sys/class/pwm/  
pwmchip0 pwmchip1 pwmchip2 pwmchip3 pwmchip4  
cat@lubancat:~$  
cat@lubancat:~$  
cat@lubancat:~$
```



pwmchip0 是 pwm4，用在背光调节，图片中后面的 pwmchip1 是我们开启设备树添加，对应硬件上的 pwm8，依次类推。

6.3 使用 python-periphery

`python-periphery` 库支持的 PWM 输出是基于 Linux 的 PWM 子系统实现的，所以要想利用该库做 PWM 输出，需要板卡提供支持。像鲁班猫板卡，就可以完美使用 `python-periphery` 库实现 PWM 输出。这样这样一来，就不需要我们在软件层面上利用 GPIO 模拟 PWM 输出。

关于 PWM 子系统相关内容，参考《PWM 控制》。

6.3.1 安装 python-periphery

重要：如在前面小节中安装了此库，务必跳过此操作。

`python-periphery` 的安装方式如下：

```
# 在板卡使用如下命令安装  
sudo pip3 install python-periphery
```

6.3.2 periphery 输出 PWM

使用 `python-periphery` 库进行 PWM 输出的示例代码如下：

列表 1: 配套代码 `io/pwm/pwm_test_periphery.py` 文件内容

```
1 from periphery import PWM
2 import time
3
4 try:
5     # 定义占空比递增步长
6     step = 0.05
7     # 定义 range 最大范围
8     rangeMax = int(1/0.05)
9     # 打开 PWM 8, channel 0 , 对应开发板上 PWM8 外设
10    pwm = PWM(1, 0)
11    # 设置 PWM 输出频率为 1 kHz
12    pwm.frequency = 1e3
13    # 设置占空比为 0%，一个周期内高电平的时间与整个周期时间的比例。
14    pwm.duty_cycle = 0.00
15    # 开启 PWM 输出
16    pwm.enable()
17    while True:
18        for i in range(0, rangeMax):
19            # 休眠 step 秒
20            time.sleep(step)
21            # 设置占空比每次加 step%，使用 round 避免浮点运算误差
22            pwm.duty_cycle = round(pwm.duty_cycle+step, 2)
23            # 常灭 1 秒
24            if pwm.duty_cycle == 0.0:
25                time.sleep(1)
26        for i in range(0, rangeMax):
```

(下页继续)

(续上页)

```
27         time.sleep(step)
28         pwm.duty_cycle = round(pwm.duty_cycle-step,2)
29 except:
30     print("Some errors occur!\n")
31 finally:
32     # 退出时熄灭 LED
33     pwm.duty_cycle = 0.0
34     # 释放资源
35     pwm.close()
```

代码说明：

- 第 10 行，创建了一个 PWM 对象
- 第 12、14 行，分别对该 PWM 对象的参数做了初始化，分别是 PWM 输出波形的频率、占空比
- 第 16 行，根据对应的参数，使能 PWM 波形输出

提示： 如果不熟悉 PWM 一些 python 模块的使用，可以进入 python 终端交互模式，使用命令 **from periphery import PWM** 和 **help(PWM)**，查看 PWM 模块相关的帮助。

6.3.2.1 实验操作

示例代码使用 LubanCat 2 板卡，操作如下：

```
# 确认已安装 python-periphery 包
# 使能了 PWM 设备树插件，确认开启的 pwm

# 在板卡 pwm_test.py 所在的目录执行如下命令
sudo python3 pwm_test.py
```

如果提示权限问题，可以切换 root 用户然后运行：

```
# 切换到 root, 是密码 root  
su root
```

如果外接 LED 可看到板载的 LED 灯会有明暗变化，或者可以通过示波器查看波形。

第 7 章 UART 通讯

本章中，我们开始为大家介绍一种常见的通讯协议 UART，在我们的鲁班猫板卡上，我们常常使用终端来与板卡进行交互，而这个终端与板卡连接的众多方式之中，UART 串口就是我们最常使用的方式之一。

关于 UART 协议的具体内容，我们在此不作过多探究，大家可以自行搜索 UART 通讯协议的相关知识学习。

本章中，我们的重点是使用前面 Python 库，来调用板卡上的串口资源。硬件上，通过板卡与 USB 转 TTL 模块连接到我们的电脑上，并通过串口上位机来进行实验现象的演示和操作。

重要：若无特殊提及，本书教程基于 Python 3.8.10 版本 (extboot 分区 Ubuntu20.04 镜像) 进行实验及讲解。

7.1 实验准备

7.1.1 添加 UART 资源

在板卡上的部分 GPIO 可能没有启用，可注释掉某些设备树节点或者设备树插件的加载，重启系统来开启，LubanCat_RK 系列板卡的引脚参考下：《[LubanCat-RK 系列-40pin 引脚对照图](#)》

如笔者使用的鲁班猫 LubanCat 2 板卡，根据引出的 40 个引脚对照图，串口资源可以使用 UART3，下面我们开启下 UART3 的设备树插件：

```
# 不同板卡的配置文件和 pwm 不同，以 LubanCat 2 为例，配置文件为 uEnvLubanCat 2.txt，  
# 使用下面命令：
```

(下页继续)

(续上页)

```
sudo vim /boot/uEnv/uEnvLubanCat 2.txt
# 进入编辑模式，取消 pwm 前面的注释，保存并退出文件，重启系统
```

```
root@lubancat:/boot/uEnv# sudo vim uEnvLubanCat2.txt
uname_r=4.19.232
size=0x1000000
#dtb=rk3568-lubancat2.dtb

enable_uboot_overlays=1
#overlay_start

#dtoverlay=dtb/overlay/lubancat-i2c3-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-i2c5-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-pwm8-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-pwm9-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-pwm10-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-pwm14-m0-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-spi3-m1-gpio-cs-overlay.dtbo
#dtoverlay=dtb/overlay/lubancat-spi3-m1-overlay.dtbo
[dtoverlay=dtb/overlay/lubancat-uart3-m1-overlay.dtbo] 取消#注释，保存重启，开启串口
#overlay_end

~
~
~
~

"uEnvLubanCat2.txt" 19L, 610C                                16,1          All
```

如果使用的是其他鲁班猫 RK 系列板卡，也是类似操作。

```
# 在终端中输入如下命令，可以查看到 UART 资源：
ls /dev/ttyS*
```

板卡上的 UART 资源示例，仅供参考：

```
cat@lubancat:~$ ls /dev/ttyS*
/dev/ttyS3
cat@lubancat:~$
```

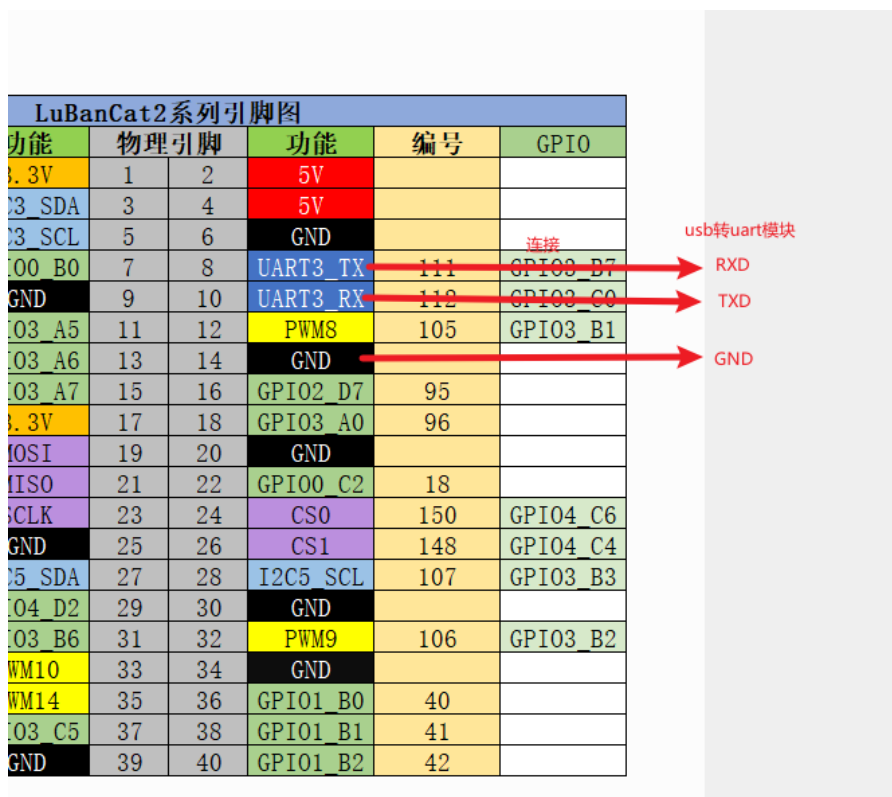
其中/dev/ttyS3 对应的就是板卡上的 uart3 资源。

7.1.2 硬件连接

注：硬件连接小节，需要大家根据自身开发环境等情况对本章内容进行参考调整。

本章中，我们通过上述 Python 库，编写使用 LubanCat 2 板卡上 UART3 资源的代码，并通过 USB 转 TTL 模块、串口上位机来进行代码正确性测试。

笔者硬件连接如图，仅供参考：



USB 转 TTL 模块连接如上图示。

7.2 方式一：使用 pyserial 库

pyserial 库封装了对串口资源的访问方法，该库兼容多种平台对串口资源使用，有许多平台特性相关的方法，官方的使用说明参考：[pyserial 库](#)代替。

在 Linux 系统上，关于 UART 子系统相关内容，参考 [《串口通讯》](#)。

7.2.1 安装 pyserial

重要：在前面小节的实验中，可能已经安装了此库，如已安装则跳过此操作。

pyserial 的安装方式如下：

```
# 在板卡使用如下命令安装
sudo pip3 install pyserial
```

7.2.2 使用 pyserial

列表 1: 配套代码 io/uart/uart_test0.py 文件内容

```
1  """ pyserial uart 测试 """
2  import serial
3
4  # 打开 uart3, 设置串口波特率为 115200, 数据位为 8, 无校验位, 停止位为 1, 不使用流控制,
   # 以非阻塞模式打开串口, 等待时间为 3s
5  with serial.Serial(
6      "/dev/ttyS3",
7      baudrate=115200,
8      bytesize=serial.EIGHTBITS,
9      stopbits=serial.STOPBITS_ONE,
10     parity=serial.PARITY_NONE,
11     timeout=3,
12 ) as uart3:
13     # 使用申请的串口发送字节流数据 "Hello World!\n"
14     uart3.write(b"Hello World!\n")
15
16     # 以非阻塞的方式打开的串口, 在读取串口接收的数据时, 该函数返回条件二者满足其一, 一、读
   取到 128 个字节, 二、读取时间超过 1 秒
```

(下页继续)

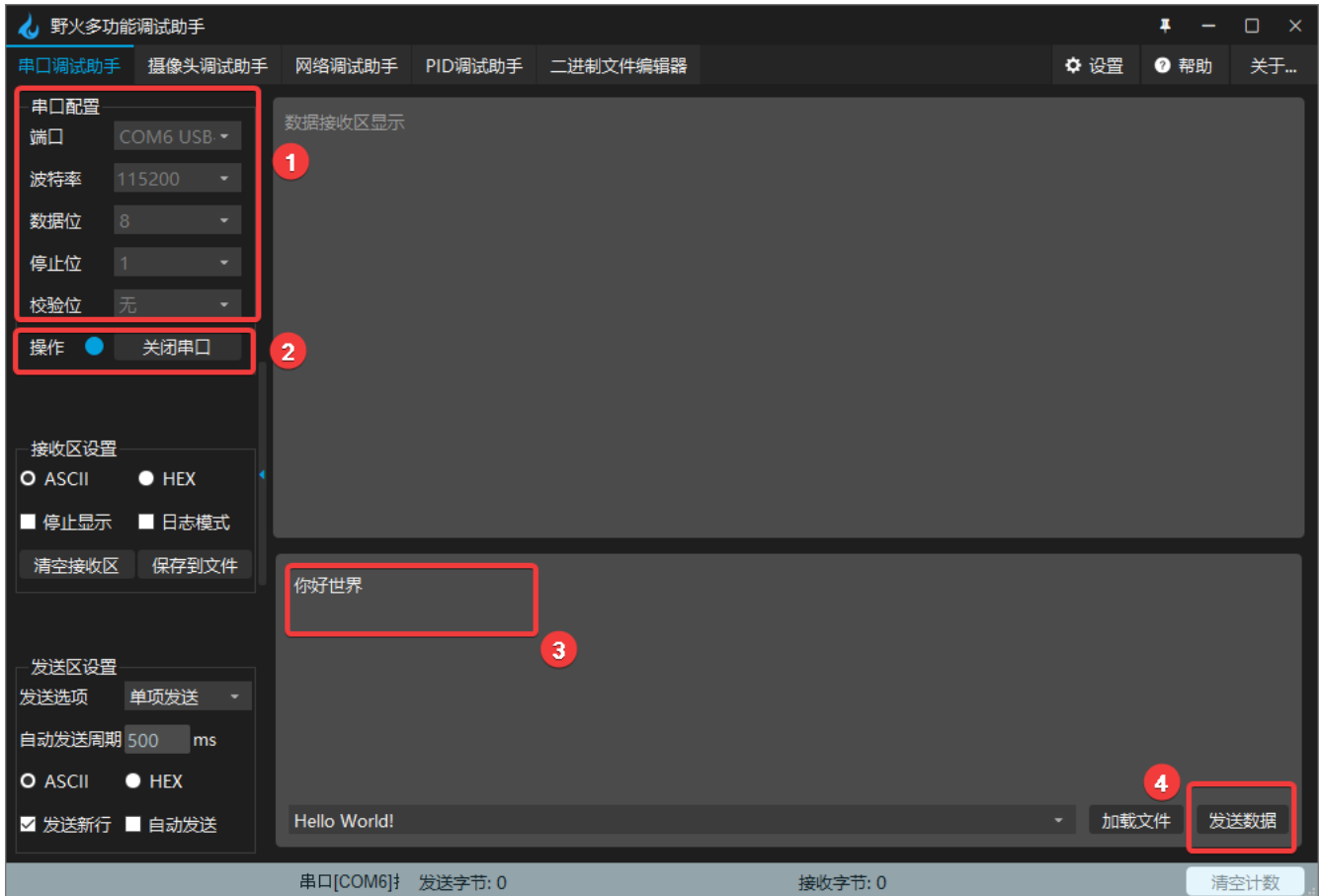
(续上页)

```
17     buf = uart3.read(128)
18
19     # 注: Python 读取出来的数据类型为: bytes
20     # 打印原始数据
21     print("原始数据:\n", buf)
22     # 转码为 gbk 字符串, 可以显示中文
23     data_strings = buf.decode("gbk")
24     # 打印读取的数据量及数据内容
25     print("读取到 {:d} 个字节 , 以字符串形式打印:\n {:s}".format(len(buf), data_
    ↪strings))
```

7.2.2.1 实验操作

由于在硬件连接时, 已经将 `uart3` 对应的 TX\RX 接口与 USB 转 TTL 模块的 RXD\TXD 接口连接, 所以要查看实验现象, 需要使用到串口上位机以显示数据方便观察实验现象。

使用野火多功能调试助手中, 就自带串口上位机。



如图，我们在串口助手设置串口的工作模式与我们程序配置的一致，并打开对应的端口，即图中的 1、2 步骤。接下来我们运行程序。

示例代码使用 LubanCat i.MX6ULL MINI 板卡，操作如下：

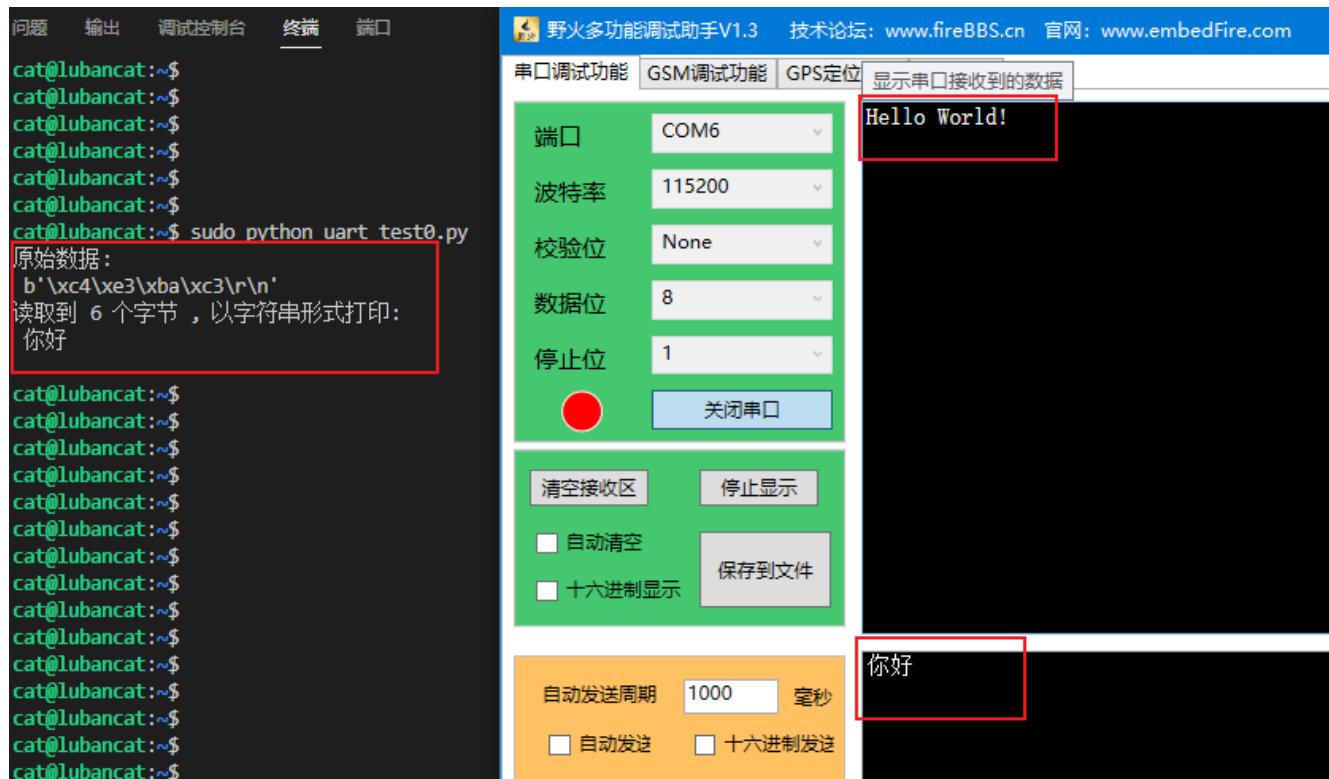
```
# 确认已安装 pyserial 包
# 确认使能了 UART 设备树插件

# 在板卡 uart_test.py 所在的目录执行如下命令
sudo python3 uart_test.py

# 可看到串口助手打印出的程序中编程好的数据：Hello World!\n
```

在终端中输入命令后，我们在串口助手中编辑要发送的数据，并发送，即图中的 3、4 步骤。

在等待超时后，可看到终端打印出的接收到的数据：你好世界



关于该库，更多的用法，参见：[pyserial API](#)。

7.3 方式二：使用 python-periphery

`python-periphery` 库支持的 UART 功能是基于 Linux 的 UART 系统实现的，所以要想利用该库使用到 UART 的功能，需要板卡提供支持。像鲁班猫板卡，就可以完美使用 `python-periphery` 库 UART 通讯功能。

7.3.1 安装 python-periphery

重要： 如在前面小节中安装了此库，务必跳过此操作。

python-periphery 的安装方式如下：

```
# 在板卡使用如下命令安装
sudo pip3 install python-periphery
```

7.3.2 periphery 使用 UART 功能

使用 python-periphery 库进行 UART 功能使用的示例代码如下：

列表 2: 配套代码 io/uart/uart_test1.py 文件内容

```
1  """ python-periphery uart 测试 """
2  from periphery import Serial
3
4  try:
5      # 申请串口资源/dev/ttyS3, 设置串口波特率为 115200, 数据位为 8, 无校验位, 停止位为
6      # 1, 不使用流控制
7      serial = Serial(
8          "/dev/ttyS3",
9          baudrate=115200,
10         databits=8,
11         parity="none",
12         stopbits=1,
13         xonxoff=False,
14         rtscts=False,
15         )
16     # 使用申请的串口发送字节流数据 "python-periphery!\n"
```

(下页继续)

(续上页)

```
16 serial.write(b"python-periphery!\n")
17
18 # 读取串口接收的数据，该函数返回条件二者满足其一，一、读取到 128 个字节，二、读取时间
  超过 1 秒
19 buf = serial.read(128, 1)
20
21 # 注：Python 读取出来的数据类型为：bytes
22 # 打印原始数据
23 print("接收的原始数据:\n", buf)
24
25 # 转码为 gbk 字符串，可以显示中文
26 data_strings = buf.decode("gbk")
27
28 # 打印读取的数据量及数据内容
29 print("读取到 {:d} 个字节，以字符串形式打印:\n {}".format(len(buf), data_
  ↳ strings))
30 finally:
31     # 释放申请的串口资源
32     serial.close()
```

代码说明：

- 第 6 行，申请 UART 资源，占用 uart3，配置对应的工作模式
- 第 16 行，使用申请到的串口资源发送数据
- 第 19 行，使用申请到的串口资源接收数据，接收方式为阻塞接收
- 第 23~29 行，将读取到的数据打印出来
- 第 32 行，释放 UART 资源，释放 uart3

7.3.2.1 实验操作

实验操作与上同。

示例代码使用 LubanCat 2 板卡，操作如下：

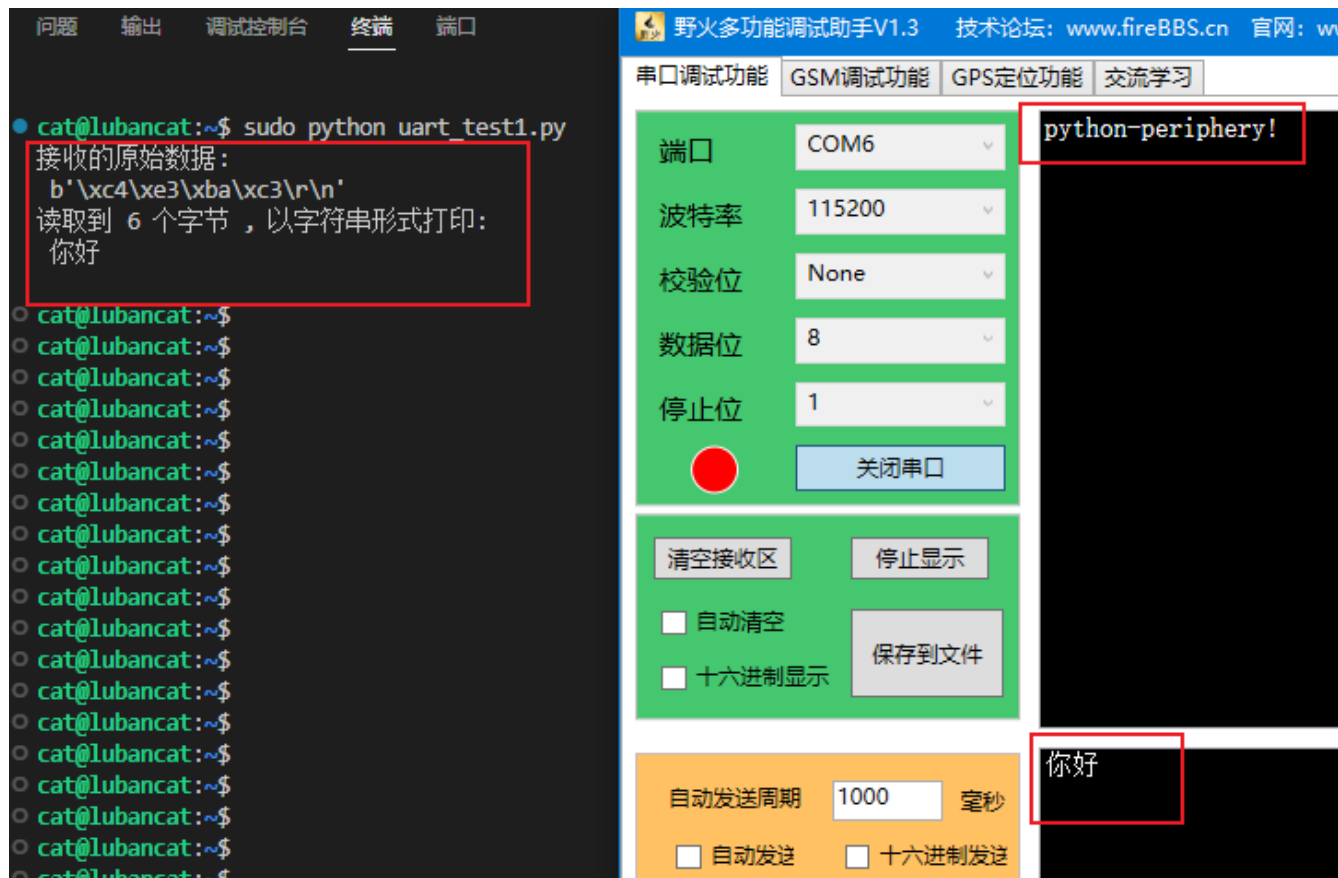
```
# 确认已安装 python-periphery 包
# 确认使能了 UART 设备树插件

# 在板卡 uart_test.py 所在的目录执行如下命令
sudo python3 uart_test1.py

# 可看到串口助手打印出的程序中编程好的数据: Hello World!\n
```

在终端中输入命令后，我们在串口助手中编辑要发送的数据，并发送，即图中的 3、4 步骤。

在等待超时后，可看到终端打印出的接收到的数据：你好



自此，实验完成。

关于 `python-periphery` 库更多的 UART 用法，关注下：[Periphery UART](#)

第 8 章 I2C 通讯

I2C (IIC) 协议是在电子设备中常用的通讯协议,通过它,我们可以对各种各样的电子器件进行控制(注:一般受控的设备为从机),如常见的 姿态传感器 MPU6050、温度图像传感器 MLX90640、0.96 寸 OLED 显示屏 SSD1306 等等,都有通过 I2C 通讯协议来和板卡(注:一般主动发起控制的为主机)进行通讯的。

关于 I2C 协议的具体内容,我们在此不作过多探究,大家可以自行搜索 I2C 通讯协议的相关知识学习。

本章中,我们的重点是使用前面介绍的 [python-periphery](#) 及 [Adafruit Blinka](#) 两个 Python 库,来使用到我们鲁班猫板卡上的 I2C 主机。

8.1 I2C 实验

一般情况下,我们需要一个具体的通讯对象来完成本节实验,即前面提到的各种类型的电子器件。

考虑到大家手中的模块或 I2C 设备各不相同,所以对上述 Python 库的使用介绍分为两部分,一部分为库官方示例 Demo 介绍,另一部分则为使用库中对应的 API,在实际的模块中使用介绍。

笔者使用的是 0.96 寸 OLED 显示屏模块 SSD1306 野火【OLED 屏_I2C_0.96 寸】,对上述提到的库进行使用示例的编写。实际的使用中,鲁班猫板卡上的 I2C 主机能通讯的器件类型并不受限制。因此笔者会在代码中对相关的代码进行详细注释,如大家手中没有相同的模块,也可进行类似的操作,对不同的模块进行实验代码编写。

8.2 实验准备

8.2.1 板卡添加 I2C 资源

在板卡上的 GPIO 可能没有复用为 I2C，可通过设置设备树插件来加载，LubanCat_RK 系列板卡的引脚参考下：《LubanCat-RK 系列-40pin 引脚对照图》

下面以 LubanCat 2 为例开启下 i2c 的设备树插件（LubanCat 2 引出的引脚有两个 i2c）：

```
# 不同板卡的配置文件和 i2c 不同，以 LubanCat 2 为例，配置文件为 uEnvLubanCat 2.txt，  
# 可以开启 I2c3、i2c5  
# 登录系统终端，打开/boot/uEnv/uEnvLubanCat 2.txt 文件  
  
sudo vim /boot/uEnv/uEnvLubanCat 2.txt  
# 进入编辑模式，取消 I2c3 前面的注释，保存并退出文件，重启系统
```

```
uname_r=4.19.232  
size=0x1000000  
#dtb=rk3568-lubancat2.dtb  
  
enable_uboot_overlays=1  
#overlay_start  
  
dtoverlay=/dtb/overlay/lubancat-i2c3-m0-overlay.dtbo  
dtoverlay=/dtb/overlay/lubancat-i2c5-m0-overlay.dtbo  
#dtoverlay=/dtb/overlay/lubancat-pwm8-m0-overlay.dtbo  
#dtoverlay=/dtb/overlay/lubancat-pwm9-m0-overlay.dtbo  
#dtoverlay=/dtb/overlay/lubancat-pwm10-m0-overlay.dtbo  
#dtoverlay=/dtb/overlay/lubancat-pwm14-m0-overlay.dtbo  
#dtoverlay=/dtb/overlay/lubancat-spi3-m1-gpio-cs-overlay.dtbo  
#dtoverlay=/dtb/overlay/lubancat-spi3-m1-overlay.dtbo  
#dtoverlay=/dtb/overlay/lubancat-uart3-m1-overlay.dtbo  
  
#overlay_end
```

取消前面#注释，开启i2c3和i2c5，并保存退出，重启

uEnvLubanCat2.txt

重启后，查看板卡的 I2C 资源是否加载：

在终端中输入如下命令，可以查看到 I2C 资源：

```
ls -l /dev/i2c-*
```

板卡上的 I2C 资源示例，仅供参考：

```
cat@lubancat:~$ ls -l /dev/i2c-*
crw-rw---- 1 root i2c 89, 0 Nov 9 09:30 /dev/i2c-0
crw-rw---- 1 root i2c 89, 1 Nov 9 09:30 /dev/i2c-1
crw-rw---- 1 root i2c 89, 3 Nov 9 09:30 /dev/i2c-3
crw-rw---- 1 root i2c 89, 5 Nov 9 09:30 /dev/i2c-5
crw-rw---- 1 root i2c 89, 6 Nov 9 09:30 /dev/i2c-6
cat@lubancat:~$
```

一些简单 i2c 相关的测试参考下 《I2C 通讯》

8.2.2 硬件连接

鲁班猫 LubanCat 2 板卡，板上无 I2C 器件可进行相关实验现象展示，故本节实验现象，就通过在硬件层面连接器件外围器件 0.96 寸 OLED 显示屏 SSD1306 为大家展示，使用的是 I2c3。

连接如图，仅供参考（LubanCat 2 的 I2c3），oled 屏和板卡 40pin 引脚连接表如下：

OLED 屏幕引脚	对应 GPIO	LubanCat 2 板卡引脚
SDA	GPIO1_A0	I2C3_SDA
SCL	GPIO1_A1	I2C3_SCL
GND	•	GND
VCC	•	3.3V

8.3 使用 python-periphery

`python-periphery` 库支持的 I2C 功能是基于 Linux 的 I2C 系统实现的，所以要想利用该库使用到 I2C 的功能，需要板卡提供支持。像鲁班猫板卡，就可以完美使用 `python-periphery` 库 I2C 通讯功能。这样这样一来，就不需要我们在软件层面上利用 GPIO 模拟 I2C 主机功能了。

8.3.1 安装 python-periphery

重要： 如在前面小节中安装了此库，务必跳过此操作。

`python-periphery` 的安装方式如下：

```
# 在板卡使用如下命令安装
sudo pip3 install python-periphery
```

8.3.2 periphery 使用 I2C 功能

先参考学习下 `python-periphery` 库进行 I2C 功能使用的示例代码：

列表 1: 官方提供的 code example -i2c (学习参考)

```
1 from periphery import I2C
2
3 # 打开 i2c1 主机
4 i2c = I2C("/dev/i2c-1")
5
6 # 以 EEPROM 器件为例，从 EEPROM 内存地址 0x100 处读取数据，EEPROM 器件的 I2C 设备地址为 0x50
7 msgs = [I2C.Message([0x01, 0x00]), I2C.Message([0x00], read=True)]
```

(下页继续)

(续上页)

```
8 # 开启 I2C 传输
9 i2c.transfer(0x50, msgs)
10 # 打印接受到的消息
11 print("0x100: 0x{:02x}".format(msgs[1].data[0]))
12 # 操作完成, 关闭 I2C 主机以释放资源。
13 i2c.close()
```

代码说明:

- 第 4 行, 申请 I2C 资源, 占用 I2C1 主机
- 第 7 行, 构造消息结构 I2C.Message。I2C.Message 中第一成员为发送的数据列表, 第二个指定此消息的作用, 默认为 I2C 的写命令, 在写命令的情况下, 会将数据列表中的数据发送出去, 如果指定 read=True, 则此条消息对应为 I2C 的读命令, 会将读取到的数据存放在 data 成员中。
- 第 9 行, 开启 I2C 传输, 发送 I2C 从机设备的地址与构造好的消息结构
- 第 17 行, 释放 I2C 资源, 释放 I2C1 主机

通过 `python-periphery` 库中给出的对 EEPROM 器件操作的代码示例, 我们可以掌握该库对 I2C 主机使用的基本方法, 那么接下来就可以针对具体的器件展开编程使用了。

这里笔者使用前面所述的 0.96 寸 OLED 显示屏 SSD1306 模块编程展示:

列表 2: 配套代码 io/i2c/i2c_test_periphery.py 文件内容

```
1 """ periphery i2c 测试 使用 0.96 寸 OLED 模块 """
2 import time
3 from periphery import I2C
4
5 # 打开 i2c-3 控制器
6 i2c = I2C("/dev/i2c-3")
7
8 # 设备从机地址 0x3c, 即 OLED 模块地址
```

(下页继续)

(续上页)

```
9 I2CSLAVEADDR = 0x3C
10
11 # 使用 periphery i2c 库读取功能测试
12 def i2c_read_reg(devregaddr):
13     """
14     使用 periphery i2c 库读取功能测试
15     """
16     # 构造数据结构
17     # 第一个 Message 为要读取的设备地址
18     # 第二个 Message 用于存放读取回来的消息，注意其中添加的 read=True
19     msgs = [I2C.Message([devregaddr]), I2C.Message([0x00], read=True)]
20     # 发送消息，发送到 i2cSlaveAddr
21     i2c.transfer(I2CSLAVEADDR, msgs)
22     print("从寄存器 0x{:02x} 读取到: 0x{:02x}".format(devregaddr, msgs[1].
↵data[0]))
23
24
25 def oled_write_cmd(cmd):
26     """
27     使用 periphery i2c 库发送功能测试
28     """
29     msgs = [I2C.Message([0x00, cmd])]
30     i2c.transfer(I2CSLAVEADDR, msgs)
31
32
33 def oled_write_data(data):
34     """
35     使用 periphery i2c 库发送功能测试
36     """
37     msgs = [I2C.Message([0x40, data])]
38     i2c.transfer(I2CSLAVEADDR, msgs)
39
```

(下页继续)

(续上页)

```
40
41 def oled_init():
42     """
43     使用 OLED 模块进行代码功能测试 0.96 寸 OLED 模块初始化
44     """
45     time.sleep(1)
46     oled_write_cmd(0xAE)
47     oled_write_cmd(0x20)
48     oled_write_cmd(0x10)
49     oled_write_cmd(0xB0)
50     oled_write_cmd(0xC8)
51     oled_write_cmd(0x00)
52     oled_write_cmd(0x10)
53     oled_write_cmd(0x40)
54     oled_write_cmd(0x81)
55     oled_write_cmd(0xFF)
56     oled_write_cmd(0xA1)
57     oled_write_cmd(0xA6)
58     oled_write_cmd(0xA8)
59     oled_write_cmd(0x3F)
60     oled_write_cmd(0xA4)
61     oled_write_cmd(0xD3)
62     oled_write_cmd(0x00)
63     oled_write_cmd(0xD5)
64     oled_write_cmd(0xF0)
65     oled_write_cmd(0xD9)
66     oled_write_cmd(0x22)
67     oled_write_cmd(0xDA)
68     oled_write_cmd(0x12)
69     oled_write_cmd(0xDB)
70     oled_write_cmd(0x20)
71     oled_write_cmd(0x8D)
```

(下页继续)

(续上页)

```
72     oled_write_cmd(0x14)
73     oled_write_cmd(0xAF)
74
75
76 def oled_fill(filldata):
77     """
78     清空 OLED 屏幕
79     """
80     for i in range(8):
81         oled_write_cmd(0xB0 + i)
82         # page0-page1
83         oled_write_cmd(0x00)
84         # low column start address
85         oled_write_cmd(0x10)
86         # high column start address
87         for j in range(128):
88             oled_write_data(filldata)
89
90
91 # 代码测试
92 try:
93     # 初始化 OLED 屏幕, SSD1306 必要
94     oled_init()
95     # 清空 OLED 屏幕
96     oled_fill(0xFF)
97     # 读取寄存器测试
98     i2c_read_reg(0x10)
99 finally:
100     print("测试正常结束")
101     # 释放资源
102     i2c.close()
```

关于通过 I2C 主机发送何种命令才能去控制 0.96 寸 OLED 显示屏 SSD1306 模块不是本教程的重点，教程目的是通过提供的示例代码，让大家对相关库的 API 功能使用有个参考对象。

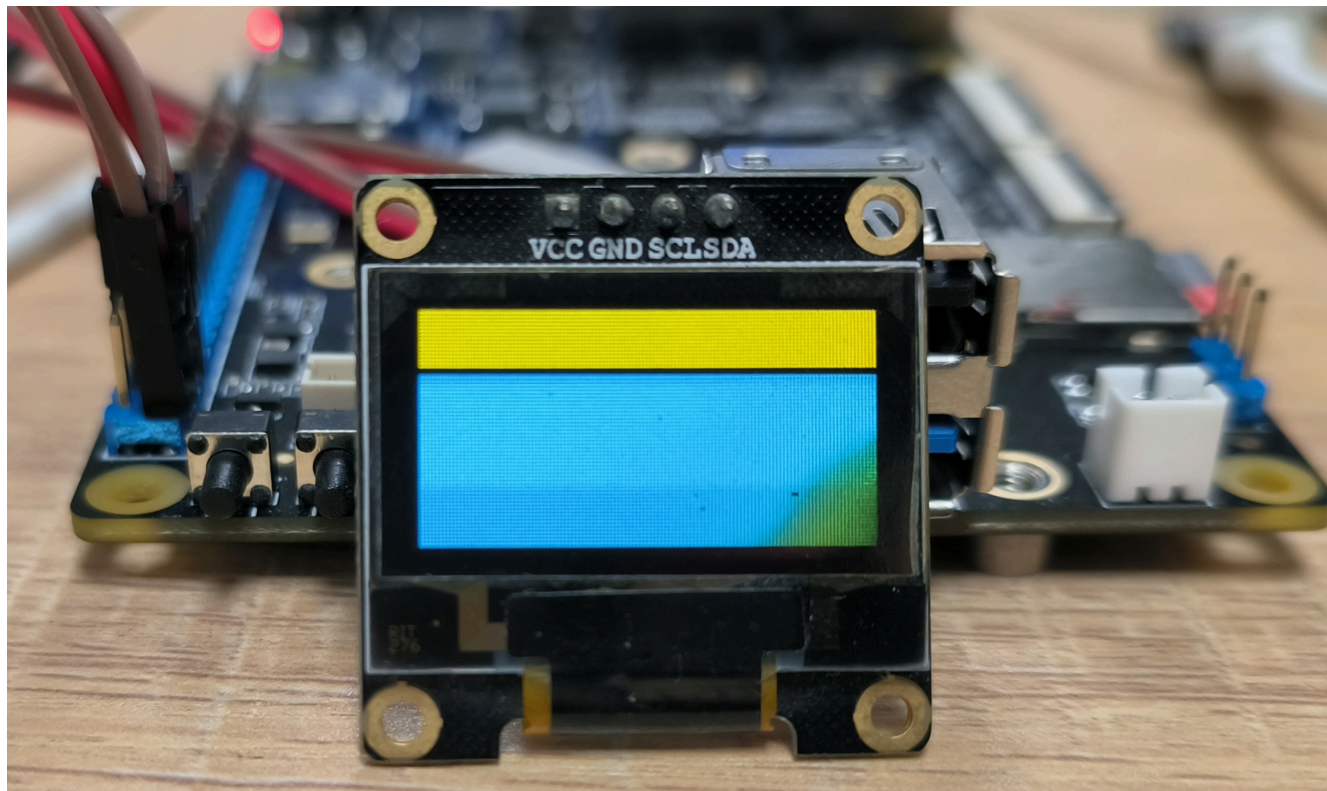
8.3.2.1 实验操作

示例代码使用 LubanCat 2 板卡，操作如下：

```
# 在板卡 i2c_test.py 所在的目录执行如下命令
sudo python3 i2c_test_periphery.py

# 可看到连接的 OLED 屏幕被清空显示色块，并在终端中打印出了读取的寄存器信息
```

显示参考（随意拿的一个旧的屏幕 ~）：



8.4 方式二：使用 Adafruit Blinka

Adafruit Blinka 与 python-periphery 功能类似，I2C 通讯也是基于 Linux 的 I2C 子系统实现的。只是在代码上的用法上有较大差异。

8.4.1 安装 Adafruit Blinka

重要： 如在前面小节中安装了此库，务必跳过此操作。

Adafruit Blinka 的安装方式如下：

```
# 在板卡使用如下命令安装
sudo apt -y install python3-libgpiod python-periphery # 注：如已安装，忽略此步
sudo pip3 install Adafruit-Blinka

sudo pip3 install machine
```

8.4.2 Adafruit-Blinka 使用 I2C 主机

使用这个 Adafruit Blinka 进行输入输出的示例代码如下：

列表 3: 官方提供的 code example - i2c

```
1 import busio
2 from board import *
3
4 # 申请 I2C 资源，占用 I2C1 主机
5 i2c = busio.I2C(SCL, SDA)
6 # 打印扫描到的挂载在 I2C1 主机下的从机设备
7 print(i2c.scan())
```

(下页继续)

(续上页)

```
8 # 操作完成, 关闭 I2C 主机以释放资源
9 i2c.deinit()
```

Adafruit Blinka 库提供的 I2C 使用示例较为简单, 仅仅只是提供了扫描挂载在 I2C1 主机下的设备, 其他的 API 功能具体的使用方法大家可以参考: [Blinka I2C](#) 下给出的介绍。

这里直接展示使用 I2C1 主机与 0.96 寸 OLED 显示屏模块 SSD1306 模块通讯的程序:

列表 4: 配套代码 io/i2c/i2c_test_adafruit.py 文件内容

```
1 """ blinka i2c 测试 使用 0.96 寸 OLED 模块 """
2 import board
3 import busio
4
5 # 初始化 OLED 命令字节数组
6 OledInitBuf = bytes(
7     [
8         0xAE,
9         0x20,
10        0x10,
11        0xB0,
12        0xC8,
13        0x00,
14        0x10,
15        0x40,
16        0x81,
17        0xFF,
18        0xA1,
19        0xA6,
20        0xA8,
21        0x3F,
22        0xA4,
23        0xD3,
```

(下页继续)

(续上页)

```
24         0x00,  
25         0xD5,  
26         0xF0,  
27         0xD9,  
28         0x22,  
29         0xDA,  
30         0x12,  
31         0xDB,  
32         0x20,  
33         0x8D,  
34         0x14,  
35         0xAF,  
36     ]  
37 )  
38  
39 # 数据发送、接受缓冲区  
40 OutBuffer = bytearray(1)  
41 InBuffer = bytearray(1)  
42  
43 try:  
44     # 申请 i2c 资源  
45     i2c = busio.I2C(board.I2C3_SCL, board.I2C3_SDA)  
46     # 扫描 I2C 设备地址, 测试  
47     print("挂载 I2C 总线上的 I2C 设备地址有")  
48     for i in i2c.scan():  
49         print("0x%02x " % i)  
50  
51     # IIC 发送命令测试 (writeto), 初始化 OLED  
52     # 初始化 OLED, 初始命令存放于字节数组 oledInitBuf 中  
53     i2c.writeto(0x3C, OledInitBuf)  
54  
55     # IIC 读取命令测试 (writeto_then_readfrom), 读取 OLED 的 0x10 寄存器
```

(下页继续)

(续上页)

```
56     # 发送数据，发送数据完成后不产生停止信号，并重新生产起始信号进行读取。
57
58     # 发送数据内容存放在字节数组 outBuffer 中，内容为 OLED 寄存器地址：0x10
59     OutBuffer[0] = 0x10
60     # 发送，发送寄存器地址 0x10，并将寄存器地址 0x10 下的内容读取到 inBuffer 中
61     i2c.writeto_then_readfrom(0x3C, OutBuffer, InBuffer)
62     # 打印数据信息
63     print("(writeto_then_readfrom) 读取到的内容为：0x%02x" % InBuffer[0])
64
65     # IIC 读取测试 (readfrom_into)，从设备内部的地址读，则不需要指定读取的地址
66     i2c.readfrom_into(0x3C, InBuffer)
67     # 打印数据信息
68     print("(readfrom_into) 读取到的内容为：0x%02x" % InBuffer[0])
69
70     # OLED 清屏
71     for i in range(8):
72         i2c.writeto(0x3C, bytearray([0x00, 0xB0 + i])) # page0-page1
73         i2c.writeto(0x3C, bytearray([0x00, 0x00])) # low column start_
↪address
74         i2c.writeto(0x3C, bytearray([0x00, 0x10])) # high column start_
↪address
75         for j in range(128):
76             i2c.writeto(0x3C, bytearray([0x40, 0xFF]))
77
78 finally:
79     # 释放 i2c 总线资源
80     i2c.deinit()
```

代码说明：

- 第 45 行，申请 I2C 资源，占用 I2C1 主机
- 第 53 行，将字节数组 OledInitBuf 里的数据，发送到挂载在 I2C1 主机下的从设备，从设备

地址为 0x3C

- 第 59~61 行，将字节数组 OutBuffer 里的数据，发送到挂载在 I2C1 主机下的从设备，并在从设备寄存器地址 0x10 处读取一个数据到字节数组 InBuffer，从设备地址为 0x3C
- 第 66 行，在从设备内部的当前地址读，读取一个数据到字节数组 InBuffer
- 第 80 行，释放 I2C 资源，释放 I2C1 主机

实验操作与上一小节相同，将程序在板卡上运行即可。

代码片段的 busio.I2C(SCL, SDA) 中的 SCL、SDA 是在 Blinky 库定义好的 LubanCat 板卡资源。

更详细的引脚定义可直接查看 [Adafruit Blinky 的源码](#)：

- [Adafruit Blinky 中 LubanCat 系列板卡引脚定义](#)
- [Adafruit Blinky 中 LubanCat RK 系列芯片引脚定义](#)

8.4.3 Adafruit_CircuitPython_SSD1306

在前面我们通过 Blinky 库，实现了通过 I2C 主机控制 0.96 寸 OLED 显示屏 SSD1306 模块，Blinky 库是基于 CircuitPython 重新实现的实时组件对象。简而言之，就是利用 Blinky 库开发的 Python 程序，可以实时去使用到板卡上的硬件资源。如果对 CircuitPython 熟悉的同学，就会知道基于 CircuitPython 库实现的、对各种各样的电子器件实施控制的代码 Demo 是非常多的。

那么也就是说，是不是我们的鲁班猫板卡适配好了 Blinky 库，即可利用 CircuitPython 丰富的各种传感器的控制 Demo 呢？以笔者做上述实验时使用的模块为例，0.96 寸 OLED 显示屏 SSD1306 模块的 Demo 可以正常使用。

那么下面笔者就以该 SSD1306 模块为例，为大家演示一下，如何将 CircuitPython 为 0.96 寸 OLED 显示屏 SSD1306 模块提供支持的程序，运行在我们的鲁班猫板卡上。

首先找到对应的仓库，这里为大家演示的是 0.96 寸 OLED 显示屏 SSD1306，那么大家可以在 Adafruit 的 Github 上 [Adafruit Github](#) 搜索关键字 SSD1306，可以看到 [Adafruit_CircuitPython_SSD1306](#)，这就是要使用到的仓库了。当然大家如果需要，也可以搜索其他常见的模块比如 dht11、mpu6050 等等等等，就需要大家自行探索了。

总之更多 Demo 仓库关注 [Adafruit Github](#)

在对应仓库下方，我们可以看到对应库的添加方法：

```
# 在板卡使用如下命令安装对应的传感器库，下列语句二选一：

# 在 root 用户下输入：
pip3 install adafruit-circuitpython-ssd1306

# 如为普通用户输入：
sudo pip3 install adafruit-circuitpython-ssd1306
```

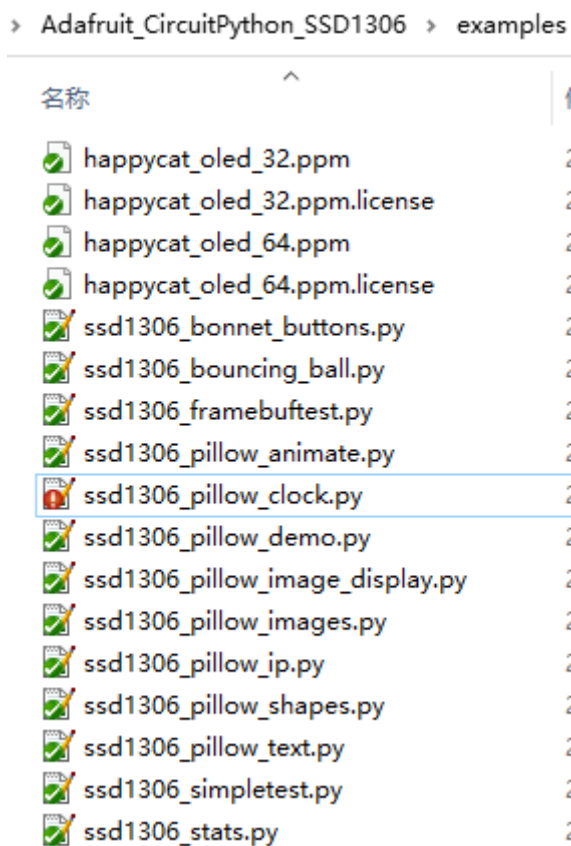
或者大家可以通过其他途径将下载好的仓库上传到板卡中，并进入仓库目录执行以下语句安装：

```
sudo python3 setup.py install
```

由于板卡性能和网络原因，安装过程会十分漫长，请耐心等待。如果安装过程中发送错误，可以更换上述方法中的另外一种再次尝试。

安装完成后，即可使用仓库中提供的各种示例 Demo 了！需要注意的是部分 Demo 中从设备的地址并非能完全适配你手中的模块，所以当出现报错信息时，可能需要修改代码 Demo 中的从设备的地址。

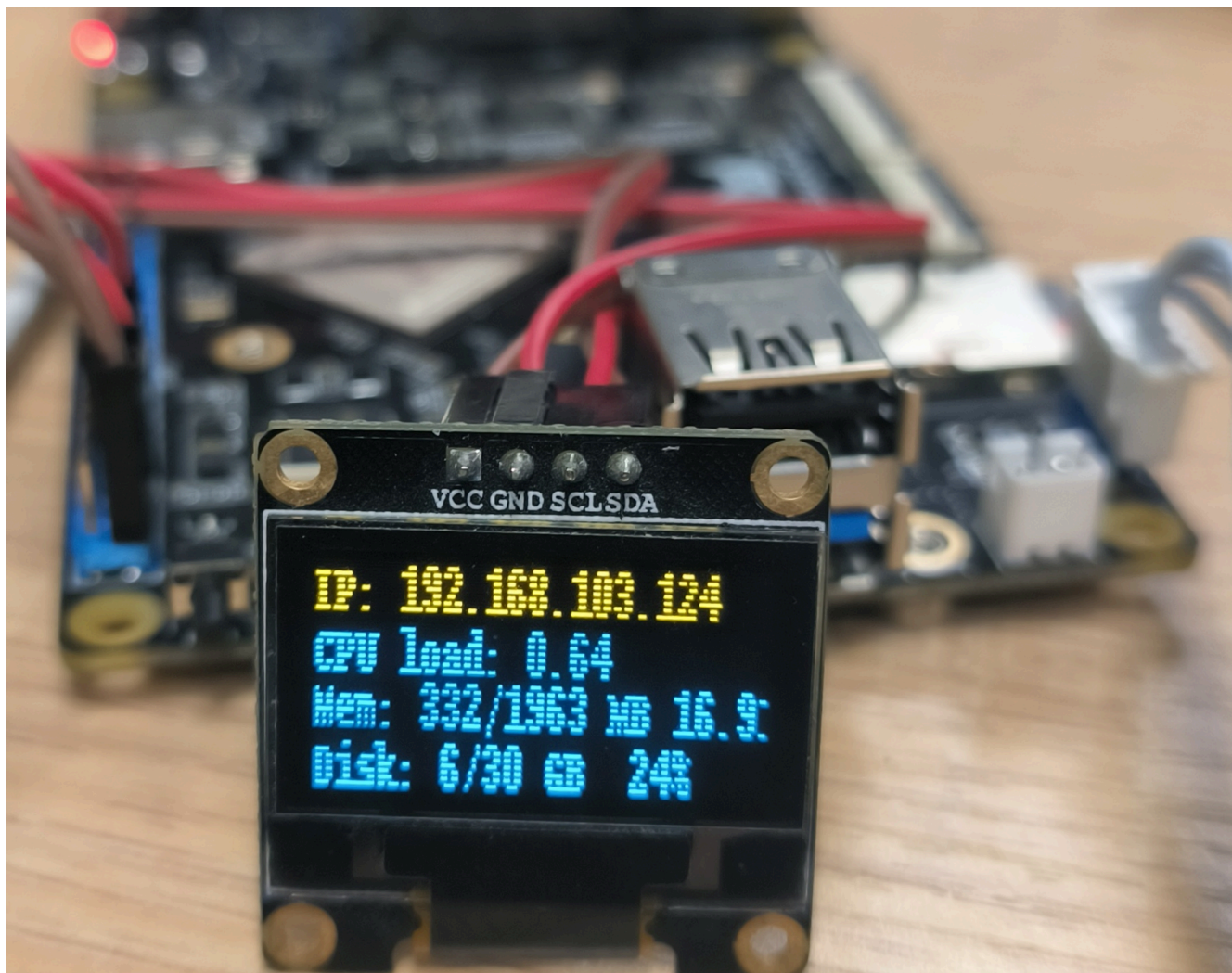
CircuitPython 为 0.96 寸 OLED 显示屏 SSD1306 模块提供的代码 Demo 如下图示例：



最后在终端中运行任意 Demo，查看一下效果：

```
sudo python3 ssd1306_stats.py
```

即可实时显示鲁班猫板卡运行状态等信息：



第 9 章 SPI 通讯

在前面章节里，我们给大家讲到了 I2C 通讯协议使用相关的内容。在本章里，我们接着为大家介绍另一种通讯协议的使用，SPI 通讯协议。SPI 通讯协议同样也是在电子器件通讯中常见的一种通讯协议。不过 SPI 通讯协议的通讯速率要更高，而且它采用的是全双工的通讯方式。所以我们常常能在对通信速率有要求的电子器件上，发现 SPI 通讯协议下规定的接口。比如一些小尺寸的 LCD 屏幕、AD 转换芯片、nRF 系列射频模块、SPI-FLASH 等等。

关于 SPI 协议的具体内容，我们在此不作过多探究，大家可以自行搜索 SPI 通讯协议的相关知识学习。

本章中，我们的重点是使用前面介绍的 `python-periphery` 库，来使用到我们鲁班猫板卡上的 SPI 主机。

9.1 SPI 实验

由于 SPI 通讯协议支持全双工的方式进行通讯，所以我们可以通过回环测试来进行实验验证，在实验中，我们将 SPI 接口的 MOSI、MISO 接口连接在一起即可。

9.2 实验准备

9.2.1 添加 SPI 资源

在板卡上的 GPIO 可能没有复用为 I2C，可通过设置设备树插件来加载，LubanCat_RK 系列板卡的引脚参考下：《[LubanCat-RK 系列-40pin 引脚对照图](#)》

下面以 LubanCat 2 为例开启下 SPI 的设备树插件（LubanCat 2 引出的引脚可使用 spi3）：

```
# 不同板卡的配置文件和 i2c 不同，以 LubanCat 2 为例，配置文件为 uEnvLubanCat 2.txt，  
# 登录系统终端，打开/boot/uEnv/uEnvLubanCat 2.txt 文件
```

```
sudo vim /boot/uEnv/uEnvLubanCat 2.txt  
# 进入编辑模式，取消 I2c3 前面的注释，保存并退出文件，重启系统
```

```
uname_r=4.19.232  
size=0x1000000  
#dtb=rk3568-lubancat2.dtb  
  
enable_uboot_overlays=1  
#overlay_start  
  
dtoverlay=/dtb/overlay/lubancat-i2c3-m0-overlay.dtbo  
dtoverlay=/dtb/overlay/lubancat-i2c5-m0-overlay.dtbo  
#dtoverlay=/dtb/overlay/lubancat-pwm8-m0-overlay.dtbo  
#dtoverlay=/dtb/overlay/lubancat-pwm9-m0-overlay.dtbo  
#dtoverlay=/dtb/overlay/lubancat-pwm10-m0-overlay.dtbo  
#dtoverlay=/dtb/overlay/lubancat-pwm14-m0-overlay.dtbo  
#dtoverlay=/dtb/overlay/lubancat-spi3-m1-gpio-cs-overlay.dtbo  
dtoverlay=/dtb/overlay/lubancat-spi3-m1-overlay.dtbo  
#dtoverlay=/dtb/overlay/lubancat-uart3-m1-overlay.dtbo  
  
#overlay_end
```

取消注释开启spi3

uEnvLubanCat2.txt

重启后，查看板卡的 SPI 资源是否加载：

```
# 在终端中输入如下命令，可以查看到 SPI 资源：  
ls -l /dev/spi*
```

板卡上的 I2C 资源示例，仅供参考：

```
cat@lubancat:~$ ls /dev/spi*  
/dev/spidev3.0  
cat@lubancat:~$
```

关于在 linux 上的 SPI 测试，可以参考下 《SPI 通讯》

9.2.2 硬件连接

注：硬件连接小节，需要大家根据自身开发环境等情况对本章内容进行参考调整。

进行回环测试，连接如下（LubanCat 2）：

LuBanCat2系列引脚图					
GPIO	编号	功能	物理引脚		功能
		3.3V	1	2	5V
GPIO1_A0	32	I2C3_SDA	3	4	5V
GPIO1_A1	33	I2C3_SCL	5	6	GND
	8	GPIO0_B0	7	8	UART3_TX
		GND	9	10	UART3_RX
	101	GPIO3_A5	11	12	PWM8
	102	GPIO3_A6	13	14	GND
	103	GPIO3_A7	15	16	GPIO2_D7
MOSI和MISO直接连接		3.3V	17	18	GPIO3_A0
GPIO4_C3	147	MOSI	19	20	GND
GPIO4_C5	149	MISO	21	22	GPIO0_C2
GPIO4_C2	146	SCLK	23	24	CS0
		GND	25	26	CS1
GPIO3_B4	108	I2C5_SDA	27	28	I2C5_SCL
	154	GPIO4_D2	29	30	GND
	110	GPIO3_B6	31	32	PWM9
GPIO3_B5	109	PWM10	33	34	GND
GPIO3_C4	116	PWM14	35	36	GPIO1_B0
	117	GPIO3_C5	37	38	GPIO1_B1
		GND	39	40	GPIO1_B2

9.3 使用 python-periphery

python-periphery 库支持的 SPI 功能是基于 Linux 的 SPI 系统实现的，所以要想利用该库使用到 SPI 的功能，需要板卡提供支持。像鲁班猫板卡，就可以完美使用 python-periphery 库 SPI 通讯功能。这样这样一来，就不需要我们在软件层面上利用 GPIO 模拟 SPI 主机功能了。

9.3.1 安装 python-periphery

重要： 如在前面小节中安装了此库，务必跳过此操作。

python-periphery 的安装方式如下：

```
# 在板卡使用如下命令安装
sudo pip3 install python-periphery
```

9.3.2 periphery 使用 SPI 功能

使用 python-periphery 库进行 SPI 功能使用的示例代码如下：

列表 1: 配套代码 io/spi/spi_test_periphery.py 文件内容

```
1  """ periphery spi 测试 """
2  from periphery import SPI
3
4  # 待发送数据列表
5  data_out = [0xAA, 0xBB, 0xCC, 0xDD]
6
7  try:
8      # 申请 SPI 资源，打开 spidev3.0 控制器，配置 SPI 主机为工作模式 0、工作速率为
      1MHz
9      spi = SPI("/dev/spidev3.0", 0, 1000000)
10
11     # 发送数据，同时接收数据到 data_in 列表中
12     data_in = spi.transfer(data_out)
13
14     # 打印发送的数据内容
15     print("发送的数据：[0x{:02x}, 0x{:02x}, 0x{:02x}, 0x{:02x}]".
      ↪format(*data_out))
```

(下页继续)

(续上页)

```
16     print("接收到的数据: [0x{:02x}, 0x{:02x}, 0x{:02x}, 0x{:02x}]".  
    ↪format(*data_in))  
17 finally:  
18     # 关闭申请的 SPI 资源  
19     spi.close()
```

代码说明:

- 第 5 行, 申请 SPI 资源, 占用 SPI3 主机, 配置对应的工作模式
- 第 9 行, 构造准备发送的消息列表
- 第 12 行, 开启 SPI 传输, 因为 SPI 为全双工通讯方式, 发送数据的同时也可接收数据
- 第 19 行, 释放 SPI 资源, 释放 SPI3 主机

9.3.2.1 实验操作

示例代码使用 LubanCat 2 板卡, 操作如下:

```
# 在板卡 spi_test.py 所在的目录执行如下命令  
sudo python3 spi_test.py  
  
# 可看到终端打印出的接受到的数据与发送出去的一致
```

```
cat@lubancat:~$ sudo python3 spi_test01.py  
发送的数据: [0xaa, 0xbb, 0xcc, 0xdd]  
接收到的数据: [0xaa, 0xbb, 0xcc, 0xdd]  
cat@lubancat:~$
```

由于在硬件连接时, 已经将 SPI 接口的 MOSI、MISO 接口连接, 所以 SPI 控制器发送出数据的同时也会将数据接收回来, 这就是回环测试。

9.4 方式二：使用 Adafruit Blinka

Adafruit Blinka 与 python-periphery 功能类似，SPI 通讯也是基于 Linux 的 SPI 子系统实现的。只是在代码上的用法上有较大差异。

9.4.1 安装 Adafruit Blinka

重要：如在前面小节中安装了此库，务必跳过此操作。

Adafruit Blinka 的安装方式如下：

```
# 在板卡使用如下命令安装
sudo apt -y install python3-libgpiod python-periphery # 注：如已安装，忽略此步
sudo pip3 install Adafruit-Blinka
```

9.4.2 Adafruit-Blinka 使用 SPI 主机

使用这个 Adafruit Blinka 进行输入输出的示例代码如下：

列表 2: 配套代码 io/spi/spi_test_adafruit.py 文件内容

```
1  """ blinka spi 测试 """
2  import busio
3  from board import *
4
5  # 数据发送、接受缓冲区
6  OutBuffer = [0xAA, 0xBB, 0xCC, 0xDD]
7  InBuffer = bytearray(4)
8
9  try:
```

(下页继续)

(续上页)

```
10     # 申请 spi 资源
11     spi = busio.SPI(SCLK, MOSI, MISO)
12
13     # 配置时先锁定 SPI
14     spi.try_lock()
15     # 配置 SPI 主机工作速率为 1MHz、时钟极性为 0、时钟相位为 0、单个数据位数为 8 位
16     spi.configure(1000000, 0, 0, 8)
17     # 配置操作完成解锁
18     spi.unlock()
19
20     # SPI 通讯, 发送的同时也进行读取, 发送的数据存放在 OutBuffer, 读取的数据存放在
InBuffer
21     spi.write_readinto(OutBuffer, InBuffer)
22
23     print(
24         "(write_readinto) 接收到的数据: [0x{:02x}, 0x{:02x}, 0x{:02x}, 0x
↪{:02x}]" .format(
25             *InBuffer
26         )
27     )
28
29     # SPI 通讯, 只写使用方法演示
30     spi.write(OutBuffer)
31     print(
32         "(OutBuffer) 发送的数据: [0x{:02x}, 0x{:02x}, 0x{:02x}, 0x{:02x}]" .
↪format(*OutBuffer)
33     )
34
35     InBuffer = [0, 0, 0, 0]
36     # SPI 通讯, 只读使用方法演示
37     spi.readinto(InBuffer)
38     print(
```

(下页继续)

(续上页)

```
39         "(readinto) 接收到的数据: [0x{:02x}, 0x{:02x}, 0x{:02x}, 0x{:02x}]" .  
↪format(*InBuffer)  
40     )  
41 finally:  
42     # 释放 spi 资源  
43     spi.deinit()
```

代码说明:

- 第 11 行, 申请 SPI 资源, 占用 SPI3 主机
- 第 14~18 行, 配置 SPI3 主机的工作模式, 在配置前需要先锁定资源
- 第 21 行, 使用 SPI3 主机发送字节数组 OutBuffer 里的数据, 同时读取数据到 InBuffer
- 第 30 行, 使用 SPI3 仅发送数据, 不读取
- 第 37 行, 使用 SPI3 仅读取数据, 不发送

运行后显示:

```
● cat@lubancat:~$ sudo python spi_test_adafruit.py  
(write_readinto)接收到的数据: [0xaa, 0xbb, 0xcc, 0xdd]  
(OutBuffer)发送的数据: [0xaa, 0xbb, 0xcc, 0xdd]  
(readinto)接收到的数据: [0x00, 0x00, 0x00, 0x00]  
○ cat@lubancat:~$
```

另外, 还有一个配套例程 `io/spi/ssd1306_spi_status.py`, 将使用 `spi` 控制屏幕显示, 具体参考下前面 `i2c` 使用 `Adafruit_CircuitPython_SSD1306` 库控制屏幕显示的小节。

`Adafruit Blinka` 库未提供 SPI 对象的使用示例, 相关的 API 功能大家可以参考: [CircuitPython SPI](#) 下给出的介绍。

第 10 章 屏幕显示-Pygame

我们的鲁班猫板卡上支持各种各样的显示设备，板卡上一般有 HDMI 接口、MIPI DSI 接口。

10.1 Pygame 库简介

Pygame 是 Python 库中用于开发游戏的一种工具，或许这么说有点限制 Pygame 的用法。Pygame 不仅可以用于游戏开发，还可以用于显示处理，多媒体设备处理等方面，该库提供了许多多媒体、显示设备的操作功能。其中的 display 模块，提供控制 Pygame 显示界面（display）的各种函数，本章将使用该模块进行屏幕显示。

10.2 实验准备

10.2.1 添加屏幕资源

在终端中输入如下命令，可以查看到显示设备资源：

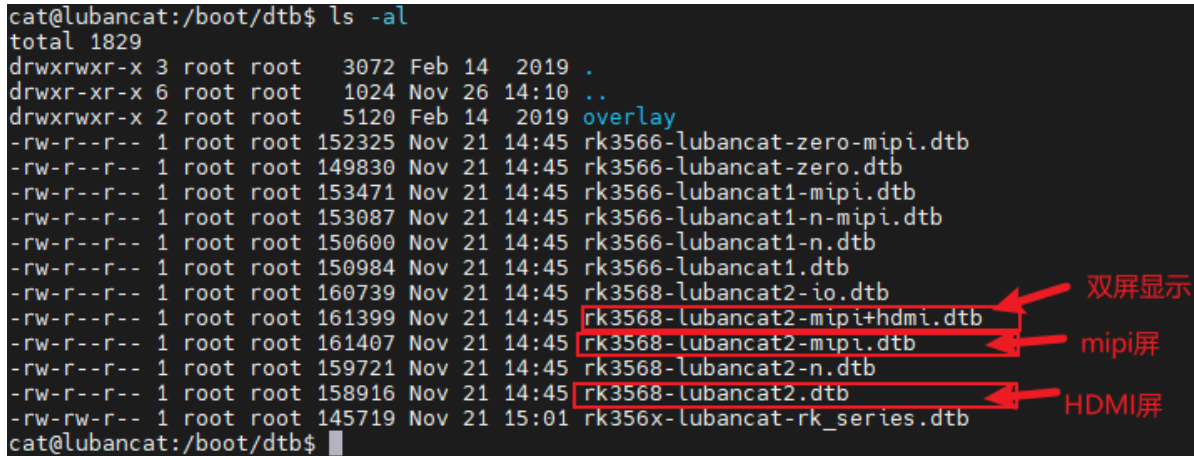
```
cat@lubancat:~$ ls /dev/dri/card*  
/dev/dri/card0  /dev/dri/card1
```

连接好屏幕，会有：

```
cat@lubancat:~/lcd$ ls /dev/fb*  
/dev/fb0
```

板卡上的显示设备资源示例，仅供参考（以 LubanCat 2，ubuntu 镜像为例）：

```
cat@lubancat:/boot/dtb$ ls -al
total 1829
drwxrwxr-x 3 root root 3072 Feb 14 2019 .
drwxr-xr-x 6 root root 1024 Nov 26 14:10 ..
drwxrwxr-x 2 root root 5120 Feb 14 2019 overlay
-rw-r--r-- 1 root root 152325 Nov 21 14:45 rk3566-lubancat-zero-mipi.dtb
-rw-r--r-- 1 root root 149830 Nov 21 14:45 rk3566-lubancat-zero.dtb
-rw-r--r-- 1 root root 153471 Nov 21 14:45 rk3566-lubancat1-mipi.dtb
-rw-r--r-- 1 root root 153087 Nov 21 14:45 rk3566-lubancat1-n-mipi.dtb
-rw-r--r-- 1 root root 150600 Nov 21 14:45 rk3566-lubancat1-n.dtb
-rw-r--r-- 1 root root 150984 Nov 21 14:45 rk3566-lubancat1.dtb
-rw-r--r-- 1 root root 160739 Nov 21 14:45 rk3568-lubancat2-io.dtb
-rw-r--r-- 1 root root 161399 Nov 21 14:45 rk3568-lubancat2-mipi+hdmi.dtb
-rw-r--r-- 1 root root 161407 Nov 21 14:45 rk3568-lubancat2-mipi.dtb
-rw-r--r-- 1 root root 159721 Nov 21 14:45 rk3568-lubancat2-n.dtb
-rw-r--r-- 1 root root 158916 Nov 21 14:45 rk3568-lubancat2.dtb
-rw-rw-r-- 1 root root 145719 Nov 21 15:01 rk356x-lubancat-rk_series.dtb
cat@lubancat:/boot/dtb$
```



默认是使用 rk3568-LubanCat 2.dtb 设备树，通过创建软连接选择对应设备树：

```
# 必须先切换到 boot 目录
cd /boot
# 切换 mipi 屏幕的设备树
ln -sf dtb/rk3568-LubanCat 2-mipi.dtb rk-kernel.dtb
# 重启开启使用 mipi 屏幕
sudo reboot
```

更多屏幕显示相关，参考下《屏幕》。

10.2.2 硬件连接

硬件连接小节，需要大家根据自身开发环境等情况对本章内容进行参考调整，这里实验是 mipi 屏（5.5 寸）连接 LubanCat 2 板卡，板卡系统带桌面。

10.2.3 Pygame 库安装

我们使用 apt 工具安装。


```
# 在终端中输入如下命令，安装 Pygame 库：
sudo apt -y install python3-pygame
# 如果之前安装过，就不需要重新安装
```

10.2.4 Pygame 库使用

安装好对应的库之后，我们就可以利用安装好的 Pygame 库编写一下测试代码。

虽然显示驱动是 DRM，但是驱动也实现了模拟 FB 设备，测试前我们可以简单使用下 Linux Framebuffer，在 Pygame 库利用 Linux Framebuffer 机制实现了调整显示设备的功能，Linux Framebuffer 把屏幕上的每个点映射成一段线性内存空间，程序可以简单的改变这段内存的值来改变屏幕上某一点的颜色。以使用以下命令不断地往 /dev/fb0 写入随机数据，使得屏幕花屏。

```
# 在终端中输入如下命令测试：
cat /dev/urandom > /dev/fb0
```

更多使用参考下《屏幕显示（framebuffer）》。

10.3 测试代码一

测试代码如下：

列表 1: 配套代码 io/lcd/lcd_test1.py 文件内容

```
1  """ 使用 pygame 进行屏幕显示测试 """
2  import os
3  import sys
4  import time
5  import pygame
6
7  # 设置系统环境，无鼠标
```

(下页继续)

(续上页)

```
8 os.environ["SDL_NOMOUSE"] = "1"
9
10 # 初始化显示设备
11 pygame.display.init()
12
13 # 设置显示窗口的范围为屏幕大小，并返回一个屏幕对象
14 screen = pygame.display.set_mode((448, 418))
15
16 # 设置窗口名称
17 pygame.display.set_caption("test")
18
19 # 使用 pygame image 模块加载图片
20 ball = pygame.image.load("test.png")
21
22 # 将图片按位拷贝到窗口
23 screen.blit(ball, (0, 0))
24
25 # 刷新窗口
26 pygame.display.flip()
27
28 time.sleep(5)
29 sys.exit()
```

以上代码会将下图显示到 mipi 显示屏的一个窗口上:



10.3.1 实验步骤

将配套代码 Lcd 目录中的测试代码及测试图片，一并上传至开发板同一目录下：

```
# 在终端中输入如下命令，因为访问硬件注意 sudo 权限：  
sudo python3 lcd_test1.py
```

10.4 测试代码二

列表 2: 配套代码 io/lcd/lcd_test2.py 文件内容

```
1 """ 使用 pygame 进行屏幕测试 """  
2 import os  
3 import sys
```

(下页继续)

(续上页)

```
4 import time
5 import pygame
6
7 class PyScope:
8     """ 定义一个 PyScope 类, 进行屏幕测试 """
9
10    screen = None
11
12    def __init__(self):
13        "PyScope 类的初始化方法, 使用 framebuffer 构造 pygame 会使用到的图像缓冲区"
14
15        # 设置系统环境为无鼠标模式
16        #os.environ["SDL_NOMOUSE"] = "1"
17        pygame.display.init()
18
19        # 创建, 设置 pygame 使用的窗口大小
20        self.screen = pygame.display.set_mode((600, 600))
21
22        # 设置窗口标题
23        pygame.display.set_caption('lcd_test2.py')
24
25        # 填充背景色为灰色
26        self.screen.fill((156, 156, 156))
27
28        # 初始化字体库
29        pygame.font.init()
30
31        # 使用默认字体显示英文
32        font=pygame.font.Font(None, 32)
33        text=font.render("Hello!", True, (255, 255, 255))
34        self.screen.blit(text, (100, 100))
35
```

(下页继续)

(续上页)

```
36     # 使用默认字体显示文字
37     text=font.render("Display Test!",True,(255,0,0))
38     self.screen.blit(text,(100,150))
39
40     # 更新屏幕内容
41     pygame.display.flip()
42
43     def __del__(self):
44         "退出 pygame 库的时候会调用该方法，可以在此添加资源释放操作"
45         pygame.display.quit()
46
47     def test(self):
48         while True:
49             # 循环获取事件，监听事件
50             for event in pygame.event.get():
51                 # 判断用户鼠标是否点了关闭按钮
52                 if event.type == pygame.QUIT:
53                     # 卸载所有模块
54                     pygame.quit()
55                     # 终止程序
56                     sys.exit()
57             # 更新屏幕内容
58             #pygame.display.flip()
59
60 # 创建一个测试实例，开始测试
61 scope = PyScope()
62 # 调用 scope 类的测试方法
63 scope.test()
```

代码功能简单看下上面注释，在 `__init__` 方法中，会初始化窗口显示，最终该代码会在弹窗显示两行英文。

10.4.1 实验步骤

将配套代码 Lcd 目录中的测试代码上传至开发板任一目录下：

```
# 在终端中输入如下命令：  
sudo python3 lcd_test2.py
```

命令执行后系统桌面会弹窗显示，鼠标点击或者关闭程序，可以关闭窗口。

10.5 参考

本章没有详细介绍 display 模块基本函数使用，关于更多 Pygame display 模块函数等，可参考 [Pygame display](#)。

关于 Python Pygame 库的使用，可参考 [Pygame 教程](#)。

第 11 章 音频播放-Pygame

LubanCat-RK 系列板卡中，LubanCat 2 和 LubanCat 1 等带有音频 rk809-codec。通过音频编解码芯片，本章我们将使用 Python-Pygame 库的 music 模块，在鲁班猫板卡，编写测试代码进行音频播放实验。

11.1 Pygame 库简介

Pygame 是 Python 库中用于开发游戏的一种工具，或许这么说有点限制 Pygame 的用法。Pygame 不仅可以用于游戏开发，还可以用于显示处理，多媒体设备处理等等方面，该库提供了许多多媒体、显示设备的操作功能。

11.2 实验准备

11.2.1 板卡上的 Sound 资源

以 LubanCat 2 为例：

```
# 在终端中输入如下命令，可以查看到显示设备资源：  
ls /dev/snd/
```

检查板卡上的音频设备资源示例，仅供参考：

```
cat@lubancat:~/sound$ ls /dev/snd/  
by-path  controlC0  pcmC0D0c  pcmC0D0p  seq  timer
```

其中，controlC0 是用于声卡的控制，如通道选择，麦克风的控制等，pcmC0D0c 是用于录音的 pcm 设备，pcmC0D0p 用于播放的 pcm 设备，seq 是音序器，timer 是定时器。

linux 更多关于鲁班猫音频系统内容，参考 《音频》。

11.2.2 硬件连接

LubanCat 1 和 LubanCat 2 板卡有耳机 + 麦克风接口，将耳机或其他播放设备与鲁班猫接口连接即可。

11.2.3 Pygame 库安装

我们使用 apt 工具安装。

```
# 在终端中输入如下命令，安装 Pygame 库：  
sudo apt -y install python3-pygame
```

11.2.4 Pygame 库使用

安装好对应的库之后，我们就可以利用安装好的 Pygame 库编写一下测试代码。

11.3 测试代码

测试代码如下：

列表 1: 配套代码../io/sound/sound_test.py 文件内容

```
1  """ pygame 音频播放测试 """  
2  import sys  
3  import time  
4  import threading  
5  
6  if len(sys.argv) == 2:
```

(下页继续)

(续上页)

```
7     m_filename = sys.argv[1]
8 else:
9     print(
10         """ 使用方法:
11         python3 sound_test.py <music_name.mp3>
12         example: python3 sound_test.py test.mp3"""
13     )
14     sys.exit()
15
16 import pygame
17
18 exitright = 'e'
19
20 def get_quitc():
21     '获取退出字符'
22     # 全局变量
23     global exitright
24     # 获取用户输入
25     exitright = input("输入字母 q 后, 按下回车以退出播放\n")
26
27 try:
28     # 初始化音频设备
29     pygame.mixer.init()
30     # 检查音频设备初始化情况
31     if pygame.mixer.get_init() is None:
32         raise RuntimeError('音频资源未正确初始化, 请检查音频设备是否可用',)
33     # 加载音频文件, 此处未播放 mp3 文件, 因为 pygam 对 MP3 格式的播放支持是有限的
34     pygame.mixer.music.load(m_filename)
35     # 设置音量为 50%
36     pygame.mixer.music.set_volume(0.2)
37     # 播放音频文件 1 次
38     pygame.mixer.music.play(0)
```

(下页继续)

(续上页)

```
39
40     # 开启子线程，等待用户输入退出字符
41     quitthread = threading.Thread(target=get_quitc, daemon=True)
42     quitthread.start()
43     # 获取音频播放状态，如果正在播放，等待播放完毕
44     while pygame.mixer.music.get_busy():
45         # 空闲延时，释放 cpu
46         if exitright in ('q', 'Q'):
47             raise
48         time.sleep(1)
49 except pygame.error as e:
50     print("执行程序时出现异常:", e, "\n等待音频资源被释放")
51     # 尝试停止播放
52     pygame.mixer.music.stop()
53     # 等待音频资源空闲
54     while pygame.mixer.music.get_busy():
55         time.sleep(1)
56 finally:
57     # 卸载初始化的音频设备
58     print("退出播放")
59     pygame.mixer.quit()
60     sys.exit()
```

代码功能已经在注释中详细给出。

11.3.1 实验步骤

在测试代码前，请先将配套代码文件夹 io/sound 目录下，各格式的 test 音频文件与代码放在同一目录下，再运行程序。

测试代码会播放用户指定音频文件。

```
# 在终端中输入如下命令：
sudo python3 sound_test.py test.mp3
# 或
sudo python3 sound_test.py test.wav
# 或
sudo python3 sound_test.py test.ogg
```

pygame 库导入时加载的资源比较多，所以会稍等一会才有实验现象。程序正常运行后，大家就可以在播放设备中听到播放的音频啦 ~

在音乐播放过程中，如要退出，请按终端打印的提示信息操作，在终端输入 q 再回车，程序即可结束。如程序异常终止，会导致音频设备资源被占用，需要重启开发板才可再次使用。

运行显示：

```
cat@lubancat:~/test/sound$ python3 sound_test.py test.wav
pygame 1.9.6
Hello from the pygame community. https://www.pygame.org/contribute.html
输入字母q后，按下回车以退出播放
q
退出播放
cat@lubancat:~/test/sound$ ls
sound_test.py  test.mp3  test.ogg  test.wav
cat@lubancat:~/test/sound$
```

11.3.2 参考

更多关于 Python Pygame 库的使用，可参考 [Pygame 教程](#)。

关于 Pygame music，可参考 [Pygame music](#)。

第 12 章 摄像头使用-Opencv

在鲁班猫 RK 系列板卡上，支持摄像头这种常见的传感器的使用，板卡都有 24pin mipi csi 接口，测试使用的摄像头是 ov5648。本章将使用 opencv-python 库使用摄像头并进行简单处理。

提示：教程基于 Python 3.8.10 版本 (镜像系统是 Ubuntu20.04) 进行实验及讲解。

12.1 opencv-python 库简介

OpenCV 是计算机视觉中经典的专用库，其支持多语言，跨平台，功能强大，于 1999 年由 Gary Bradsky 在英特尔创立，并于 2000 年发布第一个版本。OpenCV 现在支持与计算机视觉和机器学习相关的众多算法，并且正在日益扩展。OpenCV 支持各种编程语言，如 C++，Python，Java 等，可在不同的平台上使用，包括 Windows，Linux，OS X（该系统已经更名为 macOS），Android 和 iOS。基于 CUDA 和 OpenCL 的高速 GPU 操作接口也在积极开发中。

OpenCV-Python 是基于 Python 的库，或者说是 OpenCV 的 Python 应用接口，旨在解决计算机视觉问题，结合了 OpenCV C++ API 和 Python 语言的最佳特性。

12.2 鲁班猫板卡添加 camera 资源

本节实验中，测试使用鲁班猫 LubanCat 2 板卡为例演示，上电前连接摄像头连接到 mipi csi 接口，系统启动后：

```
# 在终端中输入如下命令，可以查看到 camera 设备资源：  
ls /dev/video*
```

检查板卡上的 camera 设备资源示例，仅供参考：

```
cat@lubancat:~$ ls /dev/video*  
/dev/video-dec0 /dev/video1 /dev/video4 /dev/video7  
/dev/video-enc0 /dev/video2 /dev/video5 /dev/video8  
/dev/video0 /dev/video3 /dev/video6
```

linux 系统下更多摄像头使用参考《摄像头》，该章节中会介绍 v4l-utils 等工具，可以查看到摄像头设备的相关信息。

12.3 相关工具及库安装

实验中我们会使用到一些工具，输入如下命令安装（如果有安装就不需要）：

1、安装一些依赖库或者工具：

```
# 在终端中输入如下命令：  
sudo apt update  
sudo apt -y install python3-pil python3-numpy python3-pip git wget  
  
# 可能需要的库，有些可能默认安装  
sudo apt install libavcodec-dev libavformat-dev libswscale-dev  
sudo apt install libgstreamer-plugins-base1.0-dev libgstreamer1.0-dev
```

等待安装完成后即可。

2、安装 opencv-python 库：

使用命令直接安装 (建议)：

```
# 可以使用 apt 命令安装 python3-opencv 包：  
sudo apt-get install python3-opencv  
  
# 或者使用 pip 指定 opencv 版本安装：  
pip3 install opencv-python==4.7.0.68
```

获取 whl 文件安装，可以到这里 [下载](#) 对应 python 版本和架构的 whl 文件

```
# 例如下面的 whl 文件，可以使用命令安装：  
pip3 install opencv_python-4.5.4.60-cp37-cp37m-manylinux_2_17_aarch64.  
↳manylinux2014_aarch64.whl
```

或者下载源码编译安装，可以参考此 [链接](#)。

提示：如果使用 Debian10 系统，要用 opencv_python 控制摄像头，要安装高版本的 opencv_python 和 pip3，使用命令（更新 pip3 和安装 whl 用 root 权限）：`sudo pip3 install --upgrade pip`，然后选择上面的方法安装 opencv_python。

3、验证是否安装成功：

打开交互 Python 终端 (或 IPython)：

```
cat@lubancat:~$ python3  
Python 3.8.10 (default, Nov 14 2022, 12:59:47)  
[GCC 9.4.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>> import cv2 as cv  
>>> print(cv.__version__)  
4.2.0  
>>>
```

12.4 摄像头拍照

列表 1: 配套代码 io/camera/camera_photo.py 文件内容

```
1 import os  
2 import cv2  
3
```

(下页继续)

(续上页)

```
4  # 帧宽和高度
5  width = 640
6  height = 480
7
8  num = 1
9
10 # 创建一个 VideoCapture 对象，并打开系统默认的摄像头（也可以打开视频或者指定的设备）
11 cap = cv2.VideoCapture(0)
12
13 # 不能打开摄像头
14 if not cap.isOpened():
15     raise RuntimeError('Could not open camera.')
16
17 # 设置帧宽和高度
18 cap.set(cv2.CAP_PROP_FRAME_WIDTH, width)
19 cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
20
21 while(cap.isOpened()):
22
23     # 返回两个参数，ret 表示是否正常打开，frame 是图像数组，一帧
24     ret, frame = cap.read()
25
26     # 窗口显示，名为 Lubancat_Camera_test
27     cv2.imshow("Lubancat_Camera_test", frame)
28
29     # 延迟 1ms，并根据键盘输入返回值 val，是板卡接的键盘
30     val = cv2.waitKey(1) & 0xFF
31
32     # 捕获到键盘输入 's'，图片保存到当前目录下
33     if val == ord('s'):
34         print("=====")
35         # 第一个参数是保存为的图片名，第二个参数为待保存图像，jpeg 格式
```

(下页继续)

(续上页)

```
36     cv2.imwrite("photo" + str(num) + ".jpg", frame)
37     print("width = ", width)
38     print("height = ", height)
39     print("success to save photo: " + 'photo' + str(num) + ".jpg")
40     print("=====")
41     num += 1
42
43     # 若检测到按键 'q', 退出, 是板卡接的键盘
44     if val == ord('q'):
45         break
46
47 # 释放摄像头
48 cap.release()
49 # 关闭窗口
50 cv2.destroyAllWindows()
```

使用带桌面的镜像系统，连接好键盘和屏幕，在板卡系统运行程序，使用摄像头详细看下上面注释。使用命令运行程序：

```
sudo python3 camera_photo.py
```

此时屏幕出现弹窗显示画面，当键盘按下“s”，会保存图片到当前目录下，安装“q”会退出程序。运行示例：


```
cat@lubancat:~/test/camera/test$ sudo python3 camera_photo.py
[ WARN:0] global ../modules/videoio/src/cap_gstreamer.cpp (935) open OpenCV | GStreamer warning: Cannot query video
(python3:43255): Gtk-WARNING **: 10:33:20.314: Locale not supported by C library.
    Using the fallback 'C' locale.
=====
frame width = 640
frame height = 480
success to save photo: photo1.jpg
=====
frame width = 640
frame height = 480
success to save photo: photo2.jpg
=====
cat@lubancat:~/test/camera/test$ ls
camera_photo.py camera_video.py out photo1.jpg photo2.jpg
```

12.5 摄像头捕获、录制视频

视频中每一帧代表一幅图像，我们通过捕获一帧帧的图像，进行压缩保存，来录制视频。

列表 2: 配套代码 io/camera/camera_video01.py 文件内容

```
1 import os
2 import cv2
3
4 # 帧宽，高度，帧率
5 width = 640
6 height = 480
7 fps = 25.0
8
9 # 创建一个 VideoCapture 对象，并打开系统默认的摄像头
10 video = cv2.VideoCapture(0)
11
12 # 设置帧宽和高度
13 video.set(cv2.CAP_PROP_FRAME_WIDTH, width)
14 video.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
15
```

(下页继续)

(续上页)

```
16 # 定义视频文件的写入格式, 未压缩的 YUV 颜色编码类型 (使用该格式保存的视频文件很大, 需要注意下), 有些格式可能不支持, 编码器失败, 需要安装相关库
17 # fourcc = cv2.VideoWriter_fourcc(*'MJPG')
18 fourcc = cv2.VideoWriter_fourcc('I','4','2','0')
19
20 # 用于实现多张图像保存成视频文件, 第一个参数是需要保存的视频文件名称, 第二个函数是编解码器的代码,
21 # 第三个参数为保存视频的帧率, 第四个参数是保存的视频文件的尺寸, 一定要与图像的尺寸相同
22 # out = cv2.VideoWriter('output.mp4',fourcc, fps, (width,height))
23 out = cv2.VideoWriter('output.avi',fourcc, fps, (width,height))
24
25 while(video.isOpened()):
26     ret, frame = video.read()
27     if ret is True:
28         # 窗口显示, 名为 Lubancat_Camera_video
29         cv2.imshow('Lubancat_Camera_video',frame)
30
31         # 将捕捉到的图像存储, 保存的视频是没有声音的
32         out.write(frame)
33
34         # 延迟 1ms, 如果键盘按下 "q", 退出, 是板卡接的键盘
35         if cv2.waitKey(1) & 0xFF == ord('q'):
36             break
37     else:
38         print("Unable to read camera!")
39         break
40
41 # 释放资源
42 video.release()
43 out.release()
44 cv2.destroyAllWindows()
```

运行程序，就会进行捕获和录制，视频保存在当前目录下，详细过程看下上面注释。当键盘按下“q”时退出，或者直接中断程序运行。

播放视频，也可以用 cv2.VideoCapture，参考下：

列表 3: 配套代码 io/camera/camera_video02.py

```
1 import cv2
2
3 # 创建一个 VideoCapture 对象，选择播放的视频文件
4 cap = cv2.VideoCapture('output.avi')
5
6 while (cap.isOpened()):
7     ret, frame = cap.read()
8
9     # 窗口显示，名为 Lubancat_Camera_video
10    cv2.imshow('Lubancat_Camera_video', frame)
11
12    # 延迟 1ms，如果键盘按下“q”，退出，是板卡接的键盘
13    if cv2.waitKey(1) & 0xFF == ord('q'):
14        break
15
16 # 释放资源
17 cap.release()
18 cv2.destroyAllWindows()
```

12.6 参考

https://docs.opencv.org/4.6.0/dc/d4d/tutorial_py_table_of_contents_gui.html

第 13 章 Jupyter Notebook

13.1 Jupyter Notebook 简介

Jupyter Notebook 是基于网页的用于交互计算的应用程序。Jupyter Notebook 以网页的形式打开，能够在每条命令编写好后，实时地查看到代码执行后的数据变化，如在编程过程中需要编写说明文档，可在同一个页面中直接编写，便于作及时的说明和解释。在 Python 软件开发中，Jupyter Notebook 可以说是一个利器。



Jupyter Notebook 应用程序可在 Linux 环境下运行，也有人将其部署在服务器中。那么我们的鲁班猫板卡功能与传统 PC 功能相似，也是在 Linux 环境下工作的，自然而然，我们也可以将 Jupyter Notebook 软件安装在我们的鲁班猫板卡上，来利用它开发自己的 Python 程序。

下面我们就来进行 Jupyter Notebook 软件的安装。

13.2 Jupyter Notebook 安装

13.2.1 软件安装

使用鲁班猫板卡，可以很方便的安装 Jupyter Notebook，仅需两条指令。

重要： 后续操作均在普通用户 `cat` 下执行命令。

```
# 使用命令
pip3 config set global.index-url https://mirror.baidu.com/pypi/simple
pip3 install jupyter

# 或者在终端中输入如下命令安装：
sudo apt -y install ipython3
sudo apt -y install jupyter
```

`ipython3` 是一个 python 的交互式 shell，使用 Jupyter Notebook 开发 python 应用程序时我们会使用到。

`jupyter` 则是我们要安装的 Jupyter Notebook 应用程序了。

13.2.2 配置 Jupyter

我们希望设置 Jupyter Notebook 文件存放位置（工作目录），需要先生成 Jupyter Notebook 的配置文件，然后修改配置文件，当我们启动 Jupyter Notebook 时，应用程序按照特定的配置去启动程序。在终端中输入如下命令：

```
# 生成 Jupyter Notebook 的配置文件
jupyter notebook --generate-config
```

(下页继续)

(续上页)

```
# 输入后会有如下提示，配置文件根据登录的用户不同而不同：
Writing default config to: /home/cat/.jupyter/jupyter_notebook_config.py
```

提示中，已经将默认配置文件生成在了当前登录的用户目录--cat下，那么我们根据此目录，打开对应的配置文件进行修改。

```
# 在修改配置文件前，先创建 Jupyter Notebook 的工作目录
mkdir /home/cat/jupyter

# 使用 vim 编辑器打开配置文件
vim /home/cat/.jupyter/jupyter_notebook_config.py
```

打开配置文件后（按 a 进入编辑模式）：

```
c.NotebookApp.allow_remote_access = True # 允许远程连接
c.NotebookApp.ip = '*' # 监听所有 ip 访问
c.NotebookApp.notebook_dir = '/home/cat/jupyter' # jupyter 的工作目录，如目录不存在则必须创建
c.NotebookApp.token = '' # 不使用 token 密钥方式访问，不安全但是方便
```

在默认配置中，所以选项默认被 # 号注释，我们只需取消需要修改的选项对应的注释，并按照上述配置的内容将配置修改好，或者直接添加上面四行，保存即可。

需要额外注意的时，在 `c.NotebookApp.notebook_dir` 选项指定的目录，是 Jupyter Notebook 启动时的工作目录，该目录必须存在，如不存在请先自行创建！

13.2.3 安全登录配置

在我们给大家提供的修改配置文件的示例中，我们默认关闭掉了 token 登录的选项，仅为了大家方便使用。这是一种不安全的做法，因为任何用户都可以登录使用 Jupyter Notebook 服务，文件安全得不到保障。

如大家需要安全的生产环境，大家可以按照如下方式操作，如不需要则跳过此小节：

```
# 在终端中输入如下命令：
ipython3

# 在启动的 python 编辑器中输入：
from notebook.auth import passwd

passwd()
# 出现 Enter password:，输入任意密码，并将生成的密钥复制保存下来。
Out[2]: 'sha1:274e5a04060e:e65098703b7e03836b01c261b6ca6d959049a3e9'
# ctrl+z 退出 ipython3
```

```
# 使用 vim 编辑器打开配置文件
vim /home/cat/.jupyter/jupyter_notebook_config.py

# 然后注释 token 配置选项
# c.NotebookApp.token = ''

# 添加密码登录，将刚才生成的密码填入
c.NotebookApp.password =
↪ 'sha1:274e5a04060e:e65098703b7e03836b01c261b6ca6d959049a3e9'
```

自此软件已经安装完成，下面我们可以启动 Jupyter Notebook，进行使用。

13.3 Jupyter Notebook 使用前注意事项

重要：使用 Jupyter Notebook 操作外设时，务必参考本小节内容。

在我们使用 Python 操作部分外设硬件时，是需要 root 用户权限的。同理，如果使用 Jupyter Notebook 操作部分外设硬件时，也需要 root 权限，那么解决方法就是登录 root 用户并以 root 用户启动 Jupyter Notebook。

需要注意的是，Jupyter Notebook 的启动时会检查环境配置，不同的用户拥有各自的配置文件！

所以，以 root 用户身份登录 Jupyter Notebook 前，需要以 root 用户再走一遍前面的配置流程。

即：以 root 用户启动 Jupyter Notebook 前，需要先登录 root 用户，并以 root 用户的身份使用 `jupyter notebook --generate-config` 命令生成配置文件并进行修改以配置 Jupyter Notebook 环境。

13.4 Jupyter Notebook 使用

13.4.1 启动软件

输入以下命令，以启动软件。

```
# 在终端中输入如下命令，可以启动 Jupyter Notebook:
# 允许 root 用户使用，启动软件时不打开浏览器
jupyter notebook --no-browser --allow-root
```

软件启动后会打印如下的提示：

```
cat@lubancat:~$ jupyter notebook --no-browser --allow-root
[W 14:00:12.943 NotebookApp] WARNING: The notebook server is listening on all IP addresses and not using encryption. This is not recommended
[I 14:00:12.978 NotebookApp] Serving notebooks from local directory: /home/cat/jupyter
[I 14:00:12.979 NotebookApp] The Jupyter Notebook is running at:
[I 14:00:12.979 NotebookApp] http://lubancat:8888/
[I 14:00:12.980 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
```

根据提示，我们在浏览器中输入对应网址，即可使用 Jupyter Notebook 的功能了。

以 LubanCat 2 板卡为例，板卡上有两个网口，插上网线即可，电脑和板卡在一个网段：


```
cat@lubancat:~$ ifconfig
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 2a:fa:d5:eb:2c:d3 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 38

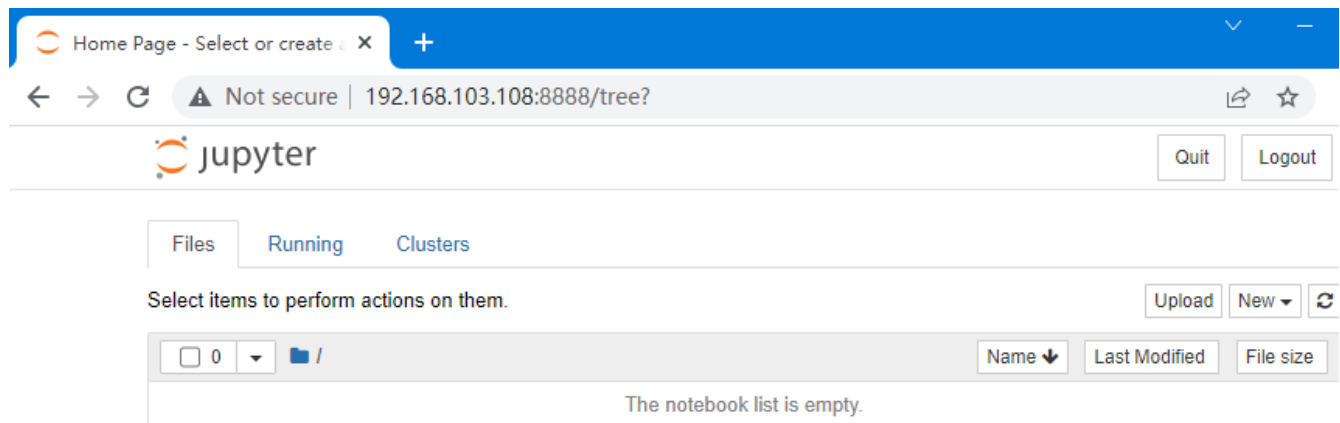
eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.103.108 netmask 255.255.255.0 broadcast 192.168.103.255
    inet6 fe80::11b4:3fba:fb2:bbbc prefixlen 64 scopeid 0x20<link>
    ether 2e:fa:d5:eb:2c:d3 txqueuelen 1000 (Ethernet)
    RX packets 786922 bytes 158359403 (158.3 MB)
    RX errors 0 dropped 20401 overruns 0 frame 0
    TX packets 9301 bytes 4816175 (4.8 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 40

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2529 bytes 1546842 (1.5 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2529 bytes 1546842 (1.5 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

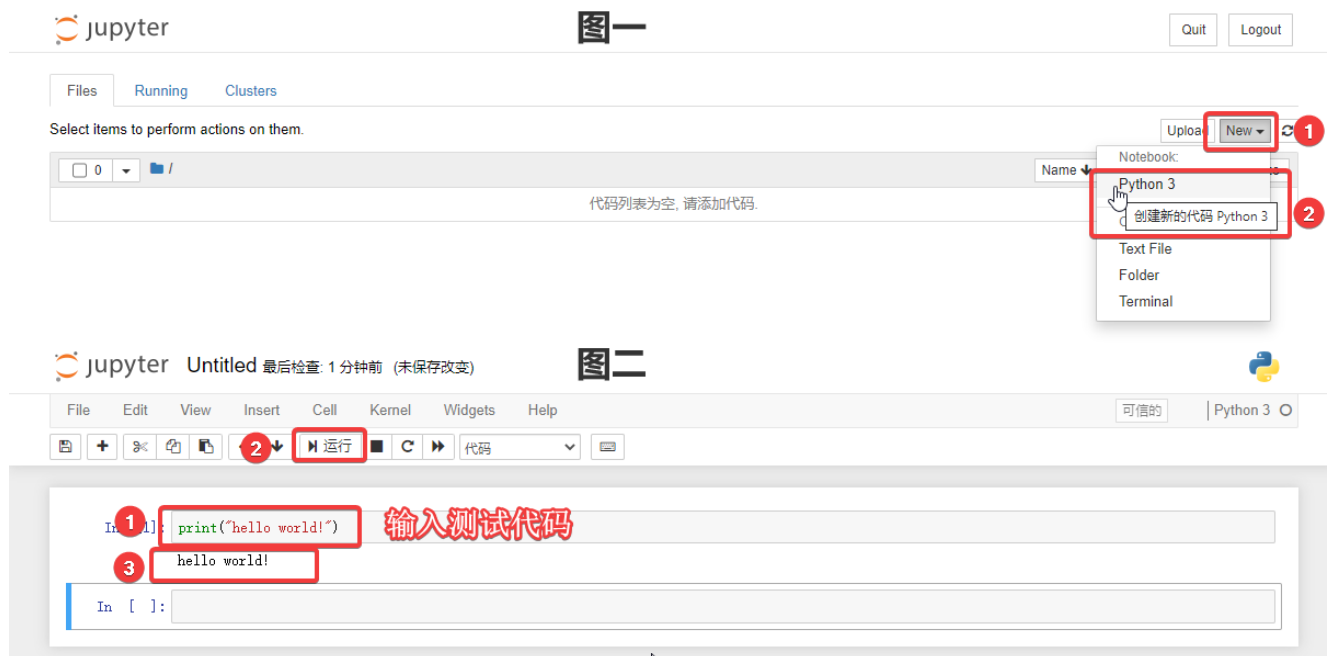
在 PC 浏览器中输入 (IP 地址根据实际板卡):

<http://192.168.103.108:8888/> 可访问 Jupyter Notebook

输入创建的密码, 然后到登录启动界面如图:

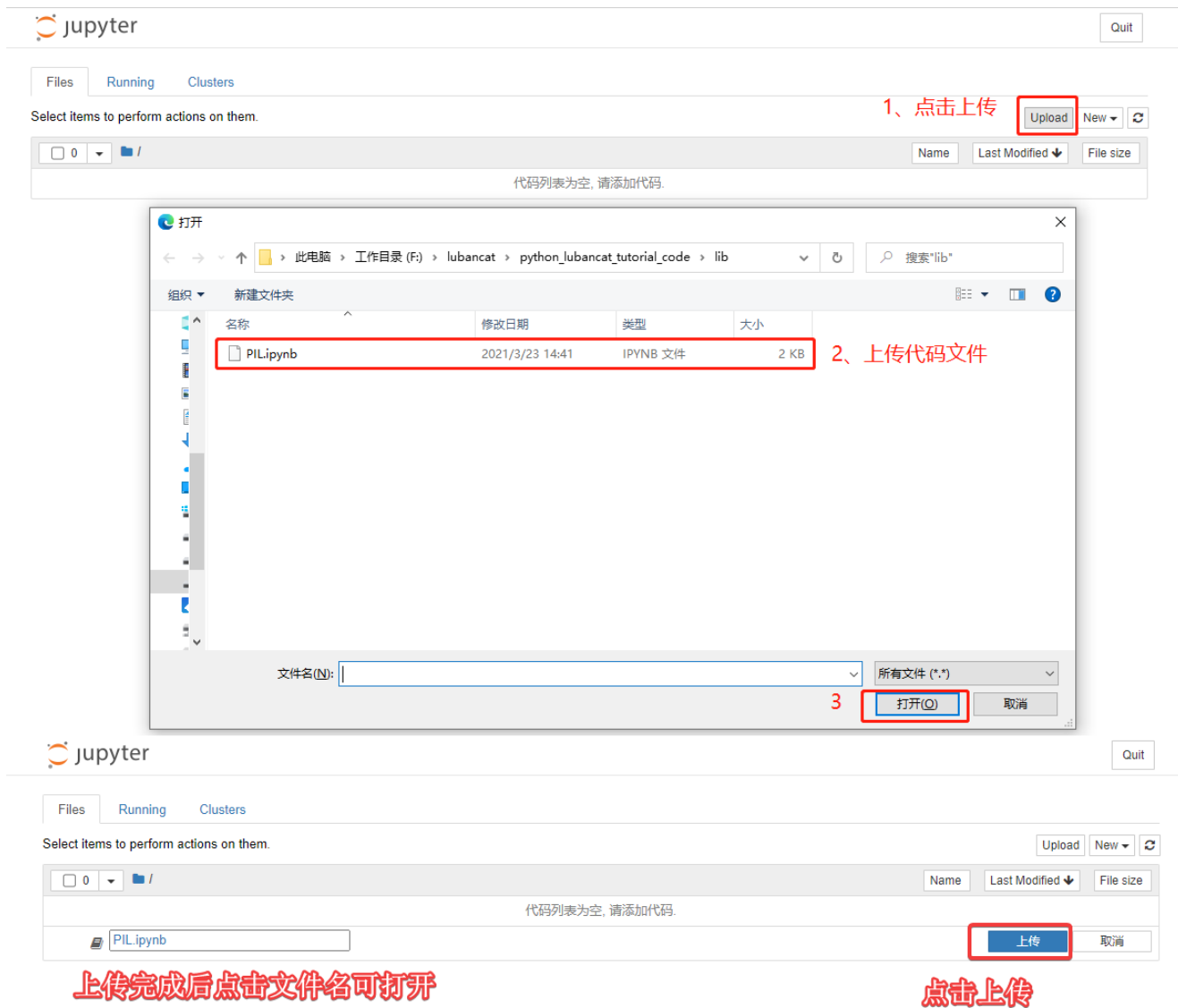


启动之后, 我们可以尝试这使用一下 Jupyter Notebook, 编写简单的代码来进行测试:



13.4.2 上传代码

有时候我们也可以使用别人写好的代码，我们只需要将对应的文件下载，并上传到 jupyter 上即可使用。



13.5 参考链接

<https://docs.jupyter.org/en/latest/>

第 14 章 PIL 库

14.1 PIL 库简介

PIL(Python Image Library)，是 python 一个强大的图像处理库，使用该库，我们可以对图像做非常多的处理工作。我们可以在鲁班猫板卡上安装 Python-PIL 库，并通过 jupyter 编写一些测试代码来使用该库。

14.2 PIL 库安装

我们使用 apt 工具安装。

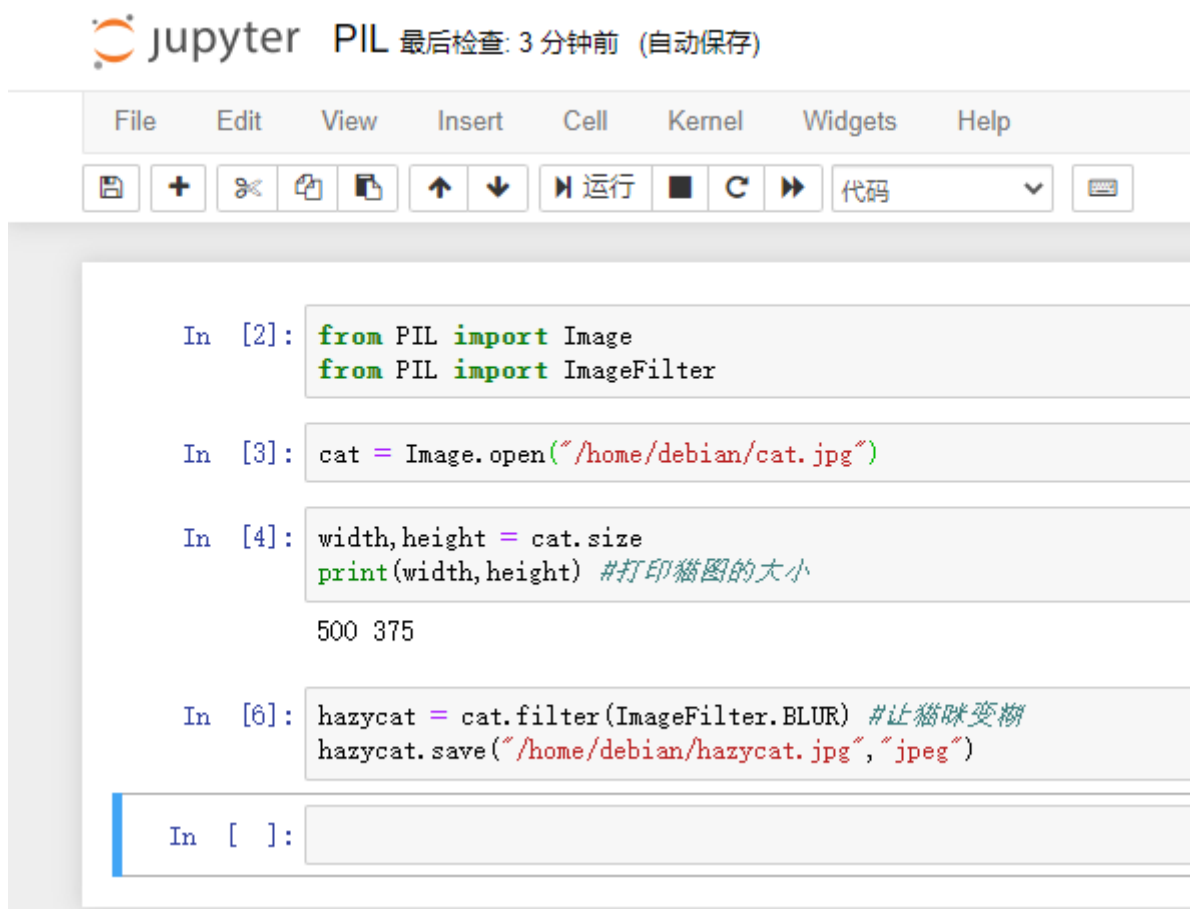
```
# 在终端中输入如下命令，安装 PIL 库：  
sudo apt -y install python3-pil
```

等待安装完成后即可。

14.3 PIL 库使用

安装好对应的库之后，我们就可以利用前面安装的 jupyter 来编写一下测试代码。

测试代码如下：



The image shows a Jupyter Notebook interface with the title "PIL 最后检查: 3 分钟前 (自动保存)". The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for saving, adding cells, undo, redo, copy, paste, and running code. The code is as follows:

```
In [2]: from PIL import Image
        from PIL import ImageFilter

In [3]: cat = Image.open("/home/debian/cat.jpg")

In [4]: width,height = cat.size
        print(width,height) #打印猫图的大小
        500 375

In [6]: hazycat = cat.filter(ImageFilter.BLUR) #让猫咪变糊
        hazycat.save("/home/debian/hazycat.jpg","jpeg")

In [ ]:
```

效果如下:

原图:



代码执行后:



大家可以将配套代码目录 libdemo\PIL\PIL.ipynb 文件，上传到 jupyter 中使用。

更多关于 Python 图像处理库相关的用法，则需要大家自行探索了，需要注意的是 Python3 版本后，图像处理库为 pillow，是在 PIL 的基础上创建了兼容的版本。

第 15 章 Numpy 库

15.1 Numpy 库简介

NumPy(Numerical Python) 是 Python 一个强大的数值计算扩展处理库，支持大量的维度数组与矩阵运算，此外也针对数组运算提供大量的数学函数库，在 Python 应用的许多领域，我们都可以见到 Numpy 的身影，比如科学运算、人工智能、图像处理等等。我们可以在鲁班猫板卡上安装 Python-NumPy 库，并通过 jupyter 编写一些测试代码来使用该库。

15.2 NumPy 库安装

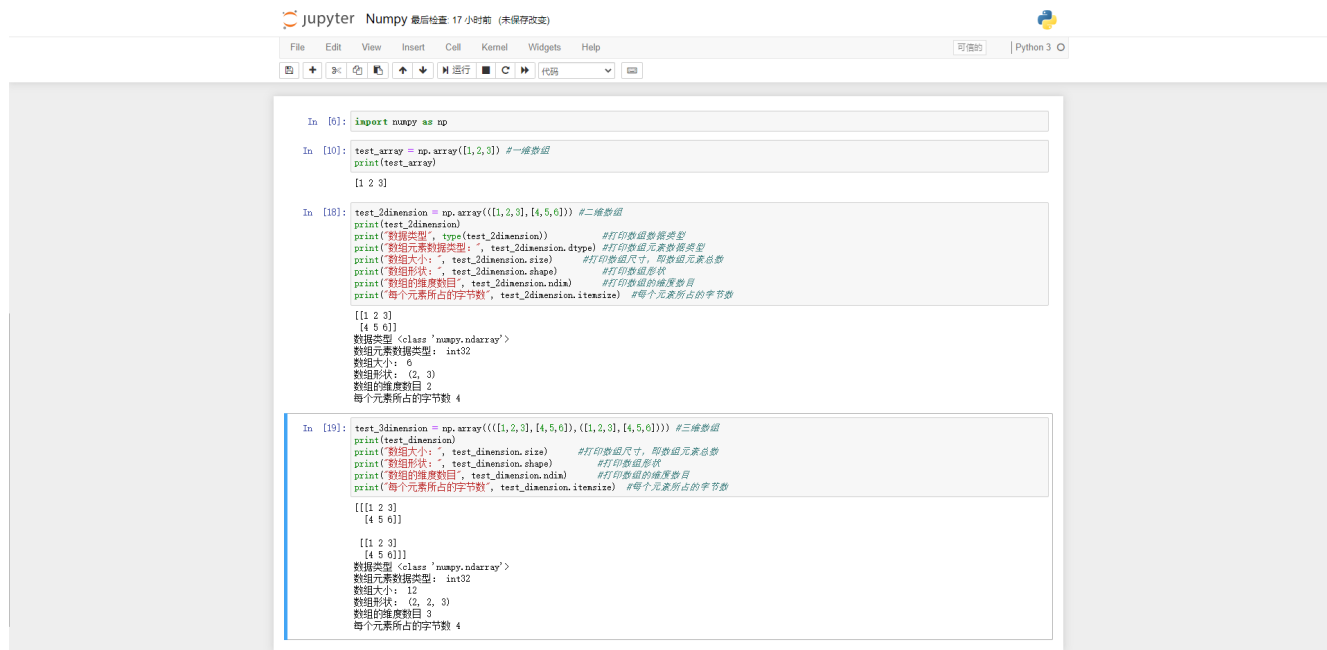
我们使用 apt 工具安装，我们此处安装的是 Python3 版本的 Numpy 库，若需要 Python2 环境的 NumPy 库，请自行将下列命令中的 3 删去。

```
# 在终端中输入如下命令，安装 numpy 库，等待安装完成后即可：  
sudo apt -y install python3-numpy
```

15.3 NumPy 库使用

安装好对应的库之后，我们就可以利用前面安装的 jupyter 来编写一下测试代码 (jupyter 安装参考前面 **Jupyter Notebook** 章节)。

测试代码及效果如下：



```
In [6]: import numpy as np

In [10]: test_array = np.array([1,2,3]) #一维数组
print(test_array)
[1 2 3]

In [18]: test_2dimension = np.array([[1,2,3],[4,5,6]]) #二维数组
print(test_2dimension)
[[1 2 3]
 [4 5 6]]
print("数据类型: ", type(test_2dimension)) #打印数组数据类型
print("数组元素数据类型: ", test_2dimension.dtype) #打印数组元素数据类型
print("数组大小: ", test_2dimension.size) #打印数组大小, 即数组元素总数
print("数组形状: ", test_2dimension.shape) #打印数组形状
print("数组的维度数目", test_2dimension.ndim) #打印数组的维度数目
print("每个元素所占的字节数", test_2dimension.itemsize) #每个元素所占的字节数

[[1 2 3]
 [4 5 6]]
数据类型: <class 'numpy.ndarray'>
数组元素数据类型: int32
数组大小: 6
数组形状: (2, 3)
数组的维度数目 2
每个元素所占的字节数 4

In [19]: test_3dimension = np.array([[[1,2,3],[4,5,6]], [[1,2,3],[4,5,6]]]) #三维数组
print(test_3dimension)
[[[1 2 3]
  [4 5 6]]
 [[1 2 3]
  [4 5 6]]]
print("数据类型: ", type(test_3dimension)) #打印数组数据类型
print("数组大小: ", test_3dimension.size) #打印数组大小, 即数组元素总数
print("数组形状: ", test_3dimension.shape) #打印数组形状
print("数组的维度数目", test_3dimension.ndim) #打印数组的维度数目
print("每个元素所占的字节数", test_3dimension.itemsize) #每个元素所占的字节数

[[[1 2 3]
  [4 5 6]]
 [[1 2 3]
  [4 5 6]]]
数据类型: <class 'numpy.ndarray'>
数组元素数据类型: int32
数组大小: 12
数组形状: (2, 2, 3)
数组的维度数目 3
每个元素所占的字节数 4
```

大家可以将配套代码目录 libdemo\Numpy\Numpy.ipynb 文件，上传到 jupyter 中使用。

更多关于 Python Numpy 库的使用，可参考 [NumPy 教程](#)

第 16 章 Pandas 库

16.1 Pandas 库简介

Pandas 是基于 NumPy 的一种工具，该工具常用于数据分析领域。Pandas 可以用于处理各种杂乱的数据或者表格数据，该库提供了许多操作大型数据集所需的工具。我们可以在鲁班猫板卡上安装 Python-Pandas 库，并通过 jupyter 编写一些测试代码来使用该库。

16.2 Pandas 库安装

我们使用 apt 工具安装。

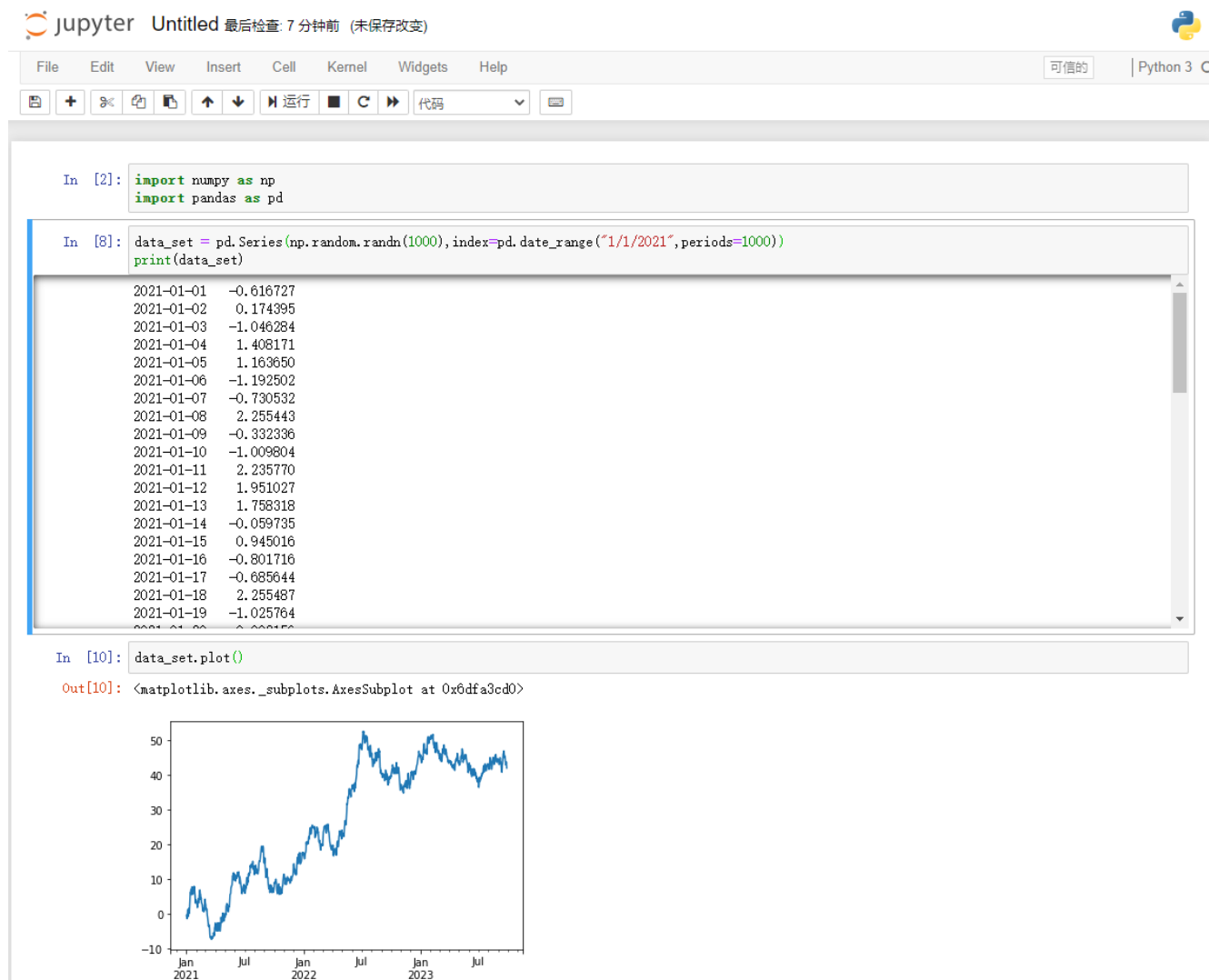
```
# 在终端中输入如下命令，安装 Pandas 库：  
sudo apt -y install python3-pandas
```

等等安装完成后即可。

16.3 Pandas 库使用

安装好对应的库之后，我们就可以利用前面安装的 jupyter 来编写一下测试代码。

测试代码及效果如下：



大家可以将配套代码目录 libdemo\Pandas\Pandas.ipynb 文件，上传到 jupyter 中使用。

更多关于 Python Numpy 库的使用，可参考 [Pandas 教程](#)

第 17 章 Web 库 - Flask

17.1 Flask 库简介

Flask 是使用 Python 构建 web 应用程序时最常见的 web 框架之一，和 Django 一样，不同的是 Flask 非常轻量级，构建基本 web 应用程序非常容易。

我们可以在鲁班猫板卡上安装 Python-flask 库，并通过编写一些测试代码来使用该库。在我们的板卡上部署一个简单的 web 页面。

17.2 Flask 库安装

我们使用 apt 工具安装。

```
# 在终端中输入如下命令，安装 flask 库：
sudo apt -y install python3-flask

# 或者

pip3 install flask
```

等安装完成后即可。

17.3 Flask 库使用

安装好对应的库之后，我们就可以利用该库部署一个简单 web 页面。

17.3.1 新建工程目录

先新建目录，方便工程管理，文件目录也比较简洁。

```
# 新建一个目录，用于存放我们的 web 项目，并进入目录
mkdir webapp_flask && cd webapp_flask

# 创建我们 webapp 的目录
mkdir -p app/static
mkdir app/templates
mkdir tmp
```

17.3.2 添加代码文件

17.3.2.1 __init__.py

为我们的 webapp_flask 包创建一个简单的初始化脚本，文件放置于刚才创建的 app 目录下：

```
# 在 app 目录下新建 __init__.py
vim __init__.py
```

内容如下：

列表 1: __init__.py

```
1 from flask import Flask
2
```

(下页继续)

(续上页)

```
3 app = Flask(__name__)
4 from app import view
```

17.3.2.2 views.py

在 Flask 中，视图被编写成 Python 函数。每一个视图函数映射到一个或多个请求的 URL。

让我们编写第一个视图函数 views.py，文件放置于创建的 app 目录下：

```
# 在 app 目录下新建 view.py
vim view.py
```

内容如下：

列表 2: view.py

```
1 from app import app
2
3 @app.route('/')
4 @app.route('/index')
5 def index():
6     return "Hello, Lubancat!"
```

17.3.2.3 run.py

最后一步是创建一个脚本，用于启动我们的应用程序的 Web 服务器。这个脚本为 run.py，并把它置于我们 webapp_flask 目录，即项目的顶层目录 webapp_flask：

```
# 在 webapp_flask 目录下新建 run.py
nano run.py
```

内容如下：

列表 3: run.py

```
1 #!/usr/bin/python3
2 from app import app
3 # 调试模式, 可通过主机地址"192.168.103.108" 访问
4 app.run(debug = True, host="192.168.103.108")
```

run.py 创建完成之后需要赋予一些权限:

```
# 添加权限
chmod a+x run.py
```

最终的文件目录组织如下:

```
cat@lubancat:~/webapp_flask$ tree ../webapp_flask/
../webapp_flask/
|-- app
|   |-- __init__.py
|   |-- static
|   |-- templates
|   `-- view.py
|-- run.py
`-- tmp

4 directories, 3 files
```

17.3.3 运行 webapp

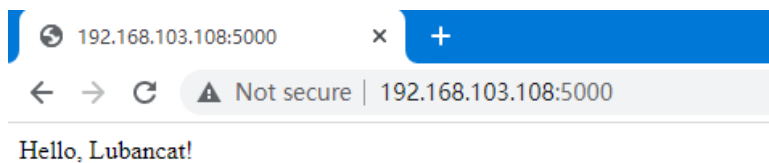
经过上述的操作, 我们的一个最简单的 webapp 已经组织完成了。下面我们可以启动 app, 通过浏览器来查看效果。

```
# 运行 app
./run.py
```

测试代码及效果如下:

```
cat@lubancat:~/webapp_flask$ ./run.py
* Serving Flask app "app" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
* Running on http://192.168.103.108:5000/ (Press CTRL+C to quit)
* Restarting with stat
* Debugger is active!
* Debugger PIN: 347-097-024
```

app 已经正常启动，终端也已打印出了服务信息。信息中提示 web 已经映射到了我们在前面代码中设置的主机 192.168.103.108 端口号为 5000，故我们可以通过此网址访问 <http://192.168.103.108:5000/> 我们部署好的网页。



大家可以将配套代码 libdemo\Flask\webapp_flask 目录，上传到鲁班猫板卡中进行测试。

17.3.4 参考

[Flask 官方文档](#)

第 18 章 Web 库 - Django

18.1 Django 库简介

Django 是 Python 编程语言驱动的一个开源模型-视图-控制器（MVC）风格的 Web 应用程序框架，也是 Web 开发的中主流的框架。我们可以在鲁班猫板卡上安装 Python-Django 库，并通过一些简单的操作来使用该库。在我们的板卡上部署一个简单的 web 页面。

18.2 Django 库安装

我们使用 apt 工具安装。

```
# 在终端中输入如下命令，安装 Django 库：  
sudo apt -y install python3-django
```

等等安装完成后即可。

18.3 Django 库使用

安装好对应的库之后，我们就可以利用该库部署一个简单 web 页面。

18.3.1 新建工程目录

先新建目录，方便工程管理，文件目录也比较简洁。

```
# 新建一个目录，用于存放我们的 web 项目，并进入目录
mkdir webapp
cd webapp
```

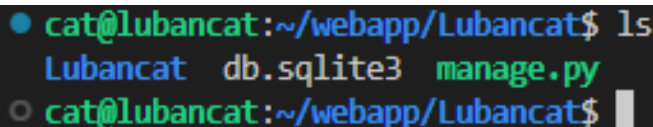
18.3.2 添加代码文件及配置

使用 Django 自带的管理器 `django-admin`，我们可以快速新建一个工程模板。

```
# 在终端中输入如下命令：
django-admin startproject Lubancat

# 待指令执行完毕后，当前目录下会自动生成项目文件夹：Lubancat，进入文件夹
cd ./Lubancat/
```

Django 自带的管理器已经为我们生成好了工程模板，在目录中可以看到如下内容：



```
cat@lubancat:~/webapp/Lubancat$ ls
Lubancat  db.sqlite3  manage.py
cat@lubancat:~/webapp/Lubancat$
```

接下来我们需要修改一下项目中的配置文件，以允许外部主机访问我们的鲁班猫板卡。进入到项目文件夹 `Lubancat` 中的 `Lubancat` 目录中，并修改 `settings.py` 文件。

```
# 在终端中输入如下命令：
cd ./Lubancat/

# 修改配置文件
vim settings.py
# 找到 ALLOWED_HOSTS 配置，并修改为如下内容，保存退出：
ALLOWED_HOSTS = ['*']
```

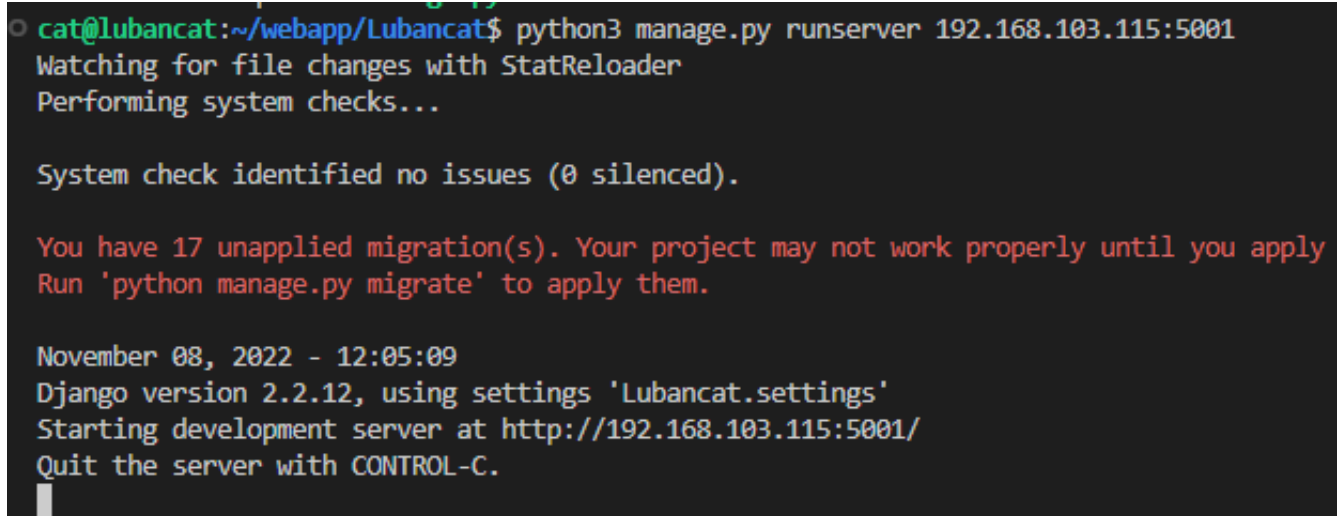
18.3.3 启动 webapp

回到项目文件夹 Lubancat 中，运行其目录下的 manage.py 文件，启动服务器：

```
# 在终端中输入如下命令, IP 地址根据板卡具体网口 ip:  
sudo python3 manage.py runserver 192.168.103.115:5001
```

稍等一会，就可以发现服务器启动信息已经正常输出啦 ~

效果如下：



```
cat@lubancat:~/webapp/Lubancat$ python3 manage.py runserver 192.168.103.115:5001  
Watching for file changes with StatReloader  
Performing system checks...  
  
System check identified no issues (0 silenced).  
  
You have 17 unapplied migration(s). Your project may not work properly until you apply  
Run 'python manage.py migrate' to apply them.  
  
November 08, 2022 - 12:05:09  
Django version 2.2.12, using settings 'Lubancat.settings'  
Starting development server at http://192.168.103.115:5001/  
Quit the server with CONTROL-C.
```

webapp 已经正常启动，终端也已打印出了服务信息。信息中提示 web 已经映射到了我们在前面代码中设置的主机 192.168.103.115 端口号为 5001，故我们可以通过此网址访问 <http://192.168.103.115:5001/> 我们部署好的网页。



The install worked successfully! Congratulations!

You are seeing this page because `DEBUG=True` is in your settings file and you have not configured any URLs.

18.3.4 Django 版本注意事项

注：鲁班猫板卡系统中使用 apt 方法安装的 Django 为 1.11.29 版本，如需安装特定版本的 Django，可使用 pip 安装，参考如下：

```
# 在终端中输入如下命令，安装指定版本 Django：
pip3 install Django==3.1.7
```

安装完成后，再根据提示，将管理工具目录放在环境变量中，或将管理工具放在环境变量中即可。

```
The script django-admin is installed in '/home/debian/.local/bin' which is not on
PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning
, use --no-warn-script-location.
```

```
# 在终端中输入如下命令，将管理工具放在环境变量中：
sudo cp /home/cat/.local/bin/django-admin /usr/bin/
```

更多关于 Python Django 库的使用，可参考 [Django 官方文档](#)

第 19 章 Modbus 协议-pymodbus 库

在前面，我们介绍了鲁班猫板卡上的各种对外连接的接口，大家可以发现，其中的一些接口在工业领域应用的是十分广泛的。那么我们鲁班猫板卡，能否在这些接口的基础上，再拓展出一些强大的功能呢？

本节实验中，我们就来给大家介绍一下在工业领域中，十分常见工业总线协议：Modbus 通讯协议，示例是 Modbus-RTU。

19.1 Modbus 协议实验

我们的鲁班猫板卡系统有丰富的开源软件支持，像我们本节要介绍的 Modbus 通讯协议，也有对应的软件可以供我们安装使用。

本节实验，我们的目标是使用 Python 语言及 Python 库-python3-pymodbus，来完成 Modbus 协议中约定的 RTU 主站，和从站进行简单通讯。

那么我们来简单了解一下 python3-pymodbus 库。

19.2 python3-pymodbus 库

pymodbus 是基于 BSD 开源协议的一个的 Modbus 协议 Python 库。它的功能十分强大，实现了 Modbus 协议中约定的所有功能，并且对通讯主机以同步及异步（asyncio、tornado、twisted）的方式进行了实现，在拥有不错的性能的同时，也为 Python 开发者在构建 Modbus 协议应用时，对应用功能进行额外拓展提供了更多可能。

pymodbus 支持以太网、串行接口来承载 Modbus 协议的物理层传输，如果你不使用串行接口（pymodbus 串行接口的实现依赖于 pyserial 库包）。那么 python3-pymodbus 库完全不依赖与第三方库包，所以该库是十分轻量的。

我们可以在鲁班猫板卡上安装 python3-pymodbus 库，并通过一些官方提供的示例代码来使用该库，完成本节的实验。

19.3 实验准备

19.3.1 添加板卡 uart 资源

重要：本节实验使用一个 uart 串口（从机）和 PC 端（主机）来适配 Modbus 底层硬件的连接，故此小节为添加 uart 资源演示，pymodbus 库提供 Modbus 协议的全栈实现，所以底层使用网口通讯也是得到支持的，大家可以根据实际情况参考本小节内容与否。

在板卡上的部分 GPIO 可能没有启用，可注释掉某些设备树节点或者设备树插件的加载，重启系统来开启，LubanCat_RK 系列板卡的引脚参考下：《[LubanCat-RK 系列-40pin 引脚对照图](#)》

本节实验中，笔者使用鲁班猫 LubanCat 2 板卡为例演示，根据引出的 40 个引脚对照图，串口资源可以使用 UART3，下面我们开启下 UART3 的设备树插件：

```
# 不同板卡的配置文件和 pwm 不同，以 LubanCat 2 为例，配置文件为 uEnvLubanCat 2.txt，  
# 使用下面命令：  
sudo vim /boot/uEnv/uEnvLubanCat 2.txt  
# 进入编辑模式，取消 pwm 前面的注释，保存并退出文件，重启系统
```

```
root@lubancat:/boot/uEnv# sudo vim uEnvLubanCat2.txt
uname_r=4.19.232
size=0x1000000
#dtb=rk3568-lubancat2.dtb

enable_uboot_overlays=1
#overlay_start

#dtoverlay=/dtb/overlay/lubancat-i2c3-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-i2c5-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm8-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm9-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm10-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-pwm14-m0-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-spi3-m1-gpio-cs-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-spi3-m1-overlay.dtbo
#dtoverlay=/dtb/overlay/lubancat-uart3-m1-overlay.dtbo
#overlay_end

~
~
~
~

"uEnvLubanCat2.txt" 19L, 610C                               16,1          All
```

取消#注释，保存重启，开启串口

如果使用的是其他鲁班猫 RK 系列板卡，也是类似操作。

```
# 在终端中输入如下命令，可以查看到 UART 资源：
ls /dev/ttyS*
```

板卡上的 UART 资源示例，仅供参考：

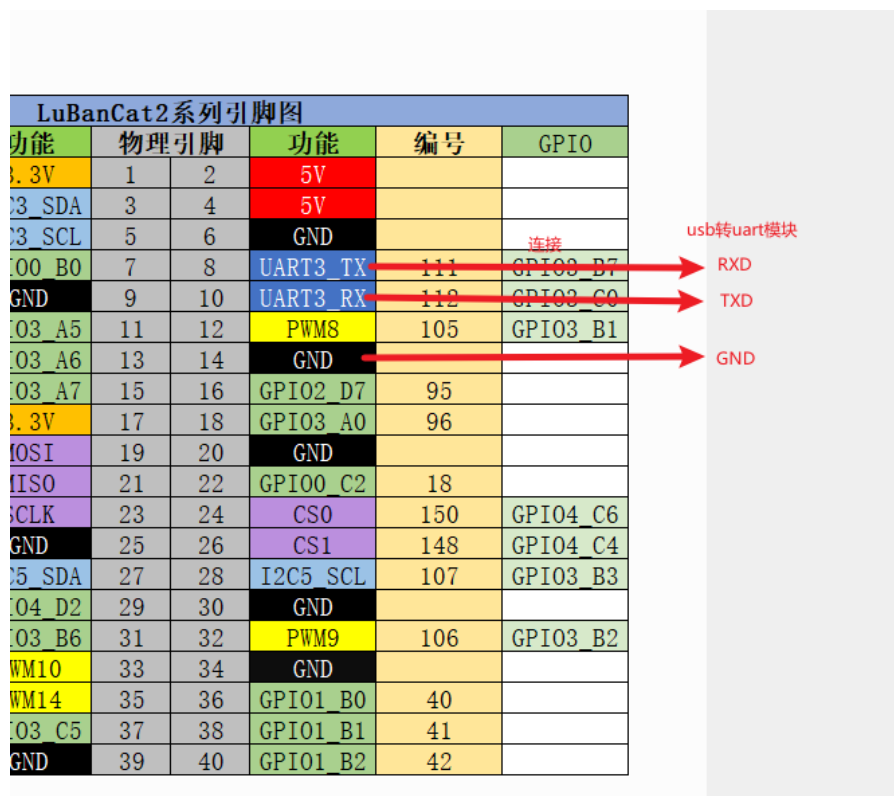
```
cat@lubancat:~$ ls /dev/ttyS*
/dev/ttyS3
cat@lubancat:~$
```

其中/dev/ttyS3 对应的就是板卡上的 uart3 资源。

提示：如出现 Permission denied 或类似字样，请注意用户权限，大部分操作硬件外设的功能，几乎都需要 root 用户权限，简单的解决方案是在执行语句前加入 sudo 或以 root 用户运行程序。

19.3.2 硬件连接

硬件连接小节，需要大家根据自身开发环境等情况对本章内容进行参考调整，本章会使用 Luban-Cat 2 中的一个串口：uart3 完成数据传输的演示。连接如下（和 **UART 通讯** 章节连接相同）：



通过 USB 转 TTL 连接到电脑。

重要： pymodbus 库对串行设备的支持，是基于 pyserial 库实现的，该库默认的串行设备支持，是基于 uart，故本节也以 uart 进行演示实验。如想 pymodbus 使用 rs485 底层电气连接的方式，需要自行修改 pymodbus 库底层源码。

19.3.3 pymodbus 库安装

在终端中输入如下命令:

```
sudo pip3 install -U pymodbus
```

或者直接拉取仓库源码, 使用 `setuptools` 工具安装等, 参考前面安装 `python` 章节

```
git clone https://github.com/riptideio/pymodbus.git
```

19.4 测试代码

安装 `pymodbus` 库之后, 就可以使用 `pymodbus` 库编程, 这里我们直接修改官方提供的 `examples`, 然后进行简单测试。此 Demo 使用了 `pymodbus` 基于 `asyncio` 异步库实现的 Modbus 协议 RTU 通讯、串口异步主机, 代码里我们设置对外的通讯端口, 是在硬件配置小节中开启的串行端口 `'/dev/ttyS3'`, 波特率 115200,。

19.4.1 RTU 从机 `slave_server.py`

代码如下:

列表 1: 配套代码 `slave_server.py` (主体部分)

```
1 def get_commandline(server=False, description=None, extras=None):
2     """Read and validate command line arguments"""
3     parser = argparse.ArgumentParser(description=description)
4     parser.add_argument(
5         "--comm",
6         choices=["serial"],
7         help="set communication",
8         default="serial",
9         type=str,
```

(下页继续)

(续上页)

```
10     )
11     parser.add_argument (
12         "--framer",
13         choices=["rtu"],
14         help="set framer, default depends on --comm",
15         type=str,
16     )
17     parser.add_argument (
18         "--log",
19         choices=["critical", "error", "warning", "info", "debug"],
20         help="set log level, default is info",
21         default="info",
22         type=str,
23     )
24     parser.add_argument (
25         "--port",
26         help="set port",
27         type=str,
28     )
29     if server:
30         parser.add_argument (
31             "--store",
32             choices=["sequential"],
33             help="set type of datastore",
34             default="sequential",
35             type=str,
36         )
37         parser.add_argument (
38             "--slaves",
39             help="set number of slaves, default is 0 (any)",
40             default=0,
41             type=int,
```

(下页继续)

(续上页)

```
42         nargs="+",
43     )
44     if extras:
45         for extra in extras:
46             parser.add_argument(extra[0], **extra[1])
47     args = parser.parse_args()
48
49     # set defaults
50     comm_defaults = {
51         "serial": ["rtu", "/dev/ptyp0"],
52     }
53     framers = {
54         "rtu": ModbusRtuFramer,
55     }
56     pymodbus_apply_logging_config()
57     _logger.setLevel(args.log.upper())
58     args.framer = framers[args.framer or comm_defaults[args.comm][0]]
59     args.port = args.port or comm_defaults[args.comm][1]
60     if args.comm != "serial" and args.port:
61         args.port = int(args.port)
62     return args
63
64 def setup_server(args):
65     """Run server setup."""
66     # The datastores only respond to the addresses that are initialized
67     # If you initialize a DataBlock to addresses of 0x00 to 0xFF, a request_
68     ↪to
69     # 0x100 will respond with an invalid address exception.
70     # This is because many devices exhibit this kind of behavior (but not_
71     ↪all)
72     _logger.info("### Create datastore")
73     # 定义 co 线圈寄存器, 存储起始地址为 0, 长度为 10, 内容为 5 个 True 及 5 个 False
```

(下页继续)

(续上页)

```
72     co_block = ModbusSequentialDataBlock(0, [True]*5 + [False]*5)
73     # 定义 di 离散输入寄存器, 存储起始地址为 0, 长度为 10, 内容为 5 个 True 及 5 个
False
74     di_block = ModbusSequentialDataBlock(0, [True]*5 + [False]*5)
75     # 定义 ir 输入寄存器, 存储起始地址为 0, 长度为 10, 内容为 0~10 递增数值列表
76     ir_block = ModbusSequentialDataBlock(0, [0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
77     # 定义 hr 保持寄存器, 存储起始地址为 0, 长度为 10, 内容为 0~10 递减数值列表
78     hr_block = ModbusSequentialDataBlock(0, [9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
79
80     if args.slaves:
81         # The server then makes use of a server context that allows the
↪server
82         # to respond with different slave contexts for different unit ids.
83         # By default it will return the same context for every unit id
↪supplied
84         # (broadcast mode).
85         # However, this can be overloaded by setting the single flag to
↪False and
86         # then supplying a dictionary of unit id to context mapping::
87         #
88         # The slave context can also be initialized in zero_mode which means
89         # that a request to address(0-7) will map to the address (0-7).
90         # The default is False which is based on section 4.4 of the
91         # specification, so address(0-7) will map to (1-8)::
92         context = {
93             0x01: ModbusSlaveContext(
94                 di=di_block,
95                 co=co_block,
96                 hr=hr_block,
97                 ir=ir_block,
98                 zero_mode=True
99             ),
```

(下页继续)

(续上页)

```
100         0x02: ModbusSlaveContext (
101             di=di_block,
102             co=co_block,
103             hr=hr_block,
104             ir=ir_block,
105         ),
106         0x03: ModbusSlaveContext (
107             di=di_block,
108             co=co_block,
109             hr=hr_block,
110             ir=ir_block,
111         ),
112     }
113     single = False
114 else:
115     context = ModbusSlaveContext (
116         di=di_block, co=co_block, hr=hr_block, ir=ir_block, unit=1
117     )
118     single = True
119
120     # Build data storage
121     args.context = ModbusServerContext(slaves=context, single=single)
122     return args
123
124
125 async def run_async_server(args):
126     """Run server."""
127     txt = f"### start ASYNC server, listening on {args.port} - {args.comm}"
128     _logger.info(txt)
129     # socat -d -d PTY,link=/tmp/ptyp0,raw,echo=0,ispd=9600
130     #             PTY,link=/tmp/ttyp0,raw,echo=0,ospd=9600
131     server = await StartAsyncSerialServer(
```

(下页继续)

(续上页)

```
132     context=args.context, # Data storage
133     #identity=args.identity, # server identify
134     timeout=1, # waiting time for request to complete
135     port=args.port, # serial port
136     # custom_functions=[], # allow custom handling
137     framer=args.framer, # The framer strategy to use
138     # handler=None, # handler for each session
139     stopbits=1, # The number of stop bits to use
140     bytesize=8, # The bytesize of the serial messages
141     # parity="N", # Which kind of parity to use
142     baudrate=115200, # The baud rate to use for the serial device
143     # handle_local_echo=False, # Handle local echo of the USB-to-RS485_
↪ adaptor
144     # ignore_missing_slaves=True, # ignore request to a missing slave
145     # broadcast_enable=False, # treat unit_id 0 as broadcast address,
146     # strict=True, # use strict timing, t1.5 for Modbus RTU
147     # defer_start=False, # Only define server do not activate
148 )
149 return server
150
151 if __name__ == "__main__":
152     cmd_args = get_commandline(
153         server=True,
154         description="Run asynchronous server.",
155     )
156     run_args = setup_server(cmd_args)
157     asyncio.run(run_async_server(run_args), debug=True)
```

使用 pymodbus 模拟 Modbus 从设备的功能，需要构造从机上下文环境，该环境由 Modbus 协议规定，需要实现 Modbus 从机的各项功能，比如各种寄存器。最终该上下文由 Server 负责调度运作，实现主机对从机数据的存入读出维护，pymodbus 还在此基础上实现了数据库的接口，感兴趣的同学可以查看 pymodbus 源码及示例。

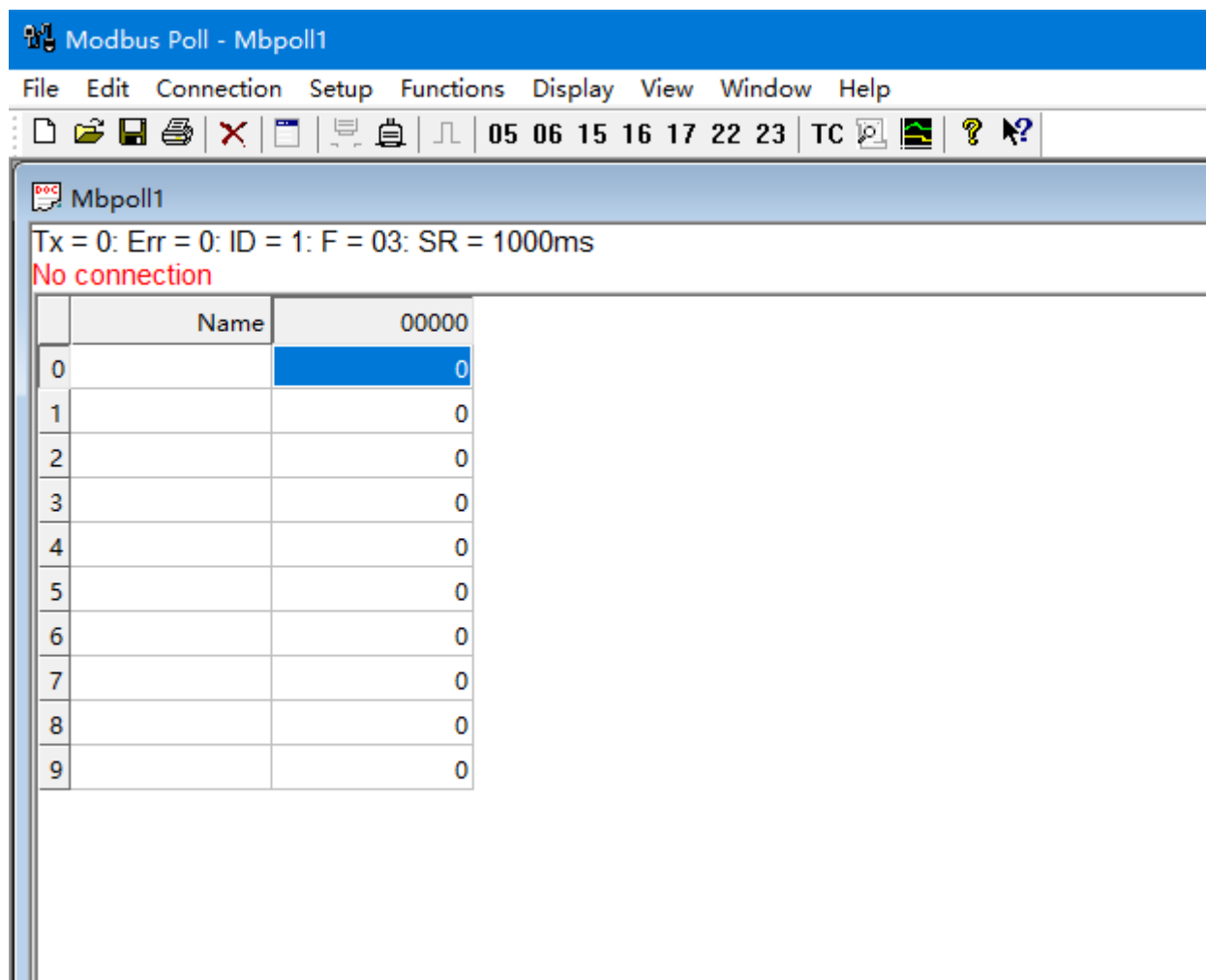
19.4.2 实验步骤

1、从机端，将配套的测试代码上传至 LubanCat 2 板卡系统中，登录系统终端，启动从机服务：

```
# 在终端中输入如下命令：
sudo python slave_server.py --comm serial --framer rtu --port /dev/ttyS3 --
↪store sequential --log debug --slaves 1
#--comm 指定 serial 通讯方式,,, --framer, 指定使用 rtu 帧,,, --port 指定串口,,, --log
指定 debug 信息
# 可以使用命令 sudo python slave_server.py -h 查看帮助

# 使用命令后在终端可以看到：
cat@lubancat:~$ sudo python server0.py --comm serial --framer rtu --port /
↪dev/ttyS3 --store sequential --log debug --slaves 1
15:10:47 INFO server0:123 ### Create datastore
15:10:47 DEBUG selector_events:59 Using selector: EpollSelector
15:10:47 INFO server0:181 ### start ASYNC server, listening on /dev/ttyS3 -
↪ serial
15:10:47 DEBUG async_io:130 Serial connection opened on port: /dev/ttyS3
15:10:47 DEBUG async_io:442 Serial connection established
```

2、主机端，按前面的硬件连接，电脑端使用 Modbus Poll 软件（试用 30 天），下载地址 <https://modbustools.com/download.html> 安装后，打开该软件：

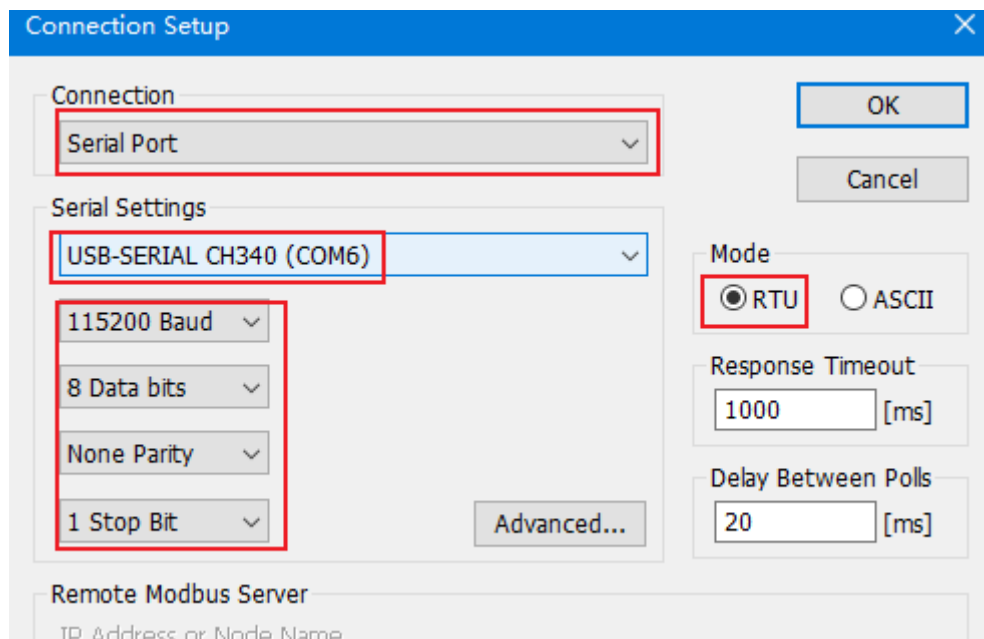


在空白处右击，选择“Read/write Definition”，然后设置从机地址，读写等。这里我们设置地址为1，从地址0开始读取10个线圈：

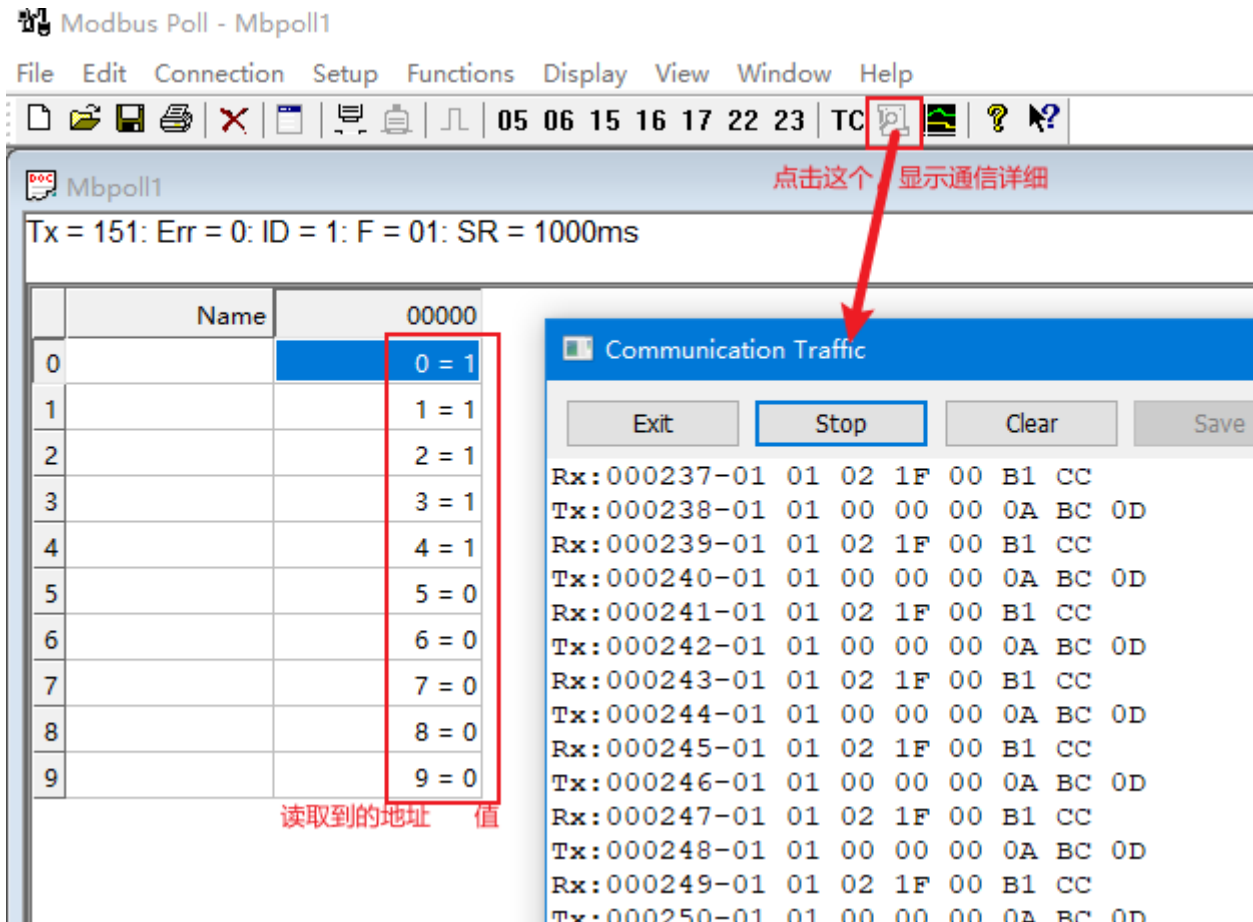
The image shows a 'Read/Write Definition' dialog box with the following fields and annotations:

- Slave ID:** 1 (Annotated: 从机地址)
- Function:** 01 Read Coils (0x) (Annotated: 功能码, 读取线圈)
- Address mode:** Dec (selected), Hex (Annotated: 读取起始地址)
- Address:** 0 (Annotated: 读取的数量)
- Quantity:** 10 (Annotated: 帧的扫描周期)
- Scan Rate:** 1000 [ms] (Annotated: 表格中显示地址)
- Disable:** Read/Write Disabled, Disable on error (Annotated: Read/Write Once)
- View:** Rows: 10 (selected), 20, 50, 100, Fit to Quantity (Annotated: Hide Name Columns, PLC Addresses (Base 1), Enron/Daniel Mode)

设置完成后，点击主窗口的“connection”，之后选择串口，这里是 COM6，具体设置如下：



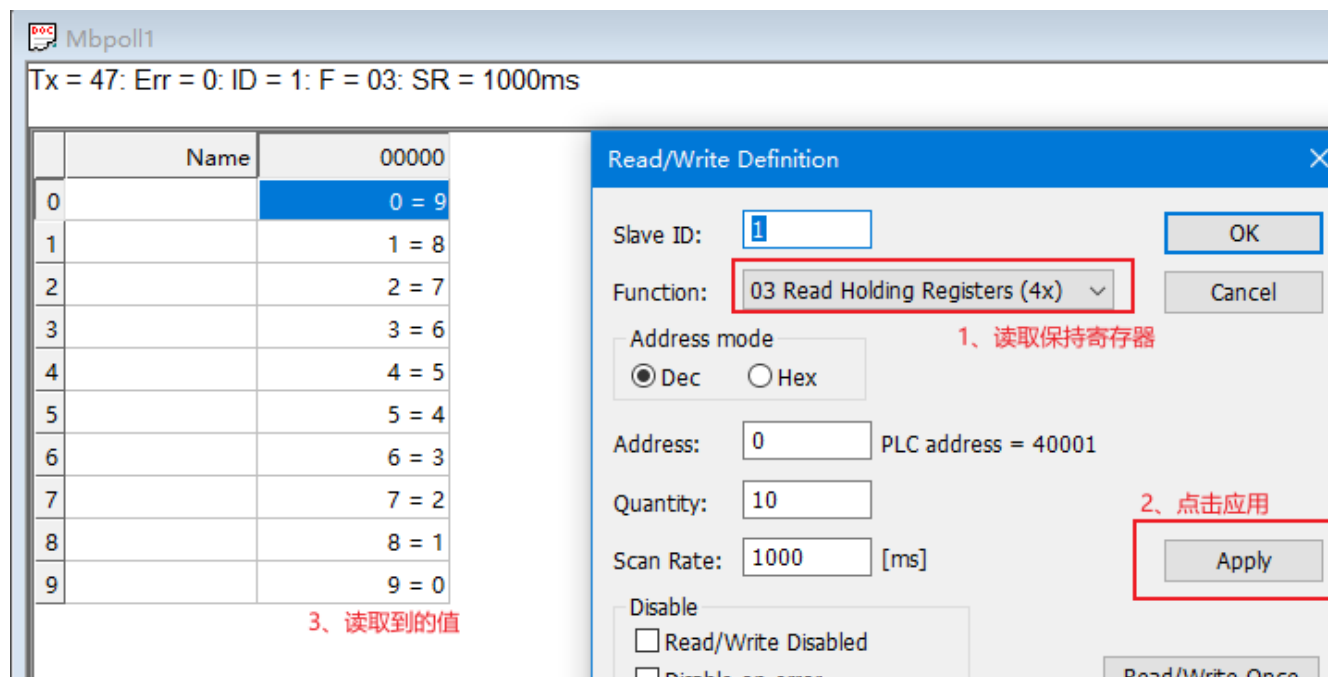
连接成功后就可以看到读取的值：



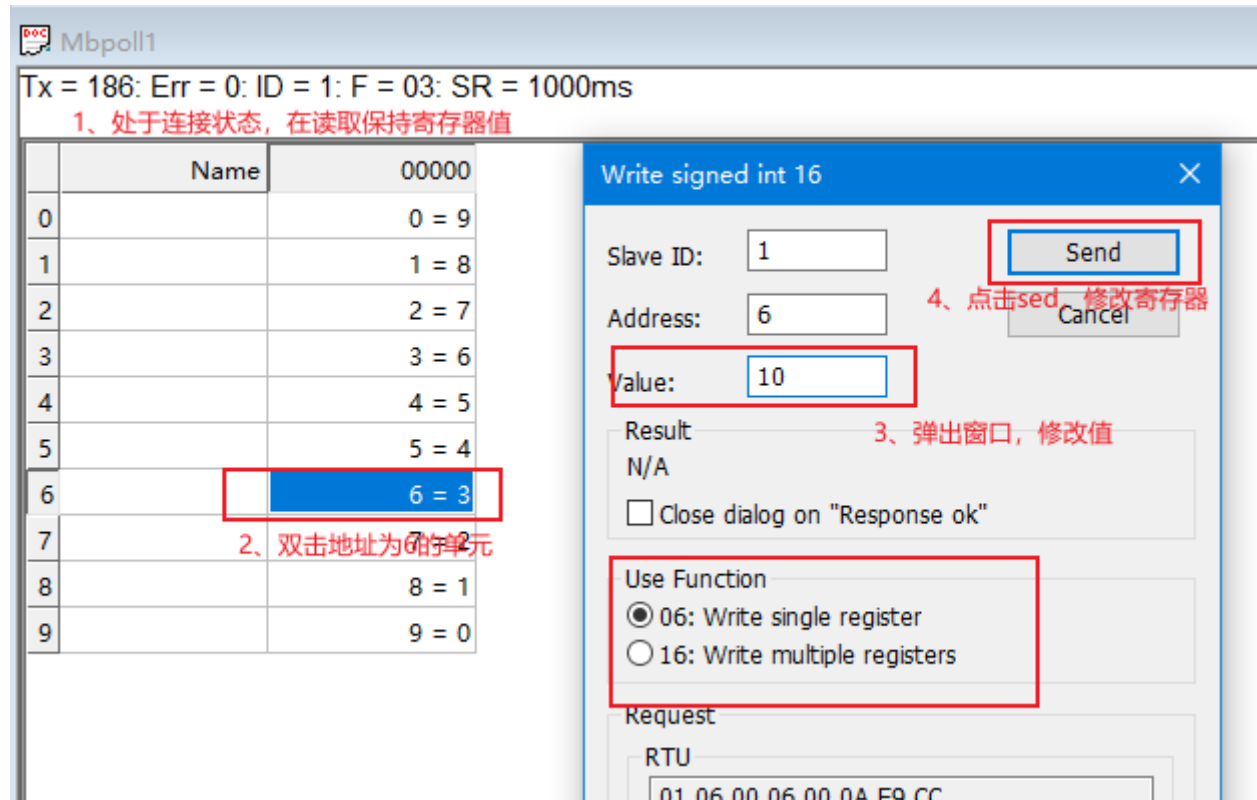
与此同时，板卡终端打印出的信息如下（执行时指定 -log debug）：

```
cat@lubancat:~$ sudo python server0.py --comm serial --framer rtu --port /dev/ttyS3 --store sequential --log debug --slaves 1
15:10:47 INFO server0:123 ### Create datastore
15:10:47 DEBUG selector_events:59 Using selector: EpollSelector
15:10:47 INFO server0:181 ### start ASYNC server, listening on /dev/ttyS3 - serial
15:10:47 DEBUG async_io:130 Serial connection opened on port: /dev/ttyS3
15:10:47 DEBUG async_io:442 Serial connection established
15:19:10 DEBUG async_io:222 Handling data: 0x1 0x1 0x0 0x0 0xa 0xbc 0xd
15:19:10 DEBUG rtu_framer:193 Getting Frame - 0x1 0x0 0x0 0xa
15:19:10 DEBUG factory:216 Factory Request[ReadCoilsRequest: 1]
15:19:10 DEBUG rtu_framer:116 Frame advanced, resetting header!!
15:19:10 DEBUG context:66 validate: fc-[1] address-0: count-10
15:19:10 DEBUG context:80 getValues: fc-[1] address-0: count-10
15:19:10 DEBUG async_io:299 send: [ReadCoilsResponse(10)]- b'0101021f00b1cc'
15:19:10 DEBUG rtu_framer:252 Frame - [b'\x01\x01\x00\x00\x00\n\xbc\r'] not ready
15:19:11 DEBUG async_io:222 Handling data: 0x1 0x1 0x0 0x0 0xa 0xbc 0xd
15:19:11 DEBUG rtu_framer:193 Getting Frame - 0x1 0x0 0x0 0xa
15:19:11 DEBUG factory:216 Factory Request[ReadCoilsRequest: 1]
```

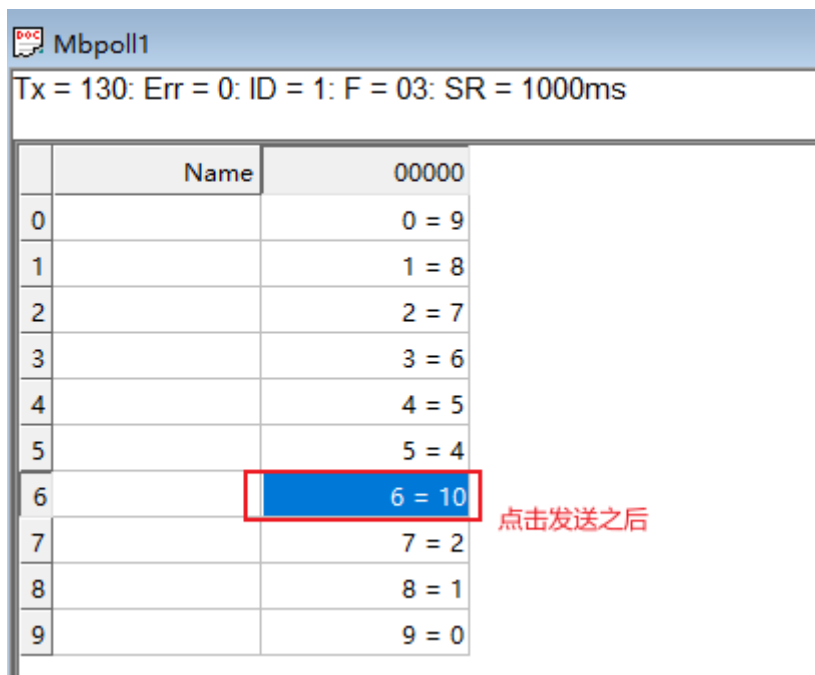
上面处于连接状态后，我们还可以进行其他寄存器的读取和写入等操作，右击空白进入“Read/write Definition”，重新设置下值：



修改保存寄存器的值:



点击发送之后，读取到的寄存器的值：



	Name	00000
0		0 = 9
1		1 = 8
2		2 = 7
3		3 = 6
4		4 = 5
5		5 = 4
6		6 = 10
7		7 = 2
8		8 = 1
9		9 = 0

更详细的一些寄存器读写调试等操作，可以参考下 Modbus Poll 软件使用帮助。在上面测试过程中，可以看到，主机对从机发送的命令、接收到从机的响应信息及各项测试结果等等。

19.4.3 现象简析

我们可以通过一张图来简单看看，Modbus-RTU 主从机在通讯过程中发生的一些请求与响应。

以 读取多个线圈测试为例（单击图片可放大）：

The screenshot displays a terminal window on the left and the Modbus Poll software interface on the right. The terminal shows the execution of a Python script named `server0.py` using the `pymodbus` library. The script defines a `setup_server` function that initializes a Modbus server with various data blocks. The terminal output includes debug messages for creating the data store, starting the server, and handling a connection. The Modbus Poll software, titled 'Modbus Poll - Mbpoll1', shows a table of registers with addresses 0 to 9. The 'Name' column is empty, and the 'Value' column shows values like 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. A 'Communication Traffic' window is open, showing a list of received and transmitted messages in hexadecimal and ASCII format. The messages are prefixed with 'Rx:' and 'Tx:' and include address, function code, and data fields.

```
117 def setup_server(args):
118     """Run server setup."""
119     # The datastores only respond to the addresses that are initialized
120     # If you initialize a DataBlock to addresses of 0x00 to 0xFF, a request to
121     # 0x100 will respond with an invalid address exception.
122     # This is because many devices exhibit this kind of behavior (but not all)
123     logger.info("### Create datastore")
124     # 定义线圈寄存器, 存储起始地址为0, 长度为10, 内容为5个True及5个False
125     co_block = ModbusSequentialDataBlock(0, [True]*5 + [False]*5)
126     # 定义离散输入寄存器, 存储起始地址为0, 长度为10, 内容为5个True及5个False
127     di_block = ModbusSequentialDataBlock(0, [True]*5 + [False]*5)
128     # 定义ir寄存器, 存储起始地址为0, 长度为10, 内容为0~10递增数值列表
129     ir_block = ModbusSequentialDataBlock(0, [0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
130     # 定义hr寄存器, 存储起始地址为0, 长度为10, 内容为0~10递减数值列表
131     hr_block = ModbusSequentialDataBlock(0, [9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
132
133
KeyboardInterrupt
cat@lubuntu:~$ sudo python server0.py --comm serial --framer rtu --port /dev/ttyS3 --store seq
15:27:14 INFO server0:123 ### Create datastore
15:27:14 DEBUG selector_events:59 Using selector: EpollSelector
15:27:14 INFO server0:181 ### start ASYNC server, listening on /dev/ttyS3 - serial
15:27:14 DEBUG async_io:130 Serial connection opened on port: /dev/ttyS3
15:27:14 DEBUG async_io:442 Serial connection established
15:27:16 DEBUG async_io:222 Handling data: 0x1 0x1 0x0 0x0 0x0 0xa 0xbc 0xd
15:27:16 DEBUG rtu_framer:193 Getting Frame - 0x1 0x0 0x0 0x0 0xa
15:27:16 DEBUG factory:216 Factory Request[ReadCoilsRequest': 1]
15:27:16 DEBUG rtu_framer:116 Frame advanced, resetting header!!
15:27:16 DEBUG context:66 validate: fc-[1] address-0: count-10
15:27:16 DEBUG context:80 getValues: fc-[1] address-0: count-10
15:27:16 DEBUG async_io:299 send: [ReadCoilsResponse(10)] - b'\x01\x02\x1f\x00\x0a\xbc\x0d'
15:27:16 DEBUG rtu_framer:252 Frame - [b'\x01\x01\x00\x00\x00\x0a\xbc\x0d'] not ready
15:27:17 DEBUG async_io:222 Handling data: 0x1 0x1 0x0 0x0 0x0 0xa 0xbc 0xd
```

图中, 在电脑上上位机端, 使用 modbus Poll 读写单个或者多个寄存器及对应参数, 当板卡从机收到命令后, 会做出对应的响应, 并将响应的 log 信息, 开启 debug 后可以看到。

pymodbus 库中还支持用户自定义的请求及响应, 即 Modbus 协议中约定的用户定义功能码相关内容。此外 pymodbus 库还是实现了一些服务器功能, 比如数据持久化, 可以让用户将从机寄存器内容通过一些常见的数据库进行存储。

在前面, 我们提到了, pymodbus 库是使用 Python 语言编写的 Modbus 协议的全栈实现, 在我们给大家提供的示例代码中, 只展示了该库的一部分功能。要测试更多, 可以关注 pymodbus 的 Github 仓库 [pymodbus](https://github.com/pymodbus-dev/pymodbus), 学习更多相关的内容。

19.4.4 参考

官方示例代码, [pymodbus-examples](https://github.com/pymodbus-dev/pymodbus-examples),

官方包和文档说明, [pymodbus-doc](https://github.com/pymodbus-dev/pymodbus-doc)。

第 20 章 GUI 库 - PyQt5

本章将简单介绍下 PyQt5，首先使用 PyQt 创建简单桌面应用程序，然后在鲁班猫板卡上使用 Qt Creator，进行简单 ui 设计，下面的简单示例用于学习参考，更深入学习看下参考资料。

- 平台：鲁班猫 RK 系列板卡
- 系统：Debian10（带桌面）
- Python 版本：Python3.7
- Qt 版本：系统默认 5.11.3
- PyQt 版本：PyQt5

20.1 PyQt5 库简介

PyQt 是 Qt 应用框架与 Python 的结合，同时支持 Python2.x 和 Python3.x 版本，本教程使用的是 Python3.7 版本，PyQt5 版本。

PyQt5 是由一系列 Python 模块组成，超过 620 个类，6000 函数和方法。能在诸如 Unix、Windows 和 Mac OS 等主流操作系统上运行，而且由于 Python 语言的开发方便性，在编写代码时比 C++ 更为方便，美中不足的是 Python 语言的执行效率会比 C++ 慢。

PyQt5 提供 GPL 版和商业版证书，自由开发者可以使用免费的 GPL 许可，如果需要将 PyQt 用于商业应用，则必须购买商业许可。

20.2 PyQt5 库安装

20.2.1 Qt 库安装

我们使用 apt 工具安装 PyQt5（或者使用 pip 安装）：

```
# 通过 apt 工具安装
sudo apt-get -y install python3-pyqt5

# 通过 pip 安装
pip3 install pyqt5

# qt 库安装
sudo apt-get install qt5-default
```

安装成功后：

```
cat@lubancat:~$ python3
Python 3.7.3 (default, Oct 31 2022, 14:04:00)
[GCC 8.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import PyQt5
>>>
```

接下来我们测试一下 PyQt5 在鲁班猫板卡上的运行。

重要： 在桌面系统中使用，实验前请确保连接桌面屏幕，HDMI 或者 mipi 屏幕都行。

20.3 PyQt5 库基础使用

我们将通过一个名字为 `pyqt5_demo.py` 的 PyQt5 应用程序，简单介绍下 pyqt5 桌面应用程序的编写。该应用程序主要是用于测试显示、触摸、信号、槽函数、以及 PyQt5 的部分组件如：QWidget、QLCDNumber、QSlider、QVBoxLayout 等的简单使用。

20.3.1 导入需要模块

```
# 导入 sys 模块，包含一些外部向程序传递参数，退出程序，获取系统平台，系统路径等操作
import sys
# 导入 QtWidgets 模块的一些子模块，QWidget 类（用户界面基类），QLCDNumber（展示 LCD
样式的数字），QSlider（滑动条控件）
# QVBoxLayout（控件垂直布局等）QApplication（管理 GUI 应用程序的控制流和主设置等）
from PyQt5.QtWidgets import QWidget,QLCDNumber,QSlider,QVBoxLayout,
    QApplication
# 导入 PyQt5.QtCore.Qt，一些基本的函数和类
from PyQt5.QtCore import Qt

# 如果你嫌麻烦，可以替换成 ``*``，占用点内存全部导入，例如：
from PyQt5.Qt import *
```

常用的模块有 QtWidgets、QtCore、QtGui 等等，其中 QtWidgets 包括一整套的 UI 元素控件，用于建立符合系统风格的界面，QtGui 模块涵盖了多种基本图形功能的类（字体，图形，图标，颜色），QtCore 涵盖了核心的非 GUI 界面的功能（时间，文件，目录，数据类型，文本流，线程进程等对象）。

20.3.2 创建一个类将作为主窗口

```
# 创建一个名为 WinForm 的类，基于 QWidget
class WinForm(QWidget):
    # 初始化类
    def __init__(self):
        # 继承父类的 init
        super().__init__()
        # 初始化本身窗口
        self.initUI()

    def initUI(self):
        # 先创建滑块和 LCD 部件
        lcd = QLCDNumber(self)
        slider = QSlider(Qt.Horizontal, self)

        # 设置滑动条最大值和默认值，以及长和宽等，设置默认 LCD 样式显示数值，
        slider.setMaximum(1000)
        slider.setValue(666)
        slider.setMinimumWidth(200)
        slider.setFixedHeight(60)
        lcd.display(666)

        # 通过 QVBoxLayout 来设置布局，垂直分布
        vBox = QVBoxLayout()
        vBox.addWidget(lcd)
        vBox.addWidget(slider)

        self.setLayout(vBox)
        # valueChanged() 是 Qslider 的一个信号函数，只要 slider 的值发生改变，
        它就会发射一个信号，然后通过 connect 连接信号的接收部件，也就是 lcd。
        slider.valueChanged.connect(lcd.display)
```

(下页继续)

(续上页)

```
style = "QSlider::groove:horizontal {border:1px solid #999999;
↪height:10px;" \
        "background-color:#666666;margin:2px 0;}" \
        "QSlider::handle:horizontal {background-color:#ff0000;
↪border:1px solid #797979;" \
        "width:50px;margin:-20px;border-radius:25px;}" \

# 设置样式
slider.setStyleSheet(style)

#self.setGeometry(0,0,800,480)
# 设置窗口标题
self.setWindowTitle("拖动滑块控制数字显示")
```

20.3.3 创建应用程序

```
# name 是当前模块名，当模块直接运行时为 main，下面的代码块就将运行
if __name__ == '__main__':
    # 每个应用程序需要一个 QApplication 实例，sys.argv 是运行该模块传入命令行参数，可
    以为空。
    app = QApplication(sys.argv)
    # 控件实例
    form = WinForm()
    # 设置大小，然后显示控件
    form.resize(800, 480)
    form.show()
    # 程序执行，并进入消息事件循环，通过 exit 函数安全结束
    sys.exit(app.exec_())
```

创建一个 QApplication 实例，运行程序，创建一个窗口 form，作为主窗口，然后显示。

20.3.4 整体程序

pyqt5_demo.py 整体代码如下:

列表 1: 配套代码 libdemo/PyQt5/pyqt5_demo.py

```
1 import sys
2 from PyQt5.QtWidgets import QWidget,QLCDNumber,QSlider,QVBoxLayout,
   ↳ QApplication
3 from PyQt5.QtCore import Qt
4
5 class WinForm(QWidget):
6     def __init__(self):
7         super().__init__()
8         self.initUI()
9
10    def initUI(self):
11        # 先创建滑块和 LCD 部件
12        lcd = QLCDNumber(self)
13        slider = QSlider(Qt.Horizontal, self)
14
15        slider.setMaximum(1000)
16        slider.setValue(666)
17        slider.setMinimumWidth(200)
18        slider.setFixedHeight(60)
19        lcd.display(666)
20
21        # 通过 QVBoxLayout 来设置布局
22        vBox = QVBoxLayout()
23        vBox.addWidget(lcd)
24        vBox.addWidget(slider)
25
26        self.setLayout(vBox)
```

(下页继续)

(续上页)

```
27     # valueChanged() 是 Qslider 的一个信号函数, 只要 slider 的值发生改变, 它就会发射一个信号, 然后通过 connect 连接信号的接收部件, 也就是 lcd。
28     slider.valueChanged.connect(lcd.display)
29
30     style = "QSlider::groove:horizontal {border:1px solid #999999;
31 ↪height:10px;" \
32         "background-color:#666666;margin:2px 0;}" \
33         "QSlider::handle:horizontal {background-color:#ff0000;
34 ↪border:1px solid #797979;" \
35         "width:50px;margin:-20px;border-radius:25px;}" \
36
37     slider.setStyleSheet(style)
38     #self.setGeometry(0, 0, 800, 480)
39     self.setWindowTitle("拖动滑块控制数字显示")
40
41 if __name__ == '__main__':
42     app = QApplication(sys.argv)
43     form = WinForm()
44     form.resize(800, 480)
45     form.show()
46     sys.exit(app.exec_())
```

终端直接运行我们的 pyqt5_demo.py 程序:

```
sudo python3 pyqt5_demo.py
```

显示效果:

大家可以将配套代码目录 libdemo\PyQt5\pyqt5_demo.py, 上传到开发板中测试。



20.4 Qt Designer 使用

一般情况下，我们自己用 python 程序创建自己的界面应用，但随着应用程序越来越大或界面越来越复杂，以编程方式定义所有窗口部件可能会很麻烦。这个情况下可以使用 Qt Designer 专门来制作 PyQt 程序中 UI 界面，下面将介绍下使用 Qt Designer 创建一个简单的登录 UI 界面。

```
# 通过 apt 安装 Qt Creator 工具
sudo apt install qtcreator

# 安装 qt designer 等工具
sudo apt install qttools5-dev-tools
```

重要：在鲁班猫中使用 Qt Creator，设计一些复杂界面时会卡住或者直接卡死，可能是内存不够，建议在虚拟机或者 PC 中设计 ui 或者编写 pyqt5 程序，可以在板卡上直接运行。

安装之后，可以在桌面菜单搜索 Qt Creator，或者直接打开 Qt Designer，然后打开应用：

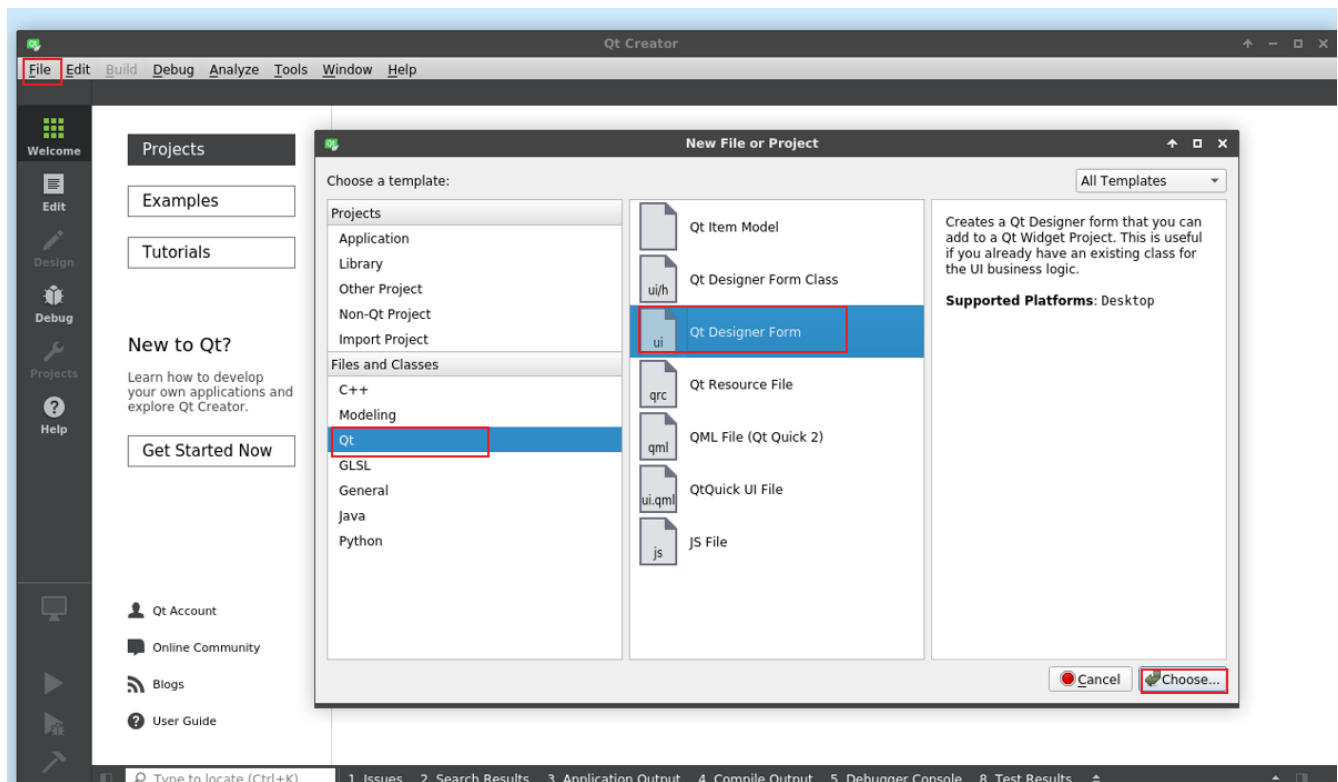
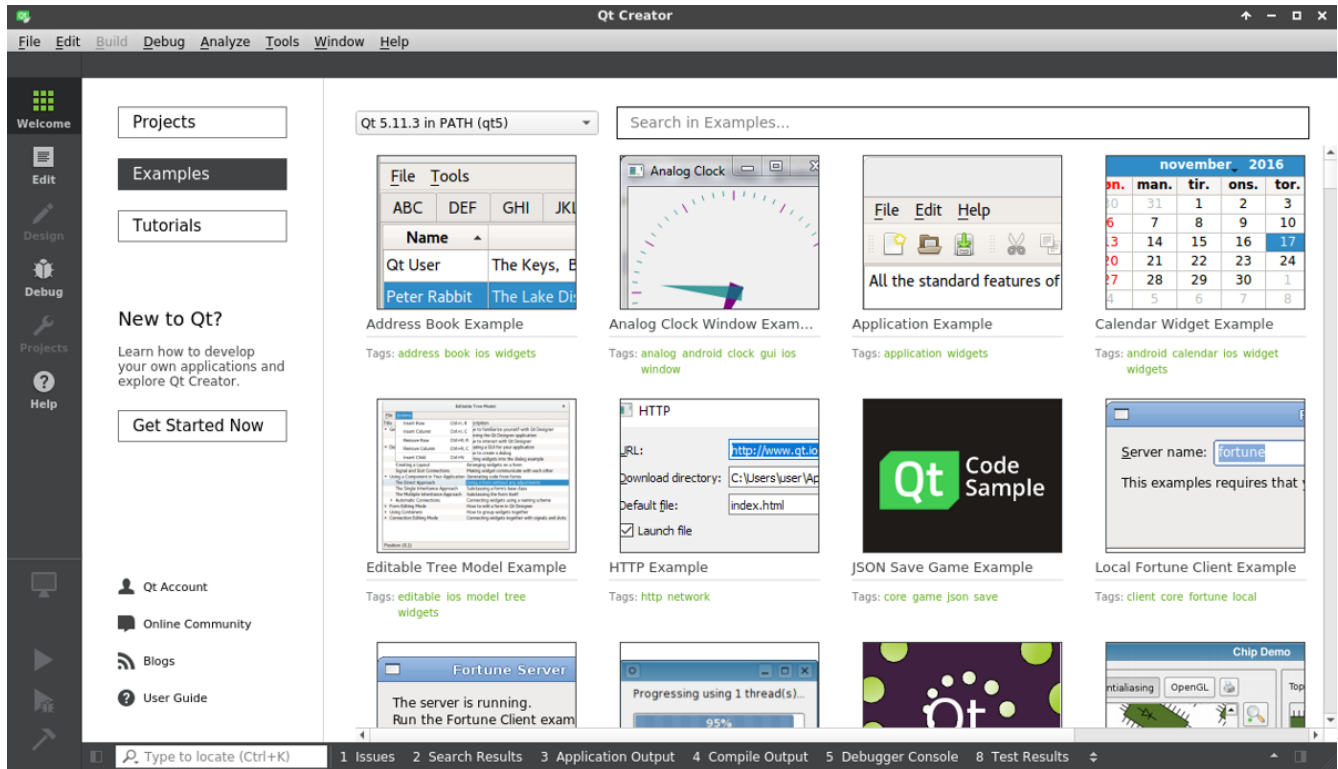
20.4.1 创建一个界面 UI 文件

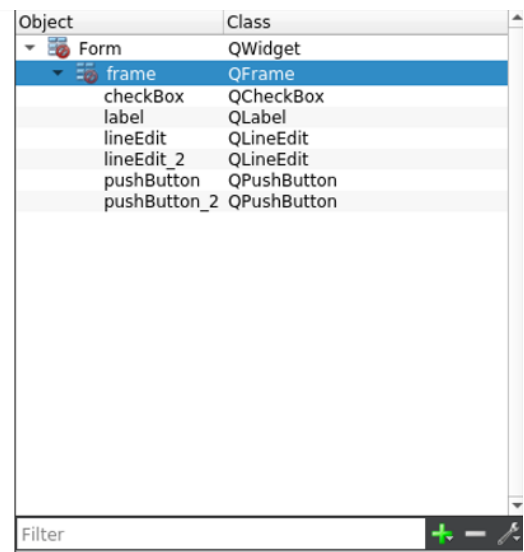
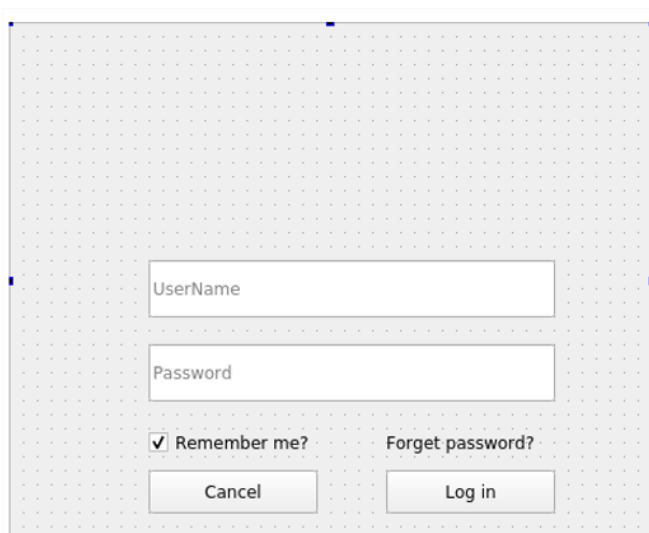
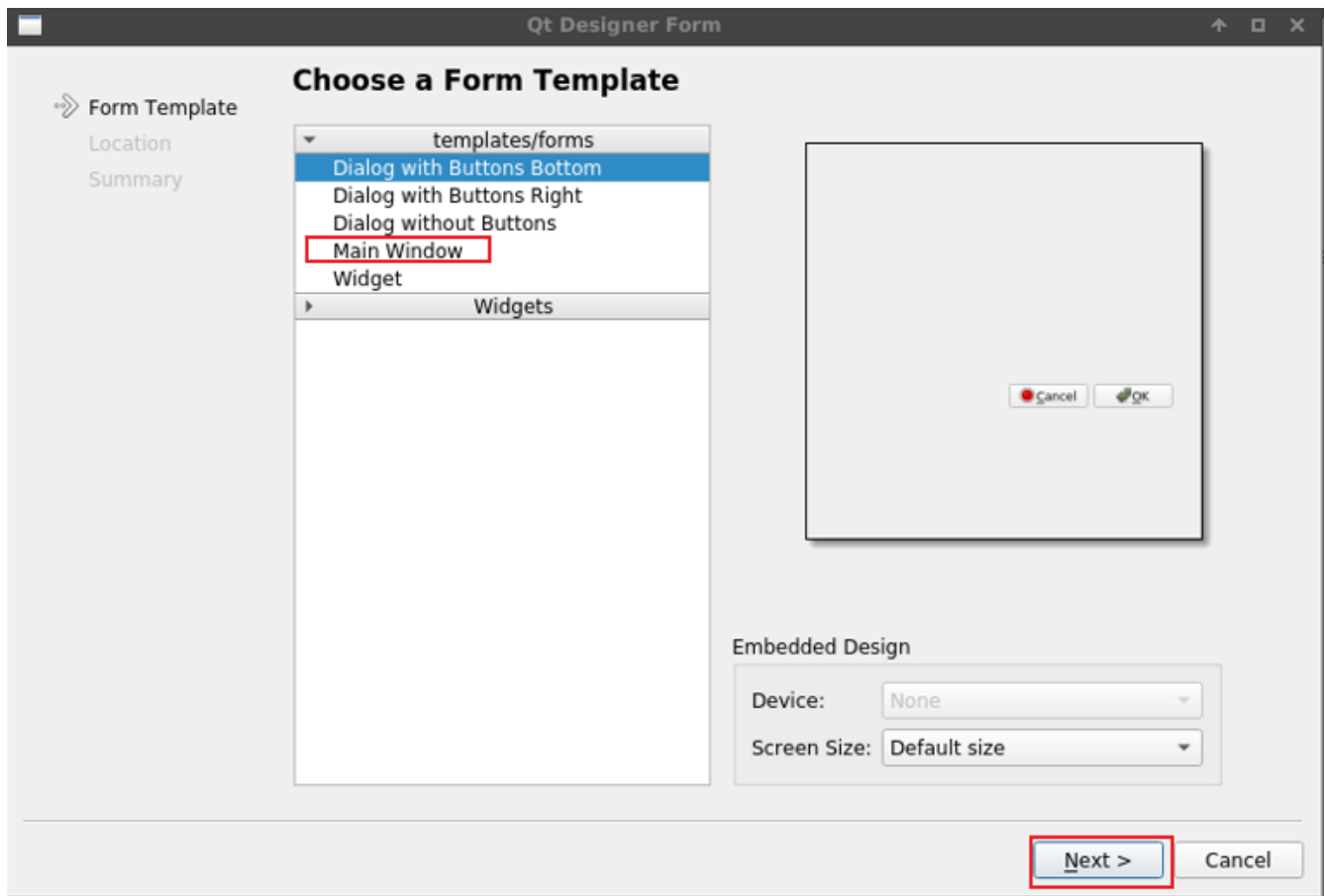
打开 Qt Creator 后点击 file，选择 go to File -> New File or Project，在 Files and Classes 栏，点击 QT，选中 QT Designer Form，之后设置创建的.ui 文件名字，路径等，如下：

接着选择 Main Window 或者其他，最后设置文件路径和文件名即可：

20.4.2 界面构建

在 ui 界面，添加一个容器 Frame 放到画布上，并调整大小。创建一个简单的登录界面，需要账号密码栏，使用 Input Widgets: Line Edit，登录和取消使用按钮 Buttons: PushButton，忘记密码框就用文本栏 Display Widgets: TextLabe，记住密码使用 Buttons: CheckBox，把这些控件拖到窗口，设置下名称，布局如下：





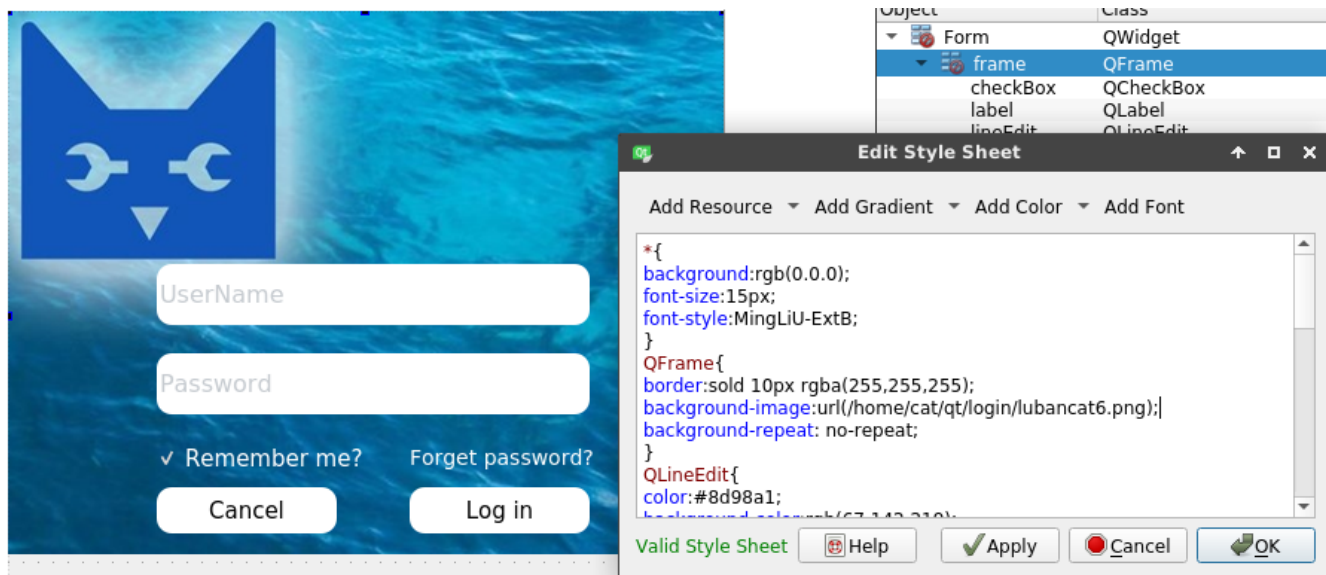
设置样式，右击整个容器，选择 edit style sheet，填入下面代码，自己可以根据喜好更改颜色、样式和选择背景图片路径，这简单配置了下，看起来还行 ~：

```
*{
background:rgb(0, 0, 0);
font-size:15px;
font-style:MingLiU-ExtB;
}
QFrame{
border:solid 10px rgba(255,255,255);
background-image:url(/home/cat/qt/login/lubancat6.png);
}
QLineEdit{
color:#8d98a1;
background-color:rgb(0, 0, 0);
font-size:16px;
border-style:outset;
border-radius:10px;
font-style:MingLiU-ExtB;
}
QPushButton{
background:#ced1d8;
border-style:outset;
border-radius:10px;
font-style:MingLiU-ExtB;
}
QPushButton:pressed{
background-color:#405361;
border-style:inset;
font-style:MingLiU-ExtB;
}
QCheckBox{
background:rgba(85,170,255,0);
```

(下页继续)

(续上页)

```
color:white;
font-style:MingLiU-ExtB;
}
QLabel{
background:rgba(85,170,255,0);
color:white;
font-style:MingLiU-ExtB;
font-size:14px;
}
```



提示: ui 的设计, 以及 qt designer 使用, 可以参考下 Qt 相关的教程。

20.4.3 调用 ui 文件

使用 qt designer 设计的.ui 文件，可以直接使用 uic 模块导入 ui 文件，或者把.ui 文件转换成 python 模块，直接导入使用。

1. 直接使用 uic 模块导入.ui 文件：

列表 2: 配套代码 libdemo/PyQt5/login/login.py

```
1 import sys
2 from PyQt5.QtWidgets import QApplication, QMainWindow
3 from PyQt5 import uic
4
5 class LoginWindow(QMainWindow):
6
7     def __init__(self, *args, **kwargs):
8         super().__init__(*args, **kwargs)
9         # 使用 uic.loadUi 导入当前目录下的.ui 文件
10        uic.loadUi("login.ui", self)
11        self.setWindowTitle("Lubancat login")
12
13 if __name__ == '__main__':
14     app = QApplication(sys.argv)
15     window = LoginWindow()
16     window.show()
17     sys.exit(app.exec_())
```

2. 或者转换成 python 文件导入

使用 pyuic5 命令将.ui 文件转换成 python 文件：

```
# 通过 apt 安装 pyqt5-dev-tools 工具
sudo apt-get install pyqt5-dev-tools
```

(下页继续)

(续上页)

```
# 使用命令将.ui 文件转换成 python 文件
pyuic5 -o login.ui login_ui.py
```

转换成 python 文件，可以直接导入使用：

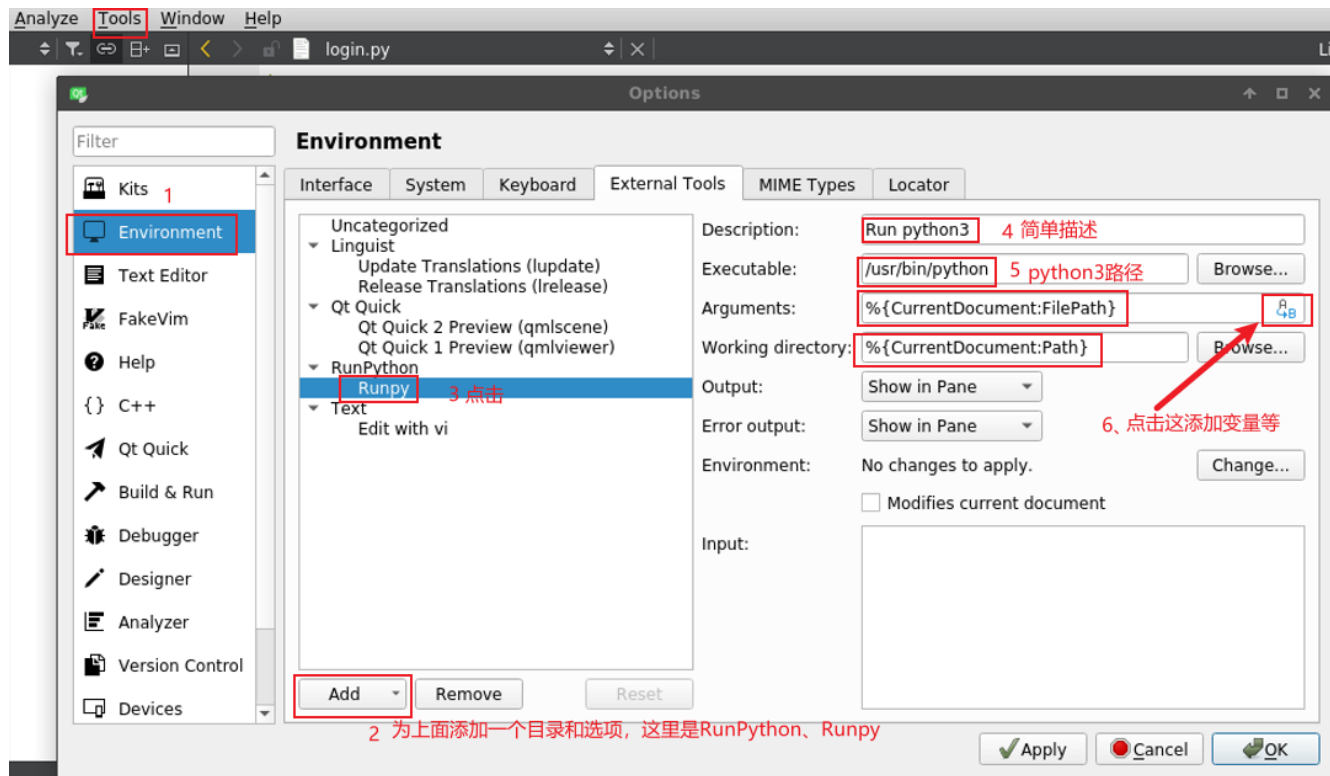
列表 3: 配套代码 libdemo/PyQt5/login/login_ui_form.py

```
1 import sys
2 from PyQt5.QtWidgets import QApplication, QMainWindow
3 # 导入转换出的 Ui_Form 类
4 from login_ui import Ui_Form
5
6 class LoginWindow(QMainWindow):
7
8     def __init__(self, parent=None):
9         super().__init__(parent)
10        # 创建一个 ui 实例
11        self.ui = Ui_Form()
12        # 运行 setupUi() 显示 ui
13        self.ui.setupUi(self)
14        self.setWindowTitle("Lubancat login")
15
16 if __name__ == "__main__":
17     app = QApplication(sys.argv)
18     win = LoginWindow()
19     win.show()
20     sys.exit(app.exec())
```

运行程序，可以在终端使用命令：

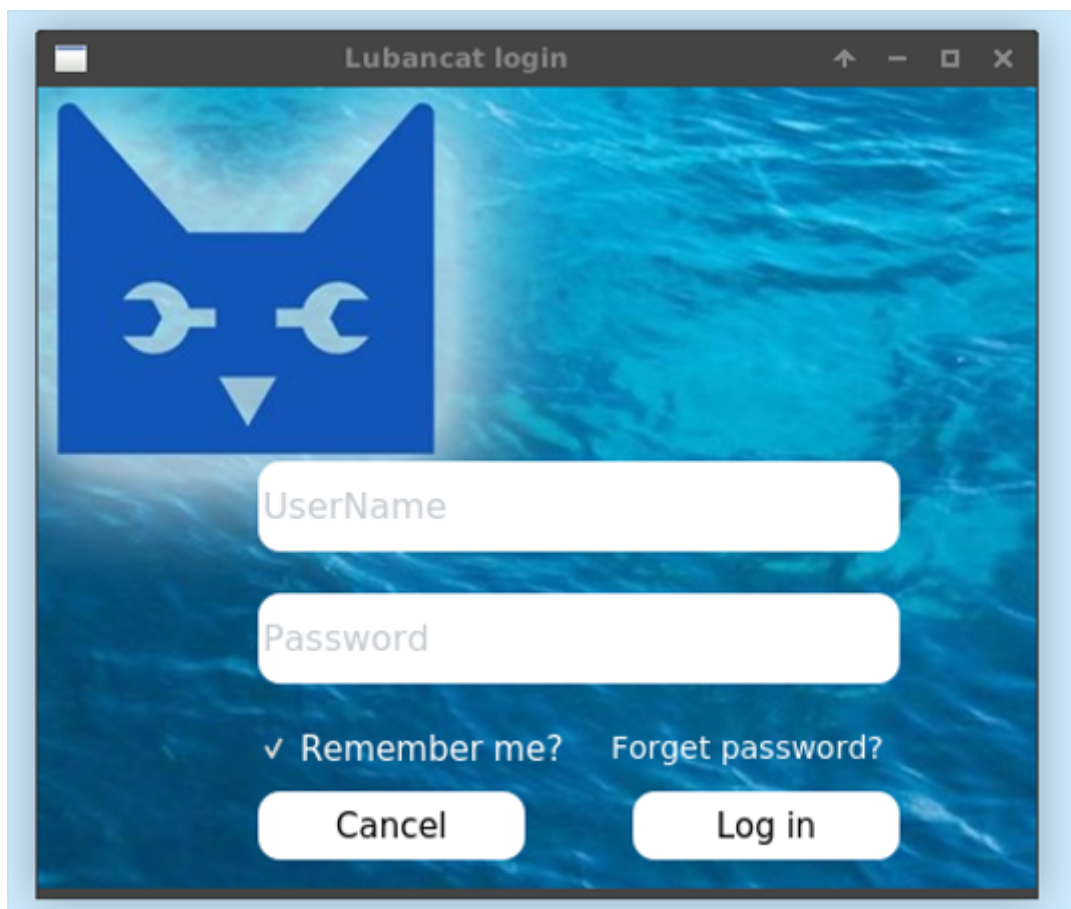
```
# 使用命令运行程序：
sudo python3 login.py
```

或者在 Qt Creator 添加一个 python 运行环境，需要点击界面的 Tool->Options... 在弹窗的界面选择 Enviroment，然后按下面图片操作，最后点击 apply 应用。



运行时，打开对应 python 文件，然后点击 Tool->External->RunPython->RunPy 即可。

运行显示效果：



程序源码参考下配套例程 libdemo/PyQt5 目录中文件，最后如果需要打包发布你的应用程序，可以看下参考链接的一些文档。

20.5 参考

更多 PyQt5 相关教程参考下：

<https://www.riverbankcomputing.com/static/Docs/PyQt5/introduction.html>

<https://www.pythonguis.com/latest/>

第 21 章 NPU 使用

NPU 是专门用于神经网络的处理单元。它旨在加速人工智能领域的神经网络算法，如机器视觉和自然语言处理。随着人工智能的应用范围正在扩大，目前提供各种领域的功能，包括面部跟踪、手势和身体跟踪、图像分类、视频监控、自动语音识别（ASR）以及高级驾驶员辅助系统（ADAS）等。

lubancat-zero、LubanCat-1/LubanCat-2 系列板卡，使用瑞芯微 rk356X 处理器，rk3568 和 rk3566 都内置 NPU 模块，最高可达 1TOPS，支持整数 8、整数 16 卷积运算，支持深度学习框架：TensorFlow、TF-lite、Pytorch、Caffe、ONNX 等等。

瑞芯微官方提供了 rknpu2 和 rknn-Toolkit2 工具，SDK 为带有 RKNPU 的芯片平台提供编程接口。Rockchip NPU 平台使用 RKNN 模型，RKNN 模型是可以使用 rknn-Toolkit2 软件开发工具包转换，部署到板卡使用 rknn-Toolkit-Lite2。

本章将简单介绍在 PC（Ubuntu 系统）上使用 RKNN-Toolkit2 进行模型转换、模型推理、性能评估等，并在板卡上使用 RKNN Toolkit Lite2 部署。

重要： 本章教程测试环境：鲁班猫 RK 系列板卡，镜像系统是 Debian10（Python 3.7.3，OpenCV 4.7.0.68），PC 环境使用 ubuntu20.04（python3.8.10）。教程编写时的 RKNN-Toolkit2 是 1.4.0 版本，板卡 npu 驱动是 0.7.2，rknnrt 是 1.4.0，adb 工具 1.0.40。

21.1 rknn-Toolkit2

RKNN-Toolkit2 工具在 PC 平台上使用，提供 python 接口简化模型的部署和运行，用户通过该工具可以便捷地完成以下功能：模型转换、量化功能、模型推理、性能和内存评估、量化精度分析、模型加密功能。

RKNN-Toolkit2 目前版本适用系统是 Ubuntu18.04/Ubuntu20.04/Ubuntu22.04，更多依赖和使用信

息可以看下《RKNN Toolkit2 快速上手指南》。

RKNN-Toolkit2 工程文件可以直接从官方 [github 地址](#) 拉取或者从 [云盘资料下载](#) (提取码 hslu), 在 1-野火开源图书 _ 教程文档-> 参考代码。(获取的 RKNN-Toolkit2 文件中包含 RKNN Toolkit Lite2)

21.1.1 rknn-Toolkit2 安装

环境安装 (PC ubuntu20.04):

```
# 安装 python 工具, ubuntu20.04 默认是安装了 python3.8.10
sudo apt update
sudo apt-get install python3-dev python3-pip python3.8-venv gcc

# 安装相关库和软件包
sudo apt-get install libxslt1-dev zlib1g-dev libglib2.0 libsm6 \
libgl1-mesa-glx libprotobuf-dev gcc
```

安装 RKNN-Toolkit2:

```
# 创建目录, 由于测试使用的 ubuntu20.04 已经安装的包可能和安装运行 RKNN-Toolkit2 所需的包版本不同, 为避免其他问题, 这里使用 python venv 隔离环境
mkdir project-Toolkit2 && cd project-Toolkit2
python3 -m venv .toolkit2_env
# 激活进入环境
source .toolkit2_env/bin/activate

# 拉取源码, 或者复制 RKNN-Toolkit2 到该目录
git clone https://github.com/rockchip-linux/rknn-toolkit2.git

# 使用 pip3 安装包时可能很慢, 设置下源
pip3 config set global.index-url https://mirror.baidu.com/pypi/simple
```

(下页继续)

(续上页)

```
# 安装依赖库, 根据 rknn-toolkit2\doc\requirements_cp38-1.4.0.txt
pip3 install numpy
pip3 install -r doc/requirements_cp38-1.4.0.txt

# 安装 rknn_toolkit2
pip3 install packages/rknn_toolkit2-1.4.0_22dcfef4-cp38-cp38-linux_x86_64.
↪whl
```

检测是否安装成功:

```
(.toolkit2_env) llh@-:~/project-Toolkit2$ python
Python 3.8.10 (default, Jun 22 2022, 20:18:18)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from rknn.api import RKNN
>>>
```

注意: 如果是使用 LubanCat-3 板卡 (rk3576), 请使用 2.0 以上版本的 [RKNN-Toolkit2](#)。

21.1.2 模型转换和模型推理

在 RKNN-Toolkit2 文件中 example 目录下有各种功能的例程, 进入 `./examples/onnx/yolov5` 目录该 Demo 展示了在 PC 上将 onnx 模型转换成 RKNN 模型, 然后导出、推理、部署到 NPU 平台运行并取回结果的过程。

执行命令:

```
# 切换到 examples/onnx/yolov5 目录下
cd examples/onnx/yolov5
# 模型转换和模型推理
python test.py
```

运行导出成功后, 会在当前目录下生成 `yolov5s.rknn` 文件, 该文件在接下来的 Toolkit Lite2 小节

将会使用到。

上面运行 yolov5 例程同时也会进行模型推理，会在当前目录下输出结果图片（out.jpg）。

21.1.3 性能和内存评估

RKNN-Toolkit2 除了可以模拟推理测试，也可以进行简单连板调试，鲁班猫一些板卡 usb 接口不能直接 usb adb 调试，但可以使用网络 adb 连接进行调试。

连板调试板端需要启动 adbd 和 rknn_server 服务。PC 端的 adb server 需要连接到 adbd，这样就可以通过客户端与设备端进行沟通。rknn_server 是一个运行在板子上的后台代理服务，用于接收 PC 通过 USB 传输过来的协议，然后执行板端 runtime 对应的接口，并返回结果给 PC 端。

```
# 复制文件到板卡（可以通过 scp sftp nfs 等方式，文件在配套例程或者看下后面的参考链接，到
RK356X 的 runtime\RK356X\Linux\rknn_server 目录下获取）

# 添加可执行权限
cd ../usr/bin
chmod +x rknn_server restart_rknn.sh start_rknn.sh

# 启动 rknn_server
restart_rknn.sh

# 查看服务 adbd 的状态，处于 active 即可
sudo systemctl status adbd.service
sudo /usr/bin/adbd &
```

板子和 PC 连接在一个局域网下，测试使用本教程的配套程序：

```
# PC 端安装 adb
sudo apt install adb

# 开启 adb server
adb start-server

# 连接板子，根据板子实际 IP，默认 5555 端口
```

(下页继续)

```
adb connect 192.168.103.115

# 查看连接的设备，确认初始化运行时的 device_id
adb devices

cd examples/onnx/yolov5

# 运行程序，连板调试
python test.py
```

```
W __init__: rknn-toolkit2 version: 1.4.0-22dcfef4
--> Config model
done
--> Loading model
done
--> Building model
Analysing : 100%|██████████████████████████████████████████████████████████████████████████████| 142/142L
↪[00:00<00:00, 2940.26it/s]
Quantizing : 100%|██████████████████████████████████████████████████████████████████████████████| 142/142L
↪[00:00<00:00, 336.83it/s]
W build: The default input dtype of 'images' is changed from 'float32' to
↪'int8' in rknn model for performance!
Please take care of this change when deploy rknn modelL
↪with Runtime API!
W build: The default output dtype of 'output' is changed from 'float32' to
↪'int8' in rknn model for performance!
Please take care of this change when deploy rknn modelL
↪with Runtime API!
W build: The default output dtype of '327' is changed from 'float32' to
↪'int8' in rknn model for performance!
Please take care of this change when deploy rknn modelL
↪with Runtime API!
```

(下页继续)

(续上页)

```

W build: The default output dtype of '328' is changed from 'float32' to
↳ 'int8' in rknn model for performance!
        Please take care of this change when deploy rknn model.
↳ with Runtime API!
done
--> Export rknn model
done
--> Init runtime environment
W init_runtime: Flag perf_debug has been set, it will affect the
↳ performance of inference!
I NPUTransfer: Starting NPU Transfer Client, Transfer version 2.1.0
↳ (b5861e7@2020-11-23T11:50:36)
D RKNNAPI: =====
D RKNNAPI: RKNN VERSION:
D RKNNAPI:   API: 1.4.0 (bb6dac9 build: 2022-08-29 16:17:01) (null)
D RKNNAPI:   DRV: rknn_server: 1.3.0 (121b661 build: 2022-04-29 11:11:47)
D RKNNAPI:   DRV: rknnrt: 1.4.0 (a10f100eb@2022-09-09T09:07:14)
D RKNNAPI: =====
done
=====
                        Performance
        ##### The performance result is just for debugging, #####
        ##### may worse than actual performance! #####
=====

Total Weight Memory Size: 7312768
Total Internal Memory Size: 7782400
Predict Internal Memory RW Amount: 134547200
Predict Weight Memory RW Amount: 7312768
ID   OpType          DataType Target InputShape
↳   OutputShape          DDR Cycles   NPU Cycles   Total Cycles
↳ Time (us)          MacUsage(%)   RW (KB)      FullName
0    InputOperator    UINT8      CPU         \
↳   (1, 3, 640, 640)          0          0          0          (下页继续)
↳   19          \          1200.00      InputOperator: images

```

(续上页)

1	Conv	UINT8	NPU	(1, 3, 640, 640), (32, 3, 6, 6), (32)			
→	(1, 32, 320, 320)		1493409	691200	1493409		
→	11956	9.64	4409.25		Conv:Conv_0		
2	exSwish	INT8	NPU	(1, 32, 320, 320)			
→	(1, 32, 320, 320)		2167674	0	2167674		
→	5408	\	6400.00		exSwish:Sigmoid_1_2swish		
3	Conv	INT8	NPU	(1, 32, 320, 320), (64, 32, 3, 3), (64)			
→	(1, 64, 160, 160)		1632022	921600	1632022		
→	4153	36.99	4818.50		Conv:Conv_3		
4	exSwish	INT8	NPU	(1, 64, 160, 160)			
→	(1, 64, 160, 160)		1083837	0	1083837		
→	2849	\	3200.00		exSwish:Sigmoid_4_2swish		
5	Conv	INT8	NPU	(1, 64, 160, 160), (32, 64, 1, 1), (32)			
→	(1, 32, 160, 160)		813640	102400	813640		
→	1455	11.73	2402.25		Conv:Conv_6		
6	exSwish	INT8	NPU	(1, 32, 160, 160)			
→	(1, 32, 160, 160)		541919	0	541919		
→	1456	\	1600.00		exSwish:Sigmoid_7_2swish		
..... (中间省略)							
142	Conv	INT8	NPU	(1, 128, 80, 80), (255, 128, 1, 1), (255)			
→	(1, 255, 80, 80)		824352	408000	824352		
→	1040	65.38	2433.88		Conv:Conv_198		
143	OutputOperator	INT8	CPU	(1, 255, 80, 80), (1, 80, 80, 256)			
→	\		0	0	0		
→	1011	\	3200.00		OutputOperator:output		
144	OutputOperator	INT8	CPU	(1, 255, 40, 40), (1, 40, 40, 256)			
→	\		0	0	0		
→	216	\	800.00		OutputOperator:327		
145	OutputOperator	INT8	CPU	(1, 255, 20, 20), (1, 20, 20, 256)			
→	\		0	0	0		
→	178	\	220.00		OutputOperator:328		
Total Operator Elapsed Time(us): 117367							

(下页继续)

(续上页)

上面测试可以看到开启了量化，并且调用了 `eval_perf` 对模型性能进行评估，获取测试模型的整体耗时及每一层的耗时情况，以上就是简单连接调试。

更多 `rknn-toolkit2` 的功能测试例程，参考下 <https://github.com/rockchip-linux/rknn-toolkit2/tree/master/examples/functions>。

21.2 RKNN Toolkit Lite2

RKNN Toolkit Lite2 为 Rockchip NPU 平台提供 Python 编程接口，用于在板端部署 RKNN 模型。

21.2.1 板卡上安装 RKNN Toolkit Lite2

Toolkit-lite2 适用于开发板系统端部署推理模型，更多依赖和使用信息可以看下 [RKNN 用户手册](#) 获取 RKNN Toolkit Lite2，可以直接从官方 github 获取，或者在配套例程中，将文件传输到板卡中，直接 git 到板卡也行。获取到的 Toolkit Lite2 目录结构如下：

```
.
├── docs
│   ├── Rockchip_User_Guide_RKNN_Toolkit_Lite2_V1.4.0_CN.pdf
│   ├── Rockchip_User_Guide_RKNN_Toolkit_Lite2_V1.4.0_EN.pdf
│   └── change_log.txt
├── examples
│   └── inference_with_lite
│       ├── resnet18_for_rk356x.rknn
│       ├── resnet18_for_rk3588.rknn
│       ├── space_shuttle_224.jpg
│       └── test.py
└── packages
    ├── rknn_toolkit_lite2-1.4.0-cp37-cp37m-linux_aarch64.whl
    ├── rknn_toolkit_lite2-1.4.0-cp39-cp39-linux_aarch64.whl
    └── rknn_toolkit_lite2_1.4.0_packages.md5sum

4 directories, 10 files
```

环境安装（以 LubanCat 2，Debian10 为例）：

```
sudo apt update

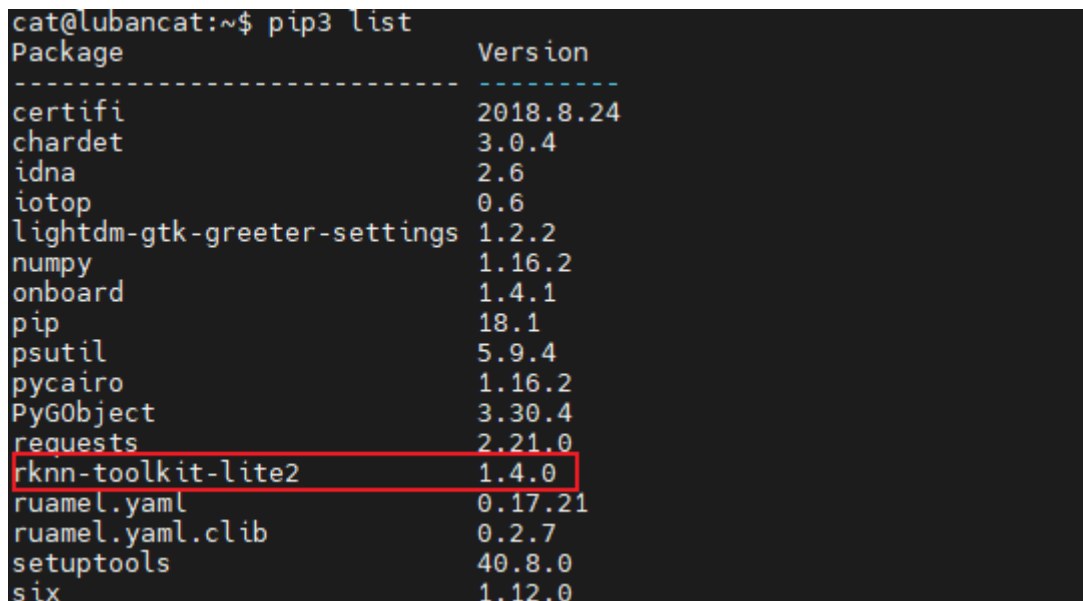
# 安装其他 python 工具
sudo apt-get install python3-dev python3-pip gcc

# 安装相关依赖和软件包
pip3 install wheel
sudo apt-get install -y python3-opencv
sudo apt-get install -y python3-numpy
sudo apt -y install python3-setuptools
```

Toolkit Lite2 工具安装:

```
# 进入到 rknn_toolkit_lite2/packages 目录下, 选择 Debian10 ARM64 with python3.
→ 7.3 的 whl 文件安装:
pip3 install rknn_toolkit_lite2-1.4.0-cp37-cp37m-linux_aarch64.whl
```

安装成功:



```
cat@lubancat:~$ pip3 list
Package                               Version
-----
certifi                               2018.8.24
chardet                               3.0.4
idna                                  2.6
iotop                                 0.6
lightdm-gtk-greeter-settings         1.2.2
numpy                                 1.16.2
onboard                               1.4.1
pip                                   18.1
psutil                               5.9.4
pycairo                              1.16.2
PyGObject                            3.30.4
requests                              2.21.0
rknn-toolkit-lite2                   1.4.0
ruamel.yaml                           0.17.21
ruamel.yaml.clib                      0.2.7
setuptools                            40.8.0
six                                   1.12.0
```


21.2.2 板端部署推理

librknnrt.so 是一个板端的 runtime 库, 运行需要该库, 板卡默认镜像/usr/lib 目录下有 librknnrt.so 库, 但是需要更新下, 选择对应版本的 librknnrt.so。

librknnrt.so 具体在 rknpu2 工程中 runtime/目录下, 根据不同板卡, 系统选择目录, 例如 LubanCat-0/1/2 板卡选择 runtime/RK356X/Linux/librknn_api/aarch64/目录中的 librknnrt.so, 然后复制该库到板卡系统/usr/lib/目录下。

rknpu2 工程文件可以从 <https://github.com/rockchip-linux/rknpu2> 获取, 或者从 [云盘资料下载](#) (提取码 hslu):



运行 RKNN Toolkit Lite2 的 Demo, 进入获取到的 Toolkit Lite2 目录, 在../examples/inference_with_lite 目录下执行命令:

```
cat@lubancat:~/rknn-toolkit2/rknn_toolkit_lite2/examples/inference_with_lite$ ls
resnet18_for_rk356x.rknn  resnet18_for_rk3588.rknn  space_shuttle_224.jpg  test.py
cat@lubancat:~/rknn-toolkit2/rknn_toolkit_lite2/examples/inference_with_lite$ python test.py
--> Load RKNN model
done
--> Init runtime environment
I RKNN: [10:25:19.339] RKNN Runtime Information: librknnrt version: 1.4.0 (a10f100eb@2022-09-09T09
I RKNN: [10:25:19.339] RKNN Driver Information: version: 0.7.2
I RKNN: [10:25:19.340] RKNN Model Information: version: 1, toolkit version: 1.4.0-c15f5e0b(compile
mework name: PyTorch, framework layout: NCHW
done
--> Running model
resnet18
-----TOP 5-----
[812]: 0.9996696710586548
[404]: 0.0002492684288881719
[657]: 1.632158637221437e-05
[833]: 1.0159346857108176e-05
[466 895]: 9.02384545042878e-06
done
cat@lubancat:~/rknn-toolkit2/rknn_toolkit_lite2/examples/inference_with_lite$
```

修改下 Toolkit Lite2 工具中的 examples/onnx/yolov5 例程，简单改下 test.py，使用 RKNN Toolkit Lite2 部署，导入前面 RKNN-Toolkit2 转换出的 yolov5s.rknn（修改后的源文件参考配套例程）：

```
cat@lubancat:~/yolov5$ python test.py
--> Load RKNN model
done
--> Init runtime environment
I RKNN: [10:29:36.521] RKNN Runtime Information: librknnrt version: 1.4.0
↳ (a10f100eb@2022-09-09T09:07:14)
I RKNN: [10:29:36.521] RKNN Driver Information: version: 0.7.2
I RKNN: [10:29:36.524] RKNN Model Information: version: 1, toolkit version:
↳ 1.4.0-22dcfef4(compiler version: 1.4.0 (3b4520e4f@2022-09-05T20:52:35)),
↳ target: RKNPU lite, target platform: rk3568, framework name: ONNX,
↳ framework layout: NCHW
done
--> Running model
done
class: person, score: 0.8352155685424805
box coordinate left,top,right,down: [211.9896697998047, 246.17460775375366,
↳ 283.70787048339844, 515.0830216407776]
```

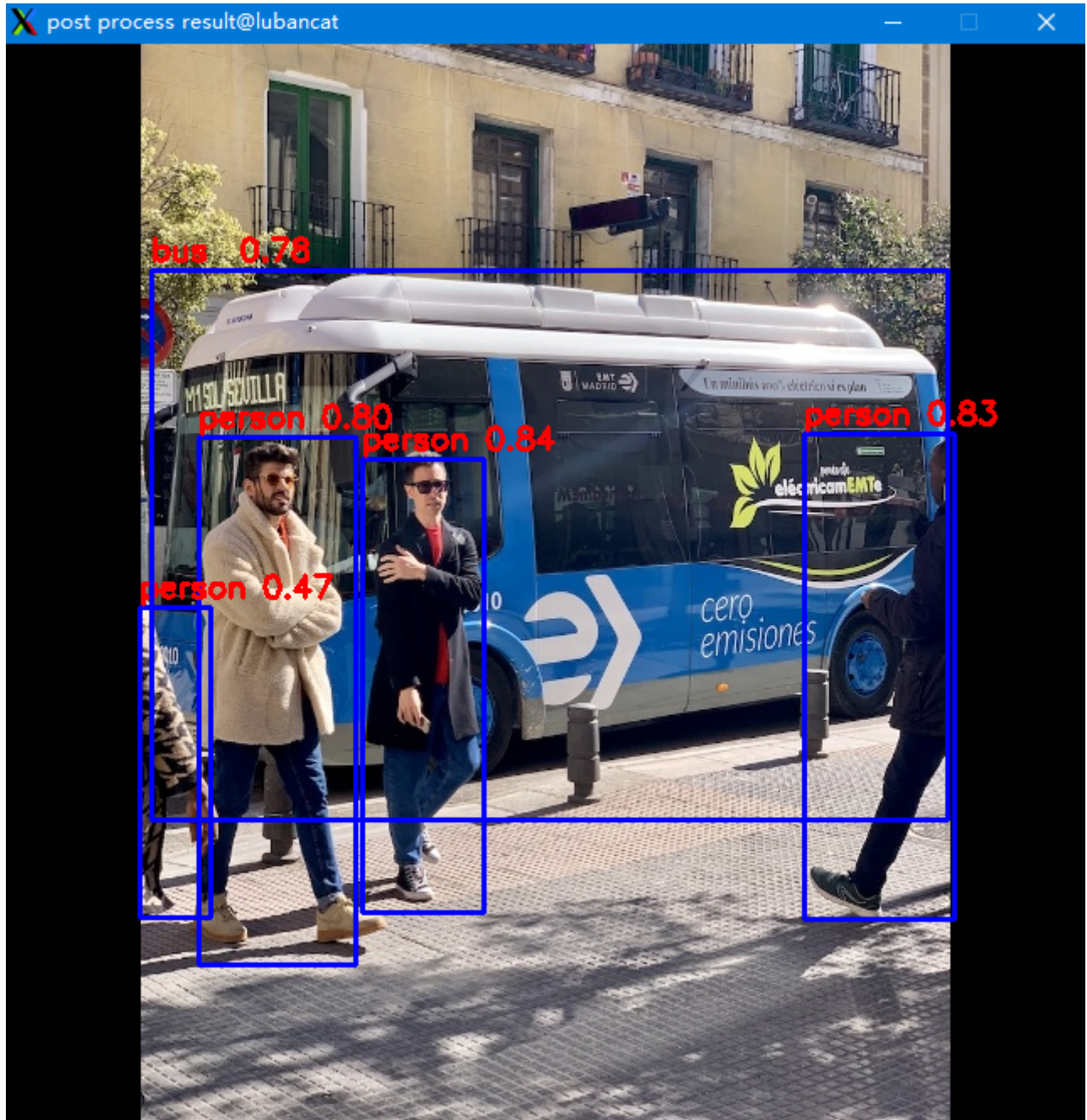
(下页继续)

(续上页)

```
class: person, score: 0.8330121040344238
box coordinate left,top,right,down: [473.26745200157166, 231.93780636787415,
↪ 562.1268351078033, 519.7597033977509]
class: person, score: 0.7970154881477356
box coordinate left,top,right,down: [114.68497347831726, 233.11390662193298,
↪ 207.21904873847961, 546.7821505069733]
class: person, score: 0.4651743769645691
box coordinate left,top,right,down: [79.09242534637451, 334.77847719192505,↪
↪121.60038471221924, 518.6368670463562]
class: bus , score: 0.7837686538696289
box coordinate left,top,right,down: [86.41703361272812, 134.41848754882812,↪
↪558.1083570122719, 460.4184875488281]

(post process result:1896): dbind-WARNING **: 10:31:30.000: Error↪
↪retrieving accessibility bus address: org.freedesktop.DBus.Error.
↪ServiceUnknown: The name org.a11y.Bus was not provided by any .service↪
↪files
cat@lubancat:~/yolov5$ ls
bus.jpg  dataset.txt  onnx_yolov5_0.npy  onnx_yolov5_1.npy  onnx_yolov5_2.
↪npy  out.jpg  test.py  yolov5s.onnx  yolov5s.rknn
```

运行正常可以显示（或者查看输出图片 out.jpg）：



提示：默认配套例程../yolov5/test.py 是没有使用 cv2.imshow() 显示图片。

21.3 相关频率设置 (rk356X)

鲁班猫板卡 CPU 默认是 interactive 状态，它会根据 CPU 使用率和目标负载来动态地调整 CPU 频率。为获得更高运行速度或者性能评估，我们需要手动固定频率，参考下这：

1、CPU 频率设置：

```
# 查看 CPU 当前频率
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq

# 查看 CPU 当前调频策略
cat /sys/devices/system/cpu/cpufreq/policy0/scaling_governor

# 查看可以设置的频率
cat /sys/devices/system/cpu/cpufreq/policy0/scaling_available_frequencies

# 设置允许 root 权限的用户通过 sysfs 的 "scaling_setspeed" 字段将 cpu 频率设置成用户想要的频率
echo userspace > /sys/devices/system/cpu/cpufreq/policy0/scaling_governor

# 设置需要固定的频率
echo 1800000 > /sys/devices/system/cpu/cpufreq/policy0/scaling_setspeed
```

2、DDR 频率设置：

```
# 查看 DDR 当前频率
cat /sys/class/devfreq/dmc/cur_freq

# 查看 DDR 当前调频策略
cat /sys/class/devfreq/dmc/governor

# 查看 DDR 可以设置的频率
cat /sys/class/devfreq/dmc/available_frequencies
```

(下页继续)

(续上页)

```
# 设置允许 root 权限的用户通过 sysfs 的 "scaling_setspeed" 字段将 cpu 频率设置成用户想要的频率
echo userspace > /sys/class/devfreq/dmc/governor

# 设置需要固定的频率, 这里是 1056000000
echo 1056000000 > /sys/devices/system/cpu/cpu0/cpufreq/policy0/scaling_setspeed
```

3、NPU 频率设置:

```
# 查看 NPU 查看可用的频率
cat /sys/class/devfreq/fde40000.npu/available_frequencies

echo userspace > /sys/class/devfreq/fde40000.npu/governor
# 设置频率
echo 900000000 > /sys/kernel/debug/clk/clk_scmi_npu/clk_rate

# 查看当前 npu 频率
cat /sys/class/devfreq/fde40000.npu/cur_freq

# 或者使用下面命令查看
cat /sys/kernel/debug/clk/clk_summary | grep npu
```

21.4 参考

<https://github.com/airockchip/rknn-toolkit2>

<https://github.com/rockchip-linux/rknpu2>

<https://github.com/rockchip-linux/rknn-toolkit2>

第 22 章 手写数字识别-飞桨 (PaddlePaddle)

22.1 飞桨 (PaddlePaddle) 简介



飞桨 (PaddlePaddle) 以百度多年的深度学习技术研究和业务应用为基础，集深度学习核心训练和推理框架、基础模型库、端到端开发套件、丰富的工具组件于一体，是中国首个自主研发、功能丰富、开源开放的产业级深度学习平台。

飞桨 (PaddlePaddle) 已凝聚 477 万开发者，基于飞桨创建 56 万个模型，服务 18 万家企事业单位。飞桨助力开发者快速实现 AI 想法，创新 AI 应用，作为基础平台支撑越来越多行业实现产业智能化升级。截至 2022 年 12 月，通信通院最新报告显示，飞桨已经成为中国深度学习市场应用规模第一的深度学习框架和赋能平台，已汇聚 535 万开发者，服务 20 万家企事业单位，基于飞

桨构建了 67 万个模型。

我们将基于飞桨（PaddlePaddle），完成一个简单的手写数字识别任务，并部署到鲁班猫板卡。通过本章节简单了解下飞桨（PaddlePaddle）以及深度学习模型。

提示：测试环境：鲁班猫板卡使用 Debian10，PC 是 ubuntu20.04。PaddlePaddle 是 CPU 版，rknn-Toolkit2 版本 1.4.0。

22.2 安装飞桨（PaddlePaddle）

```
# PC 端使用 pip3 工具安装飞桨 CPU 版
pip3 install paddlepaddle -i https://mirror.baidu.com/pypi/simple

# 验证安装，进入 python 解释器，输入：
python
import paddle
paddle.utils.run_check()

# 验证安装，打印版本信息
print(paddle.__version__)

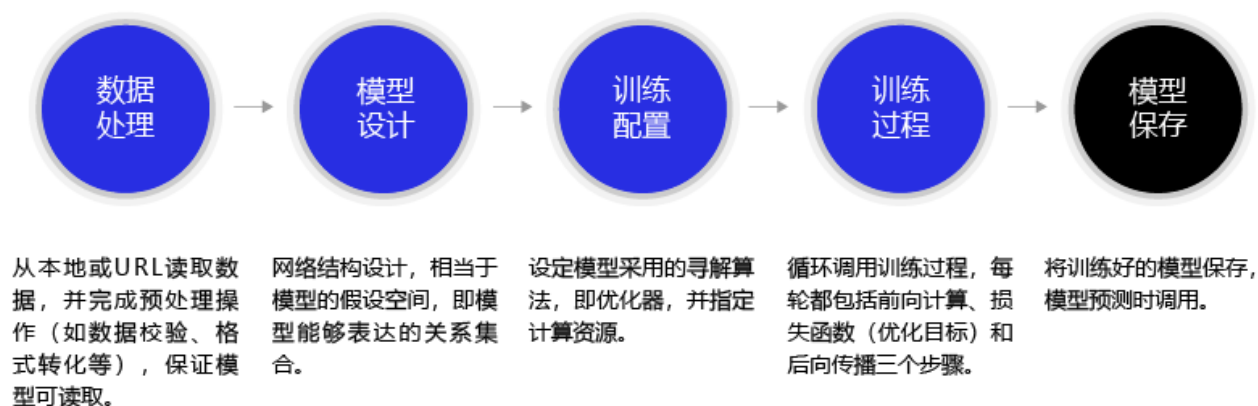
# 最后显示 PaddlePaddle is installed successfully! Let's start deep learning
↪with PaddlePaddle now.

# 说明安装成功
```

详细安装 paddlepaddle 参考下 [安装教程](#)。

22.3 手写数字识别任务

手写数字识别任务是机器学习领域使用最广泛的示例之一，相对简单的模型，非常适用初学者。任务的整体流程，参考下图片（图片来自 Paddle 教程）：



22.3.1 准备训练集和测试集

手写数字识别用灰度图表示不同人书写的数字，每张图片尺寸 28*28，我们使用 MNIST 数据集，可以到 [官网](#) 下载，或者我们直接使用 Paddle 加载 MNIST。

这里我们加载了 MNIST 训练集（mode='train'）和测试集（mode='test'），训练集用于训练模型，测试集用于评估模型效果。

```
# 下载数据集并初始化 DataSet
train_dataset = paddle.vision.datasets.MNIST(mode='train',
↪transform=transform)

test_dataset = paddle.vision.datasets.MNIST(mode='test',
↪transform=transform)
```

22.3.2 模型组网

在数字识别过程中，直接使用了飞桨内置的模型 LeNet，使用高层 API，一行代码即完成 LeNet 的网络构建和初始化，也可以自定义组网，参考 [这里](#)

```
# 模型组网并初始化网络，
lenet = paddle.vision.models.LeNet(num_classes=10)
model = paddle.Model(lenet)
#paddle.summary(lenet, (1, 1, 28, 28))
```

LeNet 模型包含 2 个 Conv2D 卷积层、2 个 ReLU 激活层、2 个 MaxPool2D 池化层以及 3 个 Linear 全连接层，取消上面 `paddle.summary(lenet, (1, 1, 28, 28))` 一行的注释，打印网络的结构和参数统计：

```
-----
Layer (type)          Input Shape          Output Shape          Param #
=====
Conv2D-1              [[1, 1, 28, 28]]     [1, 6, 28, 28]        60
ReLU-1                [[1, 6, 28, 28]]     [1, 6, 28, 28]         0
MaxPool2D-1           [[1, 6, 28, 28]]     [1, 6, 14, 14]         0
Conv2D-2              [[1, 6, 14, 14]]     [1, 16, 10, 10]       2,416
ReLU-2                [[1, 16, 10, 10]]    [1, 16, 10, 10]         0
MaxPool2D-2           [[1, 16, 10, 10]]    [1, 16, 5, 5]          0
Linear-1               [[1, 400]]           [1, 120]               48,120
Linear-2               [[1, 120]]           [1, 84]                10,164
Linear-3               [[1, 84]]            [1, 10]                850
=====
Total params: 61,610
Trainable params: 61,610
Non-trainable params: 0
-----
Input size (MB): 0.00
Forward/backward pass size (MB): 0.11
```

(下页继续)

(续上页)

```
Params size (MB): 0.24
Estimated Total Size (MB): 0.35
-----
```

22.3.3 模型训练与评估

模型训练包括多轮迭代 (epoch)，每轮迭代遍历一次训练数据集，并且每次从中获取一小批 (mini-batch) 样本，送入模型执行前向计算得到预测值，并计算预测值 (predict_label) 与真实值 (true_label) 之间的损失函数值 (loss)。执行梯度反向传播，并根据设置的优化算法 (optimizer) 更新模型的参数。观察每轮迭代的 loss 值减小趋势，可判断模型训练效果。

```
# 设置优化器及其学习率，并将网络的参数传入优化器，设置损失函数和精度计算方式
# optimizer 优化器，寻找最优解的方法；Loss 损失函数；metric 评价指标
# 这里配置使用 Adam 优化器，使用交叉熵损失函数 CrossEntropyLoss 用于分类任务评估，
model.prepare(paddle.optimizer.Adam(parameters=model.parameters()),
              paddle.nn.CrossEntropyLoss(),
              paddle.metric.Accuracy())

# 模型训练，epoch 训练轮次，batch_size 批次大小，verbose=1 打印训练过程中的日志
model.fit(train_dataset, epochs=5, batch_size=64, verbose=1)

# 模型评估，将测试数据集送入训练好的模型进行评估，得到预测值，计算预测值与真实值之间的损失
# 函数值 (loss)，
# 并计算评价指标值 (metric)，用于评估模型效果
model.evaluate(test_dataset, batch_size=64, verbose=1)
```

22.3.4 模型保存和导出 ONNX 模型

训练好的模型，可以保存到文件中，后续需要训练调优或推理部署时，再加载（load）到内存中运行。

```
1 # 保存模型
2 model.save('./output/mnist')
3 # 加载模型
4 model.load('output/mnist')
5
6 # 从测试集中取出一张图片
7 img, label = test_dataset[0]
8 # 将图片 shape 从 1*28*28 变为 1*1*28*28, 增加一个 batch 维度, 以匹配模型输入格式要求
9 img_batch = np.expand_dims(img.astype('float32'), axis=0)
10
11 # 执行推理并打印结果, 此处 predict_batch 返回的是一个 list, 取出其中数据获得预测结果
12 out = model.predict_batch(img_batch)[0]
13 pred_label = out.argmax()
14 print('true label: {}, pred label: {}'.format(label[0], pred_label))
```

Paddle 模型训练之后，转换成 ONNX 协议只需要调用 `paddle.onnx.export` 接口，便会在指定的路径下生成 ONNX 模型，也可以 Paddle 模型保存为部署模型（静态图模型），然后使用 `Paddle2ONNX` 命令行进行转换。转换原理参考下 [这里](#)。

```
# 需要保存的路径, 当前 onnx 目录下
save_path = 'onnx/lenet'
# 为模型指定输入的 shape 和数据类型
x_spec = paddle.static.InputSpec([1, 1, 28, 28], 'float32', 'x')
# 生成 ONNX 模型
paddle.onnx.export(lenet, save_path, input_spec=[x_spec], opset_version=11)
```

22.3.5 完整程序

列表 1: 配套代码 paddlepaddle/digital_recognition.py

```
1 import paddle
2 import numpy as np
3 from paddle.vision.transforms import Normalize
4
5 transform = Normalize(mean=[127.5], std=[127.5], data_format='CHW')
6 # 下载数据集并初始化 DataSet
7 train_dataset = paddle.vision.datasets.MNIST(mode='train',
8     ↪transform=transform)
9
10 test_dataset = paddle.vision.datasets.MNIST(mode='test',
11     ↪transform=transform)
12
13
14 # 模型组网并初始化网络
15 lenet = paddle.vision.models.LeNet(num_classes=10)
16 model = paddle.Model(lenet)
17
18
19 # 模型训练的配置准备, 准备损失函数, 优化器和评价指标
20 model.prepare(paddle.optimizer.Adam(parameters=model.parameters()),
21     paddle.nn.CrossEntropyLoss(),
22     paddle.metric.Accuracy())
23
24 # 模型训练
25 model.fit(train_dataset, epochs=5, batch_size=64, verbose=1)
26
27 # 模型评估
28 model.evaluate(test_dataset, batch_size=64, verbose=1)
29
30
31 # 保存模型
32 model.save('./output/mnist')
33
34
35 # 加载模型
```

(下页继续)

(续上页)

```
28 # model.load('output/mnist')
29
30 # 从测试集中取出一张图片
31 img, label = test_dataset[0]
32 # 将图片 shape 从 1*28*28 变为 1*1*28*28, 增加一个 batch 维度, 以匹配模型输入格式要求
33 img_batch = np.expand_dims(img.astype('float32'), axis=0)
34
35 # 执行推理并打印结果, 此处 predict_batch 返回的是一个 list, 取出其中数据获得预测结果
36 out = model.predict_batch(img_batch)[0]
37 pred_label = out.argmax()
38 print('true label: {}, pred label: {}'.format(label[0], pred_label))
39
40 # 需要保存的路径
41 save_path = 'onnx/lenet'
42 # 为模型指定输入的形状和数据类型
43 x_spec = paddle.static.InputSpec([1, 1, 28, 28], 'float32', 'x')
44 # 生成 ONNX 模型
45 paddle.onnx.export(lenet, save_path, input_spec=[x_spec], opset_version=11)
```

简单测试显示:

```
The loss value printed in the log is the current step, and the metric is 
↪the average value of previous steps.
Epoch 1/5
step 938/938 [=====] - loss: 0.0117 - acc: 0.9368 -
↪ 19ms/step
Epoch 2/5
step 938/938 [=====] - loss: 0.0072 - acc: 0.9778 -
↪ 18ms/step
Epoch 3/5
step 938/938 [=====] - loss: 0.0521 - acc: 0.9814 -
↪ 19ms/step
```

(下页继续)

(续上页)

```
Epoch 4/5
step 938/938 [=====] - loss: 0.0047 - acc: 0.9848 -
  ↪ 19ms/step
Epoch 5/5
step 938/938 [=====] - loss: 0.0208 - acc: 0.9857 -
  ↪ 19ms/step
Eval begin...
step 157/157 [=====] - loss: 0.0011 - acc: 0.9843 -
  ↪ 8ms/step
Eval begin...
step 157/157 [=====] - loss: 2.8939e-04 - acc: 0.
  ↪ 9825 - 8ms/step
Eval samples: 10000
true label: 7, pred label: 7
I0209 16:10:02.397411 8350 interpretercore.cc:279] New Executor is Running.
2023-02-09 16:10:02 [INFO] Static PaddlePaddle model saved in onnx/
  ↪ paddle_model_static_onnx_temp_dir.
[Paddle2ONNX] Start to parse PaddlePaddle model...
[Paddle2ONNX] Model file path: onnx/paddle_model_static_onnx_temp_dir/model.
  ↪ pdmodel
[Paddle2ONNX] Paramters file path: onnx/paddle_model_static_onnx_temp_dir/
  ↪ model.pdiparams
[Paddle2ONNX] Start to parsing Paddle model...
[Paddle2ONNX] Use opset_version = 11 for ONNX export.
[Paddle2ONNX] PaddlePaddle model is exported as ONNX format now.
2023-02-09 16:10:02 [INFO] ONNX model saved in onnx/lenet.onnx.
```

22.3.6 模拟推理和导出 RKNN 模型

使用 RKNN-Toolkit2 工具进行模型的转换和模拟推理：

列表 2: 配套代码 paddlepaddle/rknn_transfer.py

```
1 INPUT_SIZE = 28
2 IMG_PATH = 'test.jpg'
3
4 # 创建 RKNN 执行对象, 打印信息
5 #rknn = RKNN(verbose=True)
6 rknn = RKNN()
7
8 # 配置模型输入, 用于 NPU 对数据输入的预处理
9 # mean_values: 输入的均值
10 # std_values: 输入的归一化值
11 # target_platform: 目标平台
12 # 更多设置参考下 RKNN toolkit2 用户指导手册
13 print('--> Config model')
14 rknn.config(mean_values=[127.5], std_values=[127.5], target_platform='rk3568
    ↪')
15
16 # 加载 ONNX 模型
17 print('--> Loading model')
18 rknn.load_onnx(model='./lenet.onnx')
19 print('done')
20
21 print('--> Building model')
22 ret = rknn.build(do_quantization=False)
23 print('done')
24
25 # 初始化 RKNN 运行环境, 运行测试
26 print('--> Init runtime environment')
```

(下页继续)

(续上页)

```
27 ret = rknn.init_runtime()
28 print('done')
29
30 # 输入图像处理
31 img = cv2.imread(IMG_PATH)
32 img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
33 img = cv2.resize(img, (28, 28))
34 img = np.expand_dims(img, 0)
35 img = np.expand_dims(img, 0)
36
37 # 模拟推理
38 print('--> Running model')
39 outputs = rknn.inference(inputs=[img], data_format='nchw')
40 print("result: ", outputs)
41 print("本次预测数字是:", np.argmax(outputs))
42
43 # 导出 rknn 模型文件
44 print('--> Export rknn model')
45 rknn.export_rknn('./lenet.rknn')
46 print('done')
47
48 # 释放 rknn
49 rknn.release()
```

简单转换和推理测试结果:

```
1 W __init__: rknn-toolkit2 version: 1.4.0-22dcfef4
2 --> Config model
3 --> Loading model
4 done
5 --> Building model
6 done
```

(下页继续)

```
7 --> Init runtime environment
8 W init_runtime: Target is None, use simulator!
9 done
10 --> Running model
11 Analysing : 100%|██████████████████████████████████████████| 11/11
   ↳[00:00<00:00, 1931.24it/s]
12 Preparing : 100%|██████████████████████████████████████████| 11/11
   ↳[00:00<00:00, 360.11it/s]
13 result: [array([[ -3.5175781,  -4.140625 ,  -1.8232422,   0.5727539,   2.
   ↳7480469,
          -1.9824219,  -7.9726562,  -0.5292969,   3.46875 ,   7.5039062]]],
         dtype=float32)]
14
15 本次预测数字是： 9
16
17 --> Export rknn model
18 done
```

22.4.1 安装 RKNN Toolkit Lite2 和相关库

```
# 板端安装相关的库
sudo apt update
sudo apt -y install python3-pil python3-numpy python3-pip git wget
sudo apt install python3-opencv
```

22.4.2 推理测试

使用前面转换出的 rknn 模型，然后编写测试程序（也可以直接从配套例程获取文件）：

列表 3: 配套代码 paddlepaddle/rknn_test.py

```
1  IMG_PATH = '9.jpg'
2  RKNN_MODEL = 'lenet.rknn'
3
4  # 创建 RKNN
5  rknn_lite = RKNNLite()
6
7  # 导入模型
8  print('--> Load RKNN model')
9  ret = rknn_lite.load_rknn(RKNN_MODEL)
10 if ret != 0:
11     print('Load RKNN model failed')
12     exit(ret)
13 print('done')
14
15 # Init runtime environment
16 print('--> Init runtime environment')
17 ret = rknn_lite.init_runtime()
18 if ret != 0:
19     print('Init runtime environment failed!')
20     exit(ret)
21 print('done')
22
23 # load image
24 img = cv2.imread(IMG_PATH)
25 img = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
26 img = cv2.resize(img, (28, 28))
27
```

(下页继续)

(续上页)

```
28 # runing model
29 print('--> Running model')
30 outputs = rknn_lite.inference(inputs=[img])
31 print("result: ", outputs)
32 print("本次预测的数字是:", np.argmax(outputs))
33
34 rknn_lite.release()
```

运行程序:

```
--> Load RKNN model
done
--> Init runtime environment
I RKNN: [11:37:03.185] RKNN Runtime Information: librknnrt version: 1.4.0
↳ (a10f100eb@2022-09-09T09:07:14)
I RKNN: [11:37:03.185] RKNN Driver Information: version: 0.7.2
I RKNN: [11:37:03.185] RKNN Model Information: version: 1, toolkit version:
↳ 1.4.0-22dcfef4 (compiler version: 1.4.0 (3b4520e4f@2022-09-05T20:52:35)),
↳ target: RKNPU lite, target platform: rk3568, framework name: ONNX,
↳ framework layout: NCHW
done
--> Running model
result: [array([[ -3.5429688 , -4.1445312 , -1.7900391 ,  0.57128906,  2.
↳ 7421875 ,
           -2.015625  , -7.9882812 , -0.52734375,  3.4882812 ,  7.5234375 ]],
        dtype=float32)]
本次预测的数字是: 9
```

22.5 总结

以上是一个简单的示例，是用 LeNet 对手写数字数据及 MNIST 进行分类，调用封装好的训练与测试接口快速完成模型的组建与预测，用于简单学习参考。如果想一步一步自己构建模型训练，可以参考下 [PaddlePaddle 零基础实践深度学习](#)。

22.6 参考链接

<https://www.paddlepaddle.org.cn/documentation/docs/zh/install/pip/linux-pip.html>

https://www.paddlepaddle.org.cn/documentation/docs/zh/guides/beginner/quick_start_cn.html

https://www.paddlepaddle.org.cn/documentation/docs/zh/api/index_cn.html

<https://baike.baidu.com/item/%E9%A3%9E%E6%A1%A8/23472642?fromtitle=PaddlePaddle&fromid=20110894&fr=aladdin>

第 23 章 花卉图像分类-TensorFlow

TensorFlow 是一个基于数据流编程（dataflow programming）的符号数学系统，被广泛应用于各类机器学习（machine learning）算法的编程实现，其前身是谷歌的神经网络算法库 DistBelief。

本章将简单介绍 TensorFlow 的一个花卉图像分类任务，使用使用 `tf.keras.Sequential` 模型，简单构建模型，最后转换成 RKNN 模型部署到鲁班猫 RK 系列板卡上。

提示：测试环境：鲁班猫板卡使用 Debian10，PC 端是 ubuntu20.04。TensorFlow 版本 2.6.2，rknn-Toolkit2 版本 1.4.0。

23.1 安装 tensorflow 环境

在 PC 使用 Ubuntu 或者 Windows 系统安装 tensorflow，下面示例是在 ubuntu20.04 系统中：

```
# 安装 python, 更新 pip, 需要使用 Python 3.6-3.9 和 pip 19.0 及更高版本
sudo apt update
sudo apt install python3-dev python3-pip python3-venv
sudo python3 -m pip install pip --upgrade

# 一般都创建一个虚拟环境使用
sudo python3 -m venv .tensorflow_venv

# 直接命令安装最新稳定版 tensorflow, 也可以指定下版本 (GPU 和 CPU 合并)
pip3 install tensorflow
```

详细安装步骤和要求可以参考下 [TensorFlow 教程](#)。

提示：在 Windows 系统（Windows64 位操作系统）中，需要安装 Python，VC++，[Anaconda](#) 等。如需支持 NVIDIA GPU，需要安装 GPU 驱动程序，CUDA，cuDNN，关于他们之间的版本对应参考下 [这里](#)，显卡驱动与 CUDA 版本之间对应参考下 [这里](#)，网上有很多安装教程，根据自己设备，搜索较新日期文档即可。

23.2 图像分类

下面将介绍一个简单示例，使用 `tf.keras.Sequential` 模型对花卉图像进行分类。详情参考下 [这里](#)

23.2.1 准备数据集

下载数据集，包括 3700 张图像，使用 `tf.keras.utils.image_dataset_from_directory` 分 80% 用于测试集，剩下 20% 用于验证测试集。

列表 1: `tensorflow_classification.py`(参考配套例程)

```
1 # 获取数据
2 import pathlib
3 dataset_url = "https://storage.googleapis.com/download.tensorflow.org/
   ↳example_images/flower_photos.tgz"
4 data_dir = tf.keras.utils.get_file('flower_photos', origin=dataset_url,
   ↳untar=True)
5 data_dir = pathlib.Path(data_dir)
6
7 # 设置参数
8 batch_size = 32
9 img_height = 180
10 img_width = 180
11
```

(下页继续)

(续上页)

```
12 # 使用 tf.keras.utils 导入划分数据集，分为训练集和验证集
13 train_ds = tf.keras.utils.image_dataset_from_directory(
14     data_dir,
15     validation_split=0.2,
16     subset="training",
17     seed=123,
18     image_size=(img_height, img_width),
19     batch_size=batch_size)
20
21 val_ds = tf.keras.utils.image_dataset_from_directory(
22     data_dir,
23     validation_split=0.2,
24     subset="validation",
25     seed=123,
26     image_size=(img_height, img_width),
27     batch_size=batch_size)
28
29 class_names = train_ds.class_names
30 #print(class_names)
31
32 # 对数据进行处理
33 normalization_layer = layers.Rescaling(1./255)
34 train_ds = train_ds.map(lambda x, y: (normalization_layer(x), y))
35 val_ds = val_ds.map(lambda x, y: (normalization_layer(x), y))
36 num_classes = len(class_names)
```

提示：Tensorflow 的一些 API 接口说明和使用参考下 https://tensorflow.google.cn/api_docs/python/tf

23.2.2 构建和训练模型

Keras Sequential 模型构建：主要包含三个卷积块（tf.keras.layers.Conv2D）组成，每个卷积块中都有一个最大池化层（tf.keras.layers.MaxPooling2D）。最后有一个全连接层（tf.keras.layers.Dense），都以 relu 作为激活函数。

列表 2: tensorflow_classification.py(参考配套例程)

```
1 # 构建网络结构
2 model = Sequential([
3     layers.Rescaling(1./255, input_shape=(img_height, img_width, 3)),
4     layers.Conv2D(16, 3, padding='same', activation='relu'),
5     layers.MaxPooling2D(),
6     layers.Conv2D(32, 3, padding='same', activation='relu'),
7     layers.MaxPooling2D(),
8     layers.Conv2D(64, 3, padding='same', activation='relu'),
9     layers.MaxPooling2D(),
10    layers.Flatten(),
11    layers.Dense(128, activation='relu'),
12    layers.Dense(num_classes)
13 ])
14
15 # 配置优化器 'adam', 损失函数, 评价指标
16 model.compile(optimizer='adam',
17               loss=tf.keras.losses.SparseCategoricalCrossentropy(from_
18 ↪ logits=True),
19               metrics=['accuracy'])
20
21 # 查看各层网络结构
22 model.summary()
23
24 # 训练模型
25 epochs=10
```

(下页继续)

(续上页)

```

25 model.fit(
26     train_ds,
27     validation_data=val_ds,
28     epochs=epochs,
29 )

```

列表 3: 查看各层网络结构

Model: "sequential_1"

Layer (type)	Output Shape	Param #
=====		
rescaling (Rescaling)	(None, 180, 180, 3)	0
conv2d (Conv2D)	(None, 180, 180, 16)	448
max_pooling2d (MaxPooling2D)	(None, 90, 90, 16)	0
conv2d_1 (Conv2D)	(None, 90, 90, 32)	4640
max_pooling2d_1 (MaxPooling2D)	(None, 45, 45, 32)	0
conv2d_2 (Conv2D)	(None, 45, 45, 64)	18496
max_pooling2d_2 (MaxPooling2D)	(None, 22, 22, 64)	0
flatten (Flatten)	(None, 30976)	0
dense (Dense)	(None, 128)	3965056
dense_1 (Dense)	(None, 5)	645
=====		

(下页继续)

(续上页)

```
Total params: 3,989,285
Trainable params: 3,989,285
Non-trainable params: 0
```

```
92/92 [=====] - 65s 644ms/step - loss: 1.4799 -  
  accuracy: 0.3689 - val_loss: 1.1546 - val_accuracy: 0.4877  
Epoch 2/10  
92/92 [=====] - 52s 560ms/step - loss: 1.0406 -  
  accuracy: 0.5777 - val_loss: 1.0439 - val_accuracy: 0.5913  
Epoch 3/10  
92/92 [=====] - 51s 555ms/step - loss: 0.8372 -  
  accuracy: 0.6764 - val_loss: 1.0274 - val_accuracy: 0.6022  
Epoch 4/10  
92/92 [=====] - 51s 552ms/step - loss: 0.6344 -  
  accuracy: 0.7698 - val_loss: 1.0563 - val_accuracy: 0.5831  
Epoch 5/10  
92/92 [=====] - 51s 553ms/step - loss: 0.4266 -  
  accuracy: 0.8539 - val_loss: 1.2350 - val_accuracy: 0.6104  
Epoch 6/10  
92/92 [=====] - 51s 553ms/step - loss: 0.2695 -  
  accuracy: 0.9087 - val_loss: 1.4331 - val_accuracy: 0.6213  
Epoch 7/10  
92/92 [=====] - 51s 554ms/step - loss: 0.1561 -  
  accuracy: 0.9533 - val_loss: 1.6564 - val_accuracy: 0.5981  
Epoch 8/10  
92/92 [=====] - 51s 554ms/step - loss: 0.0725 -  
  accuracy: 0.9785 - val_loss: 2.0034 - val_accuracy: 0.6281  
Epoch 9/10  
92/92 [=====] - 51s 556ms/step - loss: 0.0440 -  
  accuracy: 0.9888 - val_loss: 2.1280 - val_accuracy: 0.6076
```

(下页继续)

(续上页)

```
Epoch 10/10
92/92 [=====] - 51s 555ms/step - loss: 0.0194 -
→accuracy: 0.9973 - val_loss: 2.4841 - val_accuracy: 0.5886
```

从上面训练结果看，训练精度 (accuracy) 随时间线性增加，达到 99%，而验证精度 (val_accuracy) 在 60% 左右。训练精度和验证精度之间差异过大是过拟合现象。简单的说就是该模型在训练数据上能够获得比其他数据上训练效果更好，但是在训练数据外的数据集上却达不到效果，意味着模型很难在新数据集上推广。

一般产生过拟合原因是：数据集样本数过少，样本中数据噪声干扰过大，学习模型过于复杂等。从这里的训练看，可能是训练数据集过少，模型记住了噪声等，而忽略了真实的输入输出间的关系，导致过度拟合。

我们这里扩展数据集的规模，在 Keras 预处理层进行随机翻转、旋转、缩放图像，使用 `tf.keras.layers.RandomFlip`、`tf.keras.layers.RandomRotation` 和 `tf.keras.layers.RandomZoom` 接口。另外，添加 Dropout 层，对于神经网络单元，按照一定的概率将其暂时从网络中丢弃。程序示例如下：

列表 4: tensorflow_classification.py(参考配套例程)

```
1 # 构建网络结构，添加 data_augmentation，预处理层
2 data_augmentation = keras.Sequential(
3     [
4         layers.RandomFlip("horizontal",
5                             input_shape=(img_height,
6                                             img_width,
7                                             3)),
8         layers.RandomRotation(0.1),
9         layers.RandomZoom(0.1),
10    ]
11 )
12
13 model = Sequential([
14     data_augmentation, # 添加 data_augmentation
```

(下页继续)

(续上页)

```
15     layers.Conv2D(16, 3, padding='same', activation='relu'),
16     layers.MaxPooling2D(),
17     layers.Conv2D(32, 3, padding='same', activation='relu'),
18     layers.MaxPooling2D(),
19     layers.Conv2D(64, 3, padding='same', activation='relu'),
20     layers.MaxPooling2D(),
21     layers.Dropout(0.2),    # 添加 Dropout 层
22     layers.Flatten(),
23     layers.Dense(128, activation='relu'),
24     layers.Dense(num_classes, name="outputs")
25 ])
```

简单测试结果:

```
Epoch 1/15
2023-02-20 19:39:40.074247: I tensorflow/stream_executor/cuda/cuda_dnn.
  cc:369] Loaded cuDNN version 8202
92/92 [=====] - 115s 759ms/step - loss: 1.3321 -
  accuracy: 0.4179 - val_loss: 1.0552 - val_accuracy: 0.5872
Epoch 2/15
92/92 [=====] - 58s 635ms/step - loss: 1.0306 -
  accuracy: 0.5978 - val_loss: 0.9923 - val_accuracy: 0.6267
Epoch 3/15
92/92 [=====] - 58s 635ms/step - loss: 0.9465 -
  accuracy: 0.6345 - val_loss: 0.9616 - val_accuracy: 0.6240
Epoch 4/15
92/92 [=====] - 61s 661ms/step - loss: 0.8700 -
  accuracy: 0.6604 - val_loss: 0.8555 - val_accuracy: 0.6717
Epoch 5/15
92/92 [=====] - 61s 665ms/step - loss: 0.7899 -
  accuracy: 0.6993 - val_loss: 0.8514 - val_accuracy: 0.6717
Epoch 6/15
```

(下页继续)

(续上页)

```
92/92 [=====] - 57s 620ms/step - loss: 0.7789 -  
→accuracy: 0.6979 - val_loss: 0.7658 - val_accuracy: 0.6907  
Epoch 7/15  
92/92 [=====] - 59s 640ms/step - loss: 0.7197 -  
→accuracy: 0.7159 - val_loss: 0.7940 - val_accuracy: 0.6826  
Epoch 8/15  
92/92 [=====] - 59s 644ms/step - loss: 0.6956 -  
→accuracy: 0.7367 - val_loss: 0.7710 - val_accuracy: 0.6907  
Epoch 9/15  
92/92 [=====] - 60s 657ms/step - loss: 0.6532 -  
→accuracy: 0.7510 - val_loss: 0.7250 - val_accuracy: 0.7112  
Epoch 10/15  
92/92 [=====] - 62s 672ms/step - loss: 0.6189 -  
→accuracy: 0.7657 - val_loss: 0.7893 - val_accuracy: 0.7044  
Epoch 11/15  
92/92 [=====] - 60s 652ms/step - loss: 0.6221 -  
→accuracy: 0.7674 - val_loss: 0.6982 - val_accuracy: 0.7262  
Epoch 12/15  
92/92 [=====] - 60s 651ms/step - loss: 0.5598 -  
→accuracy: 0.7888 - val_loss: 0.6821 - val_accuracy: 0.7357  
Epoch 13/15  
92/92 [=====] - 60s 649ms/step - loss: 0.5519 -  
→accuracy: 0.7885 - val_loss: 0.7939 - val_accuracy: 0.7084  
Epoch 14/15  
92/92 [=====] - 61s 667ms/step - loss: 0.5387 -  
→accuracy: 0.7997 - val_loss: 0.8331 - val_accuracy: 0.6880  
Epoch 15/15  
92/92 [=====] - 60s 653ms/step - loss: 0.5068 -  
→accuracy: 0.8151 - val_loss: 0.6627 - val_accuracy: 0.7466
```

经过在 Keras 预处理层和添加 Dropout 层后，训练后的

23.2.3 测试模型

列表 5: tensorflow_classification.py(参考配套例程)

```
1 # 获取导入图片
2 sunflower_url = "https://storage.googleapis.com/download.tensorflow.org/
   ↳example_images/592px-Red_sunflower.jpg"
3 sunflower_path = tf.keras.utils.get_file('Red_sunflower', origin=sunflower_
   ↳url)
4
5 img = tf.keras.utils.load_img(
6     sunflower_path, target_size=(img_height, img_width)
7 )
8 img_array = tf.keras.utils.img_to_array(img)
9 img_array = tf.expand_dims(img_array, 0)
10
11 # 测试
12 predictions = model.predict(img_array)
13 score = tf.nn.softmax(predictions[0])
14
15 # 输出结果
16 print(
17     "This image most likely belongs to {} with a {:.2f} percent confidence."
18     .format(class_names[np.argmax(score)], 100 * np.max(score))
19 )
```

23.2.4 TensorFlow Lite 模型

经过训练的 Keras Sequential 模型，这里使用 `tf.lite.TFLiteConverter.from_Keras_model` 生成 TensorFlow lite 模型：

列表 6: tensorflow_classification.py(参考配套例程)

```
1  # Convert the model.
2  converter = tf.lite.TFLiteConverter.from_keras_model(model)
3  tflite_model = converter.convert()
4
5  # Save the model.
6  with open('model.tflite', 'wb') as f:
7  f.write(tflite_model)
```

TensorFlow Lite 转换器转换 workflow 介绍参考下 [教程](#)

23.2.5 模型转换和模拟测试

使用 rknn-Toolkit2 导出 RKNN 模型:

列表 7: rknn_transfer.py(参考配套例程)

```
1  img_height = 180
2  img_width = 180
3  IMG_PATH = 'test.jpg'
4  class_names = ['daisy', 'dandelion', 'roses', 'sunflowers', 'tulips']
5
6  if __name__ == '__main__':
7
8      # Create RKNN object
9      #rknn = RKNN(verbose='Debug')
10     rknn = RKNN()
11
12     # Pre-process config
13     print('--> Config model')
14     rknn.config(mean_values=[0, 0, 0], std_values=[255, 255, 255], target_
    platform='rk3568')
```

(下页继续)

(续上页)

```
15     print('done')
16
17     # Load model
18     print('--> Loading model')
19     ret = rknn.load_tflite(model='model.tflite')
20     if ret != 0:
21         print('Load model failed!')
22         exit(ret)
23     print('done')
24
25     # Build model
26     print('--> Building model')
27     ret = rknn.build(do_quantization=False)
28     #ret = rknn.build(do_quantization=True, dataset='./dataset.txt')
29     if ret != 0:
30         print('Build model failed!')
31         exit(ret)
32     print('done')
33
34     # Export rknn model
35     print('--> Export rknn model')
36     ret = rknn.export_rknn('./model.rknn')
37     if ret != 0:
38         print('Export rknn model failed!')
39         exit(ret)
40     print('done')
41
42     #Init runtime environment
43     print('--> Init runtime environment')
44     ret = rknn.init_runtime()
45     #     if ret != 0:
46     #         print('Init runtime environment failed!')
```

(下页继续)

(续上页)

```
47 #         exit(ret)
48 print('done')
49
50 img = cv2.imread(IMG_PATH)
51 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
52 img = cv2.resize(img, (180, 180))
53 img = np.expand_dims(img, 0)
54
55 #print('--> Accuracy analysis')
56 #rknn.accuracy_analysis(inputs=['./test.jpg'])
57 #print('done')
58
59 print('--> Running model')
60 outputs = rknn.inference(inputs=[img])
61 print(outputs)
62 outputs = tf.nn.softmax(outputs)
63 print(outputs)
64
65 print(
66     "This image most likely belongs to {} with a {:.2f} percent confidence."
67     .format(class_names[np.argmax(outputs)], 100 * np.max(outputs))
68 )
69 #print(" 图像预测是:", class_names[np.argmax(outputs)])
70 print('done')
71
72 rknn.release()
```

测试结果显示:

```
W __init__: rknn-toolkit2 version: 1.4.0-22dcfef4
--> Config model
done
```

(下页继续)

(续上页)

```
3  img_height = 180
4  img_width = 180
5  class_names = ['daisy', 'dandelion', 'roses', 'sunflowers', 'tulips']
6
7  # Create RKNN object
8  rknn_lite = RKNNLite()
9
10 # load RKNN model
11 print('--> Load RKNN model')
12 ret = rknn_lite.load_rknn(RKNN_MODEL)
13 if ret != 0:
14     print('Load RKNN model failed')
15     exit(ret)
16 print('done')
17
18 # Init runtime environment
19 print('--> Init runtime environment')
20 ret = rknn_lite.init_runtime()
21 if ret != 0:
22     print('Init runtime environment failed!')
23     exit(ret)
24 print('done')
25
26 # load image
27 img = cv2.imread(IMG_PATH)
28 img = cv2.resize(img, (180, 180))
29 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
30 img = np.expand_dims(img, 0)
31
32 # runing model
33 print('--> Running model')
34 outputs = rknn_lite.inference(inputs=[img])
```

(下页继续)

(续上页)

```
35 print("result: ", outputs)
36 print(
37     "This image most likely belongs to {}".format(class_names[np.argmax(outputs)])
38 )
39
40
41 rknn_lite.release()
```

板卡上简单测试结果:

```
--> Load RKNN model
done
--> Init runtime environment
I RKNN: [17:28:01.594] RKNN Runtime Information: librknnrt version: 1.4.0
↳ (a10f100eb@2022-09-09T09:07:14)
I RKNN: [17:28:01.594] RKNN Driver Information: version: 0.7.2
I RKNN: [17:28:01.595] RKNN Model Information: version: 1, toolkit version:
↳ 1.4.0-22dcfef4(compiler version: 1.4.0 (3b4520e4f@2022-09-05T20:52:35)),
↳ target: RKNPU lite, target platform: rk3568, framework name: TFLite,
↳ framework layout: NHWC
done
--> Running model
result: [array([[ -3.7226562,  -1.2607422,  -0.5805664,   3.5585938,   0.296875
↳ ]],
        dtype=float32)]
This image most likely belongs to sunflowers.
done
```

23.4 总结

使用 Tensorflow 完成了一个简单花卉图像分类，并将模型转换为 RKNN 模型，部署到鲁班猫板上，模型验证精度在 70% 多，用于简单学习，该模型测试来自于 Tensorflow 官网的 [图像分类教程](#)。

23.5 参考链接

<https://tensorflow.google.cn/learn?hl=zh-cn>

<https://tensorflow.google.cn/tutorials/images/classification>

<https://tensorflow.google.cn/tutorials?hl=zh-cn>

第 24 章 ResNet18 网络-PyTorch

ResNet18 是一种卷积神经网络，它有 18 层深度，其中包括带有权重的卷积层和全连接层。它是 ResNet 系列网络的一个变体，使用了残差连接（residual connection）来解决深度网络的退化问题。

本章将简单介绍下 PyTorch 以及安装环境，然后简单分析下一个 ResNet 神经网络以及 PyTorch 的源码实现，最后我们使用 PyTorch 简单构建一个 ResNet18 网络对 Cifar-10 进行分类，并部署到鲁班猫板卡上。

提示：测试环境：鲁班猫板卡使用 Debian10，PC 是 WSL2 (ubuntu20.04)。PyTorch 是 1.10.1 CPU 版，rknn-Toolkit2 版本 1.4.0。

24.1 PyTorch 和 ResNet18

PyTorch 是一个开源的深度学习框架，该框架由 Facebook 人工智能研究院的 Torch7 团队开发，它的底层基于 Torch，但实现与运用全部是由 python 来完成。该框架主要用于人工智能领域的科学研究与应用开发。

24.1.1 PyTorch 安装

PyTorch 需要根据自己的环境安装，进入 PyTorch 官网，查看详细的安装教程。下面示例在 ubuntu20.04 上简单安装：

先到 [PyTorch 官网](https://pytorch.org/)，选择你对应环境的，比如下面是选中 linux 系统，python 语言，cpu 版本的 PyTorch。

PyTorch Build	Stable (1.13.1)		Preview (Nightly)	
Your OS	Linux	Mac		Windows
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.6	CUDA 11.7	ROCm 5.2	CPU
Run this Command:	<pre>pip3 install torch torchvision torchaudio --extra-index-url https://download.pytorch.org/whl/cpu</pre>			

该页面上面默认是最新稳定版的 PyTorch，安装以前版本可以点击该页面下面的 Previous version of PyTorch 或者直接点击 [这里](#)。

```
# 使用 pytorch, 需要先安装 python3 和 pip 基础环境, 这些可以自行搜索下即可。
# CPU 版:
pip3 install torch torchvision torchaudio --extra-index-url https://
↪download.pytorch.org/whl/cpu

# GPU 版 (CUDA 11.6):
pip3 install torch torchvision torchaudio --extra-index-url https://
↪download.pytorch.org/whl/cu116
```

安装 GPU 版本 (英伟达 GPU 显卡), 需要先根据自己的显卡安装或者更新 [显卡驱动](#), 然后安装 [CUDA 工具包](#), [cuDNN](#) 看自己情况安装, 最后点击 [这里](#) 查看 Pytorch 和 CUDA 的版本对应, 验证是否安装成功:

```
# 测试安装, 终端输入 python,
>>> import torch
>>> torch.__version__
'1.10.1+cu102'
```

(下页继续)

(续上页)

```
# 如果安装的是 CUDA 版本的 PyTorch, 命令检测 PyTorch 的安装版本以及绑定的 CUDA 版本等
>>> torch.version.cuda
'10.2'
>>> torch.cuda.is_available()
True
# PyTorch 的安装根据自己的实际环境和需求, 也可以使用 docker 等环境。
```

关于程序编辑工具, 你可以使用 Sublime Text, PyCharm, Vim 等, 这里测试环境是使用 WSL2 (ubuntu20.04), 编辑工具使用 Jupyter Notebook, 安装教程可以参考下 [这里](#), 在 Linux 系统上使用是类似的。

24.1.2 ResNet18 结构简介

ResNet (Residual Neural Network) 由微软研究院的 Kaiming He 等人在 2015 年提出, ResNet 的结构可以极快的加速神经网络的训练, 模型的准确率也有比较大的提升。

ResNet 是一种残差网络, 可以把它理解为一个子网络, 这个子网络经过堆叠可以构成一个很深的网络。ResNet 系列有多种变体, 如 ResNet18, ResNet34, ResNet50, ResNet101 和 ResNet152, 其网络结构如下 (参考 [论文](#)):

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

这里我们主要看下 ResNet18, ResNet18 基本含义是网络的基本架构是 ResNet, 网络的深度是 18

层，是带有权重的 18 层，不包括 BN 层，池化层。ResNet18 使用的基本残差单元，每个单元由两个 3x3 卷积层组成，中间有一个 BN 层和一个 ReLU 激活函数。

24.1.3 PyTorch 中的 ResNet18 实现

PyTorch 中的 ResNet18 源码实现：<https://github.com/pytorch/vision/blob/main/torchvision/models/resnet.py>

24.2 ResNet18 实现

24.2.1 数据集准备和数据预处理

接下来我们将自定义一个 ResNet18 网络结构，并使用 CIFAR-10 数据集进行简单测试。CIFAR-10 数据集由 10 个类别的 60000 张 32x32 彩色图像组成，每个类别有 6000 张图像，总共分为 50000 张训练图像和 10000 张测试图像。

列表 1: resnet18.py(截取部分，参考配套例程)

```
1 # 导入下载的数据集，使用 torchvision 加载训练集和测试集，
2 # 其中参数 download=True 表示从互联网下载数据，存放到./data 目录下，也可以自己下载放到
   指定目录下。
3 train_dataset = torchvision.datasets.CIFAR10('./data', download=True,
   ↳train=True, transform=transform_train)
4 test_dataset = torchvision.datasets.CIFAR10('./data', download=True,
   ↳train=False, transform=transform_test)
5
6 train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_
   ↳size, shuffle=True)
7 test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_
   ↳size, shuffle=False)
8
```

(下页继续)

(续上页)

```
9 # classes = ('plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',  
↪ 'ship', 'truck')
```

对划分的数据集进行预处理

列表 2: resnet18.py(截取部分, 参考配套例程)

```
1 # 预处理  
2 transform_train=torchvision.transforms.Compose([  
3     torchvision.transforms.Pad(4),  
4     torchvision.transforms.RandomHorizontalFlip(), # 图像一半的概率翻转, 一  
    半的概率不翻转  
5     torchvision.transforms.RandomCrop(32), # 图像随机裁剪成 32*32  
6     torchvision.transforms.ToTensor(), # 转为 Tensor 把灰度范围从 0-255 变  
    换到 0-1, 归一化  
7     #torchvision.transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, ↪  
    ↪0.1994, 0.2010))  
8     torchvision.transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5]) # 归  
    一化用到的均值和方差  
9 ])  
10 transform_test=torchvision.transforms.Compose([  
11     torchvision.transforms.ToTensor(),  
12     #torchvision.transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, ↪  
    ↪0.1994, 0.2010))  
13     torchvision.transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5]) # 归一  
    化用到的均值和方差  
14 ])
```

24.2.2 构建模型

列表 3: resnet18.py(截取部分, 参考配套例程)

```
1  # 残差块实现
2  class ResidualBlock(nn.Module):
3  def __init__(self, in_channels, out_channels, stride=1, downsample=None):
4      super(ResidualBlock, self).__init__()
5      self.conv1 = conv3x3(in_channels, out_channels, stride)
6      self.bn1 = nn.BatchNorm2d(out_channels)
7      self.relu = nn.ReLU(inplace=True)           # 原地替换 节省内存开销
8      self.conv2 = conv3x3(out_channels, out_channels)
9      self.bn2 = nn.BatchNorm2d(out_channels)
10     self.downsample = downsample                # shortcut
11 def forward(self, x):
12     residual=x
13     out = self.conv1(x)
14     out = self.bn1(out)
15     out = self.relu(out)
16     out = self.conv2(out)
17     out = self.bn2(out)
18     if(self.downsample):
19         residual = self.downsample(x)
20     out += residual
21     out = self.relu(out)
22     return out
23
24 # 自定义一个神经网络, 使用 nn.model,, 通过__init__ 初始化每一层神经网络。
25 # 使用 forward 连接数据
26 class ResNet(torch.nn.Module):
27     def __init__(self, block, layers, num_classes):
28         super(ResNet, self).__init__()
29         self.in_channels = 16
```

(下页继续)

(续上页)

```
30     self.conv = conv3x3(3, 16)
31     self.bn = torch.nn.BatchNorm2d(16)
32     self.relu = torch.nn.ReLU(inplace=True)
33     self.layer1 = self._make_layers(block, 16, layers[0])
34     self.layer2 = self._make_layers(block, 32, layers[1], 2)
35     self.layer3 = self._make_layers(block, 64, layers[2], 2)
36     self.layer4 = self._make_layers(block, 128, layers[3], 2)
37     self.avg_pool = torch.nn.AdaptiveAvgPool2d((1, 1))
38     self.fc = torch.nn.Linear(128, num_classes)
39
40     # _make_layers 函数重复残差块, 以及 shortcut 部分
41     def _make_layers(self, block, out_channels, blocks, stride=1):
42         downsample = None
43         if (stride != 1) or (self.in_channels != out_channels):      # 卷积核
为 1 进行升降维
44             downsample = torch.nn.Sequential(                        # _
↳ stride==2 的时候 也就是每次输出信道升维的时候
45                 conv3x3(self.in_channels, out_channels, stride=stride),
46                 torch.nn.BatchNorm2d(out_channels)
47             )
48             layers = []
49             layers.append(block(self.in_channels, out_channels, stride,
↳ downsample))
50             self.in_channels = out_channels
51             for i in range(1, blocks):
52                 layers.append(block(out_channels, out_channels))
53             return torch.nn.Sequential(*layers)
54
55     def forward(self, x):
56         out = self.conv(x)
57         out = self.bn(out)
58         out = self.relu(out)
```

(下页继续)

(续上页)

```
59         out = self.layer1(out)
60         out = self.layer2(out)
61         out = self.layer3(out)
62         out = self.layer4(out)
63         out = self.avg_pool(out)
64         out = out.view(out.size(0), -1)
65         out = self.fc(out)
66         return out
67
68 # Make model, 使用 cpu
69 model=ResNet(ResidualBlock, [2,2,2,2], num_classes).to(device=device)
70
71 # 打印 model 结构
72 print(f"Model structure: {model}\n\n")
73
74 # Loss and optimizer
75 criterion = nn.CrossEntropyLoss() # 交叉熵损失函数
76 optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate) # 优化器随
    机梯度下降
```

测试时导出模型结构:

```
Model structure: ResNet(
  (conv): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1),
↪ bias=False)
  (bn): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_
↪ running_stats=True)
  (relu): ReLU(inplace=True)
  (layer1): Sequential(
    (0): ResidualBlock(
      (conv1): Conv2d(16, 16, kernel_size=(3, 3), stride=(1, 1),
↪ padding=(1, 1), bias=False)
```

(下页继续)

(续上页)

```
(bn1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_
↪running_stats=True)
(relu): ReLU(inplace=True)
(conv2): Conv2d(16, 16, kernel_size=(3, 3), stride=(1, 1), ↪
↪padding=(1, 1), bias=False)
(bn2): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_
↪running_stats=True)
)
(1): ResidualBlock(
(conv1): Conv2d(16, 16, kernel_size=(3, 3), stride=(1, 1), ↪
↪padding=(1, 1), bias=False)
(bn1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_
↪running_stats=True)
(relu): ReLU(inplace=True)
(conv2): Conv2d(16, 16, kernel_size=(3, 3), stride=(1, 1), ↪
↪padding=(1, 1), bias=False)
(bn2): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True, track_
↪running_stats=True)
)
)
(layer2): Sequential(
(0): ResidualBlock(
(conv1): Conv2d(16, 32, kernel_size=(3, 3), stride=(2, 2), ↪
↪padding=(1, 1), bias=False)
(bn1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_
↪running_stats=True)
(relu): ReLU(inplace=True)
(conv2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), ↪
↪padding=(1, 1), bias=False)
(bn2): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_
↪running_stats=True)
(downsample): Sequential(
```

(下页继续)

(续上页)

```
(0): Conv2d(16, 32, kernel_size=(3, 3), stride=(2, 2),  
↳padding=(1, 1), bias=False)  
  (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,  
↳track_running_stats=True)  
  )  
  )  
  (1): ResidualBlock(  
    (conv1): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1),  
↳padding=(1, 1), bias=False)  
    (bn1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_  
↳running_stats=True)  
    (relu): ReLU(inplace=True)  
    (conv2): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1),  
↳padding=(1, 1), bias=False)  
    (bn2): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_  
↳running_stats=True)  
  )  
  )  
  (layer3): Sequential(  
    (0): ResidualBlock(  
      (conv1): Conv2d(32, 64, kernel_size=(3, 3), stride=(2, 2),  
↳padding=(1, 1), bias=False)  
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_  
↳running_stats=True)  
      (relu): ReLU(inplace=True)  
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1),  
↳padding=(1, 1), bias=False)  
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_  
↳running_stats=True)  
      (downsample): Sequential(  
        (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(2, 2),  
↳padding=(1, 1), bias=False)
```

(下页继续)

(续上页)

```
(1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, ↵
↵track_running_stats=True)
)
)
(1): ResidualBlock(
  (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), ↵
↵padding=(1, 1), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_
↵running_stats=True)
  (relu): ReLU(inplace=True)
  (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), ↵
↵padding=(1, 1), bias=False)
  (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_
↵running_stats=True)
)
)
(layer4): Sequential(
  (0): ResidualBlock(
    (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), ↵
↵padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_
↵running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), ↵
↵padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_
↵running_stats=True)
    (downsample): Sequential(
      (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), ↵
↵padding=(1, 1), bias=False)
      (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, ↵
↵track_running_stats=True)
```

(下页继续)

(续上页)

```
)  
)  
(1): ResidualBlock(  
    (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1),  
↪padding=(1, 1), bias=False)  
    (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_  
↪running_stats=True)  
    (relu): ReLU(inplace=True)  
    (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1),  
↪padding=(1, 1), bias=False)  
    (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_  
↪running_stats=True)  
)  
)  
(avg_pool): AdaptiveAvgPool2d(output_size=(1, 1))  
(fc): Linear(in_features=128, out_features=10, bias=True)  
)
```

24.2.3 训练和测试模型

列表 4: resnet18.py(截取部分, 参考配套例程)

```
1 if __name__ == "__main__":  
2     # 训练模型  
3     total_step = len(train_loader)  
4     for epoch in range(0, num_epochs):  
5         for i, (images, labels) in enumerate(train_loader):  
6             images = images.to(device=device)  
7             labels = labels.to(device=device)  
8             outputs = model(images)  
9             loss = criterion(outputs, labels)
```

(下页继续)

(续上页)

```
10         # 梯度清零
11         optimizer.zero_grad()
12         # 反向传播
13         loss.backward()
14         # 更新参数
15         optimizer.step()
16         if (i+1) % total_step == 0:
17             print('Epoch [{}/{}], Step [{}/{}], Loss: {:.4f}'
18                   .format(epoch+1, num_epochs, i+1, total_step, loss.
19 ↪item()))
19         print("Finished Tranining")
```

```
print('\nTest the model')
# 转换到`eval`模式
model.eval() # eval mode (batchnorm uses moving mean/variance instead of ↪
↪mini-batch mean/variance)
with torch.no_grad():

    correct = 0
    total = 0

    for images, labels in test_loader:

        images = images.to(device=device)
        labels = labels.to(device=device)
        outputs = model(images)

        _, predicted = torch.max(outputs.data, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()
```

(下页继续)

(续上页)

```
print('Accuracy of the network on the 10000 test images: {} %'.  
↪format(100 * correct / total))
```

训练模型，后再测试集上测试准确率达 89.8400%:

```
Epoch [88/100], Step [391/391], Loss: 0.0820  
Epoch [89/100], Step [391/391], Loss: 0.0185  
Epoch [90/100], Step [391/391], Loss: 0.0166  
Epoch [91/100], Step [391/391], Loss: 0.0334  
Epoch [92/100], Step [391/391], Loss: 0.0641  
Epoch [93/100], Step [391/391], Loss: 0.0359  
Epoch [94/100], Step [391/391], Loss: 0.0994  
Epoch [95/100], Step [391/391], Loss: 0.0069  
Epoch [96/100], Step [391/391], Loss: 0.0722  
Epoch [97/100], Step [391/391], Loss: 0.0182  
Epoch [98/100], Step [391/391], Loss: 0.2182  
Epoch [99/100], Step [391/391], Loss: 0.0657  
Epoch [100/100], Step [391/391], Loss: 0.0501  
Finished Tranining  
  
Test the model  
在 10000 张测试集图片上的准确率:89.8400 %
```

24.2.4 保存为 onnx 模型

这里我们使用 `torch.onnx.export` 保存模型为 onnx 模型（也可以导出 pt 模型等）:

列表 5: resnet18.py(截取部分, 参考配套例程)

```
1 # export onnx (rknn-toolkit2 only support to opset_version=12)
2 x = torch.randn((1, 3, 32, 32))
3 torch.onnx.export(model, x, './resnet18_pytorch_100.onnx', opset_version=12,
  ↳ input_names=['input'], output_names=['output'])
```

24.2.5 导出 RKNN 模型和模拟测试

列表 6: resnet18.py(截取部分, 参考配套例程)

```
1 def show_perfs(perfs):
2     perfs = 'perfs: {}\n'.format(perfs)
3     print(perfs)
4
5 def softmax(x):
6     return np.exp(x) / sum(np.exp(x))
7
8 if __name__ == '__main__':
9
10     MODEL = './resnet18_pytorch.onnx'
11
12     # 创建 RKNN
13     # 如果测试遇到问题, 请开启 verbose=True, 查看调试信息。
14     # rknn = RKNN(verbose=True)
15     rknn = RKNN()
16
17     # 配置模型, 预处理
18     print('--> Config model')
19     rknn.config(mean_values=[125.307, 122.961, 113.8575], std_values=[51.
  ↳ 5865, 50.847, 51.255], target_platform='rk3568')
```

(下页继续)

(续上页)

```
20     print('done')
21
22     # 加载模型
23     print('--> Loading model')
24     #ret = rknn.load_pytorch(model=model, input_size_list=input_size_list)
25     ret = rknn.load_onnx(model=MODEL)
26     if ret != 0:
27         print('Load model failed!')
28         exit(ret)
29     print('done')
30
31     # 构建模型
32     print('--> Building model')
33     ret = rknn.build(do_quantization=False)
34     if ret != 0:
35         print('Build model failed!')
36         exit(ret)
37     print('done')
38
39     # 导出 rknn 模型
40     print('--> Export rknn model')
41     ret = rknn.export_rknn('./resnet_18_100.rknn')
42     if ret != 0:
43         print('Export rknn model failed!')
44         exit(ret)
45     print('done')
46
47     # 输入图片处理
48     img = cv2.imread('./0_125.jpg')
49     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
50     img = cv2.resize(img, (32, 32))
51     img = np.expand_dims(img, 0)
```

(下页继续)

(续上页)

```
52
53     # 初始化运行环境
54     print('--> Init runtime environment')
55     ret = rknn.init_runtime()
56     if ret != 0:
57         print('Init runtime environment failed!')
58         exit(ret)
59     print('done')
60
61     # 模拟推理
62     print('--> Running model')
63     outputs = rknn.inference(inputs=[img])
64     np.save('./pytorch_resnet18_qat_0.npy', outputs[0])
65     #show_outputs(softmax(np.array(outputs[0][0])))
66     print(outputs)
67     print('done')
68
69     rknn.release()
```

24.2.6 板端部署测试

24.2.6.1 简单测试

列表 7: rknnlite_inference0.py

```
1  IMG_PATH = '0_125.jpg'
2  RKNN_MODEL = './resnet_18_100.rknn'
3  img_height = 32
4  img_width = 32
5  class_names = ["plane", "car", "bird", "cat", "deer", "dog", "frog", "horse", "ship",
6  ↪               "truck"]
```

(下页继续)

(续上页)

```
6
7  # Create RKNN object
8  rknn_lite = RKNNLite()
9
10 # load RKNN model
11 print('--> Load RKNN model')
12 ret = rknn_lite.load_rknn(RKNN_MODEL)
13 if ret != 0:
14     print('Load RKNN model failed')
15     exit(ret)
16 print('done')
17
18 # Init runtime environment
19 print('--> Init runtime environment')
20 ret = rknn_lite.init_runtime()
21 if ret != 0:
22     print('Init runtime environment failed!')
23     exit(ret)
24 print('done')
25
26 # load image
27 img = cv2.imread(IMG_PATH)
28 img = cv2.resize(img, (32, 32))
29 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
30 img = np.expand_dims(img, 0)
31
32 # runing model
33 print('--> Running model')
34 outputs = rknn_lite.inference(inputs=[img])
35 print("result: ", outputs)
36 print(
37     "This image most likely belongs to {}."
```

(下页继续)

(续上页)

```
38     .format(class_names[np.argmax(outputs)])
39 )
40 rknn_lite.release()
```

测试结果:

```
--> Load RKNN model
done
--> Init runtime environment
I RKNN: [16:02:15.992] RKNN Runtime Information: librknnrt version: 1.4.0
↳ (a10f100eb@2022-09-09T09:07:14)
I RKNN: [16:02:15.992] RKNN Driver Information: version: 0.7.2
I RKNN: [16:02:15.992] RKNN Model Information: version: 1, toolkit version:
↳ 1.4.0-22dcfef4 (compiler version: 1.4.0 (3b4520e4f@2022-09-05T20:52:35)),
↳ target: RKNPU lite, target platform: rk3568, framework name: ONNX,
↳ framework layout: NCHW
done
--> Running model
result: [array([[ -2.0566406, -15.234375 ,  6.6835938, -6.828125 , -9.
↳ 9921875,
           -6.5390625, -5.671875 , -15.8515625, -17.96875 , -11.90625  ]],
        dtype=float32)]
This image most likely belongs to bird.
```

24.2.6.2 使用测试集图像测试

先转换测试集为.jpg 格式图片，然后传输到板卡进行部署测试：

列表 8: cifar10_to_jpg.py

```
1 # CIFAR-10 数据集所在的绝对路径, 根据具体路径修改
2 base_dir = "/mnt/e/Users/Administrator/Desktop/wsl_user/pytorch/"
3 data_dir = os.path.join(base_dir, "data", "cifar-10-batches-py")
4 test_o_dir = os.path.join( base_dir, "Data", "cifar-10-png", "raw_test")
5
6 # 解压缩
7 def unpickle(file):
8     with open(file, 'rb') as fo:
9         dict_ = pickle.load(fo, encoding='bytes')
10        return dict_
11
12 # 生成测试集图片
13 if __name__ == '__main__':
14     print("start...")
15     test_data_path = os.path.join(data_dir, "test_batch")
16     test_data = unpickle(test_data_path)
17     for i in range(0, 10000):
18         img = np.reshape(test_data[b'data'][i], (3, 32, 32))
19         img = img.transpose(1, 2, 0)
20
21         label_num = str(test_data[b'labels'][i])
22         o_dir = os.path.join(test_o_dir, label_num)
23         if not os.path.isdir(o_dir):
24             os.makedirs(o_dir)
25
26         img_name = label_num + '_' + str(i) + '.jpg'
27         img_path = os.path.join(o_dir, img_name)
28         imwrite(img_path, img)
29     print("done.")
```

然后修改板端测试文件添加:

列表 9: rknnlite_inference1.py

```
1 def rknn_inference(root):
2     total=0
3     correct=0
4     for path in os.listdir(root):
5         image_filenames = os.listdir(root + '/' + path)
6         for image_filename in image_filenames:
7             img = cv2.imread(root + '/' + path + '/' + image_filename)
8             img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
9             outputs = rknn_lite.inference(inputs=[img])
10            total += 1
11            if np.argmax(outputs) == int(path[:1]) :
12                correct += 1
13            print("correct={}, total={}".format(correct,total))
14            print('在{}张测试集图片上的准确率:{:.2f} %'.format(total,100 * correct /
    ↪total))
```

简单测试结果:

```
1 --> Load RKNN model
2 done
3 --> Init runtime environment
4 I RKNN: [10:23:15.384] RKNN Runtime Information: librknnrt version: 1.4.0_
    ↪(a10f100eb@2022-09-09T09:07:14)
5 I RKNN: [10:23:15.385] RKNN Driver Information: version: 0.7.2
6 I RKNN: [10:23:15.385] RKNN Model Information: version: 1, toolkit version:_
    ↪1.4.0-22dcfef4(compiler version: 1.4.0 (3b4520e4f@2022-09-05T20:52:35)),_
    ↪target: RKNPU lite, target platform: rk3568, framework name: ONNX,_
    ↪framework layout: NCHW
7 done
8 --> Running model
```

(下页继续)

(续上页)

```
9 在 10000 张测试集图片上的准确率:71.21 %  
10 done
```

24.3 参考

<https://pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html>

https://pytorch.org/hub/pytorch_vision_resnet/

<https://arxiv.org/abs/1512.03385>

第 25 章 YOLOv5

Yolov5 是一种目标检测算法，属于单阶段目标检测方法，是在 COCO 数据集上预训练的物体检测架构和模型系列，它代表了 Ultralytics 对未来视觉 AI 方法的开源研究，其中包含了经过数千小时的研究和开发而形成的经验教训和最佳实践。最新的 YOLOv5 v7.0 有 YOLOv5n、YOLOv5s、YOLOv5m、YOLOv5l、YOLOv5x 等，除了目标检测，还有分割，分类等应用场景。

YOLOv5 基本原理，简单的讲是：将整张图片划分为若干个网络，每个网格预测出该网格内物体的种类和位置信息，然后根据预测框与真实框之间的交并比值进行目标框的筛选，最终输出预测框。

本章将简单使用 YOLOv5，并在鲁班猫板卡上简单部署测试。

提示：测试环境：鲁班猫 RK 板卡系统是 Debian10，PC 是 WSL2 (ubuntu20.04)，PyTorch 是 1.10.1，rknn-Toolkit2 版本 1.4.0，YOLOv5 v7.0，airockchip/yolov5 v6.2。

25.1 YOLOv5 环境安装

安装 python 等环境，以及相关依赖库，然后克隆 YOLOv5 仓库的源码。

```
# 创建虚拟环境 (名称为.yolov5_env)，这里使用 venv，也可以先安装 anaconda，使用 conda 命令创建
sudo python3 -m venv .yolov5_env
source .yolov5_env/bin/activate

# 拉取最新的 yolov5 仓库，pytorch 版
git clone https://github.com/ultralytics/yolov5
cd yolov5
```

(下页继续)

(续上页)

```
# 安装依赖库
pip3 install -r requirements.txt

# 进入 python 命令行, 检测安装的环境
import torch
import utils
display = utils.notebook_init()

# 这里的测试环境显示:
Checking setup...
YOLOv5 □ 2023-2-20 Python-3.8.10 torch-1.10.1+cpu CPU
Setup complete □ (6 CPUs, 12.4 GB RAM, 77.3/251.0 GB disk)
```

25.2 YOLOv5 简单使用

25.2.1 获取预训练权重文件

下载 yolov5s.pt, yolov5m.pt, yolov5l.pt, yolov5x.pt 权重文件, 可以直接从 [这里](#) 获取。其中后面 n、s、m、l、x 表示网络的宽度和深度, 最小的是 n, 它速度最快、精度最低。

25.2.2 YOLOv5 简单测试

进入 yolov5 源码目录, 把前面下载的权重文件放到当前目录下, 两张测试图片位于 ./data/images/,

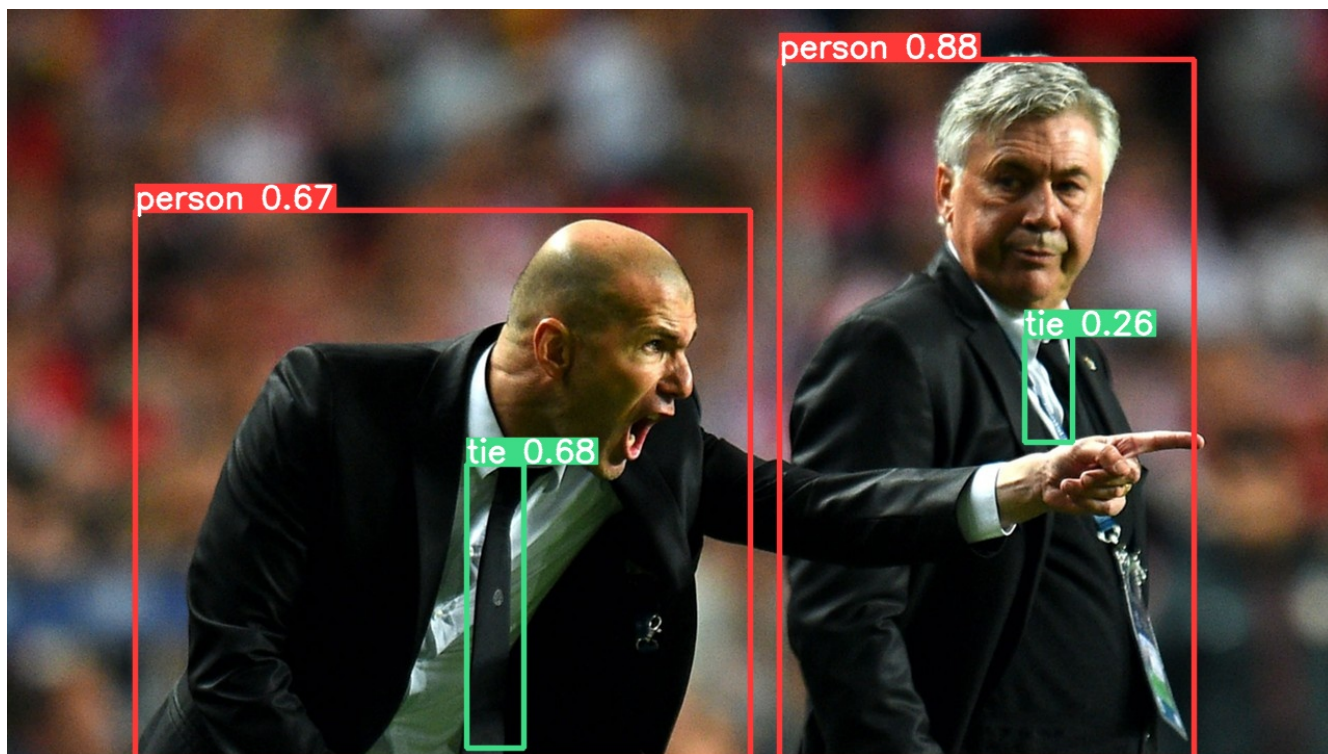
```
# 简单测试
# --source 指定测试数据, 可以是图片或者视频等
# --weights 指定权重文件路径, 可以是自己训练的, 或者直接用官网的
sudo python3 detect.py --source ./data/images/ --weights yolov5s.pt
```

测试结果：

```
(.yolov5_env) llh@YH-LONG:/mnt/e/Users/Administrator/Desktop/wsl_user/yolov5/yolov5-7.0$ python detect.py --source ./data/images/zidane.jpg --weights yolov5s.pt
detect: weights=['yolov5s.pt'], source=./data/images/zidane.jpg, data=data/coco128.yaml, imgsz=[640, 640], conf_thres=0.25, iou_thres=0.45, max_det=1000, device=, view_img=False, save_txt=False, save_conf=False, save_crop=False, nosave=False, classes=None, agnostic_nms=False, augment=False, visualize=False, update=False, project=runs/detect, name=exp, exist_ok=False, line_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False, vid_stride=1
YOLOv5 2023-2-28 Python-3.8.10 torch-1.10.1+cpu CPU

Fusing layers ...
YOLOv5s summary: 213 layers, 7225885 parameters, 0 gradients, 16.4 GFLOPs
image 1/1 /mnt/e/Users/Administrator/Desktop/wsl_user/yolov5/yolov5-7.0/data/images/zidane.jpg: 384x640 2 persons, 2 ties, 193.3ms
Speed: 2.3ms pre-process, 193.3ms inference, 4.0ms NMS per image at shape (1, 3, 640, 640)
Results saved to runs/detect/exp2
```

上面依次显示：基础配置、网络参数、检测的结果、处理速度、最后是保存的位置。我们切换到 runs/detect/exp2 目录，查看检测的结果：



25.2.3 转换为 rknn 模型

接下来将基于 yolov5s.pt，导出 rknn：

1、转成 yolov5s.onnx（也可以是 torchscript 等模型），先安装下 onnx 安装依赖环境：

```
# 安装下 onnx 的环境灯
pip3 install -r requirements.txt coremltools onnx onnx-simplifier
↪onnxruntime

# 获取权重文件，使用 yolov5s.pt
# 这里我们使用下面命令，导出 onnx 模型
sudo python3 export.py --weights yolov5s.pt --include onnx

# 或者使用下面命令，导出 torchscript
sudo python3 export.py --weights yolov5s.pt --include torchscript
```

模型导出显示：

```
(.yolov5_env) llh@YH-LONG:/mnt/e/Users/Administrator/Desktop/wsl_user/
↪yolov5/yolov5-7.0$ python export.py --weights yolov5s.pt --include onnx
export: data=data/coco128.yaml, weights=['yolov5s.pt'], imgsz=[640, 640],
↪batch_size=1, device=cpu, half=False,
inplace=False, keras=False, optimize=False, int8=False, dynamic=False,
↪simplify=False, opset=12, verbose=False,
workspace=4, nms=False, agnostic_nms=False, topk_per_class=100, topk_
↪all=100, iou_thres=0.45, conf_thres=0.25, include=['onnx']
YOLOv5 2023-2-28 Python-3.8.10 torch-1.10.1+cpu CPU

Fusing layers...
YOLOv5s summary: 213 layers, 7225885 parameters, 0 gradients, 16.4 GFLOPs

PyTorch: starting from yolov5s.pt with output shape (1, 25200, 85) (14.1 MB)
```

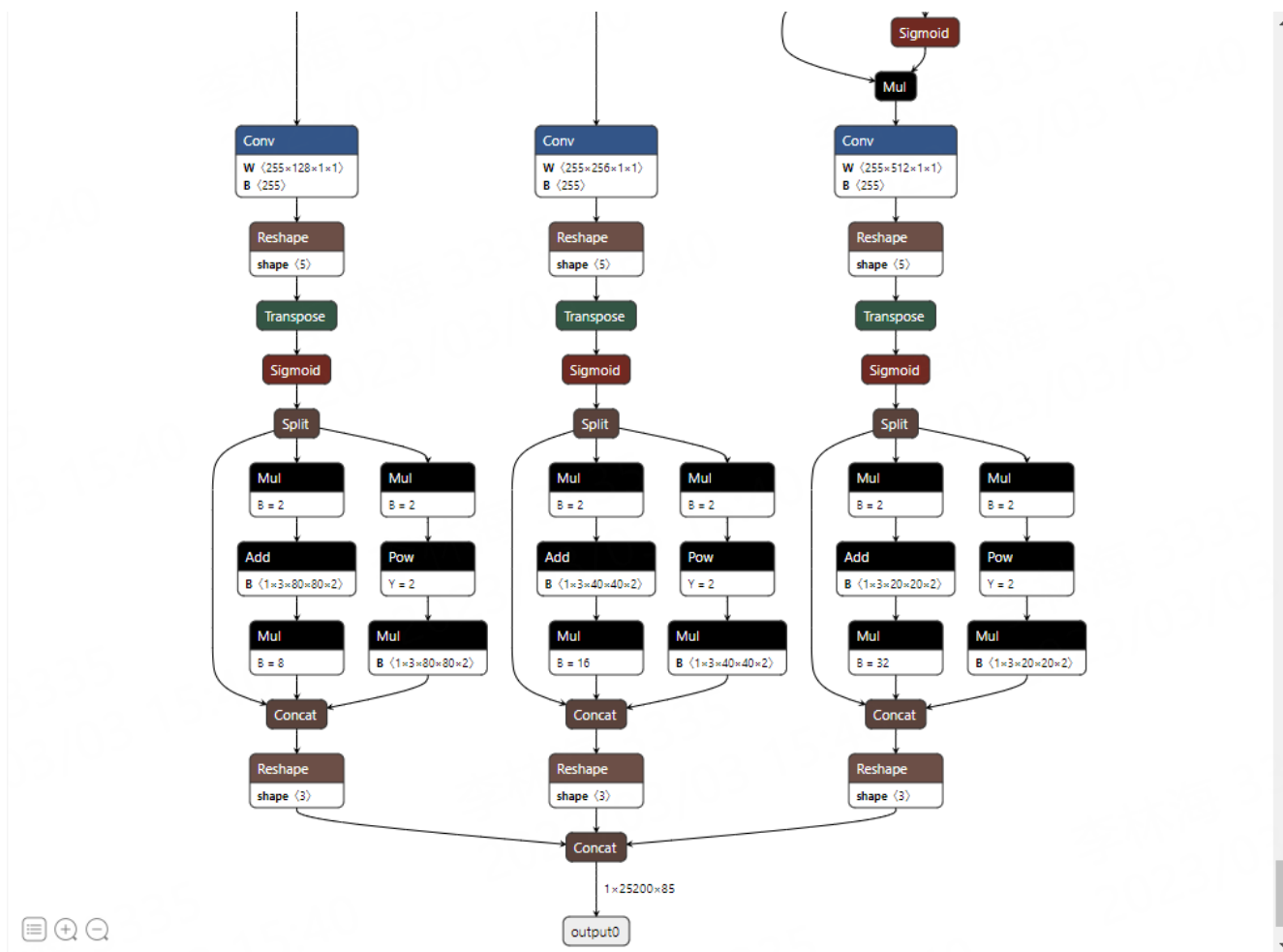
(下页继续)

(续上页)

```
ONNX: starting export with onnx 1.13.1...
ONNX: export success □ 1.8s, saved as yolov5s.onnx (28.0 MB)

Export complete (2.3s)
Results saved to /mnt/e/Users/Administrator/Desktop/wsl_user/yolov5/yolov5-
↪7.0
Detect:          python detect.py --weights yolov5s.onnx
Validate:        python val.py --weights yolov5s.onnx
PyTorch Hub:     model = torch.hub.load('ultralytics/yolov5', 'custom',
↪'yolov5s.onnx')
Visualize:       https://netron.app
```

会在当前目录下生成 yolov5s.onnx 文件。我们可以使用 [Netron](#) 可视化模型：



2、重新导出

看上面的图片，转出 onnx 模型尾部最后经过 Detect 层，为适配到 rknn 做适当处理，这里移除这个网络结构（注意模型结构尾部的 sigmoid 函数没有删除），直接输出三个特征图。

需要修改下 yolov5 文件源码：

列表 1: models/yolo.py

```
# 修改工程文件 models/yolo.py 中类 Detect 的 forward 函数为：
# def forward(self, x):
#     z = [] # inference output
#     for i in range(self.nl):
```

(下页继续)

(续上页)

```
#         x[i] = self.m[i](x[i]) # conv
#         bs, _, ny, nx = x[i].shape # x(bs,255,20,20) to x(bs,3,20,20,85)
#         x[i] = x[i].view(bs, self.na, self.no, ny, nx).permute(0, 1, 3, 4,
↪ 2).contiguous()

#         if not self.training: # inference
#             if self.dynamic or self.grid[i].shape[2:4] != x[i].shape[2:4]:
#                 self.grid[i], self.anchor_grid[i] = self._make_grid(nx,
↪ ny, i)

#             if isinstance(self, Segment): # (boxes + masks)
#                 xy, wh, conf, mask = x[i].split((2, 2, self.nc + 1, self.
↪ no - self.nc - 5), 4)
#                 xy = (xy.sigmoid() * 2 + self.grid[i]) * self.stride[i]
↪ # xy
#                 wh = (wh.sigmoid() * 2) ** 2 * self.anchor_grid[i] # wh
#                 y = torch.cat((xy, wh, conf.sigmoid(), mask), 4)
#             else: # Detect (boxes only)
#                 xy, wh, conf = x[i].sigmoid().split((2, 2, self.nc + 1),
↪ 4)
#                 xy = (xy * 2 + self.grid[i]) * self.stride[i] # xy
#                 wh = (wh * 2) ** 2 * self.anchor_grid[i] # wh
#                 y = torch.cat((xy, wh, conf), 4)
#                 z.append(y.view(bs, self.na * nx * ny, self.no))

#         return x if self.training else (torch.cat(z, 1), ) if self.export
↪ else (torch.cat(z, 1), x)

def forward(self, x):
    z = [] # inference output
    for i in range(self.nl):
        z.append(torch.sigmoid(self.m[i](x[i])))
```

(下页继续)

(续上页)

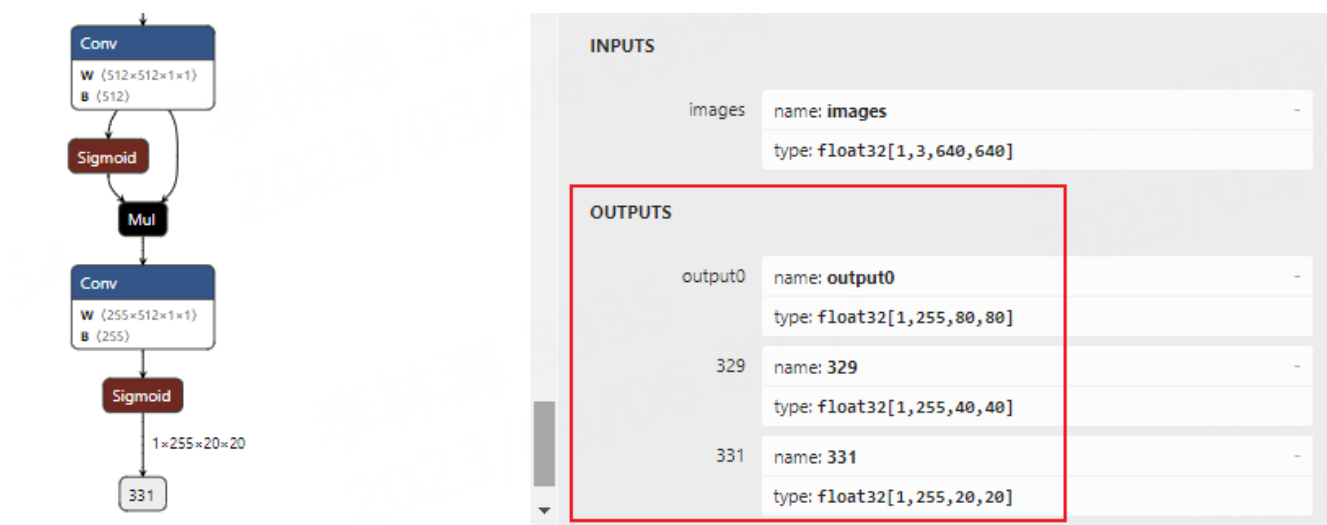
```
return z
```

列表 2: export.py

```
# 在 export.py 文件 run() 函数中修改行:

if half and not coreml:
    im, model = im.half(), model.half() # to FP16
- shape = tuple((y[0] if isinstance(y, tuple) else y).shape) # model_
↪output shape
+ shape = tuple((y[0] if (isinstance(y, tuple) or (isinstance(y, list)))
↪else y).shape) # model output shape
    metadata = {'stride': int(max(model.stride)), 'names': model.names} #
↪model metadata
    LOGGER.info(f"\n{colorstr('PyTorch:')} starting from {file} with output_
↪shape {shape} ({file_size(file):.1f} MB)")
```

修改之后重新执行命令转换: `python export.py --weights yolov5s.pt --include onnx` 使用 [Netron](#) 可视化模型:



3、转换成 rknn 模型

这里将前面的 onnx 模型转换成 rknn 模型，需要安装下 rknn-Toolkit2 等环境，参考下《NPU 使用》章节。

提示： 为避免其他问题，rknn-Toolkit2 的版本和板卡端 librknrt.so 库版本应该保持一致。

使用 rknn-Toolkit2 的模型转换功能：

列表 3: rknn_transfer.py(这里截取部分，参考配套例程)

```
1 if __name__ == '__main__':
2
3     # 创建 RKNN
4     # 如果测试遇到问题，请开启 verbose=True，查看调试信息。
5     # rknn = RKNN(verbose=True)
6     rknn = RKNN()
7
8     # 输入图像处理
9     img = cv2.imread(IMG_PATH)
10    # img, ratio, (dw, dh) = letterbox(img, new_shape=(IMG_SIZE, IMG_SIZE))
11    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
12    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
13
14    # pre-process config
15    print('--> Config model')
16    rknn.config(mean_values=[[0, 0, 0]], std_values=[[255, 255, 255]],
17    ↪target_platform="rk3568")
18    print('done')
19
20    # 如果前面转换出的是 torchscript，重命名为.pt 文件，取消下面注释，然后加载模型
21    #print('--> Loading model')
22    #ret = rknn.load_pytorch(model=PYTORCH_MODEL, input_size_list=[[1, 3, IMG_
23    ↪SIZE, IMG_SIZE]])
```

(下页继续)

(续上页)

```
22     #if ret != 0:
23     #     print('Load model failed!')
24     #     exit(ret)
25     #print('done')
26
27     # 加载 onnx 模型
28     print('--> Loading model')
29     ret = rknn.load_onnx(model=ONNX_MODEL)
30     if ret != 0:
31         print('Load model failed!')
32         exit(ret)
33     print('done')
34
35     # 构建模型，默认开启了量化
36     print('--> Building model')
37     ret = rknn.build(do_quantization=True, dataset=DATASET)
38     #ret = rknn.build(do_quantization=False)
39     if ret != 0:
40         print('Build model failed!')
41         exit(ret)
42     print('done')
43
44     # 量化精度分析。默认关闭，取消注释开启
45     #print('--> Accuracy analysis')
46     #Ret = rknn.accuracy_analysis(inputs=[img])
47     #if ret != 0:
48     #     print('Accuracy analysis failed!')
49     #     exit(ret)
50     #print('done')
51
52     # 导出 RKNN 模型
53     print('--> Export rknn model')
```

(下页继续)

(续上页)

```
54     ret = rknn.export_rknn(RKNN_MODEL)
55     #if ret != 0:
56     #     print('Export rknn model failed!')
57     #     exit(ret)
58     print('done')
59
60     # 初始化运行环境，默认没有配置 device_id 等，需要调试时开启
61     print('--> Init runtime environment')
62     ret = rknn.init_runtime()
63     # ret = rknn.init_runtime(target='rk3568', device_id='192.168.103.
↪115:5555', perf_debug=True)
64     if ret != 0:
65         print('Init runtime environment failed!')
66         exit(ret)
67     print('done')
68
69     # 调试，模型性能进行评估，取消注释打开
70     # rknn.eval_perf(inputs=[img], is_print=True)
71
72     rknn.release()
```

```
# 运行转换程序
sudo python3 test.py

# 显示结果，并在当前目录下生成 yolov5s.rknn
W __init__: rknn-toolkit2 version: 1.4.0-22dcfef4
--> Config model
done
--> Loading model
done
--> Building model
```

(下页继续)

(续上页)

```
D RKNNAPI: RKNN VERSION:
D RKNNAPI:   API: 1.4.0 (bb6dac9 build: 2022-08-29 16:17:01) (null)
D RKNNAPI:   DRV: rknn_server: 1.3.0 (121b661 build: 2022-04-29 11:11:47)
D RKNNAPI:   DRV: rknnrt: 1.4.0 (a10f100eb@2022-09-09T09:07:14)
D RKNNAPI: =====
```

done

Performance

```
#### The performance result is just for debugging, ####
#### may worse than actual performance!          ####
```

Total Weight Memory Size: 7312768

Total Internal Memory Size: 7782400

Predict Internal Memory RW Amount: 138880000

Predict Weight Memory RW Amount: 7312768

ID	OpType	DataType	Target	InputShape	OutputShape	MacUsage (%)	DDR Cycles	NPU Cycles	Total Cycles	Time (us)	FullName
0	InputOperator	UINT8	CPU	\	(1, 3, 640, 640)		0	0	0		
9							1200.00				InputOperator:images
1	Conv	UINT8	NPU	(1, 3, 640, 640), (32, 3, 6, 6), (32)	(1, 32, 320, 320)	9.14	687110	691200	691200	8407	
							4409.25				Conv:Conv_0
2	exSwish	INT8	NPU	(1, 32, 320, 320)	(1, 32, 320, 320)		997336	0	997336		
							6400.00			3737	exSwish:Sigmoid_1_2swish
..... (省略)											
145	Sigmoid	INT8	NPU	(1, 255, 80, 80)	(1, 255, 80, 80)		498668	0	498668		
							3200.00			1895	Sigmoid:Sigmoid_199

(下页继续)

(续上页)

```

146  OutputOperator  INT8      CPU      (1,255,80,80),(1,80,80,256)
      ↪          \              0              0              0
      ↪ 165          \              3200.00          OutputOperator:output0
147  OutputOperator  INT8      CPU      (1,255,40,40),(1,40,40,256)
      ↪          \              0              0              0
      ↪ 52          \              800.00          OutputOperator:329
148  OutputOperator  INT8      CPU      (1,255,20,20),(1,20,20,256)
      ↪          \              0              0              0
      ↪ 34          \              220.00          OutputOperator:331
Total Operator Elapsed Time(us): 77776
=====

```

会在当前目录下导出的 rknn 模型，如果取消 eval_perf 接口注释，会进行性能评估，简单调试。

25.2.4 部署到鲁班猫板卡

导出 rknn 模型后，使用 RKNN Toolkit Lite2 在板端进行简单部署，获得结果后进行画框等后处理等等，具体文件参考下 [这里](#)。

列表 4: rknnlite_inference.py(截取部分，参考配套例程)

```

1  if __name__ == '__main__':
2
3      # 创建 RKNNLite 对象
4      rknn_lite = RKNNLite()
5
6      # 导入 rknn 模型
7      print('--> Load RKNN model')
8      ret = rknn_lite.load_rknn(rknn_model)
9      if ret != 0:
10         print('Load RKNN model failed')

```

(下页继续)

(续上页)

```
11         exit(ret)
12     print('done')
13
14     # 初始化运行环境
15     print('--> Init runtime environment')
16     ret = rknn_lite.init_runtime()
17     if ret != 0:
18         print('Init runtime environment failed!')
19         exit(ret)
20     print('done')
21
22     # 输入图像处理
23     img = cv2.imread(IMG_PATH)
24     img, ratio, (dw, dh) = letterbox(img, new_shape=(640, 640)) # 灰度填充
25     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
26
27     # 推理
28     print('--> Running model')
29     outputs = rknn_lite.inference(inputs=[img])
30     print('done')
31
32     # 后处理
33     boxes, classes, scores = yolov5_post_process(outputs)
34
35     img_1 = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
36     if boxes is not None:
37         draw(img_1, boxes, scores, classes)
38
39     # 显示或者保存图片
40     cv2.imwrite("out.jpg", img_1)
41     # cv2.imshow("post process result", img_1)
42     # cv2.waitKey(0)
```

(下页继续)

(续上页)

```
43     # cv2.destroyAllWindows()
44
45     rknn_lite.release()
```

简单测试显示：

```
--> Load RKNN model
done
--> Init runtime environment
I RKNN: [11:59:53.211] RKNN Runtime Information: librknnrt version: 1.4.0_
↪(a10f100eb@2022-09-09T09:07:14)
I RKNN: [11:59:53.211] RKNN Driver Information: version: 0.7.2
I RKNN: [11:59:53.213] RKNN Model Information: version: 1, toolkit version:_
↪1.4.0-22dcfef4(compiler version: 1.4.0 (3b4520e4f@2022-09-05T20:52:35)),_
↪target: RKNPU lite, target platform: rk3568, framework name: PyTorch,_
↪framework layout: NCHW
done
--> Running model
done
class: person, score: 0.8889630436897278
box coordinate left,top,right,down: [370, 168, 574, 495]
class: person, score: 0.5832323431968689
box coordinate left,top,right,down: [59, 242, 362, 493]
class: tie, score: 0.668832540512085
box coordinate left,top,right,down: [221, 360, 249, 491]
```

结果会保存在当前目录下的 out.jpg，查看图片：



25.3 airockchip/yolov5 简单测试

上面是使用官方的 YOLOv5 v7.0 进行简单测试，接下来我们使用 [airockchip/yolov5](https://github.com/airockchip/yolov5)，该仓库的 yolov5 对 rknpv 设备进行了部署优化，教程测试时是 v6.2 版本分支，相关优化说明请参考下仓库的 README_rkopt.md。

拉取最新的 airockchip/yolov5 仓库，安装类似，参考下前面：

```
# 教程测试时是 v6.2 分支版本，自行测试可以使用该仓库的 master 或者其他分支
git clone https://github.com/airockchip/yolov5.git
cd yolov5
```

获取权重文件，需要我们重新训练：

```
# 重新训练，获取优化后的权重文件，这里是基于权重 yolov5s.pt,, 或者使用--cfg 指定配置文件
sudo python3 train.py --data coco128.yaml --weights yolov5s.pt --img 640
```

(下页继续)

(续上页)

```
# 训练结束输出信息很多, 其中会输出:
...
Optimizer stripped from runs/train/exp/weights/last.pt, 14.9MB
Optimizer stripped from runs/train/exp/weights/best.pt, 14.9MB

Validating runs/train/exp/weights/best.pt...
...
# 训练一些分析和权重保存在 runs/train/exp/目录下
```

会在 runs/train/exp/weights/保存文件, 我们重命名为 yolov5s_relu.pt, 并复制到源码根目录下。

接下来导出 torchscript, 使用命令:

```
# 其中--weights 指定权重文件的路径,
# --rknpu 指定平台 (rk_platform 支持 rk1808, rv1109, rv1126, rk3399pro,
↳rk3566, rk3568, rk3588, rv1103, rv1106)
# --include 指定导出 onnx 模型,, 不指定默认是 torchscript

sudo python3 export.py --weights yolov5s_relu.pt --rknpu rk3568

# 正常转换会输出:
export: data=data/coco128.yaml, weights=['yolov5s.pt'], imgsz=[640, 640],
↳batch_size=1, device=cpu, half=False, inplace=False, train=False,
↳keras=False, optimize=False, int8=False, dynamic=False, simplify=False,
↳opset=12, verbose=False, workspace=4, nms=False, agnostic_nms=False, topk_
↳per_class=100, topk_all=100, iou_thres=0.45, conf_thres=0.25, include=[
↳'torchscript'], rknpu=rk3568
YOLOv5 □ 2022-10-28 Python-3.8.10 torch-1.10.1+cpu CPU

Fusing layers...
YOLOv5s summary: 213 layers, 7225885 parameters, 0 gradients, 16.4 GFLOPs
```

(下页继续)

(续上页)

```
---> save anchors for RKNN
[[10.0, 13.0], [16.0, 30.0], [33.0, 23.0], [30.0, 61.0], [62.0, 45.0], [59.
↪0, 119.0], [116.0, 90.0], [156.0, 198.0], [373.0, 326.0]]

PyTorch: starting from yolov5s.pt with output shape (1, 255, 80, 80) (14.1
↪MB)

TorchScript: starting export with torch 1.10.1+cpu...
TorchScript: export success, saved as yolov5s.torchscript (27.8 MB)

Export complete (1.92s)
Results saved to /mnt/e/Users/Administrator/Desktop/wsl_user/yolov5/yolov5-
↪master

Detect:          python detect.py --weights yolov5s.torchscript
Validate:        python val.py --weights yolov5s.torchscript
PyTorch Hub:     model = torch.hub.load('ultralytics/yolov5', 'custom',
↪'yolov5s.torchscript')
Visualize:       https://netron.app

# 或者使用下面命令, 导出 onnx 模型, 会在当前目录下生成 yolov5s.onnx 文件。
python export.py --weights yolov5s.pt --include onnx --rknpu rk3568
```

导出 torchscript, 会在当前目录下生成 yolov5s_relu.torchscript 文件, 并重新命名为.pt 文件。

25.3.1 导出 rknn 模型

测试使用前面的转换文件, 直接运行程序导出 rknn 文件, 并测试。

```
# 运行测试, 导出 rknn 模型:
W __init__: rknn-toolkit2 version: 1.4.0-22dcfef4
```

(下页继续)

(续上页)

```
--> Config model
done
--> Loading model
PtParse: 100%|███████████████████████████████████████| 698/698_
↪[00:01<00:00, 602.67it/s]
done
--> Building model
Analysing : 100%|███████████████████████████████████████| 145/145_
↪[00:00<00:00, 381.87it/s]
Quantizing : 100%|███████████████████████████████████████| 145/145_
↪[00:00<00:00, 518.54it/s]
W build: The default input dtype of 'x.1' is changed from 'float32' to 'int8'
↪' in rknn model for performance!
        Please take care of this change when deploy rknn model_
↪with Runtime API!
W build: The default output dtype of '172' is changed from 'float32' to
↪'int8' in rknn model for performance!
        Please take care of this change when deploy rknn model_
↪with Runtime API!
W build: The default output dtype of '173' is changed from 'float32' to
↪'int8' in rknn model for performance!
        Please take care of this change when deploy rknn model_
↪with Runtime API!
W build: The default output dtype of '174' is changed from 'float32' to
↪'int8' in rknn model for performance!
        Please take care of this change when deploy rknn model_
↪with Runtime API!
done
--> Export rknn model
done
--> Init runtime environment
W init_runtime: Target is None, use simulator!
```

(下页继续)

(续上页)

```

W init_runtime: Flag perf_debug has been set, it will affect the
↳ performance of inference!
I NPUTTransfer: Starting NPU Transfer Client, Transfer version 2.1.0
↳ (b5861e7@2020-11-23T11:50:36)
D RKNNAPI: =====
D RKNNAPI: RKNN VERSION:
D RKNNAPI:   API: 1.4.0 (bb6dac9 build: 2022-08-29 16:17:01) (null)
D RKNNAPI:   DRV: rknn_server: 1.3.0 (121b661 build: 2022-04-29 11:11:47)
D RKNNAPI:   DRV: rknnrt: 1.4.0 (a10f100eb@2022-09-09T09:07:14)
D RKNNAPI: =====

```

done

Performance

```

#### The performance result is just for debugging, ####
#### may worse than actual performance!          ####

```

```

Total Weight Memory Size: 7312768
Total Internal Memory Size: 6144000
Predict Internal Memory RW Amount: 86931200
Predict Weight Memory RW Amount: 7312768

```

ID	OpType	DataType	Target	InputShape	OutputShape	DDR Cycles	NPU Cycles	Total Cycles
Time (us)	MacUsage (%)	RW (KB)	FullName					
0	InputOperator	UINT8	CPU	\	(1, 3, 640, 640)	0	0	0
8						1200.00	InputOperator:x.1	
1	ConvRelu	UINT8	NPU	(1, 3, 640, 640), (32, 3, 6, 6), (32)	(1, 32, 320, 320)	687110	691200	691200
8387		9.16				4409.25	Conv:input.4_Conv	
2	ConvRelu	INT8	NPU	(1, 32, 320, 320), (64, 32, 3, 3), (64)	(1, 64, 160, 160)	750885	921600	921600
2247		45.57				4818.50	Conv:input.6_Conv	

(下页继续)

(续上页)

```

..... (省略)
80  Conv          INT8    NPU    (1, 512, 20, 20), (255, 512, 1, 1), (255)
    ↳          (1, 255, 20, 20)      66931      102000      102000
    ↳ 180          62.96      429.50      Conv:1189_Conv
81  Sigmoid       INT8    NPU    (1, 255, 20, 20)
    ↳          (1, 255, 20, 20)      31167      0      31167
    ↳ 141          \      200.00      Sigmoid:1190_Sigmoid
82  OutputOperator INT8    CPU    (1, 255, 80, 80), (1, 80, 80, 256)
    ↳          \      0      0      0
    ↳ 161          \      3200.00      OutputOperator:172
83  OutputOperator INT8    CPU    (1, 255, 40, 40), (1, 40, 40, 256)
    ↳          \      0      0      0
    ↳ 46          \      800.00      OutputOperator:173
84  OutputOperator INT8    CPU    (1, 255, 20, 20), (1, 20, 20, 256)
    ↳          \      0      0      0
    ↳ 24          \      220.00      OutputOperator:174
Total Operator Elapsed Time(us): 44412
=====

```

提示：前面测试时的 CPU 频率为 1.8Ghz，DDR 频率为 1.056Ghz，NPU 频率为 900Mhz。

后面部署测试和前面类似就不列举了，其他部署文档请参考下 [这里](#)。

25.4 参考链接

<https://github.com/ultralytics/yolov5>

<https://github.com/airockchip/yolov5/tree/master>

https://github.com/airockchip/rknn_model_zoo

第 26 章 YOLOv10

YOLOv10 是清华大学研究人员近期提出的一种实时目标检测方法，通过消除 NMS、优化模型架构和引入创新模块等策略，在保持高精度的同时显著降低了计算开销，为实时目标检测领域带来了新的突破。

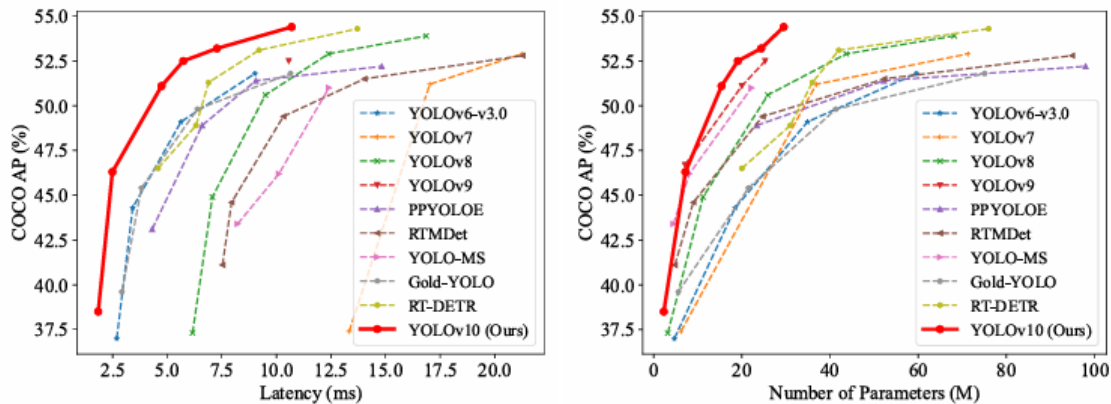


Figure 1: Comparisons with others in terms of latency-accuracy (left) and size-accuracy (right) trade-offs. We measure the end-to-end latency using the official pre-trained models.

YOLOv10 是在 Ultralytics Python 包的基础上进行了多项创新和改进，主要特点：

- 消除非极大值抑制 (NMS)：YOLOv10 通过引入一致的双重分配策略，在训练时使用一对多的标签分配来提供丰富的监督信号，在推理时使用一对一的匹配，从而消除了对 NMS 的依赖。
- 全面优化的模型架构：YOLOv10 从推理效率和准确性的角度出发，全面优化了模型的各个组成部分。这包括采用轻量级分类头、空间通道去耦下采样和等级引导块设计等，以减少计算冗余并提高模型性能。
- 引入大核卷积和部分自注意模块：为了提高性能，YOLOv10 在不增加大量计算成本的前提下，引入了大核卷积和部分自注意模块。

YOLOv10 论文地址：<https://arxiv.org/pdf/2405.14458>

YOLOv10 项目地址: <https://github.com/THU-MIG/yolov10>

本章将简单测试 YOLOv10, 并导出 rknn 模型, 在板卡上使用 RKNN Toolkit Lite2 部署测试。

26.1 YOLOv10 简单测试

26.1.1 环境配置

获取工程源码:

```
git clone https://github.com/THU-MIG/yolov10.git
```

使用 Anaconda 进行环境管理, 创建一个 yolov10 环境:

```
conda create -n yolov10 python=3.9
conda activate yolov10

# 切换到 yolov10 工程目录, 然后安装依赖
cd yolov10
pip install -r requirements.txt
pip install -e .

# 安装完成后简单测试 yolo 命令
(yolov10) xxx@anhao:~$ yolo -V
8.1.34
```

26.1.2 模型推理测试

获取模型 (这里获取 s 尺寸 yolov10 模型):

```
wget https://github.com/THU-MIG/yolov10/releases/download/v1.1/yolov10s.pt
```

然后使用命令测试：

```
(yolov10) xxx@anhao:~$ yolo predict model=./yolov10s.pt

WARNING □ 'source' argument is missing. Using default 'source=xxx'.
Ultralytics YOLOv8.1.34 □ Python-3.9.19 torch-2.0.1+cu117 CUDA:0 (NVIDIA_
↪ GeForce RTX 3060 Laptop GPU, 6144MiB)
YOLOv10n summary (fused): 285 layers, 2762608 parameters, 63840 gradients, ↪
↪ 8.6 GFLOPs

image 1/2 xxx/bus.jpg: 640x480 4 persons, 1 bus, 55.5ms
image 2/2 xxx/zidane.jpg: 384x640 2 persons, 60.5ms
Speed: 2.9ms preprocess, 58.0ms inference, 5.4ms postprocess per image at ↪
↪ shape (1, 3, 384, 640)
Results saved to /xxx/yolov10/runs/detect/predict
□ Learn more at https://docs.ultralytics.com/modes/predict
```

结果保存在 yolov10 工程文件 runs/detect/predict 目录下。

也可以测试 yolov10 工程文件中的 app.py 程序，一个网页显示，直接拖拽图像，然后推理显示结果。

26.2 板卡上部署测试

为了在板卡上部署 yolov10，参考 YOLOv8 模型的适配修改，修改下 yolov10 模型输出。

26.2.1 导出适配的 onnx 模型

拉取修改的 yolov10 工程文件：

```
git clone https://github.com/mmонтol/yolov10.git
```

修改 ultralytics/cfg/default.yaml 文件，设置模型路径为前面拉取的官方 yolov10 模型路径：

列表 1: ultralytics/cfg/default.yaml

```
1 task: detect # (str) YOLO task, i.e. detect, segment, classify, pose
2 mode: train # (str) YOLO mode, i.e. train, val, predict, export, track, ↵
   ↪ benchmark
3
4 # Train settings -----
   ↪ -----
5 model: ../yolov10n.pt # (str, optional) path to model file, i.e. yolov8n.pt,
   ↪ yolov8n.yaml
6 data: # (str, optional) path to data file, i.e. coco128.yaml
7 epochs: 100 # (int) number of epochs to train for
```

然后使用命令导出 onnx 模型：

```
export PYTHONPATH=./
python ./ultralytics/engine/exporter.py
```

最后导出模型保存在原模型同级目录下。

26.2.2 导出 rknn 模型

使用 toolkit2 工具，将前面的 onnx 模型转换成 rknn 模型，toolkit2 的安装可以参考下 [这里](#)。

编程一个简单的模型转换程序：

列表 2: onnx2rknn.py

```
1 # 省略.....
2 if __name__ == '__main__':
3     model_path, platform, do_quant, output_path = parse_arg()
4
5     # Create RKNN object
```

(下页继续)

(续上页)

```
6     rknn = RKNN(verbose=False)
7
8     # Pre-process config
9     print('--> Config model')
10    rknn.config(mean_values=[[0, 0, 0]], std_values=[
11                [255, 255, 255]], target_platform=platform)
12    print('done')
13
14    # Load model
15    print('--> Loading model')
16    ret = rknn.load_onnx(model=model_path)
17    #ret = rknn.load_pytorch(model=model_path, input_size_list=[[1, 3, 640, 640]])
18    if ret != 0:
19        print('Load model failed!')
20        exit(ret)
21    print('done')
22
23    # Build model
24    print('--> Building model')
25    ret = rknn.build(do_quantization=do_quant, dataset=DATASET_PATH)
26    if ret != 0:
27        print('Build model failed!')
28        exit(ret)
29    print('done')
30
31    # Export rknn model
32    print('--> Export rknn model')
33    ret = rknn.export_rknn(output_path)
34    if ret != 0:
35        print('Export rknn model failed!')
36        exit(ret)
```

(下页继续)

26.2.3 部署测试

在鲁班猫 RK 系列板卡上部署，这里测试使用 RKNN Toolkit Lite2 提供的 python 接口，RKNN Toolkit Lite2 安装请参考下前面教程。

列表 3: yolov10.py

```
1 if __name__ == '__main__':
2
3     # Get device information
4     host_name = get_host()
5     if host_name == 'RK3566_RK3568':
6         rknn_model = RK3566_RK3568_RKNN_MODEL_PATH
7     elif host_name == 'RK3562':
8         rknn_model = RK3562_RKNN_MODEL_PATH
9     elif host_name == 'RK3576':
10        rknn_model = RK3576_RKNN_MODEL_PATH
11    elif host_name == 'RK3588':
12        rknn_model = RK3588_RKNN_MODEL_PATH
13    else:
14        print("This demo cannot run on the current platform: {}".
15        ↪format(host_name))
16        exit(-1)
17
18    rknn_lite = RKNNLite()
19
20    # 加载 rknn 模型
21    ret = rknn_lite.load_rknn(rknn_model)
22    if ret != 0:
23        print('Load RKNN model failed')
24        exit(ret)
25
26    # 设置运行环境，目标默认是 rk3588，选择 npu0 单核
```

(下页继续)

(续上页)

```
26     if host_name in ['RK3576', 'RK3588']:
27         ret = rknn_lite.init_runtime(core_mask=RKNNLite.NPU_CORE_0)
28     else:
29         ret = rknn_lite.init_runtime()
30     if ret != 0:
31         print('Init runtime environment failed')
32         exit(ret)
33     print('done')
34
35     # 输入图像
36     img_src = cv2.imread(IMG_PATH)
37     src_shape = img_src.shape[:2]
38     img, ratio, (dw, dh) = letter_box(img_src, IMG_SIZE)
39     img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
40     img = np.expand_dims(img, 0)
41
42     # 推理运行
43     print('--> Running model')
44     outputs = rknn_lite.inference(inputs=[img])
45
46     # 后处理
47     boxes, classes, scores = post_process(outputs)
48
49     img_p = img_src.copy()
50     if boxes is not None:
51         draw(img_p, get_real_box(src_shape, boxes, dw, dh, ratio), scores,
52             ↪ classes)
53     cv2.imwrite("result.jpg", img_p)
```

修改程序中的模型路径和输入图像路径，然后运行程序，即可得到推理结果。

```
cat@lubancat:~/xxx$ python3 yolov10.py
I RKNN: [13:51:14.616] RKNN Runtime Information, librknrt version: 2.0.0b0
↳ (35a6907d79@2024-03-24T10:31:14)
I RKNN: [13:51:14.616] RKNN Driver Information, version: 0.9.2
I RKNN: [13:51:14.617] RKNN Model Information, version: 6, toolkit version:
↳ 2.0.0b0+9bab5682 (compiler version: 2.0.0b0 (35a6907d79@2024-03-
↳ 24T02:34:11)),
target: RKNPU v2, target platform: rk3588, framework name: ONNX, framework
↳ layout: NCHW, model inference type: static_shape
done
--> Running model
person @ (79 330 115 518) 0.313
bus @ (87 137 556 438) 0.936
person @ (210 240 284 512) 0.906
person @ (109 234 226 536) 0.897
person @ (477 233 560 520) 0.833
```

推理结果保存在当前目录下，名为 result.jpg。

26.3 参考链接

<https://github.com/THU-MIG/yolov10>

<https://github.com/mmontol/yolov10.git>

第 27 章 SenseVoice

SenseVoice 是具有音频理解能力的音频基础模型，包括语音识别（ASR）、语种识别（LID）、语音情感识别（SER）和声学事件分类（AEC）或声学事件检测（AED）。

Sensevoice 采用超过 40 万小时数据训练，支持超过 50 种语言，识别效果上优于 Whisper 模型。将输入的音频转换为对应的文本序列，并根据特殊的标记来指定不同的任务。

27.1 RKNN 模型转换

在 PC 上创建一个虚拟环境，安装 Sensevoice 环境

```
# 使用 conda 创建虚拟环境
conda create -n sensevoice python=3.11
conda activate sensevoice

# 安装相关库和软件
git clone https://github.com/alibaba/FunASR.git
cd FunASR
pip3 install -qq -e ./
cd ..

# 拉取 SenseVoice 源码，安装相关依赖库
git clone https://github.com/FunAudioLLM/SenseVoice
cd SenseVoice
pip install -qq -r ./requirements.txt
cd ..

pip install soundfile onnx onnxruntime kaldi-native-fbank librosa soundfile
```

(下页继续)

(续上页)

```
# 配置 pip 源 (可选)
# pip3 config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple/
```

获取 SenseVoiceSmall 模型 (可选):

```
git clone https://www.modelscope.cn/iic/SenseVoiceSmall.git
```

先简单编程将 SenseVoiceSmall 模型导出 onnx 模型, 参考了 *sherpa-onnx* <<https://github.com/k2-fsa/sherpa-onnx>> 中程序然后使用 toolkit2 工具转换成 rknn 模型。

```
# 省略.....
def main():
    # model, params = SenseVoiceSmall.from_pretrained(model="iic/
    ↪SenseVoiceSmall", device="cpu")
    model, params = SenseVoiceSmall.from_pretrained(model="../models/
    ↪SenseVoiceSmall", device="cpu")

    display_params(params)

    generate_tokens(params)

    model.__class__.forward = modified_forward
    model.encoder.__class__.forward = encoder_forward

    x = torch.randn(1, INPUT_LEN, 560, dtype=torch.float32)
    x_length = torch.tensor([100], dtype=torch.int32)
    language = torch.tensor([3], dtype=torch.int32)
    text_norm = torch.tensor([15], dtype=torch.int32)

    opset_version = 13
    filename = "model.onnx"
```

(下页继续)

(续上页)

```
torch.onnx.export(  
    model,  
    (x, x_length, language, text_norm),  
    filename,  
    opset_version=opset_version,  
    input_names=["x", "x_length", "language", "text_norm"],  
    output_names=["logits"],  
  
)  
  
if __name__ == "__main__":  
    torch.manual_seed(20250717)  
    main()
```

执行 `export_onnx.py` 程序，可自行修改模型路径和导出的 `onnx` 名称：

```
(sensevoice) llh@llh:/xxx/SenseVoice$ python export_onnx.py  
Loading remote code successfully: model  
-----params-----  
# 省略.....
```

然后使用配套例程的转换程序，将 `sensevoice` 的 `onnx` 模型成 `rknn` 模型：

```
# Usage: python3 convert_30.py onnx_model_path [platform] [dtype(optional)]  
↪ [output_rknn_path(optional)]  
#     platform choose from [rk3562, rk3566, rk3568, rk3576, rk3588]  
#     dtype choose from [fp] for [rk3562, rk3566, rk3568, rk3576, rk3588]  
  
(toolkit2_2.3) llh@llh:/xxx$ python convert.py model.onnx rk3588 fp model.  
↪ rknn  
I rknn-toolkit2 version: 2.3.2  
--> Config model
```

(下页继续)

```
done
--> Loading model
# 省略.....
I Loading :    0%|
↪ | 0/917 [00:00<?, ?it/s]
I Loading :   94%|██████████████████████████████████████      | 858/917↵
↪ [00:00<00:00, 8536.70it/s]
I Loading :  100%|██████████████████████████████████████████| 917/917↵
↪ [00:00<00:00, 6075.70it/s]
done
--> Building model
# 省略.....
I OpFusing 1:    0%|██████████████████████████████████████      | 0/100 [00:00<?,
↪ ?it/s]
I OpFusing 2 : 100%|██████████████████████████████████████████| 100/100↵
↪ [00:16<00:00, 5.94it/s]
I rknn building ...
# 省略.....
E RKNN: [15:09:24.190] Unkown op target: 0
I rknn building done.
done
--> Export rknn model
done
```

27.2 Lubancat 板卡上测试

简单测试使用 RKNN Toolkit Lite2 的接口，需要先安装 Toolkit Lite2 等环境。

教程简单测试使用 UV 创建环境，UV 的简单使用和安装参考下 [前面教程](#)，自行测试也可以直接安装，不用 UV 和虚拟环境。

```
# 使用 UV 创建一个环境，使用 python3.11
cat@lubancat:~$ uv venv --python python3.11 rknn_lite_sensevoice
Using CPython 3.11.13
Creating virtual environment at: rknn_lite_sensevoice
Activate with: source rknn_lite_sensevoice/bin/activate

# 进入环境
cat@lubancat:~$ source rknn_lite_sensevoice/bin/activate
(rknn_lite_sensevoice) cat@lubancat:~$

# 安装 RKNN Toolkit Lite2, 对应 python 版本的 whl 文件 (参考前面文档),
(rknn_lite_sensevoice) cat@lubancat:~/xxx$ uv pip install rknn_toolkit_
↳ lite2-2.3.2-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl
Using Python 3.11.13 environment at: rknn_lite_sensevoice
Resolved 5 packages in 1.17s
Prepared 5 packages in 1.64s
Installed 5 packages in 23ms
+ numpy==2.3.2
+ psutil==7.0.0
+ rknn-toolkit-lite2==2.3.2 (from xxxx)
+ ruamel-yaml==0.18.14
+ ruamel-yaml-clib==0.2.12

# 安装 soundfile, kaldia_native_fbank 等等
(rknn_lite_sensevoice) cat@lubancat:~$ uv pip install kaldia_native_fbank
↳ soundfile
```

获取测试例程 sensevoice_rknn.py:

```
# 获取测试例程，待更新
```

(下页继续)

(续上页)

```
(rknn_lite_sensevoice) cat@lubancat:~$ git clone xxx
(rknn_lite_sensevoice) cat@lubancat:~$ cd
```

将前面小节转换出的 rknn 模型传输到板卡，然后运行例程，需要指定模型路径，tokens，以及音频路径等参数

```
# 例程参数
(rknn_lite_sensevoice) cat@lubancat:~/xxx$ python sensevoice_rknn.py -h
usage: sensevoice_rknn.py [-h] --model_path MODEL_PATH --tokens TOKENS [--
↪language LANGUAGE]
--audio_path AUDIO_PATH [--use-itn USE_ITN] [--mvn_path MVN_PATH]

sensevoice Python Demo

options:
-h, --help            show this help message and exit
--model_path MODEL_PATH
                        model path, could be .rknn or .onnx file
--tokens TOKENS       Path to tokens.txt
--language LANGUAGE   the language of the input wav file. Supported values:
↪zh, en, ja, ko, yue, auto
--audio_path AUDIO_PATH
                        The input wave to be recognized
--use-itn USE_ITN     1 to use inverse text normalization. 0 to not use
↪inverse text normalization
--mvn_path MVN_PATH   The input wave to be recognized

# 执行测试例程 (Lubancat-4)
(rknn_lite_sensevoice) cat@lubancat:~/xxx$ python sensevoice_rknn.py --
↪model_path ../model/model_encoder.rknn \
--audio_path ../model/en.wav --mvn_path ../model/am.mvn --tokens ../
↪model/tokens.txt --use-itn 1
```

(下页继续)

(续上页)

```
I RKNN: [10:21:08.231] RKNN Runtime Information, librknrt version: 2.3.2
↳ (429f97ae6b@2025-04-09T09:09:27)
I RKNN: [10:21:08.231] RKNN Driver Information, version: 0.9.8
I RKNN: [10:21:08.232] RKNN Model Information, version: 6, toolkit version:
↳ 2.3.2 (compiler version: 2.3.2 (e045de294f@2025-04-07T19:48:25)),
target: RKNPU v2, target platform: rk3588, framework name: ONNX, framework
↳ layout: NCHW, model inference type: static_shape
W RKNN: [10:21:08.620] query RKNN_QUERY_INPUT_DYNAMIC_RANGE error, rknn
↳ model is static shape type, please export rknn with dynamic_shapes
W Query dynamic range failed. Ret code: RKNN_ERR_MODEL_INVALID. (If it is a
↳ static shape RKNN model, please ignore the above warning message.)

<|en|><|NEUTRAL|><|Speech|><|withitn|>The tribal chieftain called for the
↳ boy and presented him with 50 pieces of code.
<|NEUTRAL|><|NEUTRAL|><|NEUTRAL|><|NEUTRAL|><|NEUTRAL|>
```

27.3 参考链接

<https://github.com/FunAudioLLM/SenseVoice>

<https://github.com/k2-fsa/sherpa-onnx>

第 28 章 鲁班猫监控检测

本章将简单介绍一个摄像头监控检测示例，用户在浏览器上登录监控页面，登录后点击按钮可以进行视频录制和目标检测。web 程序采用的是基于 python 的 flask 框架，实现流媒体直播，图像是通过 opencv 调用摄像头获取，对图片检测处理使用 npu。

- 测试平台：lubancat 2
- 板卡系统：Debian10（带桌面）
- Python 版本：Python3.7
- opencv 版本：4.7.0.68
- Toolkit Lite2：1.4.0
- Flask：1.0.2

28.1 依赖工具及库安装

实验测试使用 lubancat 2，系统是 Debian10，一些工具和库的安装（一些工具系统已经安装就不执行）：

```
1 # 安装工具
2 sudo apt update
3 sudo apt -y install git wget
4
5 # 安装 python 相关库等，默认使用 python3
6 sudo apt -y install python3-flask python3-pil python3-numpy python3-pip
7
8 # 安装 opencv-python 相关库，测试是使用 4.7.0.68 版本，或者其他版本
9 sudo pip3 install opencv-contrib-python
```

(下页继续)

(续上页)

```
10
11 # 安装 rknn-toolkit-lite2, 参考前面 NPU 使用章节
```

28.2 视频流服务器和摄像头获取帧

28.2.1 部署视频流服务器

在这个示例中，将使用 Flask 应用框架，构建一个 web 页面，一个实时视频流服务器。

提示： flask 库简单使用可以参考前面 [教程](#)，或者 [Flask 官方文档](#)。

Flask 通过 /video_viewer 路由返回一个入参为生成器的 Response 对象。Flask 将会负责调用生成器，进入循环，持续地将摄像头中获取的帧数据作为响应块返回，并把所有部分的结果以块的形式发送给客户端。

列表 1: lubancat-flask-opencv-
rknn/controller/modules/home/views.py

```
1 from flask import session, render_template, request, redirect, url_for, \
  ↳ Response, jsonify
2 # 导入登录页面等
3 from controller.modules.home import home_blu
4 # 导入 VideoCamera 类，用于获取摄像头帧或者检测后的帧流等
5 from controller.utils.camera import VideoCamera
6 import time
7
8 video_camera = None
9 global_frame = None
```

(下页继续)

(续上页)

```
11 # 主页
12 @home_blu.route('/')
13 def index():
14     # 模板渲染
15     username = session.get("username")
16     if not username:
17         return redirect(url_for("user.login"))
18     return render_template("index.html")
19
20
21 # 获取视频流
22 def video_stream():
23     global video_camera
24     global global_frame
25
26     if video_camera is None:
27         video_camera = VideoCamera()
28
29     while True:
30         # 获取一系列单独的 JPEG 图片
31         frame = video_camera.get_frame()
32         time.sleep(0.01)
33         if frame is not None:
34             global_frame = frame
35             yield (b'--frame\r\n'
36                  b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n\r\n')
37         else:
38             yield (b'--frame\r\n'
39                  b'Content-Type: image/jpeg\r\n\r\n' + global_frame + b'\r\n\r\n')
40
41 # 视频流
```

(下页继续)

(续上页)

```
42 @home_blu.route('/video_viewer')
43 def video_viewer():
44     # 模板渲染
45     username = session.get("username")
46     if not username:
47         return redirect(url_for("user.login"))
48     # 返回生成器的 Response 对象, multipart/x-mixed-replace 类型, 边界字符串为
frame
49     return Response(video_stream(),
50                     mimetype='multipart/x-mixed-replace; boundary=frame')
```

在 index.html 这个简单的 HTML 页面中, 其中的一个图片标签 ``, 将使用 `url_for` 指向路由 `/video_viewer`, 浏览器会自动显示流中的图片, 从而保持更新图片元素:

列表 2: 示例程序 lubancat-flask-opencv-rknn/controller/templates/index.html(部分)

```
1 <body>
2 <h1 align="center" style="color: whitesmoke;">Flask+OpenCV+Rknn</h1>
3 <div class="top">
4     <div class="recorder" id="recorder" align="center">
5         <button id="record" class="btn"> 录制视频</button>
6         <button id="stop" class="btn"> 暂停录制</button>
7         <button id="process" class="btn"> 开启检测</button>
8         <button id="pause" class="btn"> 暂停检测</button>
9         <input type="button" class="btn" value=" 退出登录"
10             onclick="javascript:window.location.href='{{ url_for('user.
↵logout') }}'">
11         <a id="download"></a>
12         <script type="text/javascript" src="{{ url_for('static', filename=
↵'button_process.js') }}"></script>
```

(下页继续)

(续上页)

```
13     </div>
14 </div>
15 
16 </body>
```

28.2.2 摄像头中获取帧

从摄像头获取帧，通过导入封装好的 VideoCamera 类：

列表 3: 示例程序 lubancat-flask-opencv-rknn/controller/utils/camera.py(有省略)

```
1 class VideoCamera(object):
2     def __init__(self):
3         # 使用 opencv, 打开系统默认摄像头
4         self.cap = cv2.VideoCapture(0)
5         if not self.cap.isOpened():
6             raise RuntimeError('Could not open camera.')
7
8         # 创建 RKNNLite 对象
9         self.rknn_lite = RKNNLite()
10
11        # 设置帧宽和高度
12        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
13        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 640)
14        # .....
15
16        # 退出程序释放资源
17        def __del__(self):
18            self.cap.release()
19            self.rknn_lite.release()
```

(下页继续)

(续上页)

```
20
21 # 摄像头获取帧
22 def get_frame(self):
23     ret, self.frame = self.cap.read()
24
25     if ret:
26         # 图像检测处理
27         if self.is_process:
28             #self.image = cv2.cvtColor(self.frame, cv2.COLOR_BGR2RGB)
29             self.image = cv2.cvtColor(self.frame, cv2.COLOR_BGR2RGB)
30             self.outputs = self.rknn_lite.inference(inputs=[self.image])
31             self.frame = process_image(self.image, self.outputs)
32             #self.rknn_frame = process_image(self.image, self.outputs)
33             #ret, image = cv2.imencode('.jpg', self.rknn_frame)
34             #return image.tobytes()
35
36         if self.frame is not None:
37             # 图像编码压缩
38             ret, image = cv2.imencode('.jpg', self.frame)
39             # 将图像作为一个字节对象返回
40             return image.tobytes()
41         else:
42             return None
43     # .....
```

28.3 NPU 处理图像

使用 NPU 进行图像检测处理，本示例单独训练模型，测试直接使用官方 Toolkit Lite2 工具中的 examples/onnx/yolov5 例程中的模型，仅供实验演示使用。

提示：板卡上 RKNN Toolkit Lite2 安装使用，可以参考前面 [教程](#)，或者 [官方 github 文档](#)。

示例程序处理流程：

列表 4: 截取于鲁班猫监控检测示例程序

```
1  # 创建 RKNNLite 对象
2  self.rknn_lite = RKNNLite()
3
4  #.....
5
6  def load_rknn(self):
7      # 导入 RKNN 模型
8      print('--> Load RKNN model')
9      ret = self.rknn_lite.load_rknn(RKNN_MODEL)
10     if ret != 0:
11         print('Load RKNN model failed')
12         exit(ret)
13     # 调用 init_runtime 接口，初始化运行环境
14     print('--> Init runtime environment')
15     ret = self.rknn_lite.init_runtime()
16     if ret != 0:
17         print('Init runtime environment failed!')
18         exit(ret)
19
20     #.....
21
22     # 对摄像头获取的图片进行处理，设置图片大小
23     self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
24     self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 640)
25
26     # 转换成 RGB 格式
```

(下页继续)

(续上页)

```
27 self.image = cv2.cvtColor(self.frame, cv2.COLOR_BGR2RGB)
28
29 # 调用 inference 接口, 对图片进行检测推理, 返回结果
30 self.outputs = self.rknn_lite.inference(inputs=[self.image])
31
32 # 根据 npu 处理结果, 对图像进行后处理, 返回处理后的图像
33 self.frame = process_image(self.image, self.outputs)
34
35 # 退出程序释放资源
36 def __del__(self):
37     self.cap.release()
38     # 用 release 接口, 释放 RKNNLite 对象
39     self.rknn_lite.release()
```

不用 npu 加速处理, 图像检测识别处理可以直接使用 opencv, 感兴趣的同学可以自行研究代码, 添加数据集训练, 或者更换其他模型进行实验。使用 opencv 库进行图像处理、数字识别功能在实验代码目录: controller/utils/opencvstest.py 中。

28.4 适配具体环境

拉取实验代码:

```
# 在终端中输入如下命令, 使用 main 分支:
git clone -b main https://gitee.com/LubanCat/lubancat-flask-opencv-rknn.git
```

修改服务器监听的 IP 及端口, 大家根据实际情况修改:

```
# 进入 lubancat-flask-opencv-rknn 目录
cd ./lubancat-flask-opencv-rknn

# 修改 main.py 文件中的启动函数参数
```

(下页继续)

(续上页)

```
vim main.py

# 默认设置为 host="0.0.0.0", 就是默认网口的 ip
app.run(threaded=True, host="0.0.0.0", port=5000)

# 根据板卡环境, 设置具体的 ip
app.run(threaded=True, host="192.168.103.121", port=5000)
# 现在服务器监听的 ip 地址为 192.168.103.121、端口为 5000.
```

教程测试使用的是 ov5648, mipi csi 接口。实际使用需要确认下摄像头编号 (一般是/dev/video0):

1、先确定摄像头编号:

```
# 进入 Python3 终端:
python3

# 导入 opencv 库包
import cv2

# 输入如下命令:
cap = cv2.VideoCapture(0)
cap.isOpened()
# 如窗口中打印出了 True, 则此设备编号可用。

# 确定编号后, 释放摄像头资源
cap.release()
```

若出现任何报错, 请将摄像头编号依次递增尝试, 找到可用的设备编号。编号递增上限为 video 设备的数量, 可使用 `ls /dev/video*` 命令查看。

2、修改实验代码, 以适配自己的摄像头:

```
# 在终端中输入如下命令:
```

(下页继续)

(续上页)

```
# 在 lubancat-flask-opencv-rknn 目录下进入 opencv 操作摄像头的代码目录中:
cd ./controller/utils/

# 修改 camera.py 文件中的 VideoCamera(object) 函数:
vim camera.py

# 将 self.cap = cv2.VideoCapture(0) 语句中的函数参数 0 修改为你摄像头对应的设备编号,
或者设备文件 ("/dev/video0")
self.cap = cv2.VideoCapture(0)
```

28.5 测试实验

1. 按照上一小节中的教程内容修改后，我们就可用运行实验代码来查看实验现象。

```
# 在工程代码目录 lubancat-flask-opencv-rknn 中，执行以下命令:
sudo python3 main.py
```

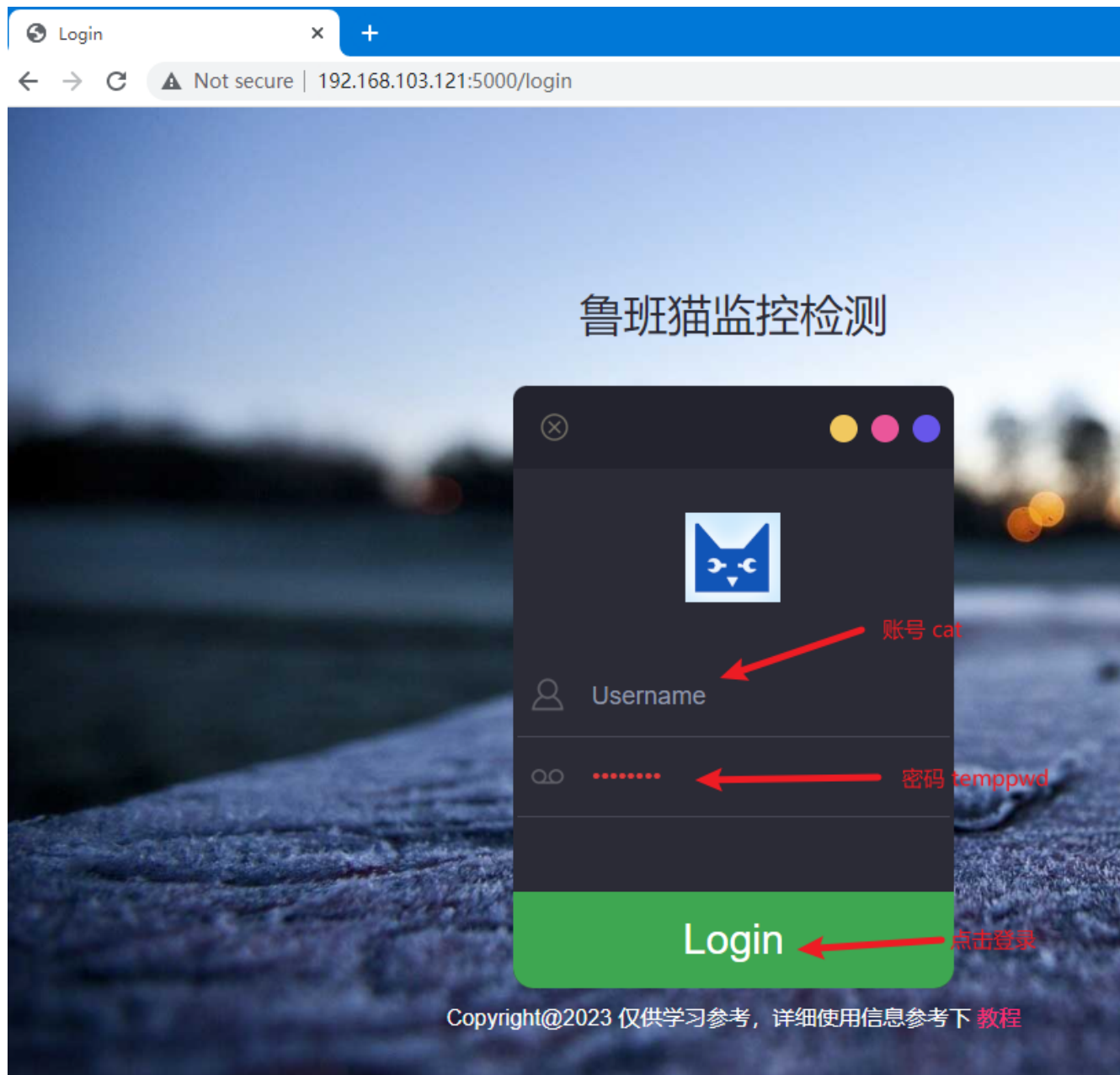
可用查看到如下实验现象：

```
cat@lubancat:~/lubancat$ sudo python3 main.py
* Serving Flask app "controller" (lazy loading)
* Environment: production
  WARNING: Do not use the development server in a production environment.
  Use a production WSGI server instead.
* Debug mode: on
I:werkzeug: * Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)
I:werkzeug: * Restarting with stat
W:werkzeug: * Debugger is active!
I:werkzeug: * Debugger PIN: 498-421-198
```

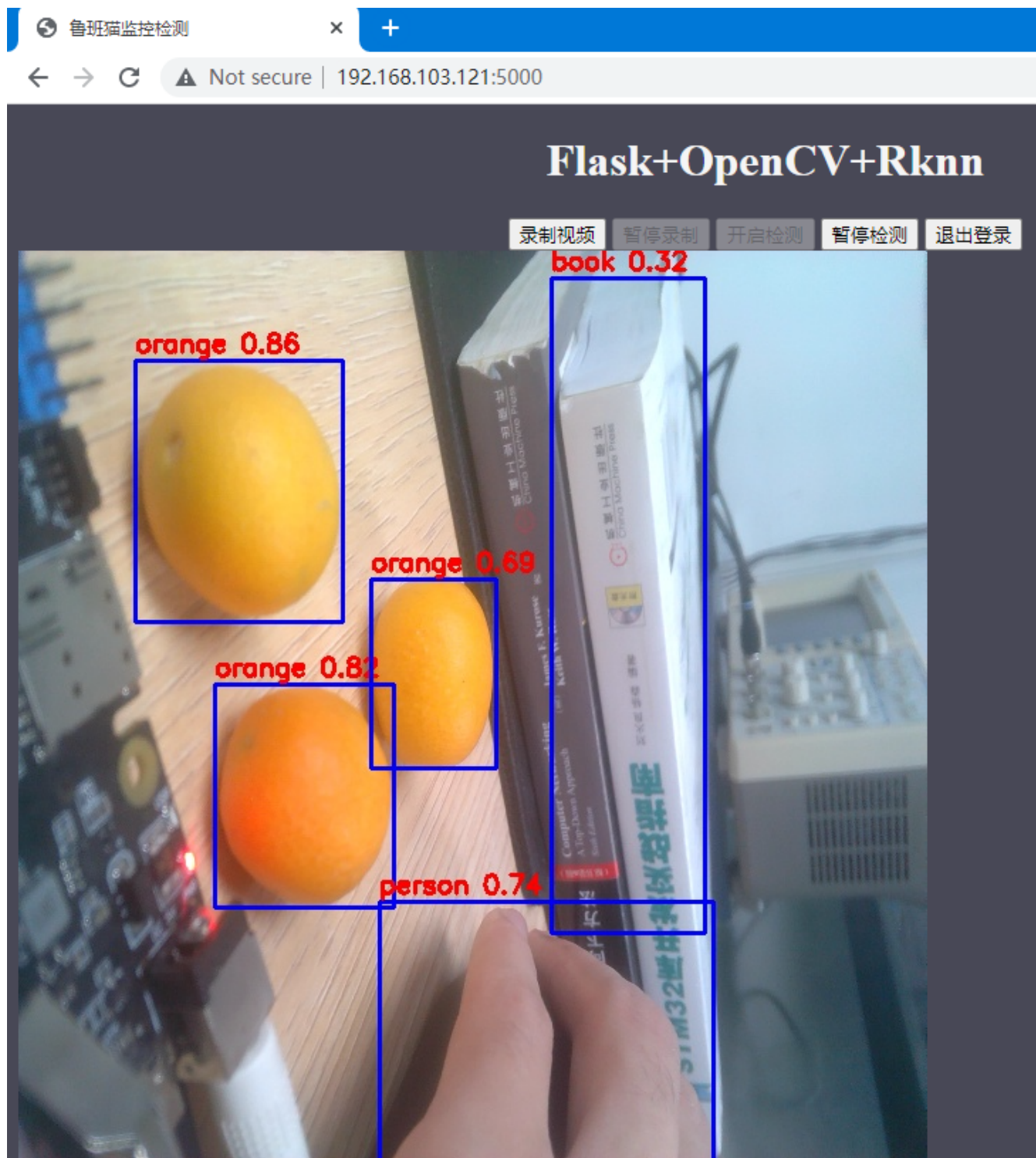
程序打印的提示信息，告诉我们服务器以及开始监听 `http://0.0.0.0:5000` 的地址，系统的默认网口 ip。如若想退出程序，按下 `CTRL+C`。

这里通过在浏览器中输入网址：<http://192.168.103.121:5000>，来观察一下实验现象。

实验现象如图：



2. 登录完成后，进入到监控界面，点击 开启检测进入到检测状态。



3. 一帧图像获取时间测试

通过 python 的 time 模块计算程序运行时间，来简单测试获取一帧图像需要的时间，以及经过 npu 处理后所需的时间。在程序中加上运行时间计算：

列表 5: controller/modules/home/views.py

```
1 # 获取视频流
2 def video_stream():
3     global video_camera
4     global global_frame
5
6     if video_camera is None:
7         video_camera = VideoCamera()
8
9     while True:
10         start_time = time.time()
11         frame = video_camera.get_frame()
12         end_time = time.time()
13         print('get_frame cost %f second' % (end_time - start_time))
14         #time.sleep(0.01)
15         if frame is not None:
16             global_frame = frame
17             yield (b'--frame\r\n'
18                   b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n\r\n')
19         else:
20             yield (b'--frame\r\n'
21                   b'Content-Type: image/jpeg\r\n\r\n' + global_frame + b'\r\n\r\n')
```

测试使用的是 lubancat 2，鲁班猫监控检测示例程序，使用 debian10 系统，默认系统配置，使用 Toolkit Lite2 工具中 yolov5 例程模型，下面测试数据仅供参考。

在不开启 npu 检测图像的情况下，正常运行程序，一帧图像大约需要 65ms，其中出现最快是 23ms，最慢是 121ms

开启 npu 处理图像，NPU 默认 600Mhz，运行：获取一帧图像大约需要 345ms（包括获取摄像头

图片，预处理图片，npu 处理图片，图片后处理画框等时间)

设置 NPU 频率 900Mhz 测试：获取一帧图像大约需要 314ms

```
# lubancat2 npu 设置 900Mhz:  
echo 900000000 > /sys/class/devfreq/fde40000.npu/userspace/set_freq
```

鲁班猫监控检测示例，实现了简单的监控显示和目标检测功能，效果不是最优的，仅用于学习参考。

28.6 参考链接

<https://gitee.com/LubanCat/lubancat-flask-opencv-rknn/tree/main/>

<https://github.com/miguelgrinberg/flask-video-streaming>

<https://gitee.com/Embedfire/flask-video-streaming-recorder>

<https://github.com/rockchip-linux/rknn-toolkit2>

第 29 章 问题反馈

限于学识和时间，文档编辑难免有误，对文档有任何编写建议或者问题，欢迎您将问题和建议发到邮箱 zgwinli555@163.com，我们将及时修改和反馈。

版权说明

野火电子保留本项目的版权所有权。